

LLM Inference / Serving

推理服务、KV cache、PagedAttention、RadixAttention 和多云 GPU 编排。

97 篇 · 2026-10-04

目录

1. LLM Inference / Serving 项目地图
2. LLM Inference
3. PagedAttention
4. RadixAttention
5. AI Infra 中文学习项目地图
6. GitHub Stars P0-P2 实现地图
7. Kubernetes DRA Design Deep Dive
8. Kubernetes GPU / Device Stack 项目地图
9. KVCacheD × SGLang × vLLM 源码深潜
10. KVCacheD × SGLang × vLLM 集成与实验手册
11. KVCacheD、SGLang 与 vLLM 三项目知识体系
12. LLM Serving / 推理引擎选型地图
13. llm-d / Kubernetes SIGs 候选项目地图
14. Agent Substrate 架构与设计思路分析
15. Agent 凭据隔离
16. agent-sandbox
17. agent-sandbox 架构与设计思路分析
18. agentgateway 架构与设计思路分析
19. AI Infra Learning 中文 Star 项目清单整理
20. AIBrix
21. AIBrix 架构与设计思路分析
22. Batch Inference
23. Container Device Interface
24. Disaggregated Serving
25. DRA Driver for NVIDIA GPUs
26. DRA Driver for NVIDIA GPUs 架构与设计思路分析
27. Dynamo
28. Elastic KV Cache

29. Gateway API Inference Extension
30. Gateway API Inference Extension 架构与设计思路分析
31. GitHub Stars P0-P2 当前状态核验
32. GPU Sharing
33. GPU 编程学习
34. GPUStack
35. GPUStack 架构与设计思路分析
36. HAMi 架构与设计思路分析
37. HiClaw
38. HiClaw 架构与设计思路分析
39. Inference Routing
40. inference-perf
41. inference-perf 架构与设计思路分析
42. K8s GPU & Device Plugins Star 项目清单整理
43. Kthena
44. KubeAI
45. KubeAI 架构与设计思路分析
46. Kubernetes Device Plugin
47. Kubernetes Dynamic Resource Allocation
48. Kubernetes Serving Stack 对比: llm-d、AIBrix、KServe、KubeAI、OME、GPUStack、RBG、Kthena
49. KV Cache Offload (KV 多级缓存)
50. KVCacheD
51. KVCacheD 与 SGLang、vLLM 架构关系及源码实现
52. LeaderWorkerSet
53. LeaderWorkerSet 架构与设计思路分析
54. LLM 网关与路由层对比
55. llm-d
56. llm-d Batch Gateway
57. llm-d Batch Gateway 架构与设计思路分析
58. llm-d Benchmark

- 59. llm-d Benchmark 架构与设计思路分析
- 60. llm-d Inference Sim
- 61. llm-d Inference Sim 架构与设计思路分析
- 62. llm-d KV Cache
- 63. llm-d KV Cache 架构与设计思路分析
- 64. llm-d Latency Predictor
- 65. llm-d Latency Predictor 架构与设计思路分析
- 66. llm-d P/D Utils
- 67. llm-d P/D Utils 架构与设计思路分析
- 68. llm-d Planner
- 69. llm-d Prism
- 70. llm-d Prism 架构与设计思路分析
- 71. llm-d Router
- 72. llm-d Router 架构与设计思路分析
- 73. llm-d Workload Variant Autoscaler
- 74. llm-d Workload Variant Autoscaler 架构与设计思路分析
- 75. llm-d 架构与设计思路分析
- 76. llm-d 核心项目群架构再调研
- 77. Model Serving Operator
- 78. MUSA Learning Notes
- 79. MUSA Learning Notes 架构与设计思路分析
- 80. NVIDIA Dynamo 架构与业务问题（文档重构版）
- 81. NVIDIA GPU Operator
- 82. NVIDIA GPU Operator 架构与设计思路分析
- 83. NVIDIA k8s-device-plugin
- 84. NVIDIA k8s-device-plugin 架构与设计思路分析
- 85. OME 架构与设计思路分析
- 86. PowerMem
- 87. PowerMem 架构与设计思路分析
- 88. RBG（RoleBasedGroup）

- 89. SGLang
- 90. SGLang 架构与推理优化
- 91. SkyPilot
- 92. SkyPilot 架构与设计思路分析
- 93. substrate
- 94. vLLM
- 95. vLLM Semantic Router
- 96. vLLM Semantic Router 架构与设计思路分析
- 97. vLLM 架构与推理优化

LLM Inference / Serving 项目地图

当前上游核验 (2026-09-22)

本页把官方仓库/文档的当前观察与既有 raw-backed Source 摘要分开。Source 摘要仍保存其分析时点；下表用于判断本次横向地图是否需要调整。

项目	核验版本	当前稳定职责	本次处理
vllm	HEAD d50723df04f7	engine scheduler、model execution、local KV	保留内部图，补外部边界
sglang	HEAD 04c0913434c4	engine/runtime、RadixCache、distributed/P-D integration	保留内部图，补集成边界
dynamo	HEAD f36d2fab37fd	distributed request/control/state runtime、routing、KV transfer、planner	更新平台层关系
llm-d	HEAD 1e9a86a3a9da	Proxy/EPP、InferencePool、Model Server 与 routing signals	更新 Gateway/EPP 热路径
aibrix	HEAD 96056b47f158	K8s routing、autoscaling、adapter/model lifecycle、KV/multi-role orchestration	纳入核心比较

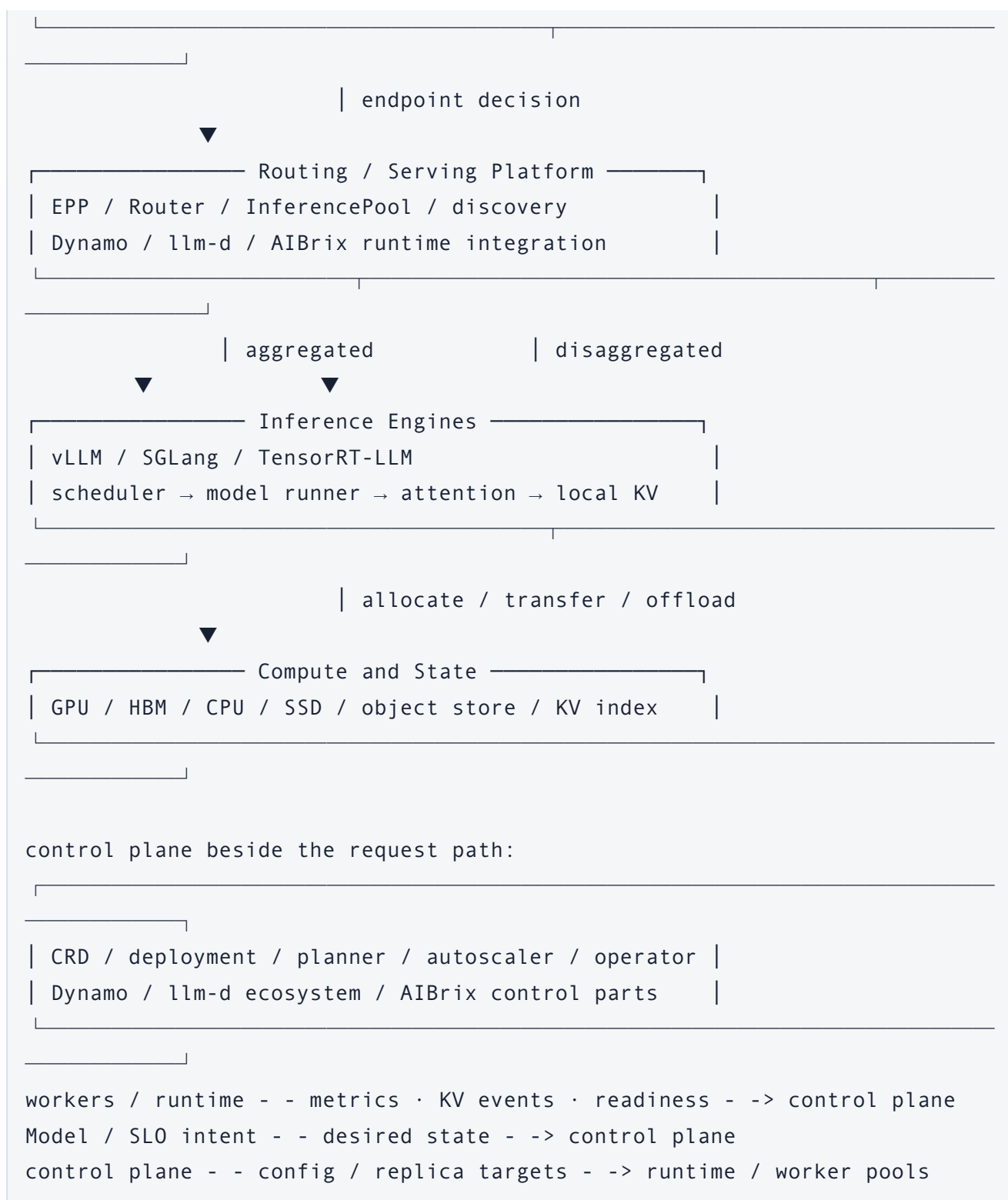
[note] 证据边界
当前职责来自本次官方仓库与文档核验；内部调用路径仍以对应 Source 页面为准。

官方架构/文档入口：[vLLM](#)、[SGLang](#)、[Dynamo](#)、[llm-d](#)、[AIBrix](#)。

这页是 M4 的 L1 职责地图，用 D1 – D5 连接入口、选点、推理引擎、KV 状态和部署控制。项目内部的完整调用图见各 Source；组合选型见 [llm-serving-engine-selection-map](#)。实线箭头只表示同步请求或数据传输，带标签的虚线箭头 - - signal / control - -> 表示异步观察或控制；控制器不串入逐请求执行链。D5 单独采用故障场景的因果记法，见该节说明。

D1 · 模块边界图





这是职责地图，不表示每个部署都包含每个方框。[vllm](#) / [sglang](#) 是推理引擎；[dynamo](#)、[llm-d](#)、[aibrix](#) 覆盖的外围 runtime、路由和控制面范围不同，不能把三者当成同一种 engine。Gateway API、HTTPRoute、InferencePool 是配置/发现契约，实际接收请求的是 Proxy/Frontend；KV index 保存位置线索，不等同于 KV 数据存储。D1 表示依赖边界，实际请求和异步循环分别展开在 D2、D3。

D2 · 在线请求热路径

aggregated:

Client → Gateway/Frontend → Router/EPP → Engine Scheduler → Model Runner → stream

disaggregated:

Client → Gateway/Frontend → Router/EPP



连接处理、endpoint 选择、token 执行是三项不同职责。Gateway/Frontend 接收连接、执行入口策略并转发流；Router/EPP 根据模型、负载和 KV locality 选 worker；引擎 Scheduler 组成 batch，Model Runner 执行模型。图中 Router/EPP 是逻辑选点步骤：在 [llm-d 的 Proxy/EPP 结构](#)和 [AIBrix 的 ext_proc 结构](#)中，Proxy 咨询选点服务后直接向 engine 转发，请求流不必穿过 EPP 进程。

P/D 模式还需要 [KV-transfer 契约](#)：请求身份、KV 布局/传输元数据、接收完成条件、超时和取消必须由 backend 组合共同定义。图中的双分支表示角色协作，不规定统一的触发顺序；具体由谁发起 prefill、谁等待 KV、谁返回 stream，要查该部署的 engine/connector/router 配置。

D3 · 扩缩与部署控制循环

Model / SLO / topology intent

|| desired state



CRD / Deployment / InferencePool / platform config

|| watch / reconcile



Operator / Planner / Autoscaler

|| config / replica targets



worker pools / engine pods

- - metrics / queue / KV / readiness - -> Operator / Planner / Autoscaler

这是后台收敛循环，竖向的 | 与横向的 -- 都表示异步观察/控制，副本目标并非同步请求调用。[dynamo](#) 的 Planner 与 Kubernetes Operator 可以分别承担容量决策和部署执行；[llm-d](#) 用 InferencePool 表达后端集合，部署和扩缩由配套的 Kubernetes 组件承担；[aibrix](#) 的 controllers 管理模型/adaptor、角色组和副本，AI

Runtime 协助 pod 内的模型生命周期。它们的 API、运行时依赖和控制对象不同，接入时要确认谁拥有副本目标，避免两个控制器竞争写入。见 [model-serving-operator](#)、[src-dynamo-architecture](#)、[src-llm-d-architecture](#)、[src-aibrix-architecture](#)。

[!warning] Conflict

2026-09-22 的 [llm-d dev 架构文档](#) 已将 Workload Variant Autoscaler 标为 deprecated，并描述 EPP metrics → KEDA Prometheus scaler → HPA 的扩缩路径。[src-llm-d-workload-variant-autoscaler-architecture](#) 与 [llm-d-workload-variant-autoscaler](#) 保存的是较早的 WVA 设计；本页将其作为历史设计参考。Source 保留原分析时点，版本迁移与实际发布适配仍需单独核验。

D4 · KV 状态生命周期

```
prompt tokens
  | prefill
  ▼
local KV blocks - - publish events/index - -> routing locality signal
  |
  ├── direct P→D transfer —————→ decode-local KV
  ├── offload → CPU / SSD / remote tier → recall/promote
  └── pressure / expiry / model change → evict or recompute
```

引擎拥有 local KV 的表示、分配和有效性判断，包括 block/page、prefix identity 与模型布局约束。外围层可以索引、路由、传输和 offload：[llm-d-kv-cache](#) 的 locality signal 帮助选点，[dynamo](#) 的传输/缓存组件和 [aibrix](#) 的 KV 能力则按具体集成参与跨 worker 或跨存储层的复用。路由索引里的“可能命中”不能代替引擎对 KV 的验证。

直接 P→D 传输、分层 offload 和事件发布是不同接口，不要求同时启用。CPU/SSD/remote tier 是候选层级；可用介质、格式兼容性、缓存失效和回收策略取决于版本及 backend。见 [kv-cache-offload](#)、[paged-attention](#)、[radix-attention](#)。

D5 · 故障与降级边界

本节的 failure → conditional response 箭头表示“故障发生后，按契约采取条件响应”的因果场景，不表示同步调用链；其中涉及的发现、摘除和扩缩仍遵循异步控制语义。

```
EPP/router unavailable → fail open / fail close / reject according to
gateway policy
worker not ready          → discovery removes/inhibits endpoint → choose
another worker
prefill/decode failure → cancel, retry, or recompute according to backend
```

contract	
KV index stale	→ lower hit quality; engine correctness remains independent
autoscaler lag	→ queue/load shedding protects the request path

这张图列的是部署时需要明确的故障契约，不表示所有项目都实现每一种 fallback。EPP 故障能否 fail open 取决于 Gateway 配置及是否存在合法的备用路由；worker 摘除依赖 readiness/discovery 的传播，新选点也可能遇到旧状态。P/D 已输出 token 后是否可重试或迁移，必须检查 backend 对流、采样状态和取消的支持，不能假定透明恢复。

KV 索引只作为优化信号且引擎独立验证缓存身份时，索引过期通常影响命中率与排队质量；这不代表 KV 数据损坏或传输失败也无害。扩缩存在启动延迟，只有配置了有界队列、admission 或 load shedding，热路径才有相应保护。故障注入应分别覆盖 Gateway、worker、KV 通道和控制器，见 [inference-routing](#)。

一句话分层

项目 / 概念	一句话定位	抽象层
vllm	推理引擎基线， paged-attention 把 KV cache 按 block 管理，调度和模型执行在 engine 内完成	推理引擎，可跨设备/节点并行
sglang	以 radix-attention 、Scheduler/ModelRunner 和执行优化组织 serving runtime	推理引擎，可跨设备/节点并行
dynamo	NVIDIA 数据中心级 LLM 推理编排层，把 vLLM/SGLang/TRT-LLM 组成协调集群	多节点 serving 编排
llm-d	CNCF Sandbox 分布式 LLM inference serving stack，围绕 Router/EPP、InferencePool、KV/P-D/autoscaling 组织	K8s serving control plane
aibrix	围绕 engine fleet 提供路由、扩缩、model/adapter lifecycle、KV 和多角色编排	K8s serving/control plane
llm-d-router	llm-d 智能入口层，用 EPP filters/scorers/scrapers 对 InferencePool endpoints 做选择	Endpoint picking / routing
llm-d-kv-cache	llm-d KV locality index / scorer，把 vLLM/SGLang KV events 转换成 cache-hit routing signal	KV-aware routing signal
llm-d-batch-gateway	OpenAI Batch API / 离线推理控制面，把 batch job/file/queue/output 接到下游 llm-d Router/model endpoint	Batch serving control plane
llm-d-benchmark	llm-d 实验编排器，把 scenario/spec、K8s lifecycle、harness 和 result workspace 串起来	Benchmark lifecycle

项目 / 概念	一句话定位	抽象层
llm-d-workload-variant-autoscaler	已摄入的多 variant 全局 allocation 设计；当前 dev 文档已标 deprecated，见 D3 冲突说明	历史 autoscaling 设计
llm-d-inference-sim	无 GPU vLLM 行为模拟器，用 OpenAI/vLLM API、KV events、latency/failure/metrics 验证控制面	Simulator / test double
SkyPilot	AI/ML 多云算力控制平面，负责跨云/K8s/Slurm 选择资源、failover、managed jobs/serve	算力控制平面
K8s GPU stack	device plugin / GPU Operator / DRA / CDI / DCGM / GPU sharing	GPU 资源基础设施

横向对比

按主要职责分组；同一项目可以跨多个边界，表中不把它们视为可直接互换的 engine。

层	项目	核心抽象与职责	KV / P-D 边界	采用代价与下一步验证
推理引擎	vllm	EngineCore / Scheduler / ModelRunner, batch 与模型执行	local KV block; prefix cache、connector 随配置启用	用目标模型/硬件验证吞吐、TTFT/ITL 和 connector; block 大小不能写成统一常量
推理引擎	sglang	Scheduler / ModelRunner / RadixCache, 执行优化与并行	prefix reuse、HiCache、P/D integration 依 backend 而定	验证投机解码、chunking、并行与传输的特性组合
分布式 runtime	dynamo	Frontend / Router / discovery / worker graph, 另有 Planner/Operator	可组织 aggregated 或 P/D workers, 集成 KV events/transfer/offload	需要运维 runtime 与控制组件; 逐项验证 backend 支持和故障契约
K8s serving/control plane	llm-d	Proxy + EPP / InferencePool / Model Server	以事件、scorer 和可选 P/D 路径连接 engine	需要 Gateway API/GAIE 兼容性; 扩缩组件按当前版本验证, 见 D3
K8s serving/control plane	aibrix	Gateway plugins / controllers / AI Runtime, 管理 engine fleet	KV、角色编排和 model/adaptor lifecycle 按需组合	验证引擎适配、CRD 生命周期、路由策略与副本归属

层	项目	核心抽象与职责	KV / P-D 边界	采用代价与下一步验证
基础设施/算力控制	SkyPilot	Task/Dag/Resources + Optimizer, 资源选择与作业/服务生命周期	部署引擎和服务, 不定义 local KV 布局	需对接云/K8s/Slurm 凭据、配额与容量; 不解决模型执行性能
基础设施/设备	K8s GPU stack	DevicePlugin / DRA / CDI / Operator / observability	提供设备、拓扑和健康信号	设备分配、隔离和驱动兼容性需独立验证

架构交叉矩阵

交叉模式	采用项目	工程含义
KV cache 显式管理	vllm , sglang , dynamo , llm-d-kv-cache , aibrix	区分 engine 内 KV 分配、外部索引与存储层; 职责见 D4
前缀复用	vllm , sglang , dynamo	system prompt、few-shot、agent template 可共享, 路由/调度要感知 prefix
Prefill/Decode 分离	vllm , sglang , dynamo , llm-d , aibrix	engine connector 与平台路由协作; 收益取决于 workload、带宽和传输契约
多级 KV offload / locality signal	dynamo , kv-cache-offload , llm-d-kv-cache	GPU 显存不够时, 把 KV 分层; 路由时则把 KV locality 变成 endpoint score
Speculative decoding	sglang , vllm	用 draft/target 或 ngram 等方法降低 decode latency
K8s 控制面	dynamo , llm-d , aibrix , SkyPilot	不同系统分别收敛 serving graph、engine fleet 或底层算力, 见 D3
Batch / benchmark / simulator	llm-d-batch-gateway , llm-d-benchmark , llm-d-inference-sim	serving 选型需要异步任务、可复现实验和低成本控制面替身
Variant allocation (历史设计)	llm-d-workload-variant-autoscaler	可研究多硬件/多角色的容量分配; 当前采用路径须考虑 D3 的 deprecated 状态
资源经济	SkyPilot , dynamo Planner, K8s GPU stack	不只是跑得快, 还要按 SLA、成本和容量调度

核心设计轴

1. KV cache：从内存优化到系统接口

LLM serving 的很多架构分歧都来自 KV cache：

- [vllm](#) 的 [paged-attention](#) 把 KV cache 切成 block，解决传统最大 seq_len 预分配浪费。
- [sglang](#) 的 [radix-attention](#) 把前缀共享做到 token 级，适合 agent template、few-shot、tree-of-thought 这类大量共享前缀场景。
- [dynamo](#) 在引擎外连接 KV events/index、传输与可选缓存层；[src-dynamo-architecture](#) 中的 KVBM/SequenceHash 细节属于该分析版本，部署时需核对 backend 集成。

结论：KV cache 已经从“GPU 内部 buffer”变成 serving 系统的一等资源。后续的路由、扩缩、迁移、offload、GPU sharing 都需要知道 KV 的存在。

2. 引擎执行和集群编排是两层问题

[vllm](#) / [sglang](#) 负责引擎内执行，实例本身也可使用跨设备/节点并行；[dynamo](#) 组织实例间 runtime，[llm-d](#) / [aibrix](#) 则从 Kubernetes serving 与 fleet 管理边界补齐路由和控制能力。

```
engine execution: tokenizer → scheduler → model runner → attention / local KV
cluster requests: frontend → route → aggregated or P/D workers → stream
cluster control: metrics - - -> capacity decision - - -> deployment / replicas
```

把这两层混淆会导致选型误判。vLLM/SGLang 的强项是内核执行与 batching；Dynamo 的强项是多节点协调、KV-aware routing、P/D disaggregation 和 SLA planner。SkyPilot 再往上，解决集群/云在哪里、怎么拉起、失败怎么换区/换云。

3. P/D 分离改变了 GPU 池形态

Prefill 和 decode 的资源特征不同：

- prefill：长 prompt，计算密集，吞吐更像大矩阵计算。
- decode：逐 token，低 batch 情况下内存带宽和 KV 访问更敏感。

[disaggregated-serving](#) 把两者拆开后，系统需要额外解决：

- prefill worker 和 decode worker 如何匹配。
- KV 如何从 prefill 传到 decode。
- decode worker 是否已有相关 prefix KV。
- prefill/decode 分别按什么指标扩缩。
- 请求迁移时哪些状态可重建，哪些不可复制。

[Dynamo 官方架构](#)覆盖 aggregated 与 disaggregated worker 组织；P/D 是需要选择的部署模式。[sglang](#) / [vllm](#) 的传输接口和 [llm-d](#) / [aibrix](#) 的外围编排应按 backend 组合核验，不能用项目名推断默认启用或完整兼容。

4. 路由不再只是负载均衡

传统 HTTP 负载均衡按连接数、延迟或权重分发。LLM serving 的路由还要考虑：

- 哪个 worker 已有 prompt prefix KV。
- prefill 队列和 decode 队列哪个更忙。
- KV 在 GPU、CPU、磁盘还是远端对象存储。
- 当前请求是否可迁移。
- LoRA/model adapter 是否匹配。

[src-dynamo-architecture](#) 描述的 KV-aware routing 用 prefix overlap 与负载估计平衡复用和排队；cost/softmax 等策略细节应随版本核验。[llm-d-router](#) 和 [aibrix](#) 也将 engine 状态纳入选点，但信号格式、策略插件和失败行为不同。见 [inference-routing](#)。

5. GPU 资源层正在变成 serving 架构的一部分

K8s GPU 资源层不再只是“NVIDIA device plugin 把 /dev/nvidia0 暴露给 Pod”。

当前材料已经显示出完整栈：

- device plugin / GPU Operator / container toolkit。
- GPU Feature Discovery / DCGM exporter / GPUD。
- GPU sharing / vGPU：HAMi、vgpu-scheduler、gpushare、Volcano vGPU。
- DRA / CDI：下一代设备声明和分配。
- KV cache 资源化：kvcached 这类项目把 KV 也推向可调度资源。

当 serving 系统进入 P/D 分离、多级 KV、GPU sharing 后，K8s 的设备 API、DRA/CDI、GPU health 和调度策略会直接影响模型服务设计。

项目工程剖面

[vllm](#)：推理引擎基线和 PagedAttention

[vllm](#) 以 Scheduler、ModelRunner 和 local KV 管理承接 token 执行。PagedAttention 的 block table 让显存按需分配；内部调用图见 [src-vllm-architecture](#)，当前职责对应上方官方核验。

适合借鉴的点：

- 把 KV cache 从连续大 buffer 变成可管理 block。
- 把 OpenAI-compatible serving 做成事实入口。
- 生态兼容和模型覆盖优先。

主要局限：

- KV page/block 粒度和 prefix reuse 需要按模型、attention/backend 与配置核验。
- 引擎的分布式执行或 KV connector 不等于完整的 Gateway、fleet lifecycle 和容量控制。
- 集群级 SLA 与跨副本路由需明确外部组件归属；offload 可以有 engine 侧接口及外围存储实现。

sglang：执行路径与 KV 复用

[src-sglang-architecture](#) 在 2026-09-14 的分析版本中展示了以下结构；进程数、backend 数和特性兼容性随版本与部署方式变化：

- HTTP / TokenizerManager / Scheduler / DetokenizerManager 的职责分离。
- RadixCache token 级 KV 复用。
- EXTEND / DECODE / MIXED 等 forward mode。
- 可选 attention backend 与融合 kernel。
- speculative decoding 的 draft/verify 路径。
- P/D KV transfer backend 集成。
- OpenAI / Anthropic / Ollama / gRPC / Engine API。

它提供了研究执行优化和缓存复用的具体路径；代价是特性组合约束复杂。speculative、chunked prefill、disagg 能否组合，需要目标版本、模型和硬件上的验证。

dynamo：数据中心级 serving 编排

[dynamo](#) 把 engine backend 接入分布式 serving runtime。请求经 Frontend/Router 到 aggregated 或 P/D workers；KV events/index 为路由提供异步信号，Planner/Operator 在请求链外调整容量。完整内部图见 [src-dynamo-architecture](#)，跨项目职责见 D1 – D4。

它的工程难点集中在三平面解耦：

- request plane：低延迟请求路径。
- control plane：discovery、desired state、部署与容量收敛。
- storage/event plane：KV visibility、事件传播与可选 offload。

它把 [vllm](#) / [sglang](#) / TensorRT-LLM 当 backend，围绕多节点路由、KV transfer 和容量管理组织协作。传输、事件后端、缓存层级及请求迁移能力都需按所选 backend 和配置确认，不能从 runtime 存在推断所有能力已启用。

llm-d：Gateway/EPP 与 Kubernetes serving 组合

[llm-d](#) 以 Proxy、EPP、InferencePool、Model Server 为主要边界：Proxy 承接连接，EPP 选 endpoint，engine 执行模型。KV、P/D、batch、benchmark 和扩缩是围绕这些边界组合的能力，具体采用路径随版本演进。当前扩缩状态见 D3；历史与内部架构见 [src-llm-d-architecture](#)、[src-llm-d-router-architecture](#)，对比见 [src-k8s-serving-stack-comparison](#)。

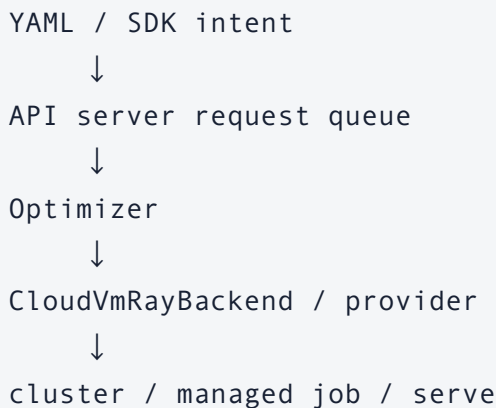
[aibrix](#): engine fleet 的路由与生命周期管理

[aibrix](#) 围绕 engine fleet 组合 Gateway routing、controllers、autoscaling 与 AI Runtime，并提供 model/adapter、KV 和多角色编排能力。它的控制资源与 pod 内适配职责有别于 llm-d 的 InferencePool/EPP 边界，也有别于 Dynamo 的分布式 runtime。接入前应明确模型/adapter 生命周期、副本写入权和引擎适配接口。见 [src-aibrix-architecture](#)、[src-k8s-serving-stack-comparison](#)。

[SkyPilot](#): 算力控制面而不是 serving engine

[SkyPilot](#) 不优化 attention kernel，也不管理 KV cache。它解决的是：在 Kubernetes、Slurm、公有云、on-prem 之间怎么选择资源、启动集群、failover、运行 managed jobs / serve。

它在 serving 地图里的位置更高：



对 LLM serving 来说，SkyPilot 适合承接“服务应该部署在哪、容量不足如何换 region/cloud、成本如何比较”的问题。它不替代 Dynamo/SGLang/vLLM，而是它们上方的资源选择和执行控制面。

[K8s GPU stack](#): 设备层正在上移

K8s GPU & Device Plugins star list 已经足够说明：GPU 资源层正在从 device plugin 扩展为完整平台。

关键分层：

- runtime/container integration: NVIDIA container toolkit、CDI、NVML bindings。
- Kubernetes exposure: NVIDIA device plugin、GPU Operator、GPU Feature Discovery。
- sharing/virtualization: HAMi、vgpu-scheduler、Volcano vGPU、DRA。
- observability/diagnostics: DCGM exporter、GPUd、fake GPU。
- workload layer: TensorRT、KV cache virtualization。

对于 serving 系统，这层决定了 GPU 能不能细粒度共享、能不能按拓扑调度、能不能自动诊断、能不能把 KV cache/显存压力暴露给调度器。

核心难点

1. 吞吐和延迟不是同一个优化目标

Continuous batching 提高吞吐，但可能增加单请求排队延迟；speculative decoding 降低 decode latency，但引入 draft model 和验证开销；P/D 分离提高资源利用，但多一次 KV transfer。Serving 系统必须按 workload profile 调优，而不是追求单一指标。

2. KV 复用和负载均衡天然冲突

把请求发给已有 prefix KV 的 worker 能省 prefill，但那个 worker 可能很忙。把请求发给空闲 worker 延迟可能更低，但要重新 prefill。[dynamo](#)、[llm-d-router](#) 和 [aibrix](#) 的策略研究都需要在实测中比较命中收益与排队代价。

3. P/D 分离引入状态转移问题

Prefill 输出的 KV 必须被 decode 节点可见。这个过程涉及 RDMA/NIXL/Mooncake/Mori/Ascend 等 transfer backend，也涉及失败恢复：worker 挂掉后请求是否可迁移，guided decoding/n>1 这种状态机是否可复制。

4. 多级 KV offload 是缓存系统，不是简单 swap

把 KV 从 GPU 放到 CPU/NVMe/S3，需要回答：

- 谁决定 evict/promote。
- 访问代价如何进入 router。
- 多 worker 是否共享。
- hash 身份如何去重。
- cache miss 是否比重新 prefill 更慢。

[src-dynamo-architecture](#) 的 KVBM 材料可作为缓存层级和身份管理的设计参考；具体层级与策略应按版本验证。跨系统讨论见 [kv-cache-offload](#)。

5. GPU 资源调度和模型调度开始交叉

GPU sharing、MIG/vGPU、DRA/CDI、KV cache offload、P/D 分离都会改变“一个 Pod 需要几张 GPU”的简单假设。Serving control plane 需要更懂 GPU 拓扑、NVLink、NUMA、显存压力、KV cache 热度。

6. 可观测必须覆盖 token、KV、队列和 GPU

普通 HTTP latency 不够。现代 LLM serving 至少要观测：

- TTFT / ITL / output token latency。
- prefill/decode queue depth。
- KV hit/miss、tier、transfer latency。

- batch size、chunked prefill、CUDA graph hit。
- GPU utilization、memory pressure、DCGM health。
- autoscaler decision 和 provider capacity error。

设计分型

分型	代表	核心问题
推理引擎	vllm 、 sglang	local KV、batching、model execution、并行与 connector
数据中心编排层	dynamo	多节点 routing、KV offload、SLA autoscale、K8s operator
K8s serving/control plane	llm-d 、 aibrix 、 kserve	分别比较 endpoint picking、engine fleet lifecycle 与模型服务 API 的边界
算力控制面	SkyPilot	多云/K8s/Slurm 资源选择、failover、managed jobs/serve
GPU 资源层	K8s GPU stack	device plugin、DRA、GPU sharing、observability

选型建议

目标	优先看	工程关注点
建立可测量的 serving baseline	vllm / sglang	先测目标模型、硬件、TTFT/ITL，再比较执行特性
研究现代推理引擎内部优化	sglang	RadixAttention、Scheduler、spec decoding、P/D transfer
构建多节点高 SLA serving 集群	dynamo	KV-aware routing、KVBM、Planner、K8s operator
标准化 Gateway/endpoint picking	llm-d	Proxy/EPP、InferencePool、KV 信号及当前扩缩集成
管理 engine fleet 与 adapter 生命周期	aibrix	Gateway plugins、controllers、AI Runtime、KV/角色编排
跨云/跨集群管理 AI workload	SkyPilot	optimizer、failover、managed jobs、serve
建生产 GPU Kubernetes 底座	k8s-gpu-device-stack	device plugin、GPU Operator、DRA/CDI、DCGM/GPUd

下一批候选

详见 [llm-d-kubernetes-sigs-candidate-map](#)。已有的 Entity/Source 不再列为待补项目；下一步聚焦尚未验证的跨层问题：

- 用 [llm-d-benchmark](#)、[inference-perf](#) 和 [llm-d-inference-sim](#) 构造相同 workload，比较路由策略、KV 命中率与 TTFT/ITL，而非只比吞吐。
- 核验 llm-d 当前 EPP/KEDA/HPA 路径与历史 WVA 的迁移边界，确认可用 release、配置差异和多 variant 场景。
- 联合 kueue、karpenter、[lws](#)、jobset 验证容量不足、模型冷启动与 worker group readiness 的时间线。
- 针对 D5 做 Gateway/EPP、P/D worker、KV 通道的故障注入，记录可重试阶段、取消传播和流式输出语义。

当前知识库缺口

- 还缺少同模型、同硬件、同 SLO 的跨项目实测，当前地图说明职责，不能据此直接给出性能排名。
- KV event schema、身份/布局兼容性和失效传播尚缺跨 engine/connector 的契约对照；“可接入”不等于任意组合可共享 KV。
- P/D 故障后在飞请求、已输出 token 和采样状态能恢复到哪一步，尚缺按 backend 分类的证据。
- 多级 KV 存储、GPU sharing 与拓扑调度如何共同影响路由/扩缩决策，仍需测量与控制权分析。

相关页面

- [llm-inference](#)
- [vllm](#)
- [sglang](#)
- [dynamo](#)
- [llm-d](#)
- [aibrix](#)
- [llm-serving-engine-selection-map](#)
- [inference-routing](#)
- [model-serving-operator](#)
- [paged-attention](#)
- [radix-attention](#)
- [disaggregated-serving](#)
- [kv-cache-offload](#)
- [kubernetes](#)
- [llm-d-kubernetes-sigs-candidate-map](#)

LLM Inference

LLM 推理 (inference / serving) 指把训练好的大语言模型部署成在线服务, 对外提供 token 生成 API。核心挑战: 高吞吐、低延迟、长 context、多并发、成本。

M4 阅读入口

- 先看 [llm-inference-serving-project-map](#): 理解 engine、routing、distributed runtime、Kubernetes control plane 和 GPU infrastructure 的职责边界。
- 再看 [llm-serving-engine-selection-map](#): 先选择缺失的架构层, 再选择项目或组合。
- 需要下钻时进入 [vllm](#)、[sglang](#)、[dynamo](#)、[llm-d](#)、[aibrix](#) 及对应 Source 页面。

系统分层

层级	代表项目	关注点
推理引擎	vllm , sglang	KV cache 管理、batching、scheduler、kernel、模型加载
Engine 内存插件	kvcached , elastic-kv-cache	保留引擎逻辑KV语义, 以GPU VMM 让physical KV pages按需占用/释放, 实现同卡多实例弹性
数据中心编排	dynamo	P/D 分离、KV transfer/offload、router、planner、operator; 把 engine 变成可扩展、可迁移的集群服务
K8s serving stack	llm-d , aibrix , kserve, kubebai , ome, gpustack , rbg , kthema	从标准模型 API、runtime/operator、LLM 路由、P/D workload 到 GPU/MaaS 的不同控制面
路由 / 网关	llm-d-router , semantic-router , routellm, gateway-api-inference-extension , ai-gateway	模型选择、endpoint picking、成本/质量/语义/KV-aware routing
KV locality / cache signal	llm-d-kv-cache , dynamo , kv-cache-offload	KV block index、cache-hit scoring、KV transfer/offload tiers
离线 / 实验 / 历史扩缩外围	llm-d-batch-gateway , llm-d-benchmark , llm-d-inference-sim , llm-d-workload-variant-autoscaler (deprecated / 历史设计)	batch job、benchmark、无 GPU simulator, 以及仅供迁移/设计参考的 variant autoscaling 历史方案

层级	代表项目	关注点
硬件资源层	hami, gpu-operator , k8s-device-plugin , dra-driver-nvidia-gpu	GPU discovery、device plugin、DRA/CDI、sharing/vGPU/MIG

K8s serving stack 扩展比较

这八个项目不是同一层的替代品，建议先按抽象层分类：

类别	项目	核心对象/入口	主要解决的问题
分布式 LLM serving	llm-d	Gateway/EPP、InferencePool、model server	K8s 标准入口下的 endpoint picking、KV/P/D 和分布式推理
GenAI 基础组件	aibrix	Gateway、CRD、Unified AI Runtime、KV/LoRA 组件	vLLM fleet 的路由、adapter、KV、autoscaling、故障检测和异构成本优化
通用模型平台	kserve	InferenceService、LLMInferenceService、InferenceGraph	用统一 API/operator 承载 predictive + generative AI，并支持 canary、缓存、KV offload 与 scale-to-zero
轻量 AI operator	kubecore	Model CRD、model proxy、loader、autoscaler	快速把 LLM/VLM/embedding/speech 模型变成 OpenAI-compatible API
Runtime/operator 抽象	ome	model agent、runtime selector、accelerator config	把模型生命周期、推理 runtime 和加速器配置解耦
GPU/MaaS 平台	gpustack	server、worker、scheduler、gateway、model service	跨本地/K8s/云管理 GPU，并提供多模型 API、计量、认证和运维
Workload 原语	rbg	RoleBasedGroup、Role、RoleInstance、CoordinatedPolicy	表达 gateway/router/prefill/decode 多角色有状态服务，保证拓扑和跨角色原子操作
一体化 LLM serving	kthelma	ModelBooster、ModelServing、ModelServer、ModelRoute	在 K8s/Volcano 内整合路由、P/D、限流、canary、扩缩、拓扑和 gang scheduling

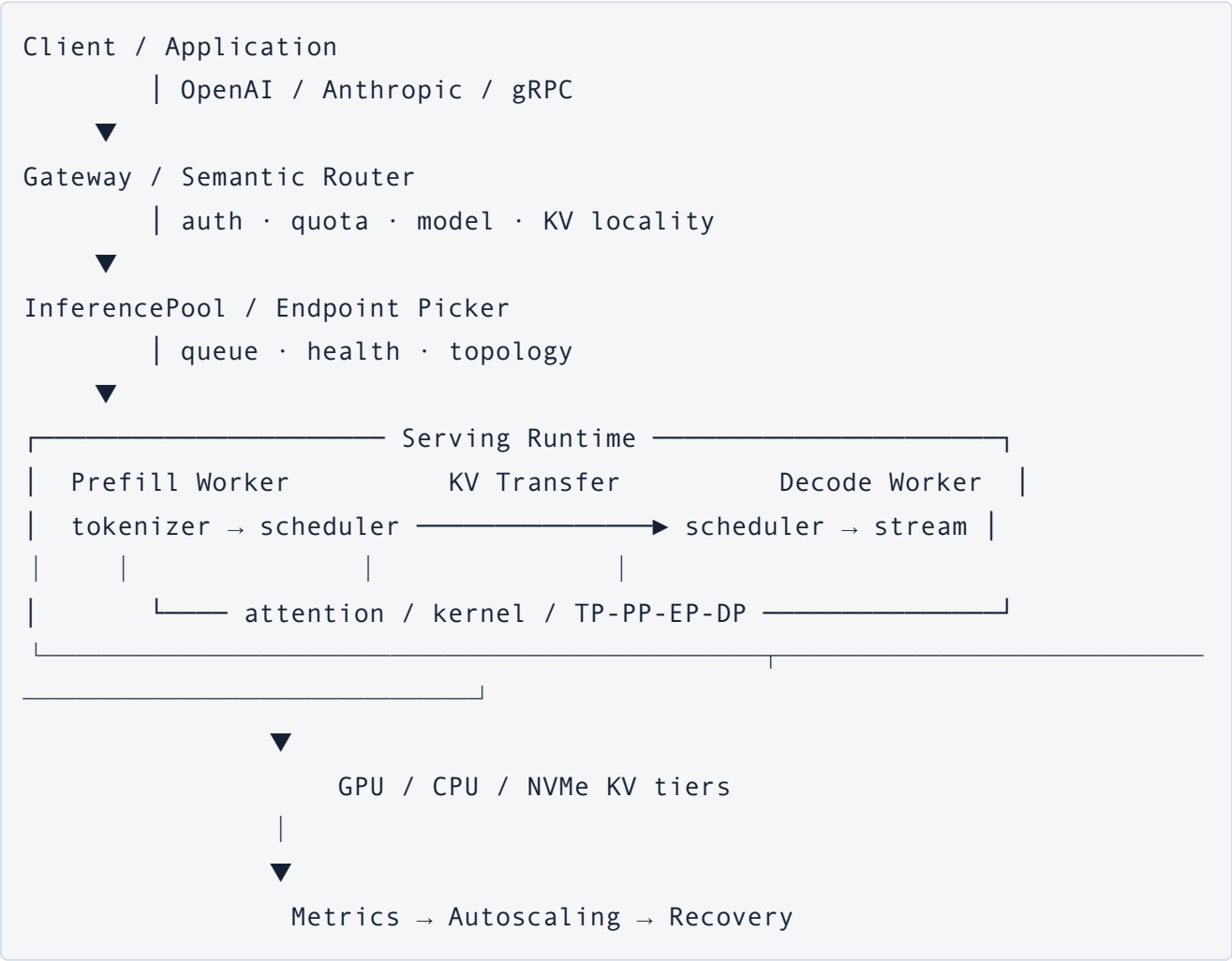
选型不要只问“哪个最好”

- 要 Gateway API + InferencePool 标准化：优先研究 [llm-d](#) / kserve。
- 要 vLLM 生态的 LoRA、KV、企业组件：研究 [aibrix](#)。
- 要 最短路径把模型暴露成 OpenAI API：研究 [kubecore](#)。
- 要 模型 runtime/accelerator 生命周期抽象：研究 ome。

- 要 GPU 集群、多云、MaaS、token/API 计量：研究 [gpustack](#)。
- 要 多角色、有状态、P/D workload 的原子升级/扩缩：研究 [rbg](#)。
- 要 K8s 原生完整 LLM serving，并深度结合 Volcano 拓扑/gang：研究 [kthena](#)。

完整的 README/文档驱动对比、能力矩阵和选型流程见 [src-k8s-serving-stack-comparison](#)。

总体架构与请求流程



核心技术主题

代码阅读入口

如果要从实现而不是概念开始，建议沿两条主链阅读：

vLLM:

EngineCore.step

- Scheduler.schedule
- KVCacheManager.allocate_slots
- GPUModelRunner.execute_model
- Attention selector/backend
- update_from_output

SGLang:

run_event_loop

- get_next_batch_to_run
- ScheduleBatch
- UnifiedRadixCache.match_prefix + allocation
- ModelRunner / ForwardMode backend
- process_batch_result

详细的文件路径、函数职责、状态对象和两者差异见 [src-vllm-architecture](#) 与 [src-sglang-architecture](#)。

Prefill vs Decode

Prefill 负责把 prompt/context 一次性编码成 KV cache，算力密集、吞吐敏感；Decode 每步生成一个 token，延迟敏感、状态持续时间长。[disaggregated-serving](#) 把两者拆到不同 GPU 池中分别扩缩，是 [dynamo](#)、[llm-d](#) 等系统的主线。

Batching

Continuous batching / inflight batching 让不同请求在 token step 之间动态进出 batch，避免传统 fixed batch 的尾部浪费。它是 [vllm](#) / [sglang](#) 这类 engine 的吞吐基础。

KV cache

长上下文和多轮对话让 KV cache 成为一等资源。[paged-attention](#) 用分页思想降低碎片，[radix-attention](#) 用 radix tree 加速 prefix 复用，[kv-cache-offload](#) 把 KV 在 GPU/CPU/SSD/远端之间迁移。到了 [dynamo](#) / [llm-d](#)，KV cache 还会反过来影响路由和调度：[llm-d-kv-cache](#) 把 KV events 变成 locality index，[llm-d-router](#) 再把 cache-hit score 和负载、profile 等信号一起纳入 endpoint picking。

Chunked Prefill

Chunked prefill 把长 prompt 的 prefill 切块，与 decode 请求交错执行，减少长 prompt 阻塞短请求。它常和 prefix cache、P/D 分离、batch scheduler 一起出现。

Speculative Decoding

Speculative decoding 用小模型或 draft head 先猜 token，再由大模型验证，目标是降低每个生成 token 的大模型前向次数。工程代价是调度、显存、accept rate 和模型兼容性变复杂。

LoRA / Adapter Serving

LoRA serving 让一个 base model 同时服务多个轻量 adapter，关键问题是 adapter 加载、batch 内 adapter 混排、cache 隔离和多租户权限。[aibrix](#) 等 K8s serving 项目会把它放到模型生命周期和 gateway 层一起处理。

Multi-modal Serving

VLM / speech / embedding / rerank 等任务把输入预处理、processor、tokenizer、模型 runtime 和输出格式变得更复杂。[kubecai](#) 这类 operator 会把 LLM/VLM/embedding/speech 纳入同一 Model CRD。

Quantization

FP8 / INT4 / AWQ / GPTQ 等量化路线降低显存和带宽压力，但会影响 kernel 支持、精度、吞吐和 serving 兼容性。选型时要看 engine 是否原生支持目标量化格式，以及 GPU 架构是否匹配。

Batch Inference

[batch-inference](#) 把大量请求作为异步 job 执行，关注文件、队列、状态、重试、取消、输出归档和成本，而不是单个请求的 streaming latency。[llm-d-batch-gateway](#) 说明 batch 层可以复用下游 [llm-d](#) Router/model endpoint，但必须额外引入 PostgreSQL、Redis/Valkey 和 object store 来承接长时状态。

Benchmark / Simulator

推理系统选型不能只看架构图，还要能复现实验。[llm-d-benchmark](#) 把 stack standup、scenario 渲染、harness 运行和结果收集做成 workspace；[llm-d-inference-sim](#) 则用无 GPU 的 vLLM 行为模拟器验证 router、autoscaling、KV event 和 benchmark 流程。二者解决的是“如何测”和“如何低成本复现控制面行为”。

Variant Autoscaling

普通 HPA/KEDA 面向单 workload 或通用 event source；LLM serving 进入 P/D 分离、多 GPU 型号、多成本池之后，如何按同一模型的多个 serving variant 做全局 allocation 是一个重要历史问题。[llm-d-workload-variant-autoscaler](#) 曾将 InferencePool、Prometheus、GPU inventory、capacity model 和 HPA/KEDA 串起来，但现在只作为 deprecated 的历史设计快照保留。当前 llm-d dev 指南为新部署采用 [EPP metrics → KEDA Prometheus scaler → HPA](#)；WVA 的冲突状态与迁移/设计历史见 [llm-d-workload-variant-autoscaler](#)。

选型入口

- 只优化单机吞吐：优先看 [vllm](#) / [sglang](#)。
- 需要 P/D 分离、KV transfer、数据中心级编排：看 [dynamo](#) / [llm-d](#)。

- 需要 Kubernetes model serving API: 看 kserve / [kubecai](#) / ome。
- 需要多租户平台和 GPU 集群管理: 看 [aibrix](#) / [gpustack](#)。
- 需要路由模型或 endpoint: 看 [inference-routing](#)、[llm-d-router](#)、[semantic-router](#)、[gateway-api-inference-extension](#)。
- 需要离线批处理、评测或仿真: 看 [llm-d-batch-gateway](#) / [llm-d-benchmark](#) / [llm-d-inference-sim](#)。
- 需要当前 llm-d autoscaling: 采用 EPP metrics → KEDA Prometheus scaler → HPA 路径, 并结合 [model-serving-operator](#) 的生命周期边界; [llm-d-workload-variant-autoscaler](#) 仅作为迁移/设计参考。

PagedAttention

[vllm](#) 论文 (Kwon et al., SOSP 2023) 提出的 block 级 KV 缓存管理机制；当前官方架构仍以 block table、attention backend 与 prefix caching 组合扩展。它把 OS 虚存分页思想（按页分配 + 页表映射）搬到 LLM KV cache：在一个确定的配置内，KV 按固定大小的逻辑/物理 block 管理，每个请求用 **block table** 记录"逻辑序列位置 → 物理 block"映射。具体 block 大小取决于配置、attention backend 和版本。

核心思想

传统 (HF transformers):

按 max_seq_len 预分配 KV 浪费严重

```
[req0  used  ][   unused   ]
[req1 used][           unused           ]
```

PagedAttention:

按 block 按需分配 (block size = B)

```
Block 0: [████████] complete (B/B tokens)
Block 1: [██████░░░░] partial tail (k/B tokens, 0 < k < B)
Block 2: [░░░░░░░░░░] free (B slots)
```

Block Table per req:

```
req0: [B0, B1]
req1: [B0, B2] ← 共享 system prompt 在 B0
```

关键机制

- **Block table**: 每个请求有一个 `int32 list[blocks]`，attention kernel 用它把"逻辑 token 索引"翻译成"物理 KV 位置"
- **Block 大小固定于当前配置**: 逻辑/物理 block 使用同一 block size B；具体 B 依配置、attention backend 和版本而定
- **Prefix sharing**: 传统/默认 full-block APC 路径按 block 边界共享 system prompt，partial tail 在形成完整 block 前不可复用。当前 vLLM 可通过 `cache_partial_block` 与可配置的 `prefix_match_unit` 启用更细粒度的 partial entry，但仍受配置、backend、版本及整除/match-unit 约束。Block table 的共享使用引用计数管理，释放时只有 `ref=0` 才回收。详见 [Prefix Caching](#) 与 [BlockPool API](#)
- **Copy-on-write**: beam search 等场景 fork 同一个 block table，写入时拷贝
- **Swap to CPU**: 内存紧张时把不活跃 block swap 到 CPU pinned memory

工程影响

- 首创性：2023 年第一个把虚存分页引入 LLM serving，HuggingFace TGI / Ray Serve / Anyscale / Together AI 都基于此或受启发
- 吞吐量：典型 $2-4\times$ over HF transformers
- 简单可靠：block table 是数组，无需锁，调度逻辑直接

逻辑地址到物理 KV 的流程

```
请求 token 位置 i → 逻辑 block = i // block_size  
    → block table[logical block]  
    → 物理 GPU KV block  
    → attention kernel gather K/V  
    → 写入新 token block 或触发扩容
```

局限与 [radix-attention](#) 的对比

- Block 边界刚性：前缀共享按当前 block size B 对齐，不能在 block 内任意分叉
- 默认 full-block APC 的 partial tail 限制：在传统/默认路径中，末尾未满的 block 在形成完整 block 前无法被前缀复用；启用 `cache_partial_block` 并配置 `prefix_match_unit` 后可以更细粒度匹配，实际边界依配置/backend/版本与整除约束而定
- 内部碎片依负载而定：程度取决于 block size、请求长度分布与实现配置

[radix-attention](#) ([sglang](#) 提出) 通过 token 级 radix 树 + flat KV pool 解决这些限制。

出处

Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention", SOSP 2023。

相关页面

- 工程实现：[vllm](#)
- 改进算法：[radix-attention](#) ([sglang](#) 提出)
- 同类对比：[sglang](#) 架构详解 → [src-sglang-architecture](#)

RadixAttention

[sglang](#) 论文 (Zheng et al., NeurIPS 2024) 提出的 token 级 KV 缓存复用机制。它用 radix 树 + 两级 KV pool 实现"任意 token 边界都能 split & share"; [vllm](#) 的 [paged-attention](#) 则使用配置内固定大小、按 block 边界复用的逻辑/物理 block, 具体大小取决于配置、attention backend 和版本。历史论文仅在其 LLaMA-7B tree-of-thought / few-shot 等工作负载上报告 $1.6 - 6.4\times$ over vLLM, 不是当前通用吞吐结论。

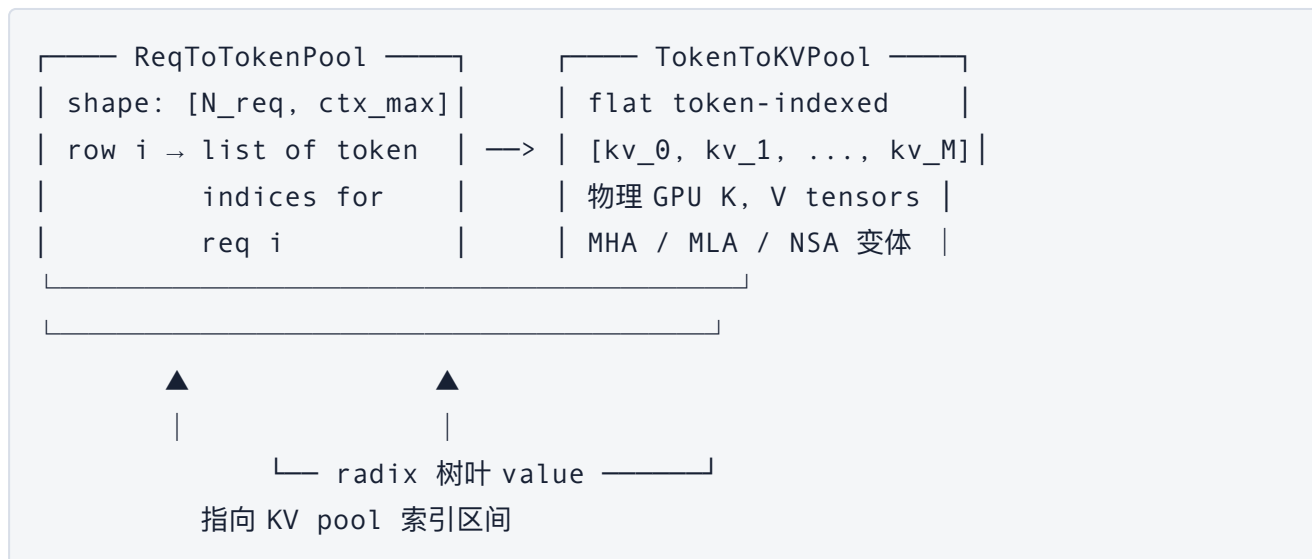
核心数据结构

TreeNode

每个节点持有:

- key: RadixKey —— 变长 token 序列 (支持 bigram 模式给 EAGLE 用)
- value: torch.Tensor —— 指向 TokenToKVPool 的 KV 索引张量
- children: dict[int, TreeNode] —— 首 token 到子节点的映射
- lock_ref: int —— 引用计数, >0 时禁止 evict (保护 in-flight 请求)
- evicted: bool —— 已 evict 的占位 (支持增量恢复)

两级 KV pool



双跳的好处: radix 树叶直接指向 KV pool 位置; attention kernel 用 device-resident req_to_token 张量做一次 gather —— 既保留 token 级粒度, 又能让 kernel 高效访存。

核心算法

match_prefix(input_ids)

```
walk from root:
  for each level:
    find child whose key shares prefix with remaining input_ids
    if full match → descend
    if partial match → SPLIT child:
      原节点截断到匹配长度
      剩余 token 移到新子节点 (保留原 value 的对应区间)
    if no match → stop
return (device_indices, last_node)
device_indices = 已命中所有节点 value 的拼接 (直接喂 attention kernel)
last_node      = 匹配链尾 (给后续 insert 用)
```

insert(req)

请求完成时调用:

1. 在 req.last_node 下挂新节点, key = req.fill_ids[len(prefix):]
2. value = req.out_cache_loc (这次 forward 新写入的 KV 槽位)
3. 父节点 lock_ref 递减 → 完成后可被 evict

Eviction

5 策略: LRU / LFU / FIFO / SLRU / Priority。

evict 时 pop evictable_leaves 堆顶 → free 该节点 value 指向的 KV 槽位 → 递归向上 unlock。

lock_ref > 0 的节点跳过 (保护 in-flight 请求)。

前缀命中与分叉流程

输入 token prefix → Radix root → longest-prefix match



↓

只对未命中 suffix 执行 prefill

↓

insert token → KV pool → radix tree

↓
eviction 从可回收叶节点释放

vs [paged-attention](#) 关键差异

维度	RadixAttention	PagedAttention
复用粒度	token 级（树节点变长）	配置内固定大小的 block（大小依配置/backend/版本）
索引结构	Radix 树 + 两级 pool	Block table（数组）
任意分叉点 split	✅ 树节点动态 split	默认 full-block APC 按 block 边界对齐；可选更细的可配置 match unit
碎片	区间连续	partial block 可有未用槽，程度依 block size 与负载而定
共享 system prompt	Radix 树支持任意 token 边界的前缀分叉	默认 full-block APC 路径需等待完整 block；当前 vLLM 可通过 <code>cache_partial_block</code> / 可配置 <code>prefix_match_unit</code> 使用更细的 partial entry，受配置/backend/版本与 match-unit 约束
实现复杂度	较高（树平衡 + lock_ref + evict）	较低（block table 哈希）
历史论文工作负载	LLaMA-7B tree-of-thought / few-shot 等工作负载报告 1.6 – 6.4× over vLLM，非当前通用结论	论文当时的对照实现

工程要点

- radix 树是 lock-free 的：单 Scheduler 进程独占，不需要锁
- device_indices 是 device-resident 张量：直接喂 attention kernel，省去 CPU↔GPU 拷贝
- bigram 模式：给 EAGLE 投机解码用 —— key 是 (token_i, token_i+1) 对而非单 token，匹配 draft tree 结构
- 5 变体并存（2026-09-14 分析快照）：`radix_cache.py` (vanilla) / `hiradix_cache.py` (分层) / `mamba_radix_cache.py` (Mamba) / `swa_radix_cache.py` (Sliding Window Attention) / `radix_cache_cpp.py` (C++ 加速版)，按 attention 类型在启动时绑定；当前 release 需重新核验该 inventory

出处

Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs", NeurIPS 2024。

相关页面

- 工程实现: [sglang](#) → `mem_cache/radix_cache.py`
- 对照算法: [paged-attention](#) ([vllm](#) 的 KV 管理)
- 架构详解: [src-sglang-architecture](#)

AI Infra 中文学习项目地图

这页把 AI Infra Learning (中文) star list 整理成实际学习路线。它解决的问题不是“哪个项目最火”，而是：如果目标是快速理解 AI Infra / LLM 工程 / Agent 工程，应该按什么顺序读、哪些项目只作索引、哪些值得后续深挖源码。

推荐学习主线

1. 先建立 AI 系统全景

优先读 Infrasy-AI/AISystem、Infrasy-AI/AIInfra、cr7258/ai-infra-learning。

这三类材料适合回答“AI Infra 到底包含哪些层”：芯片/加速器、通信、编译器、训练框架、推理引擎、服务编排、LLMOps、Kubernetes 资源管理。学习目标是先建立分层地图，不急着陷入某个框架的 API。

2. 补 LLM 基础和开源模型使用

读 datawhalechina/happy-llm、datawhalechina/self-llm、AiHubCN/Awesome-Chinese-LLM、Hannibal046/Awesome-LLM。

happy-llm 偏原理入门，self-llm 偏开源模型微调/部署实操，两个组合起来能覆盖“模型是什么”和“怎么在本地/服务器上跑起来”。Awesome list 只适合作查表，不适合从头读。

3. 进入 GPU / CUDA / kernel

顺序建议：HeKun-NVIDIA/CUDA-Programming-Guide-in-Chinese → Tony-Tan/CUDA_Freshman → xlite-dev/LeetCUDA → a-hamdi/GPU。

这一层是从“会用模型”走向“理解推理性能”的关键。LeetCUDA 最有工程训练价值，因为它覆盖 200+ CUDA kernels、Tensor Cores、HGEMM、FlashAttention 相关 MMA 练习；适合和 [paged-attention](#)、[radix-attention](#)、[kv-cache-offload](#) 的理论页一起读。

4. 读推理优化和 serving 论文/代码索引

核心入口是 xlite-dev/Awesome-LLM-Inference，再回到本 wiki 的 [llm-serving-engine-selection-map](#)。

这一层关注 FlashAttention、PagedAttention、量化、并行、KV cache、speculative decoding、prefill/decode disaggregation。读法不是把 awesome list 全部扫完，而是围绕问题选论文：吞吐、延迟、显存、长上下文、router、KV offload。

5. 做工程化和 LLMOps 横向理解

读 liguodongiot/llm-action、InftyAI/Awesome-LLMOps、CalvinXKY/InfraTech。

llm-action 适合建立工程化知识面；Awesome-LLMOps 适合作工具查表；InfraTech 更适合动手做 PyTorch/vLLM/SGLang 框架入门和性能加速练习。这个阶段可以开始回看 [dynamo](#)、[vllm](#)、[sglang](#)、SkyPilot、K8s AI Serving 相关页面。

6. 进入 Agent / Skills / 应用层

读 datawhalechina/hello-agents、panaversity/learn-agentic-ai、luzhenqian/ai-coding-lab、libukai/awesome-agent-skills。

hello-agents 是 Agent 入门主线；learn-agentic-ai 把 OpenAI Agents SDK、Memory、MCP、A2A、Knowledge Graph、Dapr、Kubernetes 串起来，适合连接本 wiki 的 mcp、agent-memory、agent-skills-plugin-system-map；ai-coding-lab 偏 AI 编程实战；awesome-agent-skills 是 Skills 生态索引。

7. 最后用面试材料查漏补缺

读 WeThinkIn/AIGC-Interview-Book、wdndev/llm_interview_note、bcefhgj/ai-agent-interview-guide。

这些材料不适合作第一遍学习主线，因为容易把概念碎片化。更适合在读完前六层后，用来补缺：模型基础、推理优化、RAG、Agent、项目表达、简历和 STAR 面试稿。

项目取舍

类型	项目	用法
核心教材	AISystem、AIInfra、happy-llm、self-llm、hello-agents	从头读，建立主干
工程练习	InfraTech、LeetCUDA、CUDA_Freshman、GPU、ai-coding-lab	边读边做，适合沉淀代码实验
查表索引	Awesome-LLM-Inference、Awesome-Chinese-LLM、Awesome-LLM、Awesome-LLMOps、awesome-agent-skills	不全量阅读，按问题检索
职业/面试	AIGC-Interview-Book、llm_interview_note、ai-agent-interview-guide	用于复盘和表达
暂缓	academicpages、ai-briefing、Java AI 示例、自动驾驶/world model 清单	与 AI Infra 主线弱相关，暂不深挖

最值得后续单独摄入的项目

- Infrasys-AI/AISystem: AI 系统全栈知识体系，适合产出一篇“AI 系统分层”概念页。
- Infrasys-AI/AIInfra: AI Infra 中文材料核心入口，适合做学习源页。

3. CalvinXKY/InfraTech: 包含 PyTorch/vLLM/SGLang/性能加速练习, 适合做工程实践地图。
4. xlite-dev/LeetCUDA: CUDA kernel 实战价值高, 适合补 GPU kernel 学习路线。
5. xlite-dev/Awesome-LLM-Inference: 可和 [llm-inference](#)、[llm-serving-engine-selection-map](#) 联动, 整理推理优化论文路线。
6. datawhalechina/self-llm: 中文开源模型微调/部署实战, 适合补模型部署入门。
7. datawhalechina/hello-agents: Agent 教程主线, 可和 ai-agent-frameworks-map、agent-framework-programming-model-map 对齐。
8. nanocoai/nanoclave: 容器化个人 Agent, 和 OpenClaw / OpenCowork / nanobot / OpenShell 有架构对比价值。

这条路线和现有 wiki 的连接

- [llm-inference](#)、[paged-attention](#)、[radix-attention](#)、[kv-cache-offload](#): 承接推理优化理论。
- [llm-serving-engine-selection-map](#): 承接 vLLM/SGLang/Dynamo/SkyPilot/llm-d 等工程选型。
- ai-agent-frameworks-map、agent-framework-programming-model-map: 承接 Agent 应用层。
- agent-skills-plugin-system-map: 承接 Agent Skills 学习材料。
- github-stars-ingest-candidates: 承接后续项目摄入 backlog。

GitHub Stars P0-P2 实现地图

这页把 github-stars-ingest-candidates 从“摄入候选清单”落实为正式分析地图：P0-P2 不再只是待办，而是被放进当前 wiki 的选型结构里，说明每个项目应该如何理解、和已有项目差在哪里、后续如果单独 ingest 应该看什么。

GitHub 当前状态见 [src-github-stars-backlog-current-state](#)。

总体分层



这批 backlog 项目的核心价值不是“再多几个开源项目”，而是把已有 wiki 的几条主线补完整：Agent 从 CLI 产品向 managed teammate 和 runtime substrate 扩展；LLM serving 从 vLLM/SGLang/Dynamo 向 K8s control plane、router 和 KV cache library 扩展；Code RAG 从搜索插件向代码图和 repo wiki 扩展；GPU 栈从 star list 进入 serving 选型的基础设施层。

P0：正式进入选型主线

主线	纳入项目	和已有 wiki 的差异
Agent Runtime / Substrate	agent-substrate/substrate, agentscope-ai/agentscope-runtime	现有 agent-runtime-sandbox-selection-map 主要覆盖 sandbox primitive、OpenShell runtime、AgentScope framework、agentgateway。Substrate 补“高密度 stateful agent workload substrate”，AgentScope Runtime 补“framework app 到生产 runtime / Agent-as-a-Service”的中间层。

主线	纳入项目	和已有 wiki 的差异
Agent Memory	mem0ai/mem0, agentscope-ai/ReMe	现有 agent-memory-selection-matrix 偏 Claude/Coding agent 插件、Markdown truth、OceanBase memory。mem0 是通用应用 memory layer，ReMe 是 AgentScope 生态内建 memory kit。
Coding Agent 执行面	openai/codex, earendil-works/pi, can1357/oh-my-pi	现有 coding-agent-selection-map 以 Claude Code/OpenCode/NemoClaw/nanobot 为骨架。Codex 是官方 Rust terminal agent；Pi 是 toolkit + TUI + coding CLI；oh-my-pi 代表 hash-anchored edits、LSP、browser、subagents 的 tool harness。
Managed / Desktop Agent	multica-ai/multica, OpenCoworkAI/open-cowork	它们不是底层 framework，而是把 coding agent 包成 teammate 或 desktop app，和 nanobot 的个人多渠道内核形成产品层对照。
LLM Serving on K8s	vllm-project/aibrix, llm-d/llm-d, llm-d/llm-d-router, llm-d/llm-d-kv-cache	现有 llm-serving-engine-selection-map 主要讲 vLLM/SGLang/Dynamo/SkyPilot。AIBrix 和 llm-d 把重点推到 K8s inference control plane、Gateway API、router 和分布式 KV cache。

Runtime / Sandbox 补层

项目	应放的位置	关键架构问题
agent-substrate/substrate	agent-runtime-sandbox-selection-map 的 substrate 层	actor/worker multiplexing、suspend/resume、snapshot、traffic routing、gVisor 如何组合成高密度 agent workload 基座。
agentscope-ai/agentscope-runtime	Agent framework 到 runtime 的服务化层	secure tool sandboxing、Agent-as-a-Service API、observability、K8s/serverless 部署如何承接 AgentScope app。
multica-ai/multica	managed teammate 产品层	coding agent task assignment、progress tracking、compound skills 如何产品化，不是 sandbox primitive。
OpenCoworkAI/open-cowork	desktop Agent OS / agent host	Claude Code、MCP tools、skills、sandbox、IM 集成在桌面端如何形成用户入口。

选型判断：如果问题是“Agent 进程怎么安全/高密度运行”，看 agent-substrate/substrate 和 OpenShell；如果问题是“Agent app 怎么服务化”，看 agentscope-runtime；如果问题是“用户如何把 agent 当同事用”，看 multica / open-cowork。

Memory 补层

项目	对比对象	关键差异
mem0ai/ mem0	powermem , agentmemory, memsearch	更像通用应用 memory layer, stars 和生态信号强, 适合补“产品/应用内用户记忆”的主流开源基线。
agentscope -ai/ReMe	AgentScope, agent-memory- selection-matrix	framework 内部 memory kit, 重点不是通用服务, 而是和 AgentScope agent/tool/workflow 生态耦合。

这两个项目应更新 agent-memory-selection-matrix 的分类: mem0 进入“通用 memory layer”, ReMe 进入“framework memory kit”。它们和 memsearch 的 Markdown truth 路线不同, 也和 tencentdb-agent-memory 的 L0-L3 语义金字塔不同。

Coding Agent 执行面与生态配套

层	项目	作用
Terminal coding agent	openai/codex	官方轻量 terminal coding agent, Rust 实现, 和 claude-code、OpenCode 同层。
Agent toolkit / TUI	earendil-works/ pi	unified LLM API + agent loop + TUI + coding agent CLI, 适合看 agent runtime/tool loop 抽象。
Tool harness	can1357/oh-my- pi	hash-anchored edits、LSP、browser、subagents, 适合拆“可靠改代码”的执行面。
Delegation plugin	openai/codex- plugin-cc	Claude Code 调用 Codex, 属于 cross-agent delegation。
Trace viewer	liaohch3/ claude-tap	拦截并查看 coding agent API traffic, 属于可观测层。
IM bridge	chenhg5/cc- connect	把本地 coding agent 接入 Feishu/Lark/Slack/Telegram 等 IM, 属于远程入口层。
Token cost	junhoyeo/ tokscale	多 agent token usage tracker, 属于成本/用量观测层。

选型时不要把这些混成“coding agent”。Codex / Pi / oh-my-pi 是执行 loop; codex-plugin-cc、claude-tap、cc-connect、tokscale 是围绕执行 loop 的 delegation、trace、IM 和 cost 层。

LLM Serving / AI Gateway / GPU 基座

子域	项目	放入地图后的含义
Serving control plane	vllm-project/aibrix, llm-d/llm-d	补 llm-serving-engine-selection-map 中“单引擎之外”的 K8s GenAI infrastructure。
Inference router	llm-d/llm-d-router, vllm-project/semantic-router, lm-sys/RouteLLM	前者偏 K8s/Gateway API 请求入口, semantic-router 偏 mixture-of-models 系统路由, RouteLLM 偏成本/质量路由算法基线。
KV cache library	llm-d/llm-d-kv-cache	让 kv-cache-offload 从 Dynamo 结论扩展到 llm-d 生态的分布式 KV cache scheduling/offloading。
Gateway API / AI Gateway	kubernetes-sigs/gateway-api-inference-extension, envoyproxy/ai-gateway, kgateway-dev/kgateway, higress-group/higress, katanemo/plano	这些项目把 mcp-gateway-tooling-map 和 agentgateway 的 gateway 讨论扩展到 GenAI access control、model routing 和 Gateway API 标准化。
Model serving operator	gpustack/gpustack, ome-projects/ome, kserve/kserve, kubeai-project/kubeai	它们不是 vLLM/SGLang 替代品, 而是在 K8s 上编排模型、GPU、autoscaling、OpenAI-compatible endpoint。
GPU / DRA / device plugin	Project-HAMi/HAMi, kubernetes-sigs/dra-driver-nvidia-gpu, NVIDIA/gpu-operator, NVIDIA/k8s-device-plugin	把 src-k8s-gpu-device-plugins-stars 里的基础设施项目升格为 serving 选型必须看的资源层。

关键取舍: vLLM/SGLang 是 engine; Dynamo/AIBrix/llm-d 是 serving system; Gateway API Inference Extension/Envoy AI Gateway/kgateway/Higress/Plano 是入口和治理; GPU Operator/device plugin/DRA/HAMi 是设备与资源分配层。

Kubernetes AI Assistant / AI Ops

项目	用户入口	适合回答的问题
kagent-dev/kagent	Cloud-native agentic AI / MCP	如何把 K8s/DevOps workflow 包装成 agentic 操作系统。
GoogleCloudPlatform/kubectl-ai	kubectl CLI	如何在命令行里做 K8s assistant。

项目	用户入口	适合回答的问题
weibaohui/k8m	Mini K8s AI dashboard	如何把多集群、权限、MCP、异常检测合进轻量 dashboard。
kubewall/kubewall	Single-binary dashboard	如何把 AI integration 放进 K8s dashboard 管理体验。

这四个项目应该进入 ai-ops / K8s assistant 专题，而不是和通用 Agent framework 混放。它们的关键差异是入口：CLI、dashboard、agentic workflow、MCP/权限模型。

Code Graph / Repo Wiki / Code Intelligence

项目	路线	和 code-semantic-search-rag-map 的关系
tirth8205/code-review-graph	local-first code intelligence graph for MCP/CLI	从 vector search 扩展到持久代码图，目标是让 coding tools 只读相关上下文。
abhigyanpatwari/GitNexus	browser-side knowledge graph + Graph RAG	零服务器、浏览器端 repo 图谱，适合对照本地优先/隐私/交互式探索。
AsyncFuncAI/deepwiki-open	AI-powered repo wiki generator	和 llm-wiki 自身工作流有镜像价值：自动总结 repo，生成结构化 wiki。

这条线说明 Code RAG 不只是 search MCP。它正在分成三类：代码搜索、代码图谱、自动 repo wiki。后续单独 ingest 时应重点看 incremental graph、truth source、line-level evidence、repo scale 和生成内容的可审计性。

后续单独 ingest 优先级

如果继续从这批项目中做完整源码架构页，优先级应调整为：

1. agent-substrate/substrate：补 runtime substrate 缺口。
2. openai/codex：补官方 terminal coding agent 执行 loop。
3. mem0ai/mem0：补通用 memory layer 主流基线。
4. vllm-project/aibrix 或 llm-d/llm-d：补 K8s GenAI inference control plane。
5. kubernetes-sigs/gateway-api-inference-extension：补 inference routing 标准化入口。
6. tirth8205/code-review-graph 或 AsyncFuncAI/deepwiki-open：补 Code Graph / Repo Wiki 实现路线。

其余项目已经在本页进入正式对比结构；是否单独 ingest 取决于你下一步要深挖哪条主线。

Kubernetes DRA Design Deep Dive

当前上游核验（2026-09-27）

本节区分当前官方证据与下文 2026-07 的 KEP 设计/状态笔记；[src-dra-driver-nvidia-gpu-architecture](#) 是 2026-06 的 raw-backed Source 快照，保留其历史仓库/HEAD 和 ASCII 图。当前默认分支 commit 不代表 release，基础 DRA 稳定也不等于所有 Kubernetes 扩展或 NVIDIA driver 功能都已稳定。完整设备栈边界见 [k8s-gpu-device-stack](#)。

项目	当前 commit	当前证据边界	
kubernetes	6c1c7702cf20	Kubernetes DRA API、scheduler allocation 与 kubelet Prepare/Unprepare；各扩展阶段按官方文档及该 commit 的 feature 定义单独核验	
dra-driver-nvidia-gpu	495bf4c59b94	NVIDIA GPU/ComputeDomain 驱动实现；通过 NVIDIA 仓库入口查询时返回此规范链接，不以 Kubernetes 的 feature stage 替代厂商支持矩阵	
能力 / feature gate		本次核验的阶段与版本	本页相关边界 / 官方依据
基础 DRA / DynamicResourceAllocation		Stable，自 Kubernetes v1.35	DRA 概览 ；基础 Claim/设备分配稳定不覆盖下列所有扩展
Device binding conditions / DRADeviceBindingConditions		Beta，自 v1.36，默认启用	延迟 Pod binding 以等待外部设备准备； 当前 feature 定义 ；仍需对应 driver/status 支持
Consumable capacity / DRAConsumableCapacity		Beta，自 v1.36，默认启用	多 Claim 消耗同一设备容量； DRA Features
Partitionable devices / DRAPartitionableDevices		Beta，自 v1.36，默认启用	通过共享计数器表达逻辑设备资源重叠； DRA Features
Prioritized list / DRAPrioritizedList		GA，自 v1.36；v1.37 锁定启用	按优先级尝试设备请求候选； 当前 feature 定义
Device taints / DRADeviceTaints		当前源码 v1.37 条目标记 GA、默认启用；v1.38 条目锁定启用	设备 taint/toleration 的阶段不同于相关附加规则； 当前 feature 定义 ，版本化源码条目不是未来 release 已发布的证明

能力 / feature gate	本次核验的阶段与版本	本页相关边界 / 官方依据
Optional node operations / DRAOptionalNodeOperations	Alpha, 自 v1.37, 默认关闭	driver 可声明跳过特定 node-local 操作, 不能推广为所有 GPU 都无需 Prepare; DRA Features

该表只列本页相关且已核验的能力, 不是完整 feature 清单。API 对象语义见 [DRA API Objects](#)。NVIDIA 侧须分别核对 [锚定 driver README](#) 和 [GPU Operator 26.7 管理路径](#): 独立仓库 GPU plugin 的支持说明与 Operator 管理版本的默认启用方式不能混用。下文 KEP 状态表属于历史演进记录, 不覆盖本节的当前阶段核验。

M5-C 中的职责边界

[k8s-gpu-device-stack](#) 是设备栈入口, 本页聚焦 [kubernetes-dra](#) 的 Claim、调度分配、节点准备与回收。

Kubernetes 提供 DRA API 和 scheduler allocation; driver 发布 ResourceSlice 库存及设备约束, 解释厂商配置并完成相应准备。DeviceClass 可由管理员或 driver 安装流程提供; node-feature-discovery 的节点标签不能替代它或 ResourceSlice。

kubelet 驱动节点侧 Prepare/Unprepare, [dra-driver-nvidia-gpu](#) 等插件操作具体设备, [cdi](#) 把已准备设备交给兼容 runtime 注入。[gpu-operator](#) 管理受管组件部署和升级, karpenter 管理节点容量; 两者都不是单次 DRA Claim 的通用分配器。是否支持特定扩展仍以页首 feature-stage 表和目标 driver/release 为准, 这里不重复或提升其阶段。

DRA Allocation / Binding Lifecycle

```

Admin / driver installation — create selectors + config —> DeviceClass
DRA driver - - publish inventory / attributes / capacities - ->
ResourceSlice
User / workload controller — create directly —> ResourceClaim
User — create ResourceClaimTemplate + referencing Pod —> API
Pod/template - - watch / reconcile - -> resourceclaim controller
resourceclaim controller — create owned Claim / record Pod claim reference
—> API

Pod + Claim + DeviceClass + ResourceSlice - - watch / scheduling input - ->
scheduler
PreFilter / Filter — evaluate feasible Node + devices —> select
compatible Node
Reserve — retain candidate allocation in memory —> PreBind
PreBind — API write —> Claim.status.allocation + reservedFor
| — no binding conditions —> continue binding

```

```

└─ binding conditions enabled + present → wait in PreBind
Claim allocation - - observed by external driver controller - -> device
preparation
driver controller ─ write device conditions → Claim.status.devices
device conditions - - observed ready / failed / timeout - -> PreBind result
PreBind success → Bind Pod to compatible Node
PreBind failure → Unreserve / cleanup this attempt - - retry feedback -
-> queue

```

图例：实线 ─ label → 表示有序调用、API 写入或节点动作，虚线 - - label - -> 表示发布、watch、反馈与重试。图按引用 Claim 的普通 Pod 路径展开：已经分配的 Claim 会被校验和复用，不能把每一轮都理解成覆盖 allocation；模板需要由 resourceclaim controller 实例化，不是 scheduler 直接把模板当作已存在的 Claim。API 写入者也分开：scheduler 负责本次分配与消费者 reservation，外部 driver/controller 按协议更新设备 conditions，resourceclaim controller 管理派生 Claim 与消费者生命周期。

锚定版本的 [DynamicResources 插件](#) 在 Reserve 记录候选，在 PreBind 的 bindClaim 写回 allocation/reservation，然后按启用的扩展与设备声明等待 binding conditions。没有该条件时直接继续 binding；出现 failure condition 或超时则本次绑定失败。后续 Filter/PostFilter 可按 Claim 的消费者限制决定是否清除失败 allocation，不能把 Unreserve 等同于无条件清空所有共享 Claim。

binding conditions 允许外部准备发生在 Pod binding 前；它不是节点侧 NodePrepareResources 的另一种名字。后者通常在 Pod 已绑定、kubelet 准备容器时发生。Claim 分配成功、外部条件就绪、Pod 绑定成功、节点准备成功和容器运行是不同检查点。对象语义与模板控制器分别见 [DRA API Objects](#) 和 [resourceclaim controller](#)。

kubelet Prepare / CDI / Release

```

Bound Pod + allocated/reserved Claim - - observed state - -> kubelet DRA
manager
kubelet ─ track Pod/Claim reference + checkpoint → node-local state
kubelet ─ NodePrepareResources → vendor DRA plugin
plugin ─ validate allocation / configure device / recover own state →
device + checkpoint
plugin ─ prepare CDI spec + return CDI device IDs → kubelet
kubelet ─ checkpoint prepared result / CRI device configuration →
runtime
runtime ─ resolve CDI spec / inject devices / start container → running
workload

Pod termination - - node reconciliation - -> kubelet drops this Pod
reference

```

```

└─ other local Pods still use Claim → retain shared prepared state
└─ no local users → NodeUnprepareResources → plugin cleanup
    └─ successful result → kubelet updates cache / checkpoint
Pod completion / deletion - - API watch - -> resourceclaim controller
controller — remove completed consumer reservation → Claim status
last consumer released — eligible deallocation / finalizer cleanup →
reusable Claim
owned Claim deletion — owner / GC lifecycle → Claim removed when cleanup
permits

```

两条终止分支分别处理节点状态和 API 状态，不是跨组件事务，也不保证总按图中文字顺序完成。当前 [kubelet DRA manager](#) 在 RPC 前后记录 Claim/Pod 与准备结果；其他本地 Pod 仍引用同一 Claim 时延迟 Unprepare。kubelet checkpoint 与 vendor plugin 自己的 checkpoint/config 是不同责任：重启恢复需同时对照 API、节点记录及真实硬件，不能把删除任一记录当作已完成设备释放。

当前 resourceclaim controller 在最后消费者从“仍使用”转为“不再使用”时，可清除相应 allocation 和不再需要的 finalizer，使保留的 Claim 以后重新分配。模板派生 Claim 随 owner 生命周期回收；用户直接创建的持久 Claim 对象可以保留，但保留对象不意味着永久保留同一设备。多消费者场景不能因其中一个 Pod 退出就清空全部 reservation 或拆掉共享准备状态。

图中 Prepare/Unprepare 是常规 node-local 路径。页首 DRAOptionalNodeOperations 仍为独立 Alpha 扩展：只有启用该能力且 allocation 携带合法 skip 声明时，kubelet 才跳过指定节点操作；这不是所有 GPU 默认行为。CDI 负责容器注入，不保证硬件准备、健康或隔离策略本身正确。正式 RPC 名称为复数 NodePrepareResources / NodeUnprepareResources，下方历史笔记的单数简写不应作为接口名使用。

DRA 失败与恢复路径

失败点 / 可见影响	自动恢复 / API 与节点边界	Operator 与人工处理边界
ResourceSlice 陈旧或 pool 不完整：设备不可选或节点准备发现不符	driver 重新发布完整库存，scheduler 根据相关更新重试；节点 plugin 仍须校验设备现状	Operator 可恢复受管 driver，但错误设备 identity/generation、权限或硬件丢失需修复
Allocation / reservation 并发冲突：PreBind status 写入失败	API 并发检查拒绝冲突写入，scheduler 重读重算并清理本次保留；不覆盖其他消费者	持续争用、错误 UID/字段所有权或自定义控制器乱写 status 需修正
Binding conditions 超时或失败：有 allocation 但 Pod 未绑定	PreBind 失败触发 Unreserve/重试；后续 deallocation 受 Claim reservation 条件限制，不保证立即换设备	修复外部准备 controller、设备条件/RBAC 协议或不可恢复资源故障；延长超时不能制造 Ready

失败点 / 可见影响	自动恢复 / API 与节点边界	Operator 与人工处理边界
Driver / kubelet 重启及 checkpoint 不一致：准备状态难以恢复	kubelet 重连插件并依据引用/准备记录重试；vendor plugin 按自己的幂等恢复规则核对硬件	Operator 重建进程不等于硬件回滚；损坏/丢失 checkpoint、升级不兼容需受控恢复
Prepare 或 Unprepare 失败：容器无法启动或节点清理未完成	kubelet 对可重试错误再次调用，插件需容忍重复 RPC 和部分完成；共享 Claim 按本地引用计数处理	检查配置、权限、设备占用和插件错误；永久错误需人工处置，不能仅删除 finalizer
CDI spec / runtime 不匹配：Claim 与 Prepare 成功但容器启动失败	修复配置后启动路径可重试；CDI 注入不负责重新选择设备	核对 CDI IDs、spec 路径、runtime 支持与 driver/toolkit 版本；Operator 只修复其管理范围
MIG / VFIO / ComputeDomain 部分变更：API、checkpoint 与硬件/IMEX 不一致	driver/controller 按具体模式重放或清理；API 重试没有通用硬件补偿保证	确认其他消费者后处理 MIG 实例、驱动绑定、域成员或排空节点；不能假设一次重启全部恢复
节点失联或非优雅关机：NodeUnprepare 可能未执行	控制面继续观察消费者和节点状态；节点返回后由 kubelet/plugin 重新协调，外部设备可能需独立清理	对无法返回的节点须核实设备/网络连接已隔离及外部状态已释放，再处理遗留 Claim；API 删除不证明旧节点已停止使用

排障应分别核对 Claim allocation/reservedFor/device conditions、Pod binding、kubelet checkpoint、vendor plugin 状态及 runtime 配置。上表是当前职责归纳；下方保留的 KEP 风险条目不能直接当成所有版本/driver 的通用恢复保证。

KEP 设计与演进（历史笔记）

以下保留 Kubernetes Dynamic Resource Allocation 的历史设计脉络。核心是 sig-node/4381-dra-structured-parameters，它把早期 3063-dynamic-resource-allocation 的 opaque driver 协商路线反转为主线：设备参数必须结构化地暴露给 scheduler 和 autoscaler，Kubernetes 才能可靠做调度和容量推理。逐个 KEP 的 Alpha/Beta/GA、是否实现和 feature gate 见 kubernetes-keps-implementation-matrix；当前阶段以页首核验表为准。

一句话定位

DRA 是 Kubernetes 对 GPU、NIC、FPGA、DPU、network-attached accelerator、可分区设备和共享容量的下一代设备资源模型。它不是 device plugin 的简单替代，而是把“设备发现、设备选择、claim 状态、节点准备、CDI 注入、scheduler 推理、autoscaler 模拟”放到同一个 API 体系里。

为什么 device plugin 不够

Device Plugin API 适合“节点本地、离散、可计数”的资源，例如 `nvidia.com/gpu: 1`。但新设备场景更复杂：

- 设备可能不在节点本地，而是通过 fabric 动态连接。
- 一个设备可能切成多个 partition，例如 MIG-like GPU。
- 多个容器或多个 Pod 可能共享同一已初始化设备。
- 用户需要选择设备属性，例如型号、内存、PCIe root、driver version、NUMA locality。
- scheduler 和 autoscaler 必须能知道“新增一个节点后是否能满足 claim”。

如果设备选择完全由 vendor driver 在调度后 opaque 决定，scheduler 只能猜，Cluster Autoscaler / karpenter 也无法模拟。

4381 的核心模型

历史模型按四个角色拆分：DRA driver 发布带 devices/attributes/capacities 的 ResourceSlice；用户/controller 创建带 requests/selectors/config 的 ResourceClaim 或模板；kube-scheduler 对照库存评估请求并写 allocation status；kubelet 调用 DRA plugin Prepare/Unprepare，再将 CDI 设备交给 runtime。当前完整顺序与模板实例化、条件等待的细分见上方两张图。

ResourceSlice 是 driver 发布的资源库存。每个 device 有名字、属性和 capacity。属性可以被 CEL selector 匹配，capacity 用 quantity 表达。

DeviceClass 是集群管理员定义的设备类别，承载通用 selector 和配置。用户侧 ResourceClaim 引用它，并可继续补充 request selector 和 config。

ResourceClaim 是用户或 controller 要的资源。它的 spec 不变，status 由系统写入 allocation result、reservation、设备结果等状态。

结构化参数的关键价值

早期 3063 让 DRA driver 通过 control-plane controller 参与 allocation。问题是 scheduler / autoscaler 看不懂 driver 的自定义逻辑。4381 的结构化参数让 Kubernetes 至少能理解：

- 有哪些设备。
- 设备在哪些节点或资源池。
- 每个设备有哪些标准化或 vendor-specific 属性。
- request 如何用 selector 表达过滤条件。
- claim 是否已经被 allocated / reserved。
- allocation 结果能否被 kubelet 和 driver 重放。

这不是为了让 Kubernetes 理解所有 vendor 细节，而是把“调度必须知道的部分”变成 API 结构，剩下的配置参数仍可 opaque 地传给 driver。

Scheduler 插件路径

DRA scheduler plugin 不是只在 Filter 阶段做一次检查。它覆盖多个 extension point:

阶段	作用
EventsToRegister	注册 ResourceClaim、ResourceSlice、DeviceClass 等事件，并通过 queueing hints 精准唤醒相关 Pod。
PreEnqueue	快速检查 Pod 引用的 claim 是否存在，不存在就先不要进入正常调度。
PreFilter	收集 claim、class、slice、已分配资源和 in-flight allocation，准备高效 filter。
Filter	判断候选节点是否能满足 claim，执行 selector/capacity/match-attribute 等检查。
Reserve	节点已选定后，在内存里计算 allocation result。
PreBind	在独立 goroutine 中把 allocation/reservation 写回 ResourceClaim status。
Unreserve	绑定失败或调度失败时释放 reservation，避免 deadlock。

这个设计把昂贵或阻塞 API 写操作放到 PreBind 旁路，减少主 scheduling cycle 的阻塞。

ResourceClaim 状态机

DRA 的状态核心在 ResourceClaim.status:

历史状态模型依次描述: unallocated; scheduler 选择 node/device 后 allocated; reservedFor 纳入消费者后被 Pod 或其他 consumer 使用; 消费者退出后进入 deallocated/reusable。它概括的是控制面生命周期，不应把其中一条状态转换当成所有节点准备状态已经同步释放，具体回收边界见上方 kubelet Prepare / CDI / Release。

几个设计点很关键:

- allocation 是否非空决定 claim 是否已分配。
- reservedFor 决定 claim 是否正被某些 consumer 使用。
- 多 scheduler 并发时，status update conflict 是正常同步机制。
- Unreserve 必须释放 reservation，否则两个 Pod 各占一个 claim 等另一个 claim，会形成永久 deadlock。
- kube-controller-manager 清理完成 Pod 的 reservation，并释放不再使用的 allocation。

kubelet 和 DRA plugin

kubelet 通过 plugin registration 发现 DRA kubelet plugin。Pod 绑定到节点并且 claim 已 allocated/reserved 后，kubelet 调用：

- NodePrepareResource：让 driver 在本节点准备设备，返回 CDI device IDs 或等价注入信息。
- NodeUnprepareResource：Pod 不再使用资源时释放节点准备状态。

这条路径的关键是：scheduler 决定“用哪个设备”，kubelet/driver 负责“把设备准备好并注入容器”。如果设备在 scheduler 决定后消失，kubelet plugin 必须二次确认，Pod 可能保持无法启动直到资源恢复或被重新处理。

DRA 与 Autoscaler

DRA 最重要的教训来自 3063 到 4381 的路线变化：Cluster Autoscaler 需要模拟未来节点上的资源可用性。对于 node-local 设备，如果 claim 参数 opaque，autoscaler 无法知道加哪类节点能满足 Pod。

结构化参数让 autoscaler 至少可以读取 ResourceSlice 模型和 claim selector，判断“创建某个 node group 的节点是否可能让 pending Pod 调度成功”。这也是 DRA 为什么不能只停留在 driver 自定义 allocation 的原因。

关键扩展 KEP

KEP	解决的问题	设计意义
5007-device-attach-before-pod-scheduled	设备 attach/初始化可能是异步的，Pod 不应先绑定再失败。	把 device binding conditions 放入 scheduler PreBind 等待路径。
5075-dra-consumable-capacity	设备不一定是离散实例，也可能有可消费容量。	支撑 GPU memory、带宽、license、共享 buffer 等容量型设备。
5941-dra-shared-consumable-capacity	多个设备共享一组 capacity。	处理多个 logical device 背后共享同一个物理资源池。
4815-dra-partitionable-devices	设备可以动态切分。	让 MIG-like 或 TPU-like partition 不再只能预先静态发布。
4816-dra-prioritized-list	用户可接受多种设备配置。	用优先级备选列表表达“首选 A，不行则 B”。
5055-dra-device-taints-and-tolerations	设备健康或策略需要暂时排除。	将 node taint 思路下沉到 device 层。

KEP	解决的问题	设计意义
6080-dra-derived-attributes	不同 driver 属性表达不一致。	用派生属性对齐调度可用的 topology/compatibility 语义。

失败模式和风险

风险	解释	设计处理
stale ResourceSlice	driver 发布的设备状态过期。	kubelet plugin 在 NodePrepare 时二次确认。
scheduler filter 成本高	CEL、设备数量、claim 数量、match attributes 会放大计算成本。	Filter timeout、queueing hints、scheduler metrics。
claim 并发 reservation	多 scheduler 或多个 Pod 争同一 claim。	ResourceVersion/UID/status conflict 作为同步点。
node reboot / kubelet restart	旧 ResourceSlice 可能残留。	kubelet 启动时删除本节点 ResourceSlices，等待 driver 重建。
opaque 参数过多	autoscaler 和 scheduler 无法推理。	structured parameters 作为 GA 主线，opaque config 仅用于 driver 准备。
non-graceful node shutdown	NodeUnprepare 不一定被调用。	driver 需要在 Deallocate 或外部状态中恢复。

和 GPU 生态的关系

- [device-plugin](#) 仍适合简单离散资源；DRA 适合需要属性、claim、共享、拓扑、partition、跨 Pod 生命周期的资源。
- [cdi](#) 是设备注入 runtime 的标准表达，DRA kubelet plugin 可以返回 CDI devices。
- [dra-driver-nvidia-gpu](#) 是观察 DRA 在真实 GPU/MIG/VFIO 场景落地的重要项目。
- node-feature-discovery 仍然有价值，但它更多发布节点能力标签；DRA 发布的是可分配设备库存。
- [k8s-gpu-device-stack](#) 应该把 DRA 作为下一代 GPU 资源抽象主线，而不是 device plugin 的旁支。

阅读顺序

1. 4381-dra-structured-parameters: DRA 主线设计。
2. 3063-dynamic-resource-allocation: 理解为什么 opaque control-plane allocation 被降级。
3. 4815 / 5075 / 5941: 理解 partition 和 capacity。
4. 5007: 理解 device attach/readiness 如何进入调度。
5. 5055 / 4816 / 6080: 理解生产化调度表达。

关键 KEP 实现状态

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
4381-dra-structured-parameters	implemented / stable, 已实现/GA	v1.30 / v1.32 / v1.34	DynamicResourceAllocation, DRASchedulerFilterTimeout	ResourceSlice/ResourceClaim/DeviceClass 结构化参数, scheduler 和 autoscaler 可推理。
3063-dynamic-resource-allocation	withdrawn / alpha, 已撤回	v1.26 / - / -	DynamicResourceAllocation, DRAControlPlaneController	opaque control-plane allocation 路线, 因调度和 autoscaler 不可推理被 4381 取代。
5007-device-attach-before-pod-scheduled	implementable / beta, 仍在 beta	v1.34 / v1.36 / v1.37	DRADeviceBindingConditions	device binding/readiness 条件进入 scheduler PreBind 等待路径。
5075-dra-consumable-capacity	implementable / beta, 仍在 beta	v1.34 / v1.36 / v1.38	DRAConsumableCapacity	设备可表达可消费 capacity, 而不是只表达离散实例。
5729-resourceclaim-support-for-workloads	implementable / beta, 仍在 beta	v1.36 / v1.37 / -	DRAWorkloadResourceClaims	ResourceClaim 支持 workload 级消费者, 连接 DRA 和 PodGroup。
5941-dra-shared-consumable-capacity	implementable / alpha, 仍在 alpha	v1.37 / v1.38 / v1.39	DRASharedConsumableCapacity	多个设备共享同一 capacity pool。
5963-device-		v1.37 /	DRADeviceCompatibilityGroups	表达多设备组合兼容性。

KEP	当前状态	Alpha / Beta / GA	Feature gate	关键实现路径
compatibility-groups	implementable / alpha, 仍在 alpha	v1.38 / v1.39		
4815-dra-partitionable-devices	implementable / beta, 仍在 beta	v1.33 / v1.36 / -	DRAPartitionableDevices	动态或逻辑分区设备进入 ResourceSlice 模型。
4816-dra-prioritized-list	implementable / stable, GA 目标已达	v1.33 / v1.34 / v1.36	DRAPrioritizedList	用户可声明设备请求的优先级备选列表。
5055-dra-device-taints-and-tolerations	implementable / stable, GA 目标已达	v1.33 / v1.36 / v1.37	DRADeviceTaints, DRADeviceTaintRules	设备级 taints/tolerations, 支持健康和策略隔离。
4817-resource-claim-device-status	implementable / stable, GA 目标已达	v1.32 / v1.33 / v1.37	DRAResourceClaimDeviceStatus	分配结果和设备信息进入 ResourceClaim status。
5304-dra-attributes-downward-api	implementable / beta, 仍在 beta	v1.36 / v1.37 / v1.38	NA	将 DRA device attributes 暴露给 Pod 内 workload。
4680-add-resource-health-to-pod-status	implementable / beta, 仍在 beta	v1.31 / v1.36 / v1.37	ResourceHealthStatus	Device Plugin/DRA resource health 进入 Pod status。

追踪重点

- DRA API 版本从 beta 到 GA 的字段稳定性。
- scheduler DRA plugin 的性能数据，尤其是 Filter latency 和 queueing hint 命中率。
- Cluster Autoscaler / karpenter 对 DRA structured parameters 的真实支持。
- NVIDIA、其他 GPU/NIC/DPU driver 是否采用 ResourceSlice/ResourceClaim 标准路径。
- DRA 与 Workload/PodGroup、Topology Manager、NUMA、CDI 的联动是否形成闭环。

Kubernetes GPU / Device Stack 项目地图

当前上游核验 (2026-09-27)

以下 commit 是执行时官方默认分支快照，不代表 release 或整套 GPU 能力的成熟度。[src-node-feature-discovery-architecture](#)、[src-gpu-operator-architecture](#)、[src-k8s-device-plugin-architecture](#)、[src-dra-driver-nvidia-gpu-architecture](#) 与 [src-hami-architecture](#) 保留 2026-06 的 raw-backed Source 及原始 ASCII 图；相关 KEP 笔记保留 2026-07 的设计/演进语境。当前 DRA 基础能力与扩展阶段分别见 [kubernetes-dra-design-deep-dive](#)。

项目	当前 commit	当前职责 / allocation 边界	M5-C 层
node-feature-discovery	386fd a4332 ba	worker/master 发布节点能力与 labels, NodeFeature/NodeFeatureRule 表达发现和规则, topology-updater/GC 管拓扑数据及清理；不执行每次 Pod 设备分配	Discovery / capability
gpu-operator	60526 e35ef ee	管理 GPU 软件组件生命周期；官方 26.7 文档区分 Device Plugin 的 ClusterPolicy 路线与 DRA 的 GPUCluster 路线，后者不直接管理 GPU driver	Node software lifecycle
k8s-device-plugin	86142 cf1a9 3f	kubelet 注册、ListAndWatch/health、Allocate 与 extended resources；按配置返回 env/mount/CDI 设备交接信息	Device Plugin allocation / injection handoff
dra-driver-nvidia-gpu	495bf 4c59b 94	DRA GPU 与 ComputeDomain 插件/控制面，发布设备库存并执行节点 Prepare/Unprepare；GPU 配置能力和 ComputeDomain 支持范围需按安装版本分别判断	DRA driver / node preparation
hami	a2dd1 91b2e 7f	scheduler/extender 计算设备共享与 reservation, annotation 交接给 device-plugin Allocate, HAMi-core 在容器侧实施相应隔离	Sharing / allocation handoff / isolation

官方依据：[NFD 当前介绍](#)、[GPU Operator 26.7 DRA 管理路径](#)、[NVIDIA Device Plugin README](#)、[NVIDIA DRA README](#) 与 [HAMi FAQ](#)。通过 NVIDIA/k8s-dra-driver-gpu 查询当前 HEAD 时，GitHub 返回的规范 commit 链接仍位于 kubernetes-sigs/dra-driver-nvidia-gpu，这里采用实际返回的链接。

共存与版本边界：GPU Operator 26.7 文档规定集群使用 GPUCluster 或 ClusterPolicy 之一，不支持两者并存或直接原地迁移；HAMi FAQ 要求避免同一节点上的插件争用同一 GPU resource。上述 NVIDIA DRA 仓库 README 仍区分受支持的 ComputeDomain 与未正式支持、默认关闭的独立 GPU plugin，不能用它推定

Operator 26.7 所管理 driver 的相同默认值。部署判断须绑定具体 release、Helm 配置和 feature gates，不能只看 [device-plugin](#)、[kubernetes-dra](#)、[cdi](#) 或 [gpu-sharing](#) 名称。

这页把 [src-k8s-gpu-device-plugins-stars](#) 从 star list 整理成 Kubernetes GPU / 异构设备资源层地图。核心结论：LLM serving 的 GPU 底座已经不只是“把 /dev/nvidia0 挂进 Pod”，而是 driver/operator、container runtime、device discovery、DRA/CDI、sharing、scheduler、observability、diagnostics 和 fake device 测试环境的组合。

D1 · GPU Device Stack

DISCOVERY / CAPABILITY

NFD: NodeFeature / NodeFeatureRule / labels / NodeResourceTopology

- - published capability: informs placement / operator policy - ->

NODE SOFTWARE LIFECYCLE

GPU Operator: reconcile selected policy and managed components

Driver / runtime prerequisites must match the chosen installation path

├─ configure Path A —> ClusterPolicy / Device Plugin

| extended resource / kubelet Allocate

| — env / mounts / optional CDI —> runtime

└─ configure Path B —> GPUCluster / DRA driver

DeviceClass + ResourceSlice / ResourceClaim

scheduler allocation / kubelet

NodePrepareResources

— CDI device handoff —> runtime

Path A OR Path B: alternative GPU Operator management modes

Telemetry / health - - observed state - -> operators / node recovery

图例：实线 — label —> 表示有序控制动作、API 调用或节点交接；虚线 - - label - -> 表示发现发布、reconcile、watch/health 反馈及重试。上层发现信号供控制器决策，不表示每次分配都同步调用 NFD。node-feature-discovery 发现能力，[gpu-operator](#) 管理组件生命周期；[device-plugin](#) / [kubernetes-dra](#) 承担各自的 allocation 路径，[cdi](#) 位于设备注入和 runtime 交接处，不是第三种 scheduler 或 allocator。

图中的分叉表示选择，不表示把两条 GPU 管理路径同时安装。按 [GPU Operator 26.7 文档](#)，ClusterPolicy 与 GPUCluster 不能在同一集群并存，也不支持从 ClusterPolicy/独立 DRA Helm 安装直接原地迁移到 GPUCluster。GPUCluster 不管理 GPU driver，需独立 NVIDIA Driver 或预装 driver，并事先准备 CDI-compatible runtime；不能把 ClusterPolicy 的 driver/toolkit 管理默认套到另一条路径。即使脱离 Operator 手动部署，也须确保同一物理设备及资源名有明确分配所有权，避免被不同 allocator 重复分配。

D2 · Device Plugin Allocation

```
Device plugin — register endpoint / resource name —> kubelet
Device plugin - - ListAndWatch: devices / health - -> kubelet device
manager
kubelet - - Node status publication - -> API: capacity / allocatable

Pod extended-resource request + observed Node allocatable
— scheduler Node fit / bind —> Pod assigned to Node
- - Pod watch - -> kubelet
kubelet — select device IDs / Allocate RPC —> device plugin
device plugin — Allocate response: env / mounts / device specs / CDI IDs
—> kubelet
kubelet — CRI container configuration —> container runtime
runtime — apply configured device injection / start —> workload

Device health change - - ListAndWatch update - -> allocatable update /
future scheduling
```

这是传统 extended-resource 路径：scheduler 先按节点资源数量等条件选 Node，节点侧 kubelet/device plugin 完成具体设备分配与容器配置。NVIDIA 的 device-list strategy 决定 envvar、volume-mounts、CDI annotation 或 CDI CRI 交接方式；不是所有安装都走 CDI。健康设备变为 unhealthy 时，kubelet 减少相应 allocatable，capacity 保持原值；这不等于自动迁移或修复已使用故障设备的容器。参见 [Kubernetes Device Plugins](#) 和 [NVIDIA plugin 配置](#)。

D3 · Discovery / Operator Control Loops

```
NFD worker — scan hardware / OS / local feature sources —> feature set
feature set - - publish NodeFeature - -> Kubernetes API
NodeFeature / NodeFeatureRule - - watch / evaluate rules - -> NFD master
NFD master — update labels / configured annotations or resources —> Node
NFD topology-updater - - publish topology - -> NodeResourceTopology
NFD GC - - observe stale / removed nodes - -> cleanup obsolete NFD objects
Node capability / topology - - consumed state - -> scheduler / operator /
policy

Admin — apply ClusterPolicy OR GPUCluster —> Kubernetes API
desired policy / operand changes - - watch / enqueue - -> GPU Operator
reconcile
GPU Operator — create / update managed operands —> DaemonSets /
```

```

Deployments / config
operand readiness / errors - - observed feedback - -> GPU Operator
GPU Operator — write conditions / status —> policy object
retryable error / further change - - requeue - -> GPU Operator reconcile

```

NFD 的 labels、feature 与 topology 数据是已观测状态，可能受扫描周期、API/watch 延迟或 worker 故障影响；NFD GC 清理过期对象不等于硬件故障恢复。Operator 的 Ready 描述所管理组件状态，不承诺每个 Pod 的 GPU 请求都能满足。分配哪块设备、Prepare 是否成功以及容器内隔离是否生效，仍属于后续路径。NFD 组件与数据对象见 [当前 Introduction](#)。

D4 · DRA Resource 生命周期

阶段	对象 / 操作与负责方	必须区分的状态
类别与库存	管理员或 driver 安装流程提供 DeviceClass；DRA driver 发布 ResourceSlice 设备库存、属性和容量	DeviceClass 是选择/配置规则，ResourceSlice 是库存；NFD labels 不能替代设备库存
请求	用户或 workload controller 创建 ResourceClaim；也可通过 ResourceClaimTemplate 为 Pod 派生 Claim	请求存在不等于已分配，模板本身不是一次 allocation
分配与绑定	scheduler 根据 Claim、class、slice 与 Node 约束选择设备并写 allocation/reservation；启用相关扩展时等待 binding conditions，再绑定 Pod	已分配、已绑定、外部设备就绪是不同状态；条件等待依赖相应 feature 和 driver
节点准备	kubelet 调用 DRA plugin 的 NodePrepareResources；driver 配置设备并维护节点侧状态，返回 CDI 设备信息	Claim status 成功不证明节点硬件配置/CDI 已完成；可选 node operations 有独立阶段限制
Runtime 交接	kubelet 经 CRI 把 CDI device IDs 交给兼容 runtime，由 runtime 按 CDI spec 应用设备配置	CDI 是注入接口，不决定 Claim 分配或共享额度
终止与回收	kubelet 在节点不再需要该 Claim 时执行 NodeUnprepareResources；控制面按 Claim/消费者生命周期处理 reservation、删除及 allocation 回收	节点 unprepare 不等于所有消费者已退出或持久 Claim 已释放；不能因一个 Pod 退出就盲删共享状态

完整状态所有权、binding conditions、Prepare/checkpoint/CDI 与回收细节见 [kubernetes-dra-design-deep-dive](#)；官方流程见 [How DRA Works](#)。基础 DRA 自 v1.35 stable，各扩展和厂商 driver 的支持阶段仍须分别核验。

Sharing / Isolation Overlay

hami 的经典路径横跨调度、分配交接和容器隔离：scheduler/extender 根据节点设备视图计算 memory/core/count 与设备 reservation，再写 Pod allocation annotations；device-plugin 在 kubelet Allocate 时读取这些信息并返回设备、环境与挂载配置；HAMi-core 在容器内执行相应 CUDA 调用拦截和限制。hami.io/vgpu-devices-to-allocate 与 hami.io/bind-phase 是分配进度协议，不是业务 Ready 标志，也不应由用户手工伪造。参见 [HAMi Protocol](#)。

同一节点不能让 HAMi 与 NVIDIA 官方 device plugin 同时争用 nvidia.com/gpu；GPU Operator 可负责其他软件组件，但要按目标版本关闭冲突的 allocation operand。HAMi-core 的软件拦截、MIG 的硬件分区以及 time-slicing 的共享策略也不能视为等价隔离保障，需验证实际调用路径、显存限制和尾延迟。[HAMi FAQ](#) 说明了插件共存及软件隔离边界。[HAMi-DRA](#) 改变的是 Claim/设备分配路径，仍可使用同一 HAMi-core 隔离层；切换分配 API 不会自动升级隔离强度。

D5 · 失败边界

失败点 / 可见影响	API、节点或 Operator 自动恢复边界	需要人工确认 / 修复的情况
NFD 状态陈旧：labels/topology 与节点不符	worker 恢复扫描、master 更新与 GC 清理可重新收敛；消费者需等待新观测	检查 worker 权限、feature source、规则和节点实际硬件；修复标签来源，不能只反复调度
Operator operand 未就绪：driver/toolkit/plugin/validator 阻塞	reconcile 可重建或更新所管理对象并持续报告 conditions	内核/driver 不兼容、镜像/权限、安装模式冲突或外部 runtime 配置需修复；重建 Pod 不能保证硬件恢复
Device Plugin 注册、Allocate 或 health 失败：资源消失/容器无法启动	plugin 按实现重注册，kubelet 更新健康和 allocatable，并重试相应启动路径	检查 socket/API 版本、NVML/driver、资源名争用及 Allocate 错误；运行中 GPU 故障可能需要节点处置或重建 workload
ResourceSlice / Claim 状态漂移：库存过期、设备无匹配或 allocation 不一致	driver 发布新库存、scheduler/controller 重读与重试可处理暂时不同步	错 driver/device identity、pool generation、selector 或存量 Claim 与硬件变化冲突需核对；不盲改已分配 Claim status
Binding conditions 等待超时：设备已选但 Pod 未绑定	scheduler 依条件和超时路径结束本次尝试、清理相应保留状态并重试	外部准备 controller、状态权限或条件协议永久失败需要修复；单纯延长超时不产生设备就绪
NodePrepare / CDI / checkpoint 错误：Pod 已绑定但不能启动或清理	kubelet/plugin 对可重试错误重试，driver 按自身 checkpoint 恢复	核对 CDI runtime/spec、checkpoint 与真实设备状态；不直接删除 checkpoint/finalizer 来掩盖未完成清理

失败点 / 可见影响	API、节点或 Operator 自动恢复边界	需要人工确认 / 修复的情况
	节点状态；Operator 可恢复受管进程	
MIG / VFIO / ComputeDomain 部分变更：API 与节点/IMEX 状态不一致	各 driver/controller 按支持的幂等与恢复路径重算；API 重试本身不回滚硬件修改	按模式检查 MIG 实例、驱动绑定、IMEX/域成员；必要时排空节点后人工修复，评估其他消费者影响
HAMi annotation 交接失败：reservation 与 Allocate 进度不一致	scheduler/device-plugin 依协议更新 pending allocation 与完成状态，可重试的 API 错误按实现恢复	检查版本、编码、目标节点、锁/缓存和设备状态；不能伪造 success 或仅重写注解当作已分配
隔离或 overcommit 不符合预期：OOM、抢占干扰或尾延迟恶化	配额记账与进程重启不能自动修复软件拦截绕过、配置错误或硬件容量不足	验证 HAMi-core 注入、调用兼容性、共享倍率与真实负载；按隔离目标调整配置或设备模式

失败矩阵是按职责归纳的排障边界，不承诺所有实现都可自动恢复。先判断故障停在 API 记账、节点准备、组件生命周期还是容器隔离，再对照 D2 – D4 和具体 driver 的 conditions、events、日志与设备状态。

一句话分层

层	代表项目	要解决的问题
节点发现	node-feature-discovery	发布能力标签、规则结果与拓扑信号，供 placement/operator 使用
驱动与节点软件栈	gpu-operator , NVIDIA container toolkit, DCGM	节点怎样安装、升级和暴露 GPU 软件栈
设备暴露	k8s-device-plugin , Intel device plugins, host device plugin	kubelet 如何发现并分配专用设备
声明式设备分配	kubernetes-dra , dra-driver-nvidia-gpu	Pod 如何声明更复杂的设备需求和分配结果
Runtime 注入	cdi , NVIDIA container toolkit	怎样把已分配设备的 runtime 配置交给容器引擎
GPU sharing / vGPU	hami, Volcano vGPU, gpushare, vgpu-scheduler	多租户如何切分、调度和隔离 GPU
调度与队列	kueue, scheduler-plugins, autoscaler	GPU batch / AI workload 如何排队和扩缩

层	代表项目	要解决的问题
观测与诊断	DCGM exporter, GPUD, fake-gpu	怎么发现 GPU 健康、利用率和测试调度逻辑

核心项目边界

node-feature-discovery

NFD 提供 worker/master、topology-updater 与 GC，发布硬件/系统能力、规则标签及资源拓扑。它帮助 scheduler 和 Operator 识别节点，不维护每个 Pod 的设备 allocation，也不替代 Device Plugin 或 DRA driver。

[k8s-device-plugin](#)

NVIDIA 官方 GPU device plugin 是传统路径的基线：通过 NVML/CUDA discovery 向 kubelet gRPC 注册 [nvidia.com/gpu](#) 等资源，并在 Allocate 阶段用 env、volume、CDI 等方式交接设备配置。它可提供配置支持的 MIG/time-slicing/MPS 资源暴露，但不负责 driver 生命周期、完整多租户治理或模型 serving。

[gpu-operator](#)

GPU Operator 是生产集群运维入口：传统 ClusterPolicy / NVIDIA Driver 路径管理 driver、container toolkit、device plugin、DCGM、MIG manager、validator 等组件；当前 GPUCluster 提供另一条 DRA 管理路径，依赖独立 driver/runtime 准备。它解决“节点 GPU 软件栈怎样持续正确”，不决定单次 Pod 获得哪块设备。

hami

HAMi 是 GPU sharing / vGPU 方向的重要样本。它通过 webhook、scheduler extender、device plugin 和多厂商 device abstraction，把 GPU 显存、算力、切分和调度策略放到 K8s 资源模型旁边。它解决“多个 workload 如何共享 GPU”，但也带来调度、隔离、观测和兼容性的复杂度。

[dra-driver-nvidia-gpu](#)

NVIDIA DRA driver 把设备库存与节点准备接入 ResourceClaim / ResourceSlice 分配模型，GPU 与 ComputeDomain 路径具有各自生命周期。基础 DRA 已稳定，但动态设备、sharing、VFIO 等能力仍需按 Kubernetes 扩展和厂商 release 分别评估，不能无条件替代现有 device plugin 部署。

和 LLM serving 的关系

[llm-inference](#) 会把 GPU 资源层进一步拉进架构设计：

- Prefill/Decode 分离要求不同 GPU 池承担不同负载。
- KV cache offload 让显存、CPU 内存、NVMe 和网络都成为资源决策。

- LoRA / adapter serving 需要更细的内存与模型资产管理。
- GPU sharing 会影响 tail latency, 不能只看平均利用率。
- DRA/CDI 会影响 serving operator 如何把“设备分配结果”交给 runtime。

所以 GPU stack 不应只作为运维背景, 而应和 [llm-d](#)、[aibrix](#)、kserve、[gpustack](#)、[kubeai](#)、kueue、node-feature-discovery 等项目分层比较, 区分 model serving、准入与节点发现。

选型提示

- 生产 NVIDIA GPU 集群基础栈: 先看 [gpu-operator](#) + [k8s-device-plugin](#)。
- 多租户共享和 vGPU: 看 hami, 同时关注隔离和性能尾延迟。
- 下一代设备 API: 看 [kubernetes-dra](#) + [dra-driver-nvidia-gpu](#) + [cdi](#)。
- LLM serving 平台: GPU 栈只是底座, 还需要 [llm-inference](#)、[inference-routing](#)、[model-serving-operator](#) 和可观测能力。

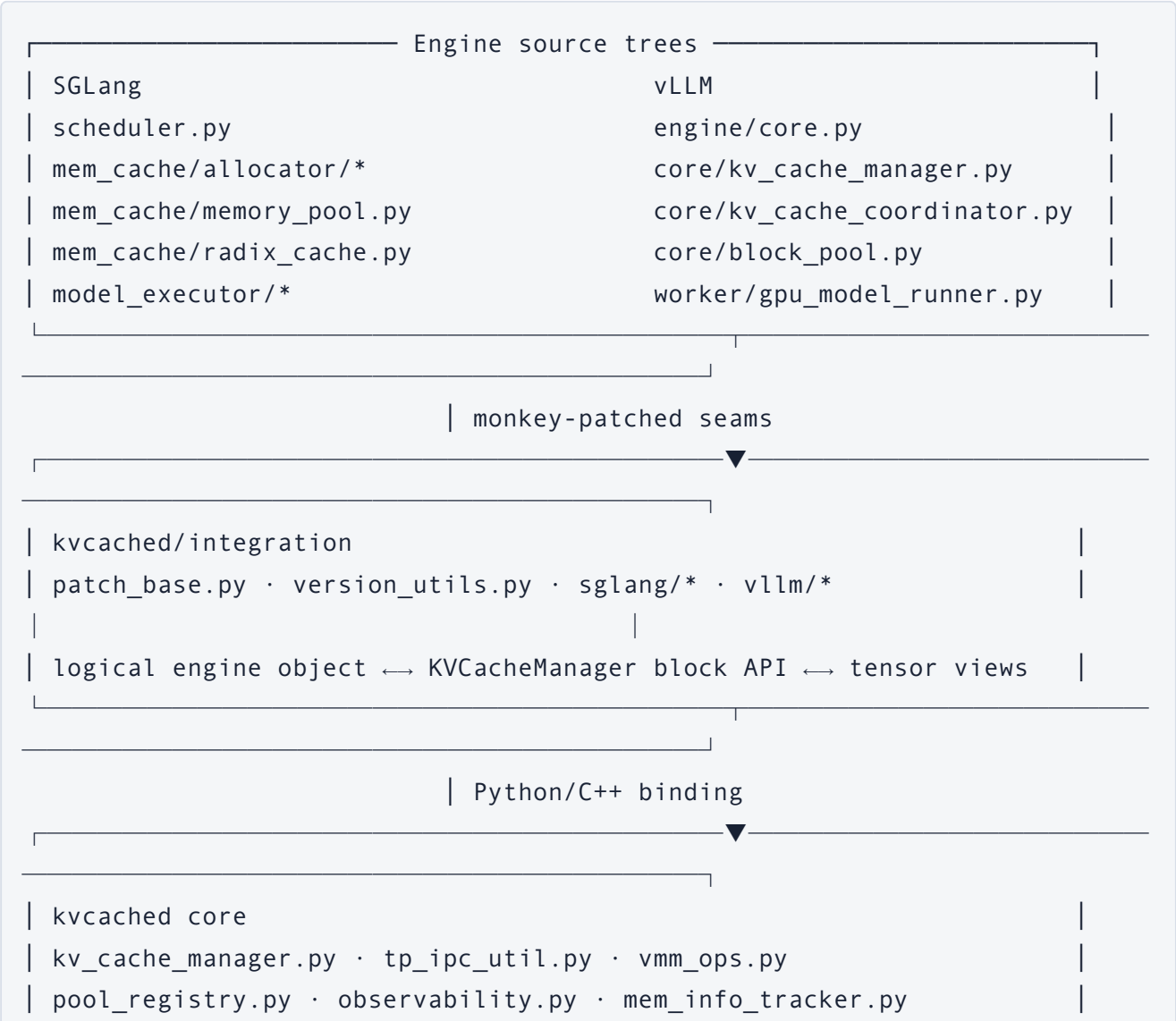
KVCacheD × SGLang × vLLM 源码深潜

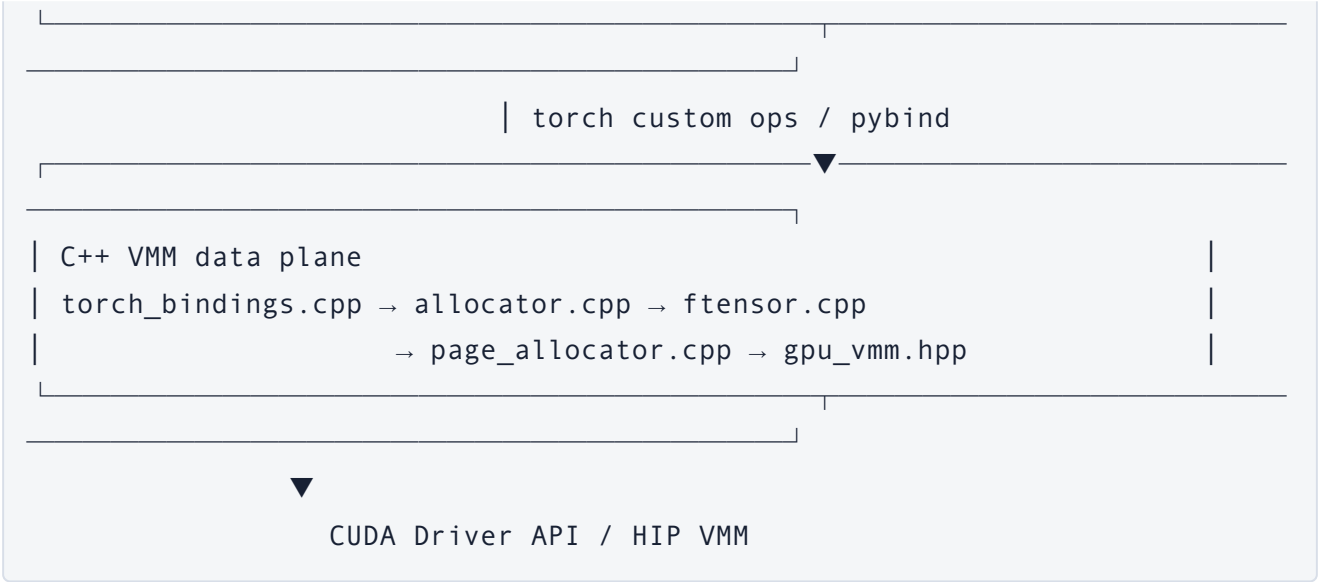
本页专门回答“代码具体怎么跑”。关系与概念先读 [kvcached-sglang-vllm-knowledge-system](#)；完整原始分析与兼容矩阵见 [src-kvcached-architecture](#)。本页所有路径基于以下本地快照：

需要把调用链落到实际部署、实验和升级验证时，继续阅读 [kvcached-integration-implementation-guide](#)。

仓库	HEAD	角色
/Users/zhenyu.jiang/kvcached	884108704f44	GPU VMM、逻辑页管理、engine patches
/Users/zhenyu.jiang/sglang	44ef8fecfe69	token/radix-oriented inference engine
/Users/zhenyu.jiang/vllm	dc36fcce902a	block-oriented inference engine

1. 源码总架构





源码阅读时沿纵向追一条调用链，不要同时横向展开三仓所有模块。核心 seam 只有两个：

1. engine 的 logical allocator：SGLang token/page allocator 或 vLLM BlockPool。
2. engine 的 physical KV tensor creation：SGLang memory pool buffer 或 vLLM GPUModelRunner KV tensors。

2. 自动注入：补丁如何在 engine import 前生效

2.1 安装期

入口文件：

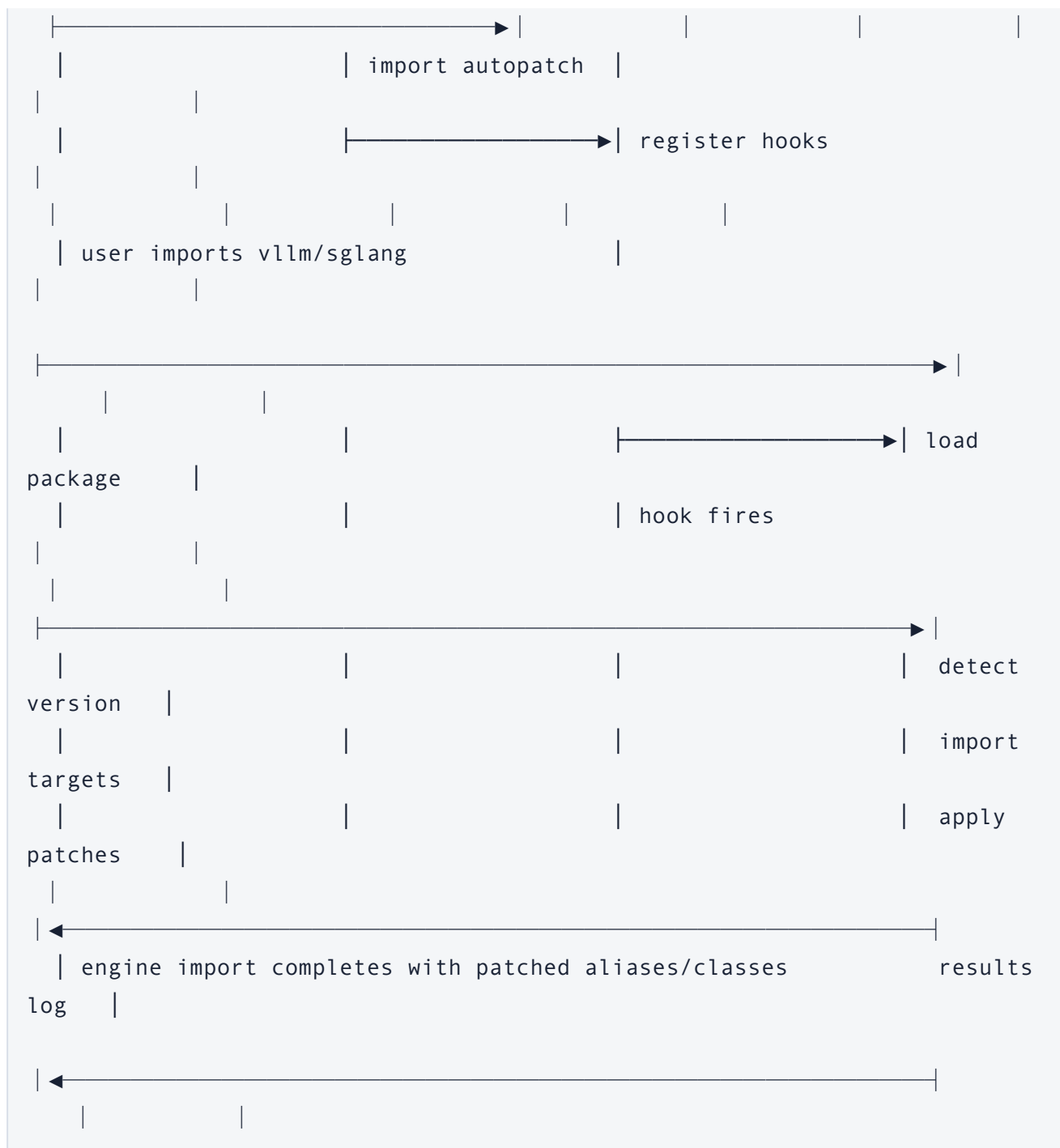
- kvcached/setup.py
- kvcached/kvcached_autopatch.pth
- kvcached/kvcached/autopatch.py

setup.py 做两件重要的事：

1. 以 PyTorch C++ stable ABI 构建 kvcached._C。CUDA build 链接 libcuda 以获得 cuMem*；HIP build 使用 CppExtension + amdhip64，避免 torch hipify 破坏已有的 backend abstraction。
2. 把 kvcached_autopatch.pth 放在 site-packages 根目录。Python 启动时 site 模块执行 .pth 中的 import，因此补丁注册发生在用户显式 import vLLM/SGLang 之前。

2.2 启动时序





kvcached/autopatch.py:6-19 只是 import 两个 engine-specific autopatch module; 异常被吞掉是为了没安装某个 engine 时不影响另一个。真正是否启用由 engine autopatch 中的 KVCACHED_AUTOPATCH 检查决定。

2.3 PatchManager 的执行模型

integration/patch base.py 定义:

- `BasePatch`: 声明 `library`、`target_module`、`target_class`、`patch_name`，并提供 `patch` marker 幂等检查。
- `PatchManager.register_patch`: 保存 `patch` 与 `version range`。

- `apply_all_patches`: 检测 engine version、按 range 跳过不兼容 patch、动态 import target_module、调用 `apply()`。
- `VersionManager/VersionRange`: 隔离跨版本签名和模块位置差异。

Patch marker 是必要的，因为 import hook、测试 reload 或多条引用链都可能重复触发；没有幂等标记会发生 wrapper 套 wrapper、重复 listener 或 alias 漂移。

3. Python/C++ API 边界

3.1 Python 暴露的 VMM 操作

`kvcached/vmm_ops.py` 把 C++ 接口分成两类：

Python 名称	实现通道	作用
<code>init_kvcached</code>	<code>torch.ops.kvcached</code>	初始化全局 <code>FTensorAllocator</code>
<code>create_kv_tensors</code>	<code>torch.ops.kvcached</code>	reserve VA 并返回 torch tensor
<code>kv_tensors_created</code>	<code>torch.ops.kvcached</code>	manager 后台初始化同步
<code>map_to_kv_tensors</code>	<code>torch.ops.kvcached</code>	映射 logical offsets
<code>unmap_from_kv_tensors</code>	<code>torch.ops.kvcached</code>	单进程直接解除映射
transactional map/unmap	<code>pybind_C</code>	返回新增 offset 或持有 transaction state
<code>PageAllocator/InternalPage</code>	<code>pybind_C</code>	Python manager 直接操作 C++ 对象

为何混用 custom ops 与 pybind: tensor 参数和 stable ABI 操作适合注册到 `torch.ops`；不接 tensor、需要返回 C++ class/复杂 transaction 结果的接口保留 pybind。

3.2 Tensor 创建调用链

```

engine adapter alloc_kv_cache(...)
→ kvcached.vmm_ops.create_kv_tensors
→ torch binding create_kv_tensors
→ FTensorAllocator::create_kv_tensors
  ├── align requested bytes to kPageSize
  ├── choose per-layer or contiguous layout
  ├── construct zero page / compound zero page
  └── FTensor(name, size, dtype, device)
      ├── alloc_virtual_mem(device, size)
      ├── init_with_zero_()
      └── torch::stable::from_blob(vaddr, sizes, strides, device, dtype)

```

- raw flat tensor(s)
- adapter uses view/as_strided/permute
- engine receives native expected KV shape

FTensor 是实现的核心。from_blob 不分配第二份 storage；它把预留的 GPU virtual address 当作 tensor storage。tensor 的可寻址范围大，不代表相同大小的 VRAM 已分配。

4. C++ VMM 数据结构

4.1 FTensorAllocator

文件: csrc/allocator.cpp、csrc/inc/allocator.hpp。

主要状态:

- static g_allocators_: 按 group_id 保存 allocator; group 0 在 init 创建, 其余 hybrid group 懒创建。
- ftensors_: non-contiguous layout 下每层一个 FTensor。
- contiguous_kv_tensor_: contiguous layout 下单一大 FTensor。
- zero_page_: 尚未backing或解除backing后的安全映射。
- pending_unmap_、finalized_unmap_transactions_: 分布式unmap幂等状态。

create_kv_tensors 的 layout 分支:

```
non-contiguous
layer0 FTensor: [K region | V region]
layer1 FTensor: [K region | V region]
...
one logical page offset maps one page in every layer and K/V target

contiguous
one FTensor: [block/page major][layer][K/V][payload]
one compound physical mapping covers num_layers × num_kv_buffers
```

4.2 FTensor

文件: csrc/ftensor.cpp、csrc/inc/ftensor.hpp。

一个 FTensor 保存:

- vaddr_: 稳定 virtual base address。
- size_、page_size_、dtype/device。
- tensor_: from_blob 创建的稳定 view。

- `mapping_`: `page_id` → physical Page handle。
- `failed_pages_`: rollback/释放不确定时保留handle，避免错误复用。

4.3 map 状态转换

```
FTensor::map(offset)
→ validate offset alignment/range
→ reject if mapping_ already owns page_id
→ GPU: unmap zero page at target VA
→ make_unique_page(device, page_id, page_size)
→ physical Page::map(target VA)
→ mapping_[page_id] = handle

failure:
if physical map happened → unmap it
remap zero page
release physical handle
any rollback failure → retain failed handle + state consistency error
```

4.4 unmap 状态转换

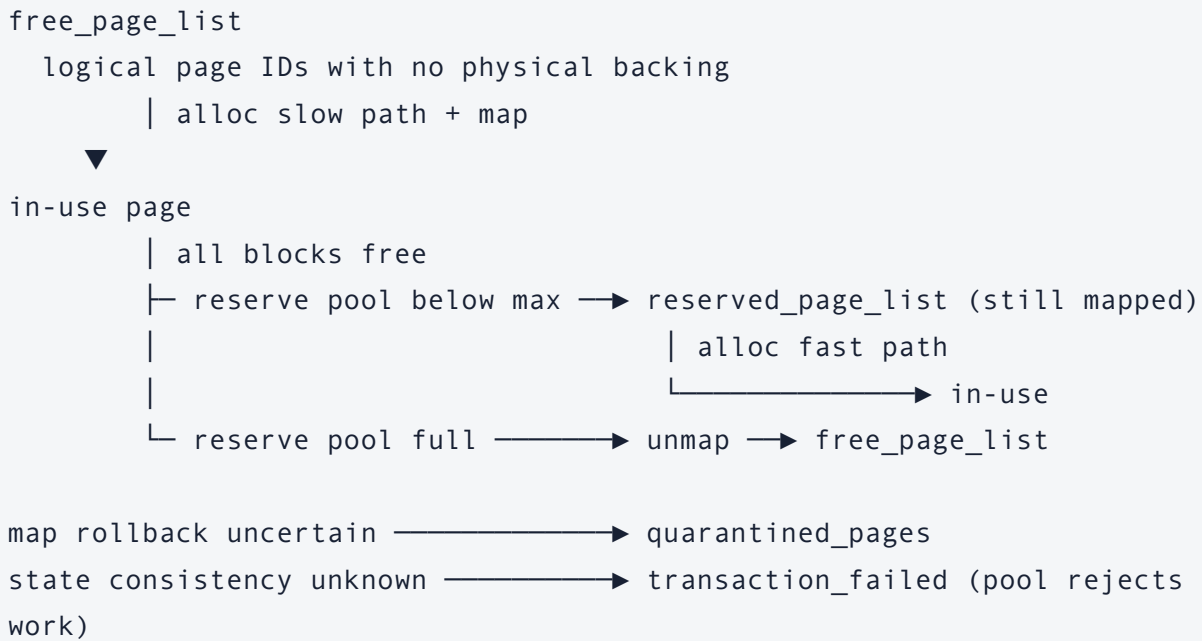
```
FTensor::unmap_retain_(offset)
→ validate currently mapped
→ cuMemUnmap physical mapping
→ map zero page back to same VA
→ move physical handle out of mapping_
→ caller decides:
    direct unmap: handle.release()
    transaction prepare: keep handle in PendingUnmapTransaction
    abort: map retained handle back
    commit: release retained handle
```

“先重新映 zero page，再释放 handle”的顺序避免 VA 留下未映射洞，也让 rollback 有明确恢复点。

5. PageAllocator 与 KVCacheManager

5.1 C++ PageAllocator 状态

文件: `csrc/page_allocator.cpp`。



constructor 根据 `mem_size_per_layer / page_size` 建 logical page id 空间，并创建 `MemInfoTracker`。`alloc_page()` 优先从 `reserved_page_list` 取，避免map latency；slow path从 free list取后调用`map_pages()`。`free_pages()`保留一部分mapped reserve，其余批量unmap。

5.2 Python KVCacheManager 状态

文件：`kvcached/kv_cache_manager.py`。

状态	内容
<code>avail_pages</code>	已mapped、有free blocks的InternalPage
<code>full_pages</code>	已mapped、没有free block的InternalPage
<code>num_avail_blocks</code>	仅统计 <code>avail_pages</code> 中的free block
<code>reserved_blocks</code>	resize/reserve逻辑中暂不参与普通allocation的block
<code>null_block</code>	SGLang/vLLM padding/null contract要求保留的block 0
<code>retired_pages</code>	logical free但等待GPU batch fence才能unmap的page
memory-limit fields	limit revision、effective bytes、shrink target

5.3 alloc 详细流程

```

KVCacheManager.alloc(need_size)
  → _wait_post_init(): tensor必须已在worker创建
  → observe PageAllocator resize target

```

```

→ available_size() >= need_size ?
  └─ no → return None
  └─ yes
→ first consume reserved_blocks
→ while remaining > 0
  └─ avail_pages non-empty
  │   → _pick_avail_page(remaining)
  │       exact/best fit first; otherwiselargest partial page
  └─ no avail page
      → PageAllocator.alloc_page()
      → InternalPage.init(block_mem_size)
      → map physical page if slow path
→ InternalPage.alloc(k)
→ move page to full_pages or avail_pages
→ return exactly need_size block IDs

```

alloc_page failure after partial work

```

→ page-derived IDs go through free()
→ reserved-derived IDs prepend back to reserved ledger
→ return None

```

_pick_avail_page 是page-affine关键：能容纳完整remaining run时选最小合适page，不能时选free最多的page。它试图让一个request的blocks集中，减少prefix cache pin住多页的概率。

5.4 free 详细流程

```

KVCacheManager.free(block_ids)
→ group block IDs by physical page
→ locate page in full_pages/avail_pages
→ InternalPage.free_batch(ids)
→ page still occupied
  └─ keep in avail_pages
→ page empty
  └─ defer_physical_release
  │   └─ append (epoch, page_ids) to retired_pages
  └─ immediate
      └─ PageAllocator.free_pages
          └─ retain mapped reserve pages
              └─ transactional/direct unmap remainder

```

逻辑free与物理release是分开的。即便request释放了block，page内其它request/prefix仍可能占用；即便整页空了，reserve pool或async fence也可能暂时保留physical mapping。

6. SGLang 源码接入

6.1 原生对象

关键上游文件：

- `sglang/python/sglang/srt/managers/scheduler.py`
- `sglang/python/sglang/srt/mem_cache/allocator/base.py`
- `.../allocator/token.py`
- `.../allocator/paged.py`
- `sglang/python/sglang/srt/mem_cache/memory_pool.py`
- `sglang/python/sglang/srt/mem_cache/radix_cache.py`
- `sglang/python/sglang/srt/mem_cache/kv_cache_configurator.py`

原生调用关系：

```
Scheduler.run_event_loop
→ get_next_batch_to_run
→ RadixCache.match_prefix
→ allocation helpers
    → TokenToKVPoolAllocator.alloc
    or PagedTokenToKVPoolAllocator.alloc_extend/alloc_decode
→ ScheduleBatch
→ run_batch / ModelRunner
→ process_batch_result
→ RadixCache.cache_finished_req or allocator.free
```

6.2 patch 注册清单

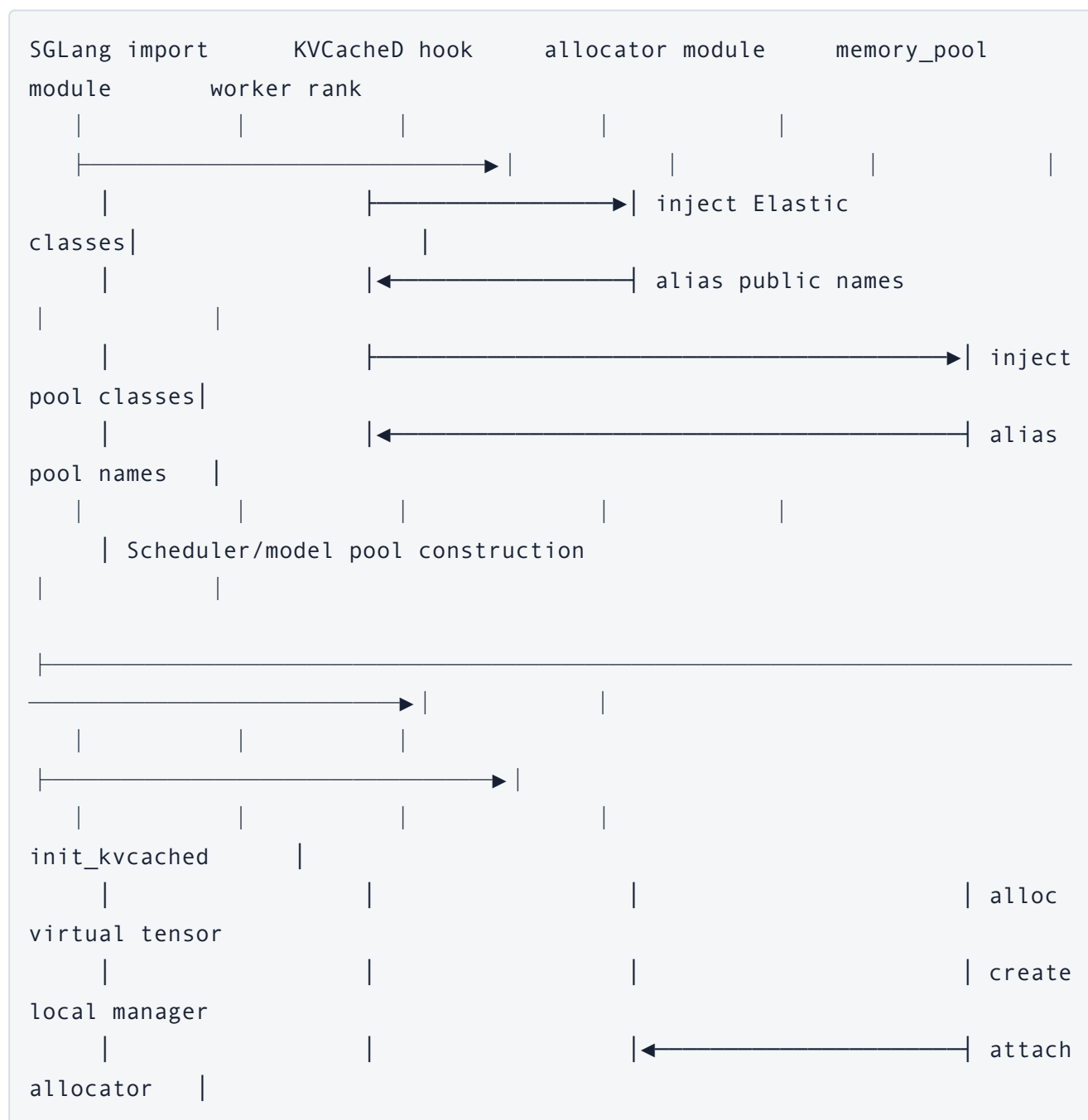
`integration/sglang/autopatch.py` 注册顺序是有语义的：

1. `ElasticAllocatorPatch`
2. `ElasticSWAAllocatorPatch`
3. `ElasticMemoryPoolPatch`
4. `ElasticMLAMemoryPoolPatch`
5. `ElasticMambaPoolPatch`
6. `ElasticHybridLinearKVPoolPatch`
7. legacy/current virtual capacity patch
8. Scheduler memory leak patch

9. RadixCache limit patch

allocator和pool alias必须先于捕获这些class alias的capacity owner模块；否则目标模块在import时已经把原生class保存到module global，后续只改package alias无法生效。

6.3 SGLang 初始化时序



6.4 Elastic token allocator

ElasticTokenToKVPoolAllocator 继承 BaseTokenToKVPoolAllocator，但不维护原生free tensor：

- available_size() → min(manager.available_size(), self.size)。

- `alloc(n)` → manager返回Python block/token IDs，再转GPU int64 tensor。
- `free(indices)` → 根据SGLang free-group语义立即free或延迟到group end。
- `clear()` → `manager.clear`并重置free group。

page_size > 1 时, ElasticPaged allocator把manager block id视作SGLang page id:

```
manager block IDs [4, 9], SGLang page_size = 4
→ token indices [16,17,18,19, 36,37,38,39]
```

extend/decode location 计算仍复用 SGLang 原生 kernel, KVCacheD 只供应新 page ids。

6.5 Elastic MHA/MLA pool

原生 MHATokenToKVPool 会创建每层 K/V torch buffer。elastic subclass 跳过这段 allocation，调用：

```
sglang.interfaces.init_kvcached
    → vmm_ops.init_kvcached(device, PAGE_SIZE, layout)

sglang.interfaces.alloc_kv_cache(shape, dtype, device, layers, page_size,
type)
    → compute bytes per engine block
    → reserve virtual bytes per layer
    → create_kv_tensors
    → build K/V or MLA views

sglang.interfaces.get_kv_cache_manager(...)
    → world_size=1 local manager
    → reserve_null_block=True
    → attach as pool.kvcached allocator
```

`world_size=1` 是刻意选择：每个SGLang TP worker的pool/tensor由本rank拥有，本地map只改变本rank，不做vLLM式coordinator fan-out。

6.6 Mamba super-cell

alloc mamba states 把一个slot的多种state布局为:

[illegible]

每种state按dtype itemsize对齐，整个cell再pad到PAGE_SIZE的最小可用因数。原因是InternalPage只交付完整block；若cell不能整除page，会出现跨页block被跳过、manager却按num_slots高估容量。

6.7 RadixCache bound

KVCacheD 不重写 match_prefix/insert/evict。RadixCacheLimitPatch 只包装 cache_finished_req:

```
original cache_finished_req(req)
→ radix tree owns finished token indices
→ excess = evictable_size - MAX_CACHED_TOKENS
→ if excess > 0: call native RadixCache.evict(excess)
→ native allocator.free(indices)
→ elastic allocator → KVCacheManager.free
```

7. vLLM 源码接入

7.1 原生对象

关键上游文件:

- vllm/v1/engine/core.py
- vllm/v1/core/kv_cache_manager.py
- vllm/v1/core/kv_cache_coordinator.py
- vllm/v1/core/single_type_kv_cache_manager.py
- vllm/v1/core/block_pool.py
- vllm/v1/worker/gpu_worker.py
- vllm/v1/worker/gpu_model_runner.py

原生调用关系:

```
EngineCore.step
→ Scheduler.schedule
→ KVCacheManager.allocate_slots
→ KVCacheCoordinator / specialized managers
→ BlockPool.get_new_blocks / touch / free_blocks
→ SchedulerOutput
→ model_executor.execute_model
```

- GPUModelRunner.execute_model
- attention backend using block table + KV tensors

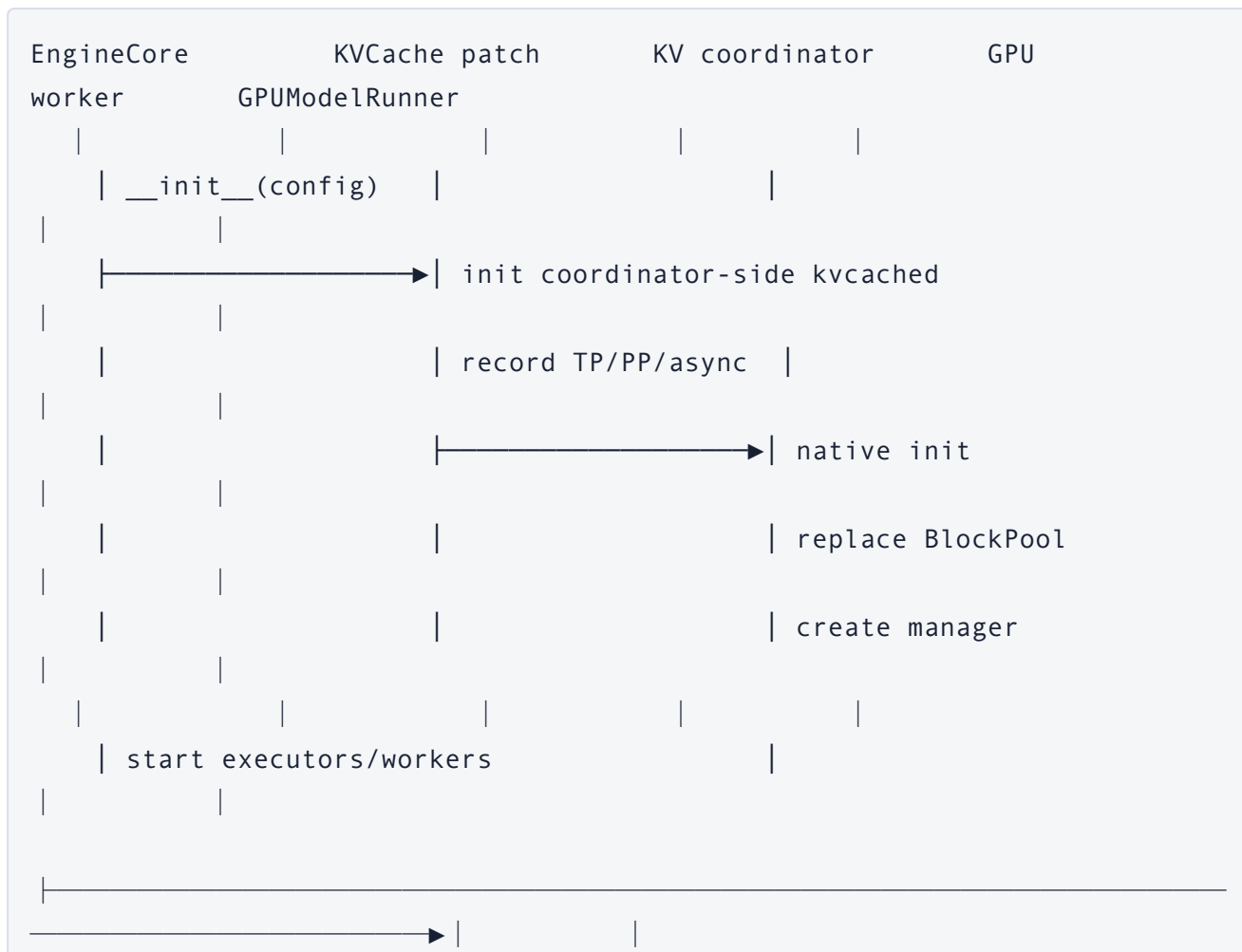
7.2 patch 覆盖面

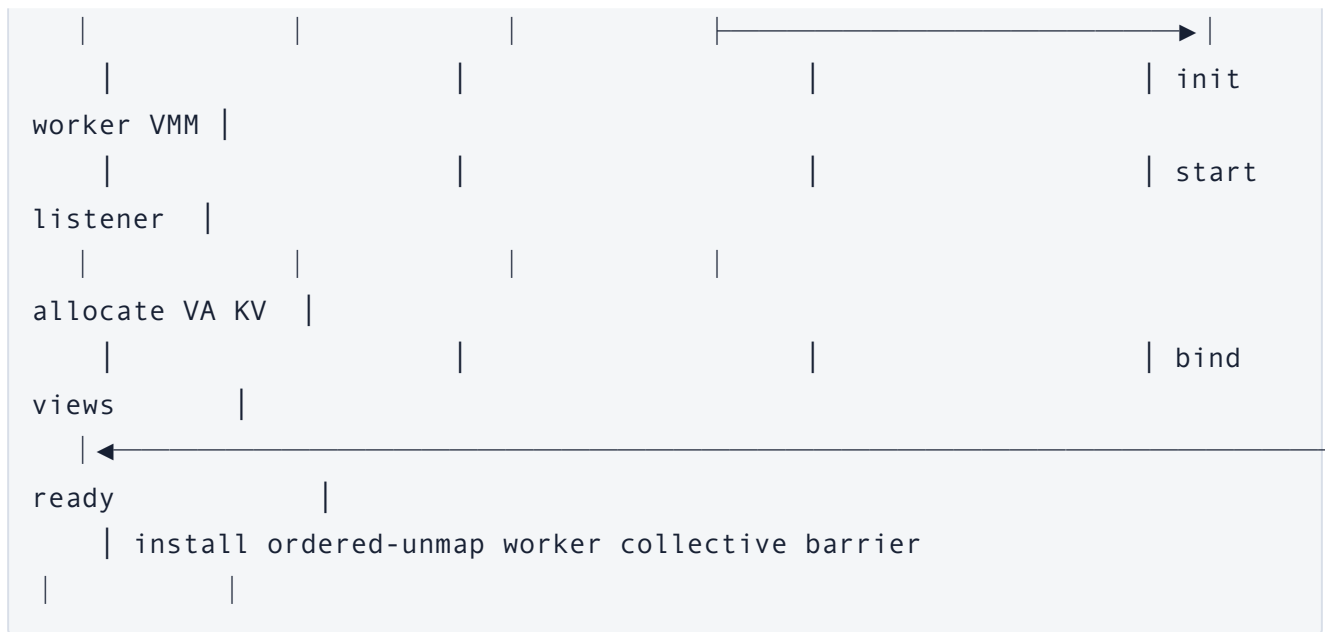
integration/vllm/autopatch.py 注册的关键patch:

- ElasticBlockPoolPatch
- EngineCorePatch、MPCClientPatch、CoreEngineProcManagerPatch
- GPUModelRunnerPatch, 以及0.28/0.29 MRV2专用patch
- GPUWorkerPatch
- HybridBlockSizeAlignPatch
- KVCacheCoordinatorPatch 或旧版 KVCacheManagerPatch
- KVCacheManagerAllocateSlotsPatch
- Mamba tail、Triton scale、NIXL compatibility patches

vLLM patch更多, 不是因为VMM机制不同, 而是 scheduler-side block ownership 与 worker-side CUDA mapping 分离。

7.3 vLLM 初始化时序





7.4 ElasticBlockPool

类在 `integration/vllm/patches.py:620-1121` 动态定义。它不能只实现 `alloc/free`，因为 `vLLM` 上层依赖 `BlockPool` 的隐式 `contract`：

Contract	实现
block object identity	预建 <code>KVCacheBlockClass(i)</code> 数组，按 <code>manager</code> 返回 <code>id</code> 索引
null block	block 0 永久保留并标 <code>is_null</code>
prefix lookup	<code>(block_hash, group_id) → blocks</code> ，支持 <code>duplicate materialization</code>
active ownership	<code>ref_cnt</code> ， <code>touch</code> 时从 <code>evictable LRU</code> 移除
cached free	full cached block <code>ref=0</code> 后进入 <code>evictable LRU</code> ，不立即释放
uncached free	partial/无hash block 直接 <code>manager.free</code>
cache cap	cached blocks 超限时 <code>evict</code> 并 <code>manager.free</code>
scheduler capacity	<code>physical manager free + evictable cached blocks</code>

7.5 vLLM allocation 时序





race: another process consumes last physical page
 KVCached alloc returns None
 → ElasticBlockPool raises KVCachedPoolExhausted
 → patched allocate_slots rolls back partial group allocation
 → returns None
 → Scheduler treats as temporary miss/preempts/retries

只捕获KVCachedPoolExhausted。shape/layout/invariant ValueError仍然fatal，避免把真实bug伪装成负载压力。

7.6 GPUModelRunner tensor allocation

vLLM 0.9+ patch 给GPUModelRunner增加 `_allocate_kv_cache_from_kvcached`:

1. 验证各 KV group 的 physical block geometry。
2. 排除 cross-layer sharing 的borrower layer，只给tensor owner分配storage。

3. 找代表性的attention group/spec/backend。
4. 取得backend KV shape和kernel block size。
5. 对packed KV、MLA、HYBRID_LINEAR、heterogeneous attention做专门geometry检查。
6. 调 `vllm.interfaces.alloc_kv_cache` 创建raw FTensor。
7. 根据group/layer构建per-layer view/as_strided/permute。
8. shared layer alias到owner tensor。
9. 交还原生 `bind_kv_cache` 注册到attention modules。

关键点：物理bytes布局必须和backend计算地址的方式一致。tensor shape“看起来正确”不够，stride、K/V offset、kernel-block ratio、inline scale/padding都必须匹配。

7.7 Worker capacity profile

vLLM原生 `Worker.determine_available_memory` 用初始化前后whole-device free-memory delta；共置进程在profile期间分配/释放会破坏这个假设。patch改用：

```
virtual_budget
- model_runner.model_memory_usage
- process-local torch activation peak
= available KV virtual bytes
```

显式 `kv_cache_memory_bytes` 和upstream startup plan仍优先，避免patch覆盖用户明确预算。CUDA graph的whole-device delta不计入可用KV预算，但相关字段仍填充给upstream warmup contract。

7.8 APC 与physical page eviction

ElasticBlockPool区分两种eviction目标：

- allocation shortage：free的逻辑slot马上复用，按纯LRU即可；page-aware排序不会多释放物理页，反而可能牺牲更新prefix。
- 主动cache cap/trim：目标是归还VRAM，优先选择所有occupancy都在evictable集合中的page；整页清空才有physical收益。

7.9 async/queued batch 释放

```
EngineCore step N submits worker batch reading page P
|
Scheduler logically frees P during/after later bookkeeping
|
KVCacheManager marks P retired with epoch E
|
EngineCore.step_with_batch_queue returns after submitting ordered work
```

```

|
capture marker E
|
model_executor.collective_rpc(_worker_physical_release_barrier)
| every worker synchronizes prior work
▼
release_retired_pages_through(E)
|
transactional unmap P

```

没有这条barrier，scheduler CPU状态可能已经free block，但GPU队列中旧batch仍在异步读取相同page，提前unmap会产生illegal memory access或silent output corruption。

8. TP/PP IPC 协议

文件：kvcached/tp_ipc_util.py。

8.1 地址与所有权

socket root: /tmp/kvcached-tp-<ipc-name>-<hash>。

```

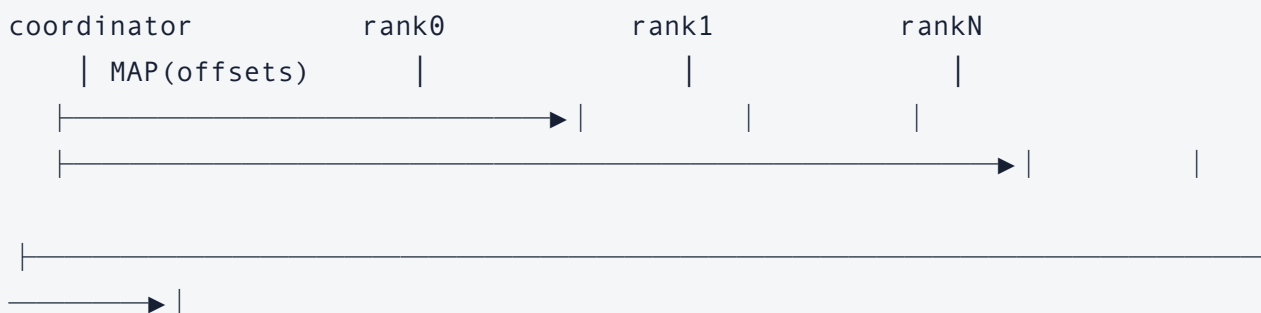
PP=1:
  root/w0.sock
  root/w1.sock

PP>1:
  root/w0.sock, root/w1.sock      # pp0
  root/pp1/w0.sock, root/pp1/w1.sock
  root/pp2/w0.sock, root/pp2/w1.sock

```

pp_rank=-1 表示coordinator operation要覆盖所有PP stage；KVCACHED_PP_SIZE决定stage集合。

8.2 map 协议



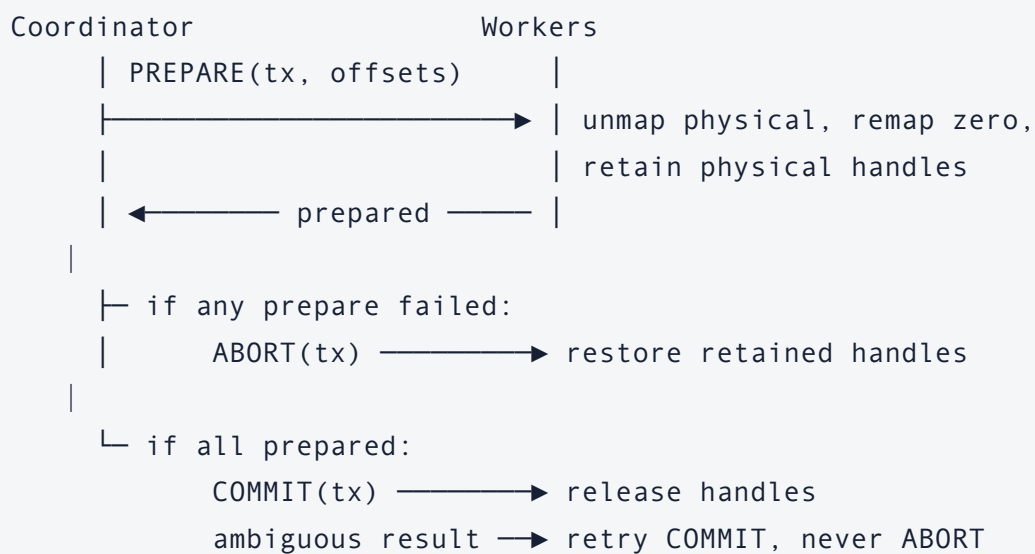


any failure:

coordinator sends rollback/unmap for offsets newly mapped by successful ranks

rollback uncertainty → StateConsistencyError / quarantine

8.3 unmap 两阶段协议



transaction outcome有bounded history，使重复commit/abort幂等。prepare未到达某worker而全局abort时，该worker也记录ABORTED，防止延迟prepare重新打开transaction。

8.4 listener shutdown

`_WorkerListener.stop`:

1. 设置stop event，拒绝新connection dispatch。
2. `shutdown()`已accept sockets，唤醒blocked recv。
3. 主动连接自己，唤醒blocked accept。
4. join listener thread，等待正在执行的VMM call结束。
5. 只有thread确认后退出后，才unlink socket和空目录。

若5秒内无法drain，返回False并保留资源供重试；不能一边handler还在VMM里，一边销毁allocator。

9. shared memory limit 与 resize

csrc/inc/mem_info_tracker.hpp 的shared record保存KV limit、used、preallocated等。kvctl修改limit/revision后:

```
kvctl limit
→ update /dev/shm record + revision
→ PageAllocator resize_watcher observes change
→ compute new per-layer target
→ KVCacheManager next allocation observes resize target
→ grow: extend logical page space
→ shrink:
    stop issuing blocks beyond target
    move/free empty pages
    wait until in-use tail blocks drain
    trim reserve mappings
```

resize不是强制抢占正在运行request; 收缩可能是渐进的。上层scheduler/cache eviction决定多快释放live/cached blocks, KVCacheD不能无视ownership直接unmap。

10. Prefix cache 的源码所有权

vLLM

```
Request.block_hashes
→ KVCacheManager/Coordinator finds cached full blocks
→ ElasticBlockPool._cached_blocks[(hash, group)]
→ touch increments ref_cnt
→ free_blocks decrements ref_cnt
    cached full block → _evictable_blocks LRU
    uncached partial → KVCacheManager.free immediately
```

SGLang

```
RadixKey(token_ids, extra_key, cache_salt)
→ RadixCache.match_prefix
→ TreeNode.value = token-slot tensor
→ lock_ref protects in-flight prefix
```

```
→ cache_finished_req inserts suffix
→ eviction frees TreeNode.value through allocator
```

KVCacheD

KVCacheD不理解hash或radix key，只看最终block IDs。它唯一能做的prefix-aware事情，是要求engine cache layer在bound/pressure下释放ID；page ledger随后判断是否能归还physical page。

11. Layout 计算

11.1 基本公式

```
engine block bytes = tokens_per_block × per_token_cell_bytes
```

MHA/GQA per-token cell

```
= num_kv_heads × head_dim × dtype_bytes
(K/V数量由num_kv_buffers或shape中的K/V维表示)
```

manager physical accounting

```
= mapped_pages × page_size × num_layers × num_kv_buffers
```

11.2 vLLM backend shape

adapter识别：

- FlashAttention: (2, num_blocks, block_size, heads, dim)。
- FlashInfer: (num_blocks, 2, block_size, heads, dim)。
- packed KV: (num_blocks, heads, block_size, 2*dim)。
- MLA: (num_blocks, block_size, latent_head_size)。

virtual block可能比kernel block大，例如256-token virtual block拆成4个64-token kernel blocks。adapter必须按kernel-block粒度建立stride，否则zero/copy/attention kernel会跳错地址。

11.3 contiguous offset

```
non-contiguous offset = page_id × PAGE_SIZE
```

contiguous compound offset

```
= page_id × PAGE_SIZE × num_layers × num_kv_buffers
```

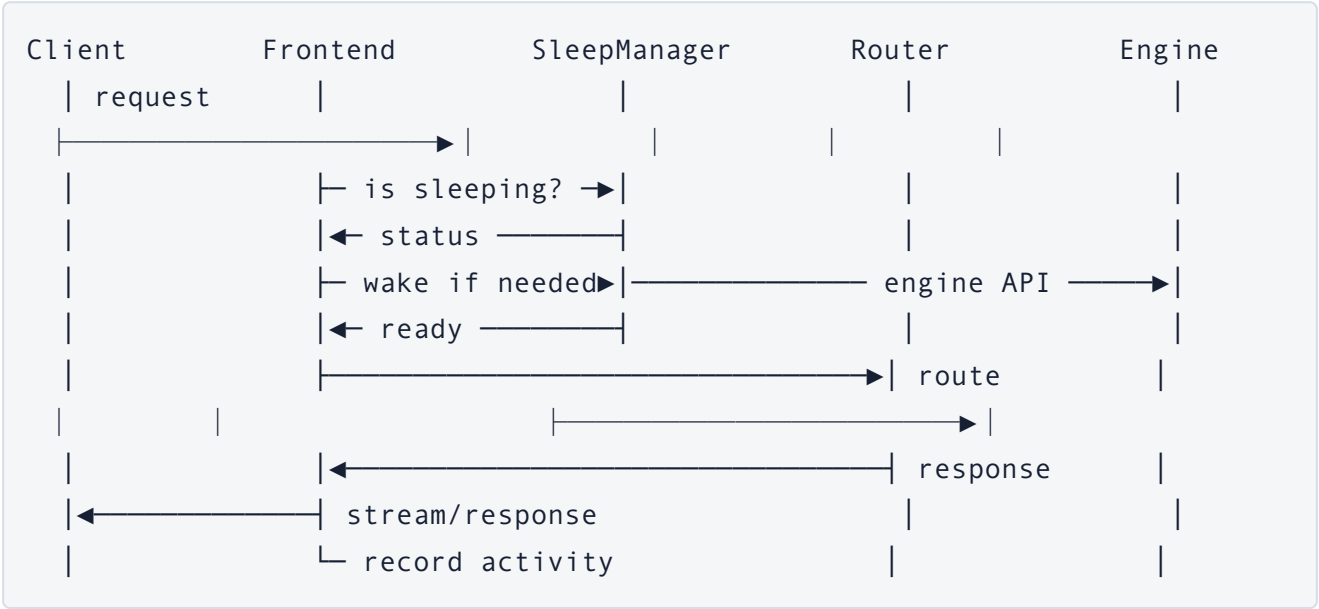
manager和FTensor创建时的num_layers/num_kv_buffers必须完全一致；只要compound stride不同，即使总bytes够，也会map到错误offset。vLLM interface因此保存created tensor capacity record，并让manager-first/tensor-first两种顺序都做交叉验证。

12. Controller 源码

controller不是VMM核心，但提供完整多模型应用示例：

文件	职责
controller/launch.py	解析YAML，构造vLLM/SGLang命令和env，以tmux启动实例/router
controller/frontend.py	aiohttp OpenAI-compatible入口和管理API
controller/router.py	model name → endpoint，转发stream/non-stream请求
controller/traffic_monitor.py	request start/end、rate、idle duration
controller/sleep_manager.py	idle检测，调用vLLM sleep/wake或SGLang release/resume

请求时序：



13. 错误如何传播

错误	层	处理
physical page暂时不足	manager/ElasticBlockPool	返回None或专用exhaustion，转scheduler retry

错误	层	处理
block/page geometry非法	adapter/manager init	fail early，提示调整page size/config
部分map失败且可回滚	FTensorAllocator	rollback新增映射，allocation失败
map rollback失败	FTensor/PageAllocator	quarantine page，避免发布未知mapping
unmap prepare失败	TP IPC	对已prepare worker发送abort恢复
unmap commit不明	TP IPC	幂等重试commit；仍不明则fail pool
queued GPU work未完成	vLLM EngineCore patch	worker barrier后才drain retired pages
worker listener未drain	shutdown	返回False，保留allocator供重试
engine版本/runner无adapter	EngineCore/autopatch	fail loud，不允许半接入

“资源压力”可恢复，“状态一致性未知”不可恢复，这是整套错误语义的主线。

14. 源码文件索引

KVCacheD 必读

顺序	文件	重点符号
1	kvcached/autopatch.py	autopatch_all
2	integration/ patch_base.py	BasePatch, PatchManager
3	integration/sglang/ autopatch.py	patch顺序和version ranges
4	integration/sglang/ interfaces.py	init_kvcached, alloc_kv_cache, alloc_mamba_states, get_kv_cache_manager
5	integration/sglang/ patches.py	Elastic allocator/pool、capacity、Radix bound
6	integration/vllm/ autopatch.py	V1/MRV2/worker/coordinator patch矩阵
7	integration/vllm/ interfaces.py	KV shape/view、created capacity guard
8		ElasticBlockPool、EngineCore、runner、worker、allocate_slots

顺序	文件	重点符号
	integration/vllm/ patches.py	
9	kv_cache_manager.py	_alloc, _pick_avail_page, free, resize, available_size, shutdown
10	tp_ipc_util.py	listener、broadcast map、2PC unmap
11	csrc/ page_allocator.cpp	page state、prealloc、map/unmap dispatch
12	csrc/allocator.cpp	FTensorAllocator create/map/transaction
13	csrc/ftensor.cpp	VA reservation与single tensor mapping
14	csrc/inc/gpu_vmm.hpp	CUDA/HIP API abstraction

SGLang 对照

文件	为什么读
srt/managers/scheduler.py	request/batch主循环和memory pool初始化
srt/mem_cache/allocator/base.py	allocator contract/free-group/eviction接口
srt/mem_cache/allocator/token.py	原生token free tensor
srt/mem_cache/allocator/paged.py	extend/decode page allocation
srt/mem_cache/memory_pool.py	MHA/MLA/Mamba/hybrid真实tensor shape
srt/mem_cache/radix_cache.py	prefix key/node/match/insert/evict
srt/mem_cache/kv_cache_configurator.py	virtual capacity如何转成pool size

vLLM 对照

文件	为什么读
v1/engine/core.py	EngineCore初始化、step、sleep/shutdown、batch queue
v1/core/block_pool.py	native block/APC/null/ref/free queue contract
v1/core/kv_cache_manager.py	allocate_slots返回None的scheduler语义
v1/core/kv_cache_coordinator.py	multi-group/hybrid block ownership

文件	为什么读
v1/worker/gpu_worker.py	capacity profile、sleep、worker lifecycle
v1/worker/gpu_model_runner.py	KV tensor allocation/bind/backend initialization
v1/kv_cache_interface.py	KV spec/config/tensor/group geometry

15. 调试与验证路径

15.1 确认 patch 真正生效

1. 检查 .pth 在解释器执行的 site-packages。
2. 两个env都设置：ENABLE_KVCACHED=true、KVCACHED_AUTOPATCH=1。
3. 日志出现PatchManager检测版本和成功patch列表。
4. 运行时class/module alias指向Elastic实现。
5. kvctl list能看到pool，idle VRAM接近权重+reserve而不是完整KV reservation。

15.2 alloc 热路径观察点

```
engine allocator get_new/alloc
→ KVCacheManager.available_size
→ KVCacheManager._alloc
→ PageAllocator.alloc_page
→ PageAllocator.map_pages
→ tp_ipc broadcast or FTensorAllocator.map
```

需要记录：requested blocks、logical free、physical free pages、selected page IDs、offsets、rank responses和耗时。

15.3 free 不降显存

按顺序排查：

1. block是否仍由running request引用。
2. 是否进入vLLM APC/SGLang RadixCache。
3. page occupancy是否仍非0。
4. empty page是否进入mapped reserve pool。
5. 是否在retired_pages等待async fence。
6. transactional unmap是否成功commit。
7. driver显示是否还有其它非KV allocation。

15.4 最重要测试簇

- `test_alloc_rollback.py`、`test_vmm_failure_policy.py`: 失败与quarantine。
- `test_tp_ipc_atomic_map.py`、`test_tp_ipc_unmap_sync.py`、`test_tp_ipc_pp_fanout.py`: rank一致性。
- `test_vllm_pool_exhaustion.py`、`test_vllm_async_unmap_ordering.py`: scheduler miss与lifetime。
- `test_vllm_native_block_pool*.py`、`test_prefix_cache.py`: block/APC契约。
- `test_sglang_allocator_patch.py`、`test_sglang_mem_pool_seam.py`: SGLang seam。
- `test_*mamba*`、`test_*hybrid*`、`test_*packed*`: 复杂layout。
- `test_shared_segment_lifecycle*`、`test_vllm_engine_core_shutdown.py`: 资源清理。

16. 从源码得出的核心结论

1. KVCacheD 的本质不是“另一个KV allocator”，而是 engine logical ownership 与 GPU physical backing 之间的 translation layer。
2. FTensor让attention kernel透明；KVCacheManager让engine allocator透明；version patches让上游源码透明。这三层透明性分别对应VMM、ledger和compatibility成本。
3. SGLang适配复杂度主要来自pool种类和token/radix语义；vLLM适配复杂度主要来自BlockPool隐式contract、hybrid groups和EngineCore/worker进程分离。
4. correctness难点不在成功路径，而在partial map rollback、allocation race、queued GPU lifetime和distributed unmap。
5. prefix cache是上层metadata policy，physical elasticity是下层page policy；只有两层共同释放ownership，VRAM才真正归还。
6. 升级一个engine版本时，必须重新验证class alias捕获时机、method signature、KV shape/stride、runner选择和shutdown路径，不能只看import是否成功。

相关入口

- 三项目关系地图: [kvcached-sglang-vllm-knowledge-system](#)
- 完整架构来源: [src-kvcached-architecture](#)
- 核心概念: [elastic-kv-cache](#)
- 集成与实验: [kvcached-integration-implementation-guide](#)
- 项目实体: [kvcached](#)、[sglang](#)、[vllm](#)
- 引擎缓存算法: [radix-attention](#)、[paged-attention](#)

KVCacheD × SGLang × vLLM 集成与实验手册

本页把 [kvcached-source-code-deep-dive](#) 的实现知识转成可执行的学习和验证路径。目标不是给出一条“复制即可生产上线”的命令，而是让每一步都有可观察的系统假设、预期状态和失败解释。

1. 使用前的边界判断

先确认问题确实属于 [elastic-kv-cache](#)：

问题	若答案为“是”	若答案为“否”
同卡是否有两个以上模型/实例？	继续评估	单实例持续满载时收益可能有限
模型权重能否同时驻留？	KV峰值可弹性共享	权重都放不下时，KVCacheD本身解决不了
KV负载是否错峰或有空闲期？	有物理页回收机会	同时满峰仍受真实VRAM上限约束
是否愿意锁定engine版本？	可维护adapter矩阵	rolling latest会带来高兼容风险
是否可以做GPU correctness测试？	可验证layout/lifetime	只看启动成功不够安全

不要把下列目标误配给KVCacheD：

- 跨节点共享相同prefix内容：研究[kv-cache-offload](#)、KV connector和distributed serving。
- Kubernetes GPU allocation或强隔离：研究[gpu-sharing](#)、MIG、MPS、DRA/device plugin。
- 降低模型权重显存：使用量化、offload、sleep或模型并行。
- 提升单次attention kernel FLOPs：选择/优化FlashAttention、FlashInfer、Triton等backend。

2. 建立可复现版本清单

每次实验先记录：

```
hardware
GPU model / count / VRAM
CUDA or ROCm driver/runtime

software
PyTorch version and torch.version.cuda/hip
KVCacheD package version + git commit
vLLM or SGLang package version + git commit
Python version
```

```
engine configuration
  model + revision
  dtype / quantization / KV dtype
  attention backend
  TP / PP / DP / EP
  block/page size
  prefix cache flags
  async scheduling / sleep mode / P-D connector
```

```
KVCacheD configuration
  IPC name
  VMM page size
  contiguous layout
  preallocation settings
  prefix cache bound
  memory limit
```

只写“vLLM latest + kvcached”无法复现实验。KV tensor layout 和runner选择可能在minor release间改变。

3. 环境准备

3.1 基础前提

当前KVCacheD build逻辑要求：

- Linux GPU环境；VMM核心不是macOS本地可执行路径。
- Python 3.9 – 3.13。
- PyTorch >=2.10，用于stable ABI build。
- CUDA或ROCm build能够提供对应VMM API。
- 目标vLLM/SGLang版本在adapter范围内。

3.2 安装顺序

```
# 1. 进入目标engine虚拟环境
source /path/to/venv/bin/activate

# 2. 先安装匹配GPU backend的PyTorch和engine
python -c 'import torch; print(torch.__version__, torch.version.cuda,
torch.version.hip)'
```

```
# 3. 安装KVCached; build isolation会看不到已安装的torch, 因此关闭
pip install kvcached --no-build-isolation

# 或源码开发安装
cd /path/to/kvcached
pip install -e . --no-build-isolation --no-cache-dir
```

如果构建报错, 先分辨是哪一层:

错误位置	常见原因
import torch失败	安装顺序错误或build isolation
stable ABI/version guard	PyTorch低于目标版本
libcuda/CUDA headers	driver/toolkit/build环境不完整
HIP symbols	ROCm/PyTorch backend不一致
wheel安装后无.pth	自定义packaging/install path没有把root data安装到site-packages

3.3 验证自动注入文件

```
python - <<'PY'
import site
from pathlib import Path

for directory in [*site.getsitepackages(), site.getusersitepackages()]:
    path = Path(directory) / "kvcached_autopatch.pth"
    if path.exists():
        print(path)
PY
```

源码只在PYTHONPATH可见, 不代表.pth会执行。 .pth必须位于解释器启动时处理的site-packages目录。

4. 环境变量分层

4.1 生效开关

```
export ENABLE_KVCACHED=true
export KVCACHED_AUTOPATCH=1
```

- KVCACHED_AUTOPATCH: engine import时是否改class/function/alias。
- ENABLE_KVCACHED: patched wrapper运行时是否走elastic分支。

两者缺一都可能表现为“服务能启动，但实际仍走原生内存”。

4.2 资源域

```
export KVCACHED_IPC_NAME=my-serving-pool
```

IPC name影响:

- /dev/shm memory information/limit segment。
- TP/PP socket root的namespace。
- kvctl显示和控制目标。

同一物理共享域应使用明确、稳定且长度适中的名称。独立租户或实验需要不同名称，避免limit和socket相互影响。

4.3 VMM page与layout

```
export KVCACHED_PAGE_SIZE_MB=2
export KVCACHED_CONTIGUOUS_LAYOUT=true
```

规则:

- page size必须是2 MiB的正整数倍。
- block/super-cell大于page时必须增大page。
- NVIDIA/CUDA默认contiguous；ROCm默认non-contiguous。
- P-D/NIXL、per-layer pointer注册或特定backend可能要求non-contiguous。
- layout改变tensor stride，是correctness配置，不只是性能配置。

4.4 reserve与cache策略

```
export KVCACHED_PAGE_PREALLOC_ENABLED=true
export KVCACHED_MIN_RESERVED_PAGES=5
export KVCACHED_MAX_RESERVED_PAGES=10
export KVCACHED_MAX_CACHED_TOKENS=16000
```

实验阶段建议：

1. 先关闭prefix cache或将cached tokens设为0。
2. 验证基本map/unmap和输出正确。
3. 再逐步增加cache bound，观察命中率和mapped pages。
4. 分别开/关preallocation，量化首page latency和idle footprint。

5. 实验 1：确认没有“假启用”

5.1 vLLM 最小启动

```
export ENABLE_KVCACHED=true
export KVCACHED_AUTOPATCH=1
export KVCACHED_IPC_NAME=vllm-lab
export VLLM_USE_V1=1
export VLLM_ATTENTION_BACKEND=FLASH_ATTN

vllm serve "$MODEL" \
  --no-enable-prefix-caching \
  --port 12346
```

5.2 SGLang 最小启动

```
export ENABLE_KVCACHED=true
export KVCACHED_AUTOPATCH=1
export KVCACHED_IPC_NAME=sglang-lab

python -m sglang.launch_server \
  --model "$MODEL" \
  --disable-radix-cache \
  --trust-remote-code \
  --port 30000
```

5.3 成功证据

必须同时看到：

- PatchManager检测到正确engine版本。
- 成功patch列表包含对应allocator/pool/runner/worker组件。
- C++ PageAllocator打印合理的layers、page size、block capacity和layout。
- kvctl list出现该IPC segment。
- idle状态下KV physical bytes接近reserve页，而不是完整virtual KV容量。
- 请求输出与关闭KVCacheD的固定seed基线一致。

只看到服务端口ready不算验证成功。

6. 实验 2：单实例按需增长和回收

6.1 流量设计

按顺序执行：

1. 启动后不发请求，记录idle状态。
2. 发一个短prompt/短output请求。
3. 发长prompt或高max_tokens请求，使更多KV pages映射。
4. 请求完成后等待cache/free路径。
5. 重复相同请求，观察prefix cache关闭和开启时的差异。

6.2 观察命令

```
kvctl list
kvctl watch -n 1
kvtop
nvidia-smi --query-compute-apps=pid,used_memory --format=csv
```

6.3 应建立的因果关系

```
longer active context / more concurrent tokens
→ more engine blocks/tokens
→ more KVCacheManager pages become occupied
→ more VMM physical mappings
→ mapped/used KV bytes increase

request completion
→ logical IDs released or cached
```

- only fully empty pages become reclaimable
- reserve/cache policy may keep part of memory mapped

如果请求结束后显存不降，不能直接判断leak。先查prefix ownership、page occupancy、reserve pages和async retirement。

7. 实验 3：两个同引擎实例共置

7.1 两个 vLLM

分别使用不同端口。是否使用相同IPC name取决于要测试的控制域：同名用于统一观察/limit语义，独立命名用于隔离实验；底层GPU physical headroom无论如何都由同一device竞争。

```
phase A: instance 1 idle, instance 2 idle
phase B: only instance 1 peak
phase C: instance 1 drains, instance 2 peak
phase D: both peak
phase E: one process graceful shutdown
```

验证：

- A阶段不应各自长期占满完整KV reservation。
- B/C阶段physical capacity应随活跃实例迁移。
- D阶段总需求超过VRAM时应出现scheduler miss/preemption/等待，而非silent corruption。
- E阶段对应segment/socket被清理，其它实例继续工作。

7.2 两个 SGLang

重点观察每个worker本地pool ownership。TP>1时，不应把一个rank的local manager operation误广播给其它rank；日志中的pool和device必须与rank对应。

8. 实验 4：vLLM 与 SGLang 跨引擎共置

这是最能验证项目关系的实验：两者没有共享block table或radix tree，但会竞争同一GPU VRAM。

```
vLLM instance
  BlockPool/APC metadata → its KVCacheManager ↘
                                          ↘ same GPU physical
headroom
SGLang instance                          |
  RadixCache/token metadata → its KVCacheManager ↙
```

测试阶段：

1. 两者prefix cache均关闭，建立纯allocation基线。
2. 交替流量，验证物理页能在两者之间复用。
3. 同时压力，观察两个scheduler如何分别处理allocation miss。
4. 只开vLLM APC，测其cached pages是否压缩SGLang headroom。
5. 只开SGLang RadixCache，做相反实验。
6. 两者均开cache并设置不同bound，观察公平性与尾延迟。

KVCacheD没有全局request priority scheduler。两个engine的公平性来自physical竞争、各自cache bound和外部limit配置，不应假设自动实现SLO公平。

9. 实验 5： prefix cache边界

9.1 三组设置

```
KVCACHED_MAX_CACHED_TOKENS=-1
    unlimited; highest reuse potential, weakest elasticity

KVCACHED_MAX_CACHED_TOKENS=0
    immediate eviction after cache ownership becomes evictable

KVCACHED_MAX_CACHED_TOKENS=N
    bounded reuse; tune N against page occupancy and workload prefixes
```

9.2 流量

使用三类请求：

- 完全相同长system prompt。
- 共享大部分prefix、尾部不同。
- 完全不同prompt。

记录：prefix hit、prefill tokens、TTFT、mapped pages、eviction blocks/tokens、page occupancy。

9.3 解释差异

- vLLM APC通常在full logical block边界缓存；partial tail通常不能同样复用。
- SGLang RadixCache可按token或page-aligned边界split/match。
- KVCacheD physical page比两者logical unit大，一个cache entry可能只占page的一部分。
- page-aware eviction只在能清空整页时直接降低physical bytes。

10. 实验 6：动态memory limit

10.1 操作

```
kvctl list
kvctl limit "$IPC" 8G
kvctl limit-percent "$IPC" 25
```

命令格式以当前kvctl help为准。实验时逐步收紧，不要在未知pool上直接设极小值。

10.2 预期状态机

```
limit revision increases
→ C++ resize watcher notices target
→ manager enters shrink state
→ new allocations avoid retired logical tail
→ running/cached blocks gradually release
→ empty/reserved pages unmap
→ effective capacity converges to target
```

limit不是强杀running request，也不会绕过prefix/cache ownership直接释放live page。收敛速度取决于流量、cache eviction和page fragmentation。

10.3 需要验证

- 收缩期间已有请求输出正确。
- 新请求在容量不足时进入engine正常backpressure/preemption路径。
- limit放宽后logical capacity可以恢复。
- resize中不存在超出FTensor reserved range的page id。

11. 实验 7：TP/PP

11.1 TP

启动TP=2或更高时，记录：

- rank → GPU device映射。
- worker socket path。
- 每次map的offset集合和rank response。
- 任一rank失败时其它rank的rollback。

vLLM的关键不变量：同一logical page在相关worker上必须全部mapped或全部unmapped。SGLang当前是每个worker local pool，各rank操作各自tensor。

11.2 PP

PP stage使用独立socket namespace。检查：

```
pp0/w0.sock ...  
pp1/w0.sock ...  
...
```

coordinator `pp_rank=-1` 的map/unmap必须覆盖所有stage。测试一个stage listener缺失时，应得到明确失败而不是只更新部分stage。

11.3 async/queued execution

对于vLLM PP或async scheduler：

1. 制造长GPU batch。
2. CPU scheduler尽早逻辑free部分blocks。
3. 验证page进入retired而非立即unmap。
4. worker collective barrier完成后才physical release。

12. 实验 8：故障注入

12.1 map失败

可以通过测试stub或受控fault injection让一个target mapping失败。预期：

- 已成功target反向rollback。
- 已成功rank收到rollback。
- 若rollback完整，allocation失败但pool仍可继续。
- rollback不确定时page quarantine或pool fail-loud。

12.2 unmap prepare失败

预期：

- 已prepare worker保留physical handle。
- coordinator发送abort。
- worker恢复原mapping。
- 没有rank提前release handle。

12.3 commit响应丢失

预期：

- coordinator重试commit。
- worker根据finalized transaction幂等返回committed。
- 不允许从ambiguous commit回退abort。

12.4 进程退出

分别测试：

- graceful engine shutdown。
- SIGTERM。
- SIGKILL。
- worker在VMM handler中退出。

观察 /dev/shm、socket目录、GPU memory和其它实例健康。SIGKILL无法执行Python/C++ cleanup，需要外部生命周期清理策略。

13. 输出正确性验证

显存曲线正常不代表KV地址正确。至少做：

13.1 固定输出对照

同一model revision、prompt、seed、sampling参数，比较：

- KVCacheD关闭。
- KVCacheD开启、prefix关闭。
- KVCacheD开启、prefix开启。
- 单实例与共置压力。

允许的数值差异要符合backend/dtype已知行为，不能接受随机乱码、重复token、长上下文后漂移。

13.2 场景覆盖

- short/long prefill。
- long decode。
- request abort/preemption。
- prefix hit/miss。
- sliding window/hybrid attention。
- MLA或Mamba模型（若目标使用）。
- TP/PP。
- sleep/wake。

- P-D/NIXL（若目标使用）。

13.3 检测silent layout corruption

高风险信号：

- 只有某attention backend错误。
- 短上下文正常，跨block/page边界后错误。
- 只有odd page id、某layer或某KV group错误。
- prefix miss正常，prefix hit错误。
- Mamba state跨request复用后错误。
- async模式偶发，sync模式正常。

这些分别指向stride/compound offset、group geometry、cache ownership和unmap lifetime。

14. 性能实验设计

14.1 不要只测平均吞吐

建议指标：

类别	指标
服务体验	TTFT p50/p95/p99、ITL、E2E latency、throughput
KV逻辑层	allocated blocks/tokens、prefix hit、evictions、preemptions
VMM层	mapped/reserved/free/quarantined pages、map/unmap latency
GPU层	process VRAM、device free memory、SM utilization
稳定性	allocation misses、IPC timeouts、rollback、failed transactions

14.2 对照组

A: engine native static memory split
 B: KVCacheD, prealloc off, prefix off
 C: KVCacheD, prealloc on, prefix off
 D: KVCacheD, bounded prefix cache
 E: KVCacheD, unlimited prefix cache

14.3 工作负载

- steady single model：证明KVCacheD自身开销。

- alternating peaks: 证明跨模型弹性。
- simultaneous peaks: 暴露真实physical上限和backpressure。
- prefix-heavy: 测cache bound trade-off。
- highly fragmented request lengths: 测page occupancy。
- burst after idle: 测preallocation对cold map latency影响。

README的 $2 - 28 \times$ TTFT是特定A100-80G三模型间歇峰值实验。复现时必须保留其workload结构，不能当作任意模型的固定加速倍数。

15. 版本升级流程

升级vLLM/SGLang时按以下顺序：

15.1 静态差异

检查上游是否改变：

- patch target module/class路径。
- constructor/method signature。
- allocator/pool alias被捕获的import时机。
- KVCacheConfig/spec/group结构。
- block size与kernel block size。
- tensor shape/stride/layout。
- runner默认选择。
- prefix cache/ref count/null block contract。
- worker/executor/shutdown lifecycle。

15.2 patch coverage

确认每个注册patch：

- version range包含目标版本。
- can_apply找到目标class。
- patch marker正确。
- 成功日志不是“部分成功后静默fallback”。
- 不兼容runner/layout能fail early。

15.3 回归层级

```
CPU unit tests
→ single-GPU allocation/layout tests
→ engine smoke inference
→ prefix/hybrid/model-specific tests
```

- TP
- PP/async
- dual-instance contention
- shutdown/fault injection

不要从“server能启动”直接跳到生产共置。

16. 常见故障决策树

服务启动失败

- └─ 没有patch日志
 - | └─ .pth不存在/未执行
 - | └─ AUTOPATCH开关未设
- └─ patch target missing
 - | └─ engine版本超出adapter或上游模块移动
- └─ geometry/layout error
 - | └─ block > VMM page
 - | └─ num_layers/buffers与tensor不一致
 - | └─ backend layout/packed/inline-scale不支持
- └─ worker socket错误
 - | └─ IPC name不一致
 - | └─ rank/device错误
 - └─ PP namespace/listener未ready

运行时错误

- └─ allocation miss
 - | └─ 正常物理竞争 → scheduler retry
 - | └─ prefix cache/reserve占用过高
- └─ 显存不回收
 - | └─ live/cached block
 - | └─ partial page occupancy
 - | └─ reserve page
 - | └─ async retired page
- └─ illegal memory access
 - | └─ stride/layout错误
 - | └─ premature unmap
- └─ 输出错误
 - | └─ K/V offset或kernel-block geometry
 - | └─ hybrid group/shared layer绑定
 - └─ Mamba state/partial-tail处理

17. 扩展一个新engine版本的方法

1. 找到logical allocator seam：谁决定block/token ID ownership。
2. 找到KV tensor seam：谁创建实际GPU storage，谁保存view。
3. 写最小adapter，先只支持MHA、单GPU、prefix关闭。
4. 明确null/padding block契约。
5. 建立engine block bytes到VMM page geometry。
6. 保持上游scheduler的allocation miss语义。
7. 再增加prefix cache，复制其hash/ref/eviction contract。
8. 再增加TP/PP，定义coordinator和CUDA-context owner。
9. 再增加async lifetime fence。
10. 最后扩展MLA/hybrid/Mamba/packed/quantized/P-D。

每一步都应有fail-loud guard。不要让不支持的layout悄悄落回一半原生、一半KVCacheD的状态。

18. 学习完成标准

能够独立完成以下任务，才算真正理解三项目关系：

- 从一次vLLM `allocate_slots`追到`FTensor::map`，再追`free/unmap`返回。
- 从`SGLang RadixCache.match_prefix`解释哪些token需要新physical backing。
- 根据model KV shape手算block bytes、blocks/page和compound offset。
- 解释为什么logical free不能保证physical allocation成功。
- 解释为什么vLLM需要worker IPC而SGLang当前采用worker-local manager。
- 从日志区分资源压力、配置不兼容和state consistency failure。
- 设计一组能发现premature unmap和stride corruption的测试。
- 在升级engine后列出必须复核的patch targets和数据契约。

相关入口

- 三项目知识体系：[kvcached-sglang-vllm-knowledge-system](#)
- 源码逐函数深潜：[kvcached-source-code-deep-dive](#)
- 完整架构分析：[src-kvcached-architecture](#)
- 项目边界：[kvcached](#)、[vllm](#)、[sglang](#)
- 核心概念：[elastic-kv-cache](#)

KVCacheD、SGLang 与 vLLM 三项目知识体系

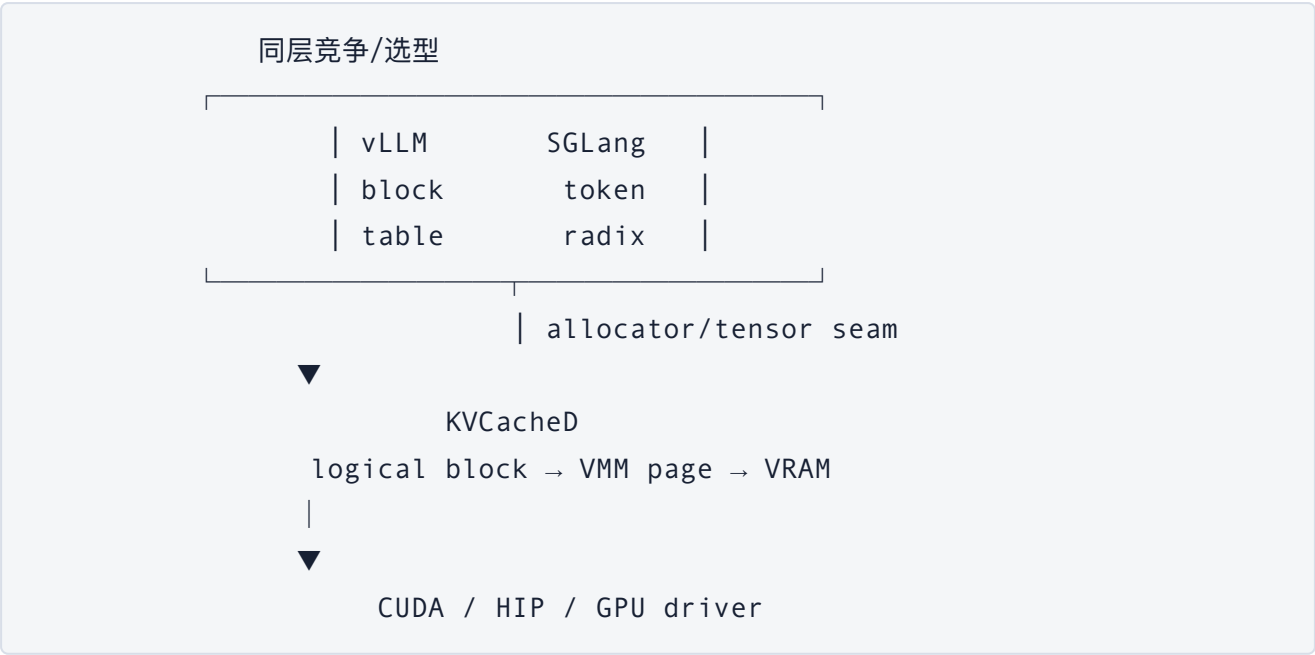
这页是三个项目的关系入口。目标不是把三仓文件逐个罗列，而是建立一套可重复使用的认知结构：先定位每个项目在哪一层，再把同一份请求从 scheduler、逻辑 KV、物理 page 一直追到 GPU VMM，最后理解两条集成路径为什么不同。

源码细节和行号证据集中在 [src-kvcached-architecture](#)；逐函数调用链、对象状态和时序图见 [kvcached-source-code-deep-dive](#)；可执行的安装、验证、压力、故障与升级路径见 [kvcached-integration-implementation-guide](#)；两套引擎自身的完整架构分别见 [src-sglang-architecture](#) 与 [src-vllm-architecture](#)。

0. 最先建立的结论

三个项目不是并列竞争关系：

- [vllm](#) 与 [sglang](#) 是同层的推理引擎，负责 API、调度、batch、模型执行、attention 和本地 KV 语义。
- [kvcached](#) 是下插到两套引擎内部的 KV physical-memory plugin，负责稳定虚拟地址和按需 VRAM backing。
- vLLM/SGLang 可以不用 KVCacheD 独立运行；KVCacheD 不能脱离引擎独立完成 LLM serving。
- KVCacheD 让两个实例共享的是 GPU physical capacity，不是同一份 KV 内容、同一棵 radix tree 或同一张 block table。



1. 知识树

```
LLM inference memory system
├─ Request execution
│   ├── prefill / decode / continuous batching
│   ├── scheduler / admission / preemption
│   └─ TP / PP / async execution lifetime
├─ Engine KV semantics
│   ├── vLLM: Request → KVCacheManager → Coordinator → BlockPool → block
table
│   └─ SGLang: Req → RadixCache → token allocator → KV pool → req_to_token
├─ Prefix reuse
│   ├── vLLM APC: fixed logical blocks + hashes + ref count/LRU
│   └─ SGLang RadixCache: token/page-aligned radix nodes + lock_ref/eviction
├─ KVCacheD adaptation
│   ├── .pth / import hook / version-aware patch
│   ├── ElasticBlockPool or ElasticToken/PagedAllocator
│   └─ VMM-backed tensor shape/stride adapters
├─ Elastic physical memory
│   ├── KVCacheManager block ledger
│   ├── PageAllocator free/reserved/in-use/quarantined pages
│   └─ FTensorAllocator VA reservation + physical map/unmap
└─ Distributed correctness and operations
    ├── TP/PP worker IPC + transactional unmap
    ├── async release barrier
    ├── /dev/shm limit + kvctl/kvtop
    └─ compatibility, failure recovery and shutdown cleanup
```

沿这棵树学习，可以避免两个常见误区：把 KVCacheD page 当成 PagedAttention block；把 elastic memory 当成跨节点 KV offload。

2. 三个项目各自解决什么

vLLM: block-oriented serving engine

核心对象链：

```
API / LLM
→ EngineCore
→ Scheduler
```

```
→ KVCacheManager
→ KVCacheCoordinator / single-type managers
→ BlockPool
→ SchedulerOutput block tables
→ GPUModelRunner
→ attention backend
```

vLLM 的 KV 语义围绕 fixed-size logical block: Request 预先计算 prefix block hashes; BlockPool 管 block object、free queue、ref count、cache hash 和 null block; KVCacheManager/Coordinator 为不同 KV group 分配 block; GPUModelRunner 将 block table 绑定到实际 per-layer KV tensor。

它擅长把 request scheduling、continuous batching、PagedAttention、多个 attention backend 和广泛模型生态组合成通用 serving baseline。KVCacheD 不改变这些核心对象，只把 BlockPool 的 block id 接到动态物理页，并替换 worker tensor allocation。

SGLang: token/radix-oriented serving runtime

核心对象链:

```
HTTP / Engine / DSL
→ TokenizerManager
→ Scheduler
→ ScheduleBatch
→ RadixCache.match_prefix
→ TokenToKVPoolAllocator / PagedTokenToKVPoolAllocator
→ MHA/MLA/Mamba/Hybrid KV pool
→ ModelRunner / attention backend
→ DetokenizerManager
```

SGLang 的 KV 语义围绕 token slot 和 RadixCache: radix node 的 value 指向 token-to-KV pool index, prefix match 可在 token 或 page-aligned 边界复用; allocator 为 extend/decode 分配位置; KV pool 保存每层真实 K/V 或 recurrent state。

它的差异化能力是 token-level prefix reuse、复杂生成程序、丰富 speculative/disaggregation/backend 组合。KVCacheD 保留 radix tree 与 scheduler, 只替换 allocator 和底层 buffer。

KVCacheD: physical backing and elasticity layer

核心对象链:

```
engine alloc/free
→ engine adapter
```

- KVCacheManager
- InternalPage / PageAllocator
- FTensorAllocator / FTensor
- CUDA/HIP VMM

它解决的是“逻辑 KV 容量很大，但平均真实活跃量较小”时的静态显存浪费。虚拟 tensor 可以覆盖 engine 计划的完整 KV 容量；physical page 只随活跃 block 映射。多个进程通过 GPU driver 的全设备 free memory 和 shared limit 协作竞争。

3. 同一个词在三个项目中的不同含义

词	vLLM	SGLang	KVCacheD
Request	Request 状态、token、hash、computed tokens	Req、prefix、output、radix node	不理解请求语义，只接收 block/token allocation
Block/ Page	KV logical block，进入 block table	token page，page_size 可为1或更大	VMM physical page，默认2 MiB
Pool	BlockPool + KV groups	token allocator + KV pool	device-wide physical headroom + per-instance page ledger
Cache	APC full-block cache	RadixCache prefix tree	不创建新 prefix 算法，只给两者设 memory bound
Capacity	num_gpu_blocks / KV config	max tokens/pages	virtual、logical、mapped、physical headroom四种容量
Free	block进入free queue，可能仍缓存	token index被allocator回收	page内全部block释放后才可能unmap
Layout	backend-specific KV tensor shape	MHA/MLA/Mamba pool shape	contiguous compound或per-layer FTensor映射

阅读源码时必须先判断当前代码谈的是哪一层的 block/page/free。

4. 对象映射：KVCacheD 替换了什么

生命周期位置	vLLM 原生对象	KVCacheD 接管方式	SGLang 原生对象	KVCacheD 接管方式
启动容量	Worker.determine_ava	改为进程本地 virtual budget，避免 whole-device delta被邻居干扰	KVCacheConfigurator._profile	用 total × mem_fraction -

生命周期位置	vLLM 原生对象	KVCacheD 接管方式	SGLang 原生对象	KVCacheD 接管方式
	ilable_memory		le_available_bytes	process reserved, rank间取min
逻辑 allocator	BlockPool	ElasticBlockPool 复制 block/APC contract, id来自 KVCacheManager	token/paged allocator	动态类替换, alloc/free委托 KVCacheManager
KV tensor	GPUModelRunner allocation/reshape	创建raw FTensor, 再按backend shape/stride构造view并bind	MHA/MLA/Mamba/hybrid pool	跳过原生真实buffer, 建立VMM-backed view
prefix meta data	block hash/cache/ref count	保留并加bounded LRU/page-aware eviction	RadixCache/tree node/lock_ref	保留并在完成插入后限制evictable tokens
分配失败	native free count通常权威, 异常视为bug	shared headroom race转成 allocate_slots=None	allocator返回 None/evict/retry	保持SGLang原有 allocation miss语义
多 rank map	worker拥有 tensor	coordinator通过socket fan-out	每worker本地pool	当前不广播本地pool操作
物理释放	native tensor 常驻	queued batch fence + transactional unmap	native tensor常驻	本地page空闲后unmap
shutdown	engine/worker teardown	stop listener、pool shutdown、unlink segment	scheduler/worker teardown	stop listener和allocator

5. 一次请求的完整生命周期

vLLM + KVCacheD

1. API 层生成 Request, 计算 token 和 prefix block hashes。
2. EngineCore.step() 调 Scheduler; scheduler 查询 computed/cached blocks。
3. KVCacheManager.allocate_slots() 进入 coordinator/BlockPool。
4. ElasticBlockPool 先复用 APC hit; 缺块时调用 KVCacheD manager。
5. manager 检查 logical slots 与 device physical pages, 选择已有 page 或申请新 page。

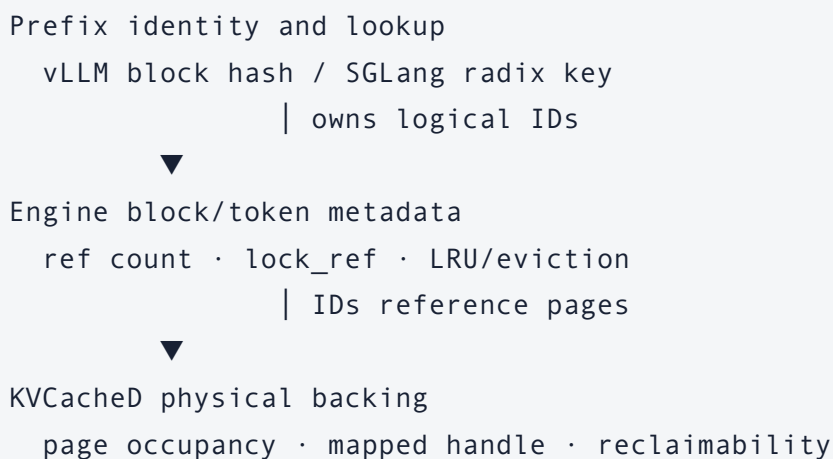
6. 新 page offset 经 Unix socket 发到相关 TP/PP worker；每个 worker 在本地 FTensor 相同 offset map 物理 handle。
7. block ids 写入 SchedulerOutput；GPUModelRunner 更新 block table。
8. 原生 attention backend 通过稳定 tensor pointer 和 block table 写/读 KV。
9. 请求结束时 partial/uncached block立即free；full cached block ref降到0后留在ElasticBlockPool LRU。
10. cache bound或物理压力触发evict；page所有block都free后进入reserve或unmap。
11. async/PP队列存在时，先做worker collective barrier，再执行transactional unmap。

SGLang + KVCacheD

1. TokenizerManager 创建 tokenized request，Scheduler 收到 Req。
2. RadixCache longest-prefix match 返回已缓存 token-slot indices。
3. extend/decode 对未命中 suffix 请求 token/page slots。
4. Elastic allocator 将 token需求换成 KVCacheManager block需求；paged path再把block id展开为token indices。
5. KVCacheManager 在本 worker 的 FTensor map physical page。
6. ScheduleBatch 仍以 EXTEND/DECODE/MIXED 方式执行；ModelRunner 和 backend不感知VMM。
7. 完成时 RadixCache 插入新 prefix并持有indices；未缓存indices返回allocator。
8. cached-token bound超限时，RadixCache原生evict释放indices。
9. page全部空闲后保留少量reserve或unmap。

6. prefix cache：三层所有权

prefix cache 不是一个单独组件，而是三层状态同时成立：



上层 cache hit 不会因为 KVCacheD 自动消失；下层也不能在 metadata 仍引用 block 时擅自 unmap。
bounded prefix cache 是两层策略的结合点。

7. 多进程所有权模型

vLLM ownership

```
EngineCore process
  owns scheduler-side logical block allocation
  owns KVCacheD manager / shared page decision
    | map/unmap commands
    ▼
GPU worker processes (TP × PP)
  own local CUDA context and FTensor mapping
  must apply same logical offset transition
```

因此 vLLM 需要 worker listeners、PP namespace、transaction id、async batch fence。manager 的 decision 与 worker 的 CUDA context 分离，是复杂性的根因。

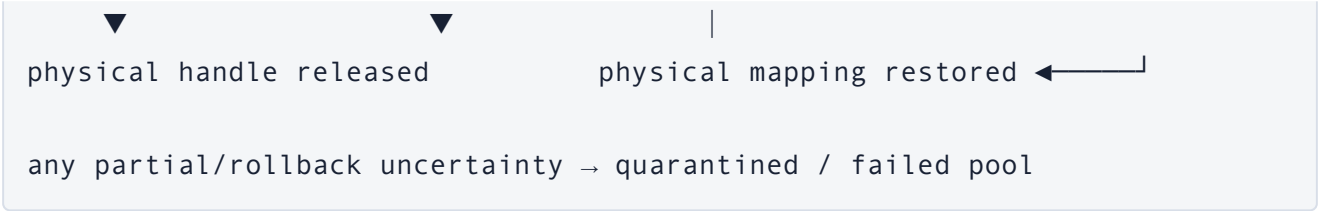
SGLang ownership

```
SGLang scheduler/worker rank
  owns local token allocator
  owns local KV pool tensor
  owns local KVCacheD manager and mapping
```

每个 TP worker 处理本 rank 的本地 pool，不能把本地 page operation 再广播给 peers，否则会重复/错误操作不同 ownership 的 pool。

8. GPU VMM 状态机

```
virtual range reserved
  | initialize
  ▼
zero-page mapped
  | alloc logical blocks needs backing
  ▼
physical page mapped — free but reserve budget → reserved mapped page
  | all blocks free and no reserve need | reuse
  ▼
unmap-retain (distributed prepare) |
  | commit | abort |
```



zero page 的作用是让尚未真实 backing 的地址仍保持有效映射，但它不是可写入实际 KV 的容量替代。真正使用前必须 map physical page。

9. Layout 与模型类型

MHA/GQA

每层通常有 K 与 V 两个 buffer。non-contiguous 可以在每层 FTensor 前后半区存 K/V；contiguous 把 token/page、layer、K/V interleave 在同一 reservation，再用 stride 暴露每层 view。

MLA

K/V 往往组合为单一 latent buffer，num_kv_buffers=1。FTensor 大小和 VMM page 需要额外对齐，不能套用普通 MHA 的 K/V split。

Hybrid attention / sliding window

多个 KV group 可能共享 BlockPool，但每组有不同 attention rule；只要物理 block bytes 统一，可以为不同 geometry 建立不同 view。hash 还必须带 group id，避免相同 prefix 在不同 group 碰撞。

Mamba / linear attention

状态不再只是 K/V，而是 conv 和 temporal/SSM state。KVCacheD 将一个 slot 的多类 state 打包成 super-cell，或在 vLLM hybrid unified pool 中给同一 physical bytes 建立 attention 与 mamba 两种解释。cell/page 和 partial-tail 处理是 correctness 关键。

10. 与其它项目层的关系

层	项目/概念	与三项目体系的关系
Engine	vllm , sglang	本页两套上层运行时
Elastic KV backing	kvcached , elastic-kv-cache	同卡 physical KV 弹性
KV offload/transfer	kv-cache-offload , Dynamo/NIXL/connector	跨 memory tier 或 P/D 节点传输 KV 内容
Routing	inference-routing , llm-d KV index	根据 endpoint 负载/KV locality 选择实例

层	项目/概念	与三项目体系的关系
GPU sharing	gpu-sharing 、MIG/MPS/HAMi	更外层的设备分配、执行/内存隔离
Serving control plane	llm-d/AIBrix/KServe等	部署、扩缩、endpoint选择和生命周期

KVCacheD 可以与这些层组合，但不能替代它们。尤其是“同卡物理页弹性”和“跨节点 KV transfer”经常都叫 KV cache management，实际数据路径和故障模型完全不同。

11. 采用时的决策框架

最佳匹配

- 多模型权重可同时驻留，KV峰值错开。
- 工作负载有明显idle/突发周期。
- 静态gpu_memory_utilization切分导致活跃模型容量不足、空闲模型浪费。
- 团队可以维护固定engine版本和GPU测试矩阵。

避免条件

- 单实例持续满载，没有可回收的KV空闲期。
- 目标是跨实例复用同一prefix KV，或远端KV持久化。
- 目标backend/layout/quantized format未被适配器明确支持。
- 必须依赖KV cache events，而当前ElasticBlockPool不支持。
- 多租户要求硬隔离或强资源quota。

采用成本

成本	内容
版本成本	上游内部API变化需要更新patch
测试成本	model × backend × layout × TP/PP × cache × engine version矩阵
运维成本	/dev/shm、worker socket、IPC timeout、shutdown cleanup
性能成本	首次map、IPC、transactional unmap、少量reserve footprint
正确性成本	stride/layout、async lifetime、partial allocation rollback

12. 学习路径

路线 A：先理解关系

1. 本页的 0 – 4 节。
2. [vllm](#) 和 [sglang](#) 实体页，建立引擎差异。
3. [elastic-kv-cache](#)，掌握四类容量和两级分页。
4. [src-kvcached-architecture](#) 的核心架构图、关键数据流和项目关系。

路线 B：追一条源码热路径

1. KVCacheD autopatch.py / patch_base.py。
2. 任选 vLLM ElasticBlockPool.get_new_blocks 或 SGLang ElasticTokenToKVPoolAllocator.alloc。
3. KVCacheManager._alloc。
4. PageAllocator.alloc_page/map_pages。
5. FTensorAllocator.map_to_kv_tensors_with_result 与 FTensor::map。
6. 反向追 free → empty page → unmap。

路线 C：专攻正确性

1. page geometry validation。
2. partial alloc rollback 与 MapQuarantinedError。
3. TP/PP prepare/commit/abort unmap。
4. vLLM queued/async release barrier。
5. manager/tensor capacity guard、null block、hybrid group hash。
6. shutdown drain 与 segment/socket cleanup tests。

路线 D：专攻集成

1. SGLang allocator/pool class alias 与版本范围。
2. vLLM EngineCore/coordinator/worker/runner 四个进程边界。
3. MHA → MLA → hybrid → Mamba，逐级增加geometry复杂度。
4. contiguous vs non-contiguous、CUDA vs ROCm。
5. APC/RadixCache bound、NIXL/P-D smoke path。

13. 快速自测问题

- vLLM block id、SGLang token index 和 KVCacheD page id 是否一一对应？为什么？
- logical free blocks 很多时，为什么 physical allocation 仍会失败？
- 为什么一个 cached block 可能阻止整张 2 MiB page 回收？
- FTensor unmap 后为什么 tensor 对象仍存在？

- vLLM allocation miss 为什么要转成 `None`，而状态不一致必须抛错？
- SGLang 为什么保留 RadixCache而只替换allocator/pool？
- vLLM为什么必须在EngineCore和GPU worker两侧都patch？
- transactional unmap 的prepare阶段保留了什么？
- contiguous layout的compound page如何改变offset计算？
- KVCacheD为什么不是KV offload，也不是通用GPU隔离方案？

如果这些问题都能从对象所有权、逻辑/物理分层和生命周期三个角度回答，三个项目之间的关系就已经形成稳定知识结构。

相关入口

- 主源码分析: [src-kvcached-architecture](#)
- 逐函数源码深潜: [kvcached-source-code-deep-dive](#)
- 集成与实验手册: [kvcached-integration-implementation-guide](#)
- KVCacheD: [kvcached](#)
- 引擎: [vllm](#)、[sglang](#)
- 核心概念: [elastic-kv-cache](#)
- 引擎缓存: [paged-attention](#)、[radix-attention](#)
- 相邻系统: [kv-cache-offload](#)、[gpu-sharing](#)、[llm-inference](#)

LLM Serving / 推理引擎选型地图

已有 [llm-inference-serving-project-map](#) 把 D1-D5 的边界和证据铺开。这页面向选型：先判断问题属于哪一层，再选择可组合的项目；不要把 engine、distributed runtime、K8s routing/control plane 和 GPU 基础设施当成同类替代品。

当前上游核验（2026-09-22）

截至 2026-09-22，通过 GitHub API 与官方文档核验以下 HEAD；详细架构证据见 [llm-inference-serving-project-map](#)。

项目	核验版本	当前职责
vllm	HEAD d50723df04f7	engine scheduler、model execution、local KV
sglang	HEAD 04c0913434c4	engine/runtime、RadixCache、distributed/P-D integration
dynamo	HEAD f36d2fab37fd	distributed request/control/state runtime、routing、KV transfer、planner
llm-d	HEAD 1e9a86a3a9da	Proxy/EPP、InferencePool、Model Server 与 routing signals
aibrix	HEAD 96056b47f158	K8s routing、autoscaling、adapter/model lifecycle、KV/multi-role orchestration

选型结论

没有一个项目能在所有层胜出。先选 engine，再按确实存在的集群、路由、运维和基础设施需求叠加能力；每加一层都要用收益覆盖其控制面和故障域成本。

选择	best fit	avoid-if	adoption cost	下一步核验
vllm	需要成熟的 OpenAI-compatible 基线、广泛模型覆盖和 paged-attention	目标模型/硬件在 sglang 上有已验证的显著优势	低到中；先处理镜像、模型、监控和容量	用真实模型、量化、并发和输出长度压测吞吐、P99、显存与稳定性
sglang	前缀复用、结构化生成、speculative decoding	团队更看重保守生态基线，或关键模	低到中；高级特性和 distributed/P-D	

选择	best fit	avoid-if	adoption cost	下一步核验
	或其执行路径更匹配 workload	型/硬件适配尚未验证	backend 会增加调优面	对同一流量的回放，核验 radix-attention 命中、尾延迟、正确性和故障恢复
dyn am o	多节点 runtime、P/D、KV transfer、KV-aware routing 需要统一协调	单机或普通副本服务已满足目标，暂不需要跨节点状态协同	高；引入 router、worker pools、KV/control state 与额外可观测性	做端到端 P/D/KV transfer 基准，并演练 worker、网络和 KV tier 故障
llm -d	K8s 上需要 Gateway API、EPP endpoint picking 和 InferencePool	不使用 K8s/Gateway API，或普通 Service/LB 已足够	中到高；新增 CRD、Gateway/EPP、升级兼容与运维责任	核验目标 Gateway/InferencePool 版本、路由信号、扩缩交互和故障回退
aib rix	需要 autoscaling、LoRA/adapter、model lifecycle 及更广的 K8s inference operations	现有组件已覆盖所需能力，或针对该需求的集成/运维成本超过收益	中到高；应按组件渐进采用，不必部署全部能力	逐项验证所选 controller/CRD 的边界、升级路径、状态恢复和与现有平台的重叠

先选层，再选项目

当前问题	应选择的层	代表项目
scheduler、kernel、local KV、单实例吞吐	推理引擎	vllm , sglang
multi-node runtime、P/D、KV transfer	distributed serving runtime	dynamo
Gateway API、endpoint picking、InferencePool	K8s routing/serving stack	llm-d
autoscaling、LoRA/model lifecycle、K8s inference operations	K8s inference control plane	aibrix
GPU allocation、sharing、health、capacity	infrastructure	k8s-gpu-device-stack

不同层的项目不是直接替代关系；生产系统通常组合一个 engine、一个 routing/control layer 和一个 infrastructure layer。distributed runtime 只在多节点协调收益明确时加入，routing 与 control plane 也可以按需求择一或组合，而不是默认全栈采用。

架构区别

层	负责什么	不负责什么	典型组合边界
vllm / sglang engine	scheduler、kernel、model execution、local KV	完整 K8s routing/control plane 和 GPU 集群底座	向上暴露服务端点和运行指标
dynamo distributed runtime	多节点 request/control/state、P/D pools、KV transfer/routing	替代底层 engine 或通用 K8s 平台	组织 engine workers；可与 K8s 层集成
llm-d routing/serving stack	Gateway API、EPP、InferencePool、Model Server 与 routing signals	engine kernel、通用 model lifecycle 平台	在 K8s 请求入口选择合适的 engine endpoint
aibrix inference control plane	autoscaling、adapter/model lifecycle、routing 与多角色编排	替代 engine kernel 或 GPU device layer	按需选择 controller，与 engine、Gateway 和基础设施衔接
k8s-gpu-device-stack infrastructure	GPU allocation、sharing、health、capacity 和队列	模型执行、KV-aware request routing	为所有上层提供设备与容量约束

详细的 D1-D5 组件图、控制流和证据集中在 [llm-inference-serving-project-map](#)；本页只保留影响选型的层间边界。

决策轴

- 执行适配：先用真实模型、硬件、量化方式、scheduler/KV 行为和运维约束比较 [vllm](#) 与 [sglang](#)，不要只看通用 benchmark。
- 跨节点协调：只有在 P/D、KV transfer 或多节点 runtime 是明确瓶颈时评估 [dynamo](#)；同时比较 engine-native integration 的能力和复杂度。
- K8s 请求路由：需要 Gateway API、EPP、InferencePool 时评估 [llm-d](#)，普通 Service/Gateway 足够时不增加该层。
- K8s 推理运维：需要 autoscaling、adapter/model lifecycle 和更广控制面时评估 [aibrix](#)，按组件采用并检查职责重叠。
- 设备与容量：用 [k8s-gpu-device-stack](#) 处理 GPU allocation、sharing、health 和 capacity；它会约束上层部署，但不替代引擎或路由。
- 跨云资源位置：需要跨云/K8s/Slurm 选择资源和启动 workload 时再看 [SkyPilot](#)；它是资源控制面，不是 inference engine。

决策流程图

以下分支不是互斥答案：先确认或选择 engine，后续命中的层可以继续组合；任何一步收益不明确，都停在当前最小充分栈。

engine 起点

└─ 尚未选择 → 按模型、硬件、scheduler/KV 行为和运维适配选择 vLLM / SGLang

└─ 已有 engine → 保留并核验它是否满足当前 workload

|

└─ 两条路径汇合 → 独立评估以下能力（可组合、无先后依赖）

└─ 需要 multi-node runtime 或 P/D、KV transfer 协调?

|└─ 是 → 评估 Dynamo 与 engine-native integrations

|└─ 否 → 不增加 distributed runtime

└─ K8s 上需要 Gateway / EPP / InferencePool routing?

|└─ 是 → 评估 llm-d

|└─ 否 → 使用普通 Service / Gateway

└─ 需要 autoscaling、adapters、model lifecycle 和更广 K8s operations?

└─ 是 → 评估 AIBrix 的所需组件

└─ 否 → 不增加 inference control plane

所有分支结束 → 保留满足需求的最小组合；不因“平台完整”而默认叠加全部项目

组合方案

模式	组成	best fit	avoid-if	adoption cost	下一步核验
最小 engine service	vllm 或 sglang + 普通 Service/ Gateway	单集群、普通副本路由，重点是尽快提供稳定推理 API	已确认需要 KV-aware endpoint picking、P/D 或复杂 lifecycle	低；主要是 engine、镜像、模型存储、指标和入口配置	回放真实流量，核验容量、P99、滚动升级和实例故障回退
K8s 智能路由	engine + llm-d Router/ InferencePool	K8s/Gateway API 环境需要根据 KV、负载或模型信号选择 endpoint	普通 Service/ LB 已达标，或不能承担 CRD/Gateway/ EPP 运维	中到高；增加 routing control/data path 与版本兼容面	核验路由增益、信号陈旧行为、扩缩期间 endpoint 一致性和 fallback
distributed	engine + dynamo	多节点 P/D、KV transfer 或跨 worker 状态	单机/普通副本已满足 SLO，网络或 KV	高；增加 runtime 服务、worker	比较共置与分离部署，测端到端 TTFT/ TPOT/P99，并做网

模式	组成	best fit	avoid-if	adoption cost	下一步核验
runtime		协调能带来可测收益	transfer 成本抵消收益	pools、状态/KV tier 和故障域	络、worker、KV 故障演练
inference operations platform	engine + 选定的 aibrix 组件	需要 autoscaling、LoRA/adapters、model lifecycle 或多角色 K8s 运维	现有组件已覆盖所需能力，或针对该需求的集成/运维成本超过收益	中到高；按所选 controller/CRD 计，不要求全量采用	建立组件级 PoC，核验 reconciliation、升级/回滚、状态恢复及与现有 autoscaler/Gateway 的所有权

避坑条件

- 不要把 [dynamo](#) 当成“另一个 vLLM”；它是 serving 编排层。
- 不要把 [llm-d](#) 当成 engine 或完整 inference operations 平台；它当前的选型核心是 Gateway/EPP/InferencePool routing/serving stack。
- 不要把 [aibrix](#) 当成必须整套部署的发行版；只采用能覆盖明确 operations 缺口且所有权清晰的组件。
- 不要把 [SkyPilot](#) 当成 inference engine；它是资源控制面。
- P/D 分离只有在 prompt/decode 负载、KV transfer、路由和扩缩都配套时才有收益。
- KV cache 已经是一等资源，路由、迁移、offload 都要显式建模。
- 不要把不同层的项目塞进单选题，也不要为了“完整架构”默认部署所有层；从最小充分栈开始，用实测瓶颈决定下一层。

llm-d / Kubernetes SIGs 候选项目地图

这页把 llm-d 组织和 kubernetes-sigs 组织里值得继续加入 wiki 的项目，按工程维度拆成候选清单。它不是 star 排行榜，而是服务于当前知识库的补全路线：哪些项目能帮助理解 [llm-inference](#)、[model-serving-operator](#)、[kubernetes-dra](#)、[inference-routing](#)、cloud-native-security 和 Kubernetes 控制器生态。

[!warning] 2026-09-22 当前状态
[llm-d-workload-variant-autoscaler](#) 的摄入/架构研究已完成，但它现为 deprecated 的历史设计，不应用于当前采用。llm-d 新部署的当前路径是 [EPP metrics](#) → [KEDA Prometheus scaler](#) → [HPA](#)。

本次核验基于 GitHub API 当前公开仓库元数据（2026-06-13）。当前已收录的同域项目包括 [llm-d](#)、[llm-d-batch-gateway](#)、[llm-d-benchmark](#)、[llm-d-workload-variant-autoscaler](#)（已完成摄入的 deprecated/历史研究）、[llm-d-inference-sim](#)、gateway-api、[gateway-api-inference-extension](#)、[agent-sandbox](#)、[dra-driver-nvidia-gpu](#)。

总体优先级

- P0：直接补当前 AI Infra / LLM serving / K8s control plane 选型缺口
- P1：强相关，但更偏评测、诊断、迁移、配套或特定场景
- P2：后续补充，先进入 backlog，不急于做完整源码架构页

优先级	项目	维度	加入价值
P0 已完成	llm-d-batch-gateway	LLM batch serving	已补正式源码架构页，把 llm-d 从在线 inference 扩展到 OpenAI Batch API / 离线 batch workload。
P0 已完成	llm-d-benchmark	LLM serving benchmark	已补正式源码架构页，作为 llm-inference-serving-project-map 的性能评测入口。
P0 已完成（历史研究）	llm-d-workload-variant-autoscaler	deprecated serving autoscaling	已补正式源码架构页，保留 variant / workload autoscaling 和资源经济的历史设计；不用于当前采用。
P0 已完成	llm-d-inference-sim	inference simulator	已补正式源码架构页，无 GPU 模拟 vLLM 行为，适合研究调度、benchmark 和路由策略。
	kueue		

优先级	项目	维度	加入价值
P0 已完成		调度 / 队列	K8s 原生 Job queueing, 是 AI/HPC/batch workload 控制面的核心项目。
P0 已完成	karpenter	节点弹性 / 成本	Node autoscaler, 连接 serving SLO、GPU 成本和容量弹性。
P0 已完成	controller-runtime	Operator SDK	现代 K8s controller 的通用抽象层, 当前大量实体页都依赖它。
P0 已完成	kubebuilder	CRD / controller 脚手架	和 controller-runtime 一起构成 Operator 工程主线。
P0 已完成	metrics-server	可观测 / autoscaling	HPA/VPA 基础指标源, 解释 autoscaling 体系必需。
P0 已完成	external-dns	网络 / DNS	K8s service/ingress/gateway 到 DNS record 的控制器代表。
P0 已完成	secrets-store-csi-driver	存储 / 凭据	CSI 方式注入外部 secret, 连接凭据治理和 runtime security。
P0 已完成	kind	计算 / 测试集群	本地 K8s 测试底座, 很多 controller/operator 项目的开发环境基线。
排除	aws-load-balancer-controller / aws-ebs-csi-driver / aws-efs-csi-driver	AWS 专项	用户明确本轮不需要。

llm-d 组织候选

项目	优先级	当前定位	应放入的 wiki 主题
llm-d-batch-gateway	P0 已完成	OpenAI-compatible /v1/batches 和 /v1/files, 把 batch job 分成 API server、processor、queue、storage、GC 等组件。	llm-inference-serving-project-map 、 batch-inference 、 model-serving-operator
llm-d-benchmark	P0 已完成	llm-d benchmark lifecycle/workspace/harness 编排。	LLM serving benchmark、 llm-inference

项目	优先级	当前定位	应放入的 wiki 主题
llm-d-workload-variant-autoscaler	P0 已完成（历史研究）	Deprecated 的 distributed inference workload variant autoscaler 架构快照，仅供迁移/设计研究。	历史 autoscaling、K8s resource economics、 model-serving-operator
llm-d-inference-sim	P0 已完成	轻量模拟 vLLM 行为，不需要 GPU 或真实大模型。	simulator、scheduler、benchmark
llm-d-latency-predictor	P1 已完成	给 inference scheduler 的 ML-based latency scoring service。	latency predictor、 inference-routing
llm-d-prism	P1 已完成	分布式推理性能分析 dashboard，把 benchmark 数据做交互式分析。	observability、performance analysis
llm-d-pd-utils	P1 已完成	Agentic Skills + scripts，用于 P/D 部署 preflight、GPU topology、RDMA/NCCL/network tests。	P/D diagnostics、GPU/network validation
llm-d-batch-gateway 组件拆分	P2	API server / batch processor / GC / storage backend 可后续拆成细页。	batch serving internals
llm-d-inference-payload-processor	P2	inference payload processor，目前公开说明较薄。	router/batch 辅助组件
llm-d-infra / templates / .github	暂缓	组织 CI、模板和治理基础设施。	不建议单独建架构页
archived deployer / model-service / routing-sidecar	暂缓	旧组件或已归档项目。	仅在追溯历史设计时引用

llm-d 的知识结构缺口

当前 wiki 已经覆盖 [llm-d](#) 总入口、[llm-d-router](#)、[llm-d-kv-cache](#) 和 P0 四类外围能力：

- Batch inference: [llm-d-batch-gateway](#) 让在线 serving 与离线 batch workload 可以共享下游 router/model server，但 SLO、队列、存储和计费边界不同。
- Performance evaluation: [llm-d-benchmark](#) / llm-d-prism / [llm-d-inference-sim](#) 可以把“架构上可行”推进到“如何测、如何复现、如何调参”。
- Autoscaling/resource economics（历史）：[llm-d-workload-variant-autoscaler](#) 是 deprecated 的历史设计研究，曾探索与 Kueue、Karpenter、GPU 资源层的连接；当前新部署采用 EPP metrics → KEDA/HPA。
- P/D deployment diagnostics: llm-d-pd-utils 把 GPU topology、RDMA、NCCL、NIXL 等实际部署问题显性化。

Kubernetes SIGs 候选：按工程维度

网络

项目	优先级	为什么值得加入
external-dns	P0 已完成	从 Service/Ingress/Gateway 等 K8s 对象动态维护外部 DNS，是典型“声明式网络控制器”。
aws-load-balancer-controller	排除	AWS LB / Ingress / Gateway API 的生产代表，适合和 gateway-api 放在同一条网络入口路线。
gateway-api	已有	已收录，作为新一代流量入口 API 基础。
gateway-api-inference-extension	已有	已收录，作为 LLM inference endpoint picking 标准层。
ingress2gateway	P1 已完成	从 Ingress 迁移到 Gateway API 的工具，适合补迁移路线。
apiserver-network-proxy	P1 已完成	control plane 到节点网络代理，适合理解托管 K8s / 私有网络控制面连接。
kube-agentic-networking	P1 已完成	Agent/tool networking policy governance，和 agent-sandbox 、agentgateway 方向高度相关。
dragnet / cni-dra-driver	P2	DRA 和高性能网络结合，适合后续连接 GPU/RDMA/AI workload。

存储 / Secret / 数据面挂载

项目	优先级	为什么值得加入
secrets-store-csi-driver	P0 已完成	把外部 secret store 通过 CSI volume 注入 Pod，连接 agent-credential-isolation 和 cloud-native-security。
aws-ebs-csi-driver	排除	块存储 CSI driver 的云厂商代表。
aws-efs-csi-driver	排除	文件存储 CSI driver 的云厂商代表，适合与 EBS 对比。
nfs-subdir-external-provisioner	P1 已完成	轻量 NFS 动态 PV，适合本地/实验集群。
sig-storage-local-static-provisioner		

项目	优先级	为什么值得加入
	P1 已完成	local PV 静态 provisioner，适合理解本地盘和调度绑定。
sig-storage-lib-external-provisioner	P1 已完成	CSI/external provisioner 的库和控制器抽象。
vsphere-csi-driver / gcp-* / azure* / alibaba-cloud-csi-driver	P2	云厂商矩阵补充，按需要选择代表，不必全量展开。

调度 / 资源 / 队列

项目	优先级	为什么值得加入
kueue	P0 已完成	K8s-native Job queueing，是 AI/HPC/batch workload 多租户队列核心。
karpenter	P0 已完成	节点级弹性和容量经济，适合接 LLM serving 的成本与 SLO。
scheduler-plugins	P0 已完成	kube-scheduler framework 的 out-of-tree 插件集合，理解调度扩展必读。
descheduler	P1 已完成	通过策略触发重调度，补“调度后优化”维度。
kwok	P1 已完成	无 kubelet 模拟大量节点/集群，适合调度和控制面扩展测试。
node-feature-discovery	P1 已完成	发现节点硬件能力，连接 GPU、NUMA、加速器调度。
kube-scheduler-simulator	P1 已完成	可视化/模拟 scheduler 行为。
dra-driver-nvidia-gpu	已有	已收录，作为 DRA + GPU 设备分配代表。

可观测 / 性能 / 诊断

项目	优先级	为什么值得加入
metrics-server		

项目	优先级	为什么值得加入
	P0 已完成	HPA/VPA 的基础资源指标源，K8s autoscaling 入口。
prometheus-adapter	P0 已完成	把 Prometheus 指标暴露为 custom/external metrics API，连接高级 autoscaling。
inference-perf	P1 已完成	GenAI inference performance benchmarking tool，可与 llm-d-benchmark 对比。
headlamp	P1 已完成	K8s UI / debugging / monitoring，可和 kubewall、k8m 对比。
usage-metrics-collector	P2	容量和使用率指标收集，适合补平台容量管理。
resource-state-metrics / logtools / instrumentation-tools	P2	作为 SIG Instrumentation 工具链补充。

计算 / Runtime / 节点

项目	优先级	为什么值得加入
kind	P0 已完成	本地 Kubernetes in Docker，是 controller/operator 开发和 CI 的事实基线。
kubespray	P0 已完成	生产集群部署自动化，适合理解集群生命周期和裸金属/on-prem。
cri-tools	P0 已完成	CRI CLI + validation，理解 kubelet/runtime 边界。
security-profiles-operator	P1 已完成	seccomp/AppArmor/SELinux profile operator，连接 runtime security。
agent-sandbox	已有	已收录，AI Agent runtime 的 sandbox CRD。
lws	P1 已完成	LeaderWorkerSet，用一组 Pod 表达 leader/worker 分布式 workload。
jobset	P1 已完成	分布式 ML/HPC workload API，和 Kueue、batch serving 强相关。

API / Operator / 控制器开发

项目	优先级	为什么值得加入
controller-runtime	P0 已完成	Manager、cache、client、reconcile、webhook、envtest 的现代 controller 抽象层。
kubebuilder	P0 已完成	CRD/controller 项目脚手架和代码生成路径。
controller-tools	P0 已完成	CRD、RBAC、webhook、object deepcopy 等生成工具链。
cluster-api	P0 已完成	声明式集群生命周期管理，适合接多集群和平台工程。
kustomize	P1 已完成	Kubernetes YAML 定制工具链，GitOps/配置管理基础。
kro	P1 已完成	Kube Resource Orchestrator，适合与 Crossplane Composition / higher-level API 对比。
apiserver-builder-alpha	P2	aggregated apiserver/controller 旧路线，可作为历史参考。

AI Infra / Agent 交叉

项目	优先级	为什么值得加入
mcp-lifecycle-operator	P1 已完成	声明式管理 MCP Servers，和 mcp、agentgateway、declarative-agent-management 直接相关。
kube-agentic-networking	P1 已完成	Agent/tool 网络策略治理，适合补 Agent runtime 安全边界。
ai-conformance	P2	AI conformance definition/proposals/tests，待生态更稳定后再摄入。
inference-perf	P1 已完成	GenAI inference benchmark，与 llm-d 、 aibrix 、kserve 有直接交集。

推荐实施顺序

第一批：直接补 AI Infra 选型缺口

- 1. [llm-d-batch-gateway](#)（已完成）

2. [llm-d-benchmark](#) (已完成)
3. [llm-d-workload-variant-autoscaler](#) (历史研究已完成; deprecated, 不用于当前采用)
4. [llm-d-inference-sim](#) (已完成)
5. kueue (已完成)
6. karpenter (已完成)
7. metrics-server (已完成)
8. prometheus-adapter (已完成)
9. [inference-perf](#) (已完成)
10. [lws](#) (已完成)
11. jobset (已完成)

这批可以让 [llm-inference-serving-project-map](#) 从“引擎/serving stack”扩展到 batch、benchmark、queueing、当前 EPP metrics → KEDA/HPA autoscaling 与 distributed workload API; WVA 仅保留为历史研究。

第二批：补 Kubernetes 控制器和平台工程底座

1. controller-runtime (已完成)
2. kubebuilder (已完成)
3. controller-tools (已完成)
4. cluster-api (已完成)
5. kind (已完成)
6. cri-tools (已完成)
7. external-dns (已完成)
8. secrets-store-csi-driver (已完成)
9. scheduler-plugins (已完成)
10. node-feature-discovery (已完成)

这批可以让 k8s-core-controller-map 从学习路径升级成更完整的平台工程架构图。

第三批：按专项补齐

- 网络专项: ingress2gateway、apiserver-network-proxy、kube-agentic-networking 已完成; aws-load-balancer-controller 按用户要求排除; dranet 留作 P2。
- 存储专项: nfs-subdir-external-provisioner、sig-storage-local-static-provisioner、sig-storage-lib-external-provisioner 已完成; aws-ebs-csi-driver、aws-efs-csi-driver 按用户要求排除。
- 安全专项: security-profiles-operator、secrets-store-csi-driver、kube-agentic-networking 已完成。
- 工具/测试专项: kustomize、kwok、kube-scheduler-simulator、headlamp、kro、mcp-lifecycle-operator 已完成; krew、e2e-framework、kubetest2 留作 P2。

和现有页面的关系

- 更新 [llm-inference-serving-project-map](#): 加入 batch gateway、benchmark、simulator、Kueue/Karpenter/LWS/JobSet 与当前 EPP metrics → KEDA/HPA 扩缩路径; deprecated WVA 只作为历史 autoscaler 研究。
- 更新 k8s-core-controller-map: 从 controller-runtime/kubebuilder 学习路径, 扩展成网络、存储、调度、可观测、计算、API/operator 的维度地图。
- 更新 [k8s-gpu-device-stack](#): 把 node-feature-discovery、Kueue、Karpenter、JobSet、LWS 放入 GPU/AI workload 的调度前置层。
- 更新 cloud-native-security: 把 secrets-store-csi-driver、security-profiles-operator、kube-agentic-networking 作为 runtime/credential/network policy 三条线。
- 更新 [model-serving-operator](#): 把 [llm-d](#) 外围组件与 K8s SIG 的 queueing/autoscaling/distributed workload API 连起来。

Agent Substrate 架构与设计思路分析

原文: raw/substrate-architecture-analysis.md · 仓库: <https://github.com/agent-substrate/substrate> · 分析版本 HEAD a3f4474

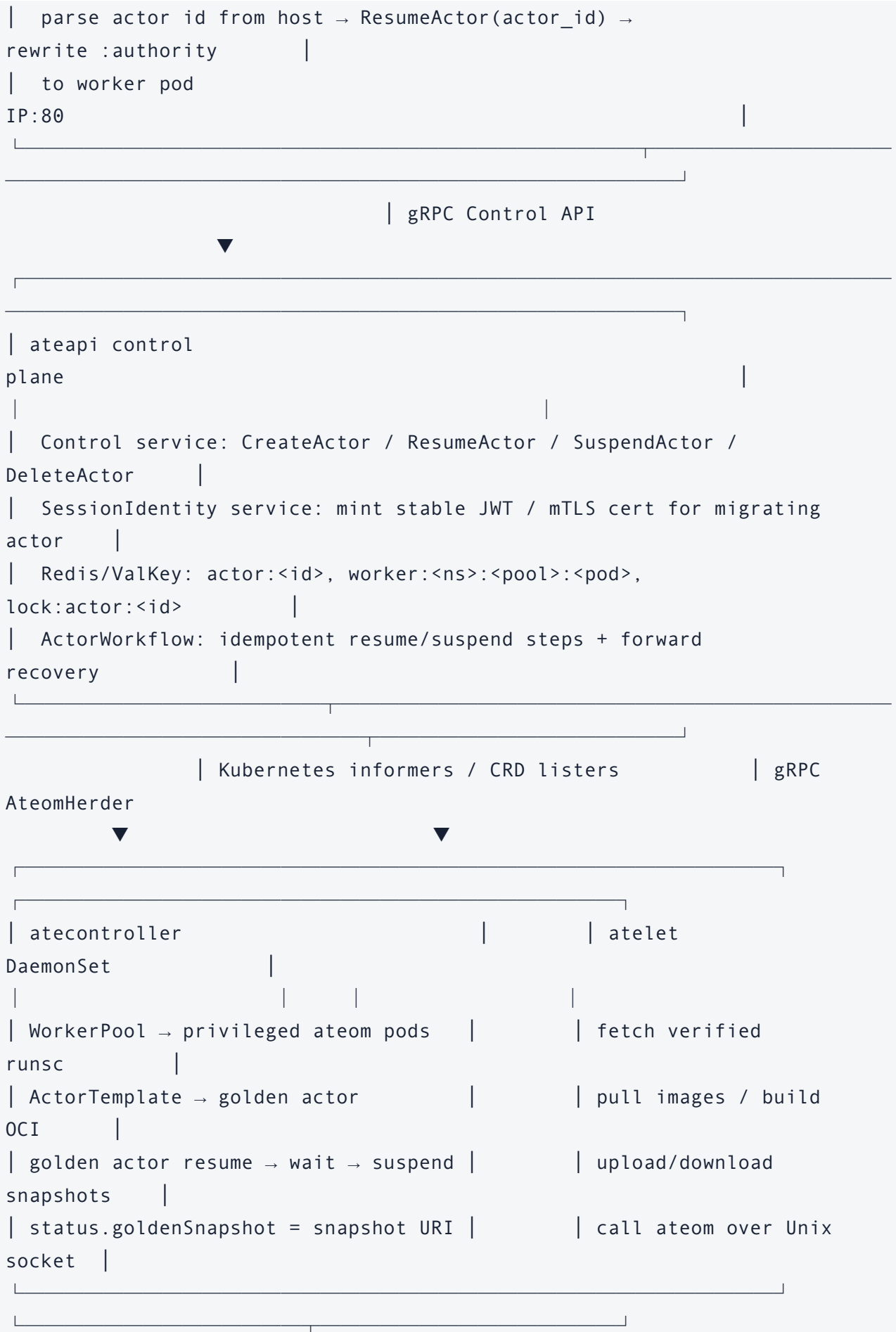
一句话定位

Agent Substrate 是一个建立在 kubernetes 之上的高密度 Agent Runtime / Sandbox substrate: Kubernetes 只负责声明式容量和基础设施, Actor 的高频生命周期、resume/suspend、worker 分配和路由唤醒由 Substrate 自己的 Redis/ValKey 控制面、Envoy ext_proc router、atelet node supervisor 和 gVisor atecom worker 承担。

它补的是 [agent-sandbox](#) 和 agentcube 之间更底层、更激进的一层: 不是“创建一个有状态 sandbox”, 而是把大量 idle actor multiplex 到较少 warm worker 上, 通过 golden snapshot、runtime snapshot、DNS/router 唤醒和 gVisor restore 降低激活延迟。官方 architecture 文档也明确标注“Much of this architecture is aspirational”, 所以要把愿景和当前代码实现分开看。

核心架构图







```

| ateom-gvisor worker
pod
|
| privileged pod, one active actor at a
time
| link-local veth: pod netns ateom0 ↔ actor netns
eth0
| nftables DNAT/MASQ compatibility
bridge
| runc create/start/checkpoint/restore pause + app
containers

```



```

| Object storage
snapshots
|
| gs:// or s3://
prefix
| checkpoint.img.zstd / pages.img.zstd /
pages_meta.img.zstd

```

模块分层

层 / 模块	职责
CRD / public API	定义 WorkerPool、ActorTemplate 和 actor lifecycle gRPC API；K8s CRD 承载低频配置，gRPC 承载高频 actor 操作。
K8s controller	把 WorkerPool 物化为 privileged ateom Deployment；把 ActorTemplate 驱动成 golden actor → golden snapshot → Ready。

层 / 模块	职责
Control plane / DB	对外提供 Control / SessionIdentity gRPC; 用 Redis/ValKey 存 actor/worker 状态和 lock; 用 workflow step 实现 resume/suspend forward recovery。
Node supervisor	每节点/每 worker 管理 runsc、镜像拉取、OCI bundle、snapshot 上传下载, 并转调 ateam。
Sandbox herder	privileged worker pod 内的 gVisor 控制进程, 串行调用 runsc create/start/checkpoint/restore, 管理 actor netns/veth/nftables。
Router / DNS	用 CoreDNS stub domain + Envoy xDS + ext_proc, 把 actor hostname 转成 ResumeActor 调用和 worker pod IP 转发。

关键边界: kubernetes API 不存每个高频 actor transition; WorkerPool/ActorTemplate 是声明式配置, Actor/Worker 的运行状态在 Redis/ValKey, 真正 runsc 操作在 worker pod 本地完成。

关键数据流

ActorTemplate 到 golden snapshot

```

ActorTemplate CR created
|
▼
atecontroller ActorTemplateReconciler
|   PhaseInitial
▼
ateapi.CreateActor(goldenActorID)
|
▼
status.phase = ResumeGoldenActor
|
▼
ateapi.ResumeActor(goldenActorID)
|   no actor snapshot + no golden snapshot
▼
atelet.Run → ateam.RunWorkload → runsc create/start
|
▼
wait 20s
|
▼

```



```

ateapi.SuspendActor(goldenActorID)
|
▼
atelet.Checkpoint → ateam.CheckpointWorkload → upload checkpoint files
|
▼
ActorTemplate.status.goldenSnapshot = snapshot URI
ActorTemplate.status.phase = Ready

```

请求唤醒和路由

```

HTTP request
Host: <actor-id>.actors.resources.substrate.ate.dev
|
▼
CoreDNS stub domain routes to atenet-router
|
▼
Envoy RequestHeaders ext_proc
|
▼
parseActorID(host)
|
▼
singleflight ResumeActor(actor-id)
|
▼
ateapi resume workflow
|
├─ LoadActorForResume
├─ AssignWorker
├─ CallAteletRestore / Run
└─ FinalizeRunning
|
▼
actor.ateom_pod_ip returned
|
▼
mutate :authority to <worker-ip>:80
|

```

▼
Envoy dynamic_forward_proxy forwards request

Resume / Suspend workflow

```
ResumeActor(actor-id)
  |
  ▼
  Acquire lock: lock:actor:<id>
  |
  ▼
  Load actor + ActorTemplate
  |
  ▼
  Find existing assigned worker or random free worker
  |
  ▼
  Update worker(actor_id=...) then actor(status=RESUMING, worker fields)
  |
  ▼
  if actor.last_snapshot: atelet.Restore(last_snapshot)
  else if template.goldenSnapshot and !boot: atelet.Restore(goldenSnapshot)
  else: atelet.Run(template spec)
  |
  ▼
  Update actor(status=RUNNING)

SuspendActor(actor-id)
  |
  ▼
  Acquire lock: lock:actor:<id>
  |
  ▼
  Load actor + ActorTemplate
  |
  ▼
  Mark actor SUSPENDING + in_progress_snapshot
  |
  ▼
  atelet.Checkpoint(snapshot URI)
```



```
Free worker, move in_progress_snapshot → last_snapshot,  
clear worker fields, status=SUSPENDED
```

设计决策与哲学

- K8s 管低频配置，Substrate 管高频 actor 状态：WorkerPool/ActorTemplate 是 CRD，actor/worker 运行态进入 Redis/ValKey。Redis store 注释明确说明跨 slot 不能原子更新 actor 和 worker，所以系统必须接受 forward recovery。
- Golden snapshot 是冷启动优化根：ActorTemplate 会生成一个 golden actor，启动后暂停成 snapshot；后续 actor 可从 golden snapshot 恢复，而不是每次重新跑初始化。
- Resume/suspend 采用 client-driven forward recovery：每个 workflow step 都能判断是否已完成，失败后靠客户端重试继续推进；每个 actor 还有 lock:actor:<id> 防同 actor 并发操作。
- worker 是 warm capacity，不是 actor 本体：WorkerPool 控制 privileged atoom pods 的数量；Actor 只是逻辑记录，运行时才绑定某个 worker pod。
- gVisor/runsc 是当前 snapshot 执行后端：atelet 负责准备 runsc、OCI bundle 和 snapshot 文件；ateom-gvisor 串行调用 runsc 的 create/start/checkpoint/restore。
- 网络唤醒还是兼容桥阶段：当前路由按 actor DNS 唤醒后把请求改写到 worker IP:80；ateom 用 veth + nftables 做 actor ingress/egress 兼容，后续仍需要更严格的 gateway/policy 层，和 agentgateway 一类项目会形成互补。

关键组件深入解读

ateapi ActorWorkflow

ActorWorkflow 是当前源码里最能体现 Substrate 设计哲学的部分。它不是写一个大函数完成 resume/suspend，而是把流程拆成可跳过的步骤：resume 依次加载 actor、分配 worker、调用 atelet restore/run、最终置 running；suspend 依次加载 actor、标记 suspending、调用 atelet checkpoint、释放 worker 并提交 snapshot。

这个拆法服务于两个现实约束：actor 与 worker 状态跨 Redis Cluster key，不可能用一个强事务同时修改；调用 atelet/ateom/runsc 是外部副作用，失败可能发生在任意位置。因此 workflow 采用“状态已写入就能被下一次请求识别并继续”的模型。

atelet + atoom-gvisor

atelet 和 atoom 的分工很清楚：atelet 是 node supervisor，接收 ateapi 的 Run/Checkpoint/Restore 请求；ateom 是 worker pod 内的 gVisor herder，真正执行 runsc。atelet 负责下载/校验 runsc、拉 OCI image、展开 rootfs、生成 OCI config、下载/上传 checkpoint zstd 文件。ateom 负责创建 interior network namespace、建立 veth、安装 nftables、执行 runsc create/start/checkpoint/restore。

当前实现假设一个 worker pod 同时只运行一个 active actor；ateom 的 mutex 和 nftables 表清理逻辑都体现了这一点。

相关页面

- [agent-runtime-sandbox-selection-map](#)
- [agent-runtime-sandbox-project-map](#)
- [agent-sandbox](#)
- [agentcube](#)
- [kubernetes](#)
- [agentgateway](#)

Agent 凭据隔离

让 AI Agent 永远拿不到真实凭据（LLM API key / GitHub PAT / OSS AK），它只持一个网关侧颁发的 consumer key，所有外部访问经网关鉴权和路由。

[HiClaw](#) 通过 Higress AI Gateway 实现这个模式：

- 用户的真 API key 只存 Higress 的 secret 里
- 每个 Agent ([Worker](#) / Manager) 被分配一个 Higress consumer + key
- Agent 调 LLM / OSS / MCP server 都经过 Higress，consumer key 决定能访问哪些路径
- 抗 prompt injection：即便 Agent 被劫持，攻击者拿到的也只是可被即时 revoke 的 consumer key——而不是真凭据

为什么重要

社区 skill 市场（如 [skills.sh](#) 上 80,000+ 社区作品）能否被企业放心装载，取决于这一点——传统框架（autogen / langgraph / crewai）让 Agent 直接持真凭据，prompt injection = 凭据失窃。

模式拆解

```
Agent runtime
| only has consumer key / short-lived token
▼
AI Gateway / Tool Gateway
| owns real provider keys, GitHub PAT, OSS AK, MCP credentials
| applies authn/authz/rate limit/audit/policy
▼
LLM provider / MCP server / storage / code host
```

关键点不是“把 key 放到另一个地方”这么简单，而是把权限决策从 prompt 可影响的 Agent 进程移到网关控制面：

- Agent 只看到统一 endpoint 和可撤销身份；
- 真凭据只在 gateway/backend secret/credential provider 内部使用；
- route / policy / consumer 决定 Agent 能访问哪些模型、工具、bucket、repo；
- audit log 记录的是“哪个 Agent 身份调用了哪个能力”，而不是一堆共享 API key。

代表实现

项目	隔离方式	适合借鉴点
HiClaw	Worker / Manager 只持 Higress consumer key，真实 LLM / OSS / MCP 凭据在网关侧	多 Agent 协作平台的默认安全边界
agentgateway	AgentgatewayBackend 保存 provider/backend secret，LLM/MCP/A2A 统一走 gateway policy	AI Gateway 作为出口流量和工具访问控制面
openkruse-agents	E2B API key storage、namespace-scoped team、identity provider 和 security token propagation，把 sandbox API 身份与运行时 token 分开	K8s sandbox platform 的 API key / short-lived token / runtime propagation 边界
loongsuite-pilot	不托管业务凭据，但在 telemetry 上统一执行 content policy / secret mask	观测链路里的隐私边界

凭据轮转

凭据轮转应该发生在网关侧，而不是要求 Agent 重新学习/保存真实 key：

1. credential provider 或 secret manager 生成/刷新真实 provider 凭据；
2. gateway 更新 backend secret 或 consumer key 映射；
3. Agent 侧只需要继续用自己的 consumer key，或接收一个短期 token；
4. 旧 consumer key 可按 Agent、Team、workspace 维度 revoke；
5. audit log 用于确认旧 key 不再被使用。

HiClaw 的 RefreshCredentials 思路属于这个模式：Worker 身份稳定，真实凭据和 STS 权限可在外部滚动。

网关单点风险

凭据隔离把风险从“每个 Agent 都可能泄露真 key”收敛为“网关是强信任边界”。需要配套：

- 网关高可用，避免所有 Agent 出口依赖单 Pod；
- RBAC / namespace / consumer policy 分层，避免一个 consumer key 访问全部能力；
- secret manager 或 KMS 托管真凭据，避免明文落盘；
- access log + anomaly detection，发现异常工具调用或模型调用；
- break-glass 机制，网关不可用时明确哪些能力允许降级，哪些必须 fail-closed。

与插件设计原则的关系

这对应 ai-agent-plugin-patterns 中的“宿主权限边界”和“协议通道纪律”：插件、skill、MCP server 可以扩展 Agent 能力，但不应把真实凭据直接暴露给 Agent prompt 或第三方插件代码。

agent-sandbox

kubernetes-sigs/agent-sandbox 是 K8s SIG Apps 官方孵化的 Sandbox CRD + controller。把 AI Agent runtime 那种"长寿命、有状态、单实例、可暂停、有稳定身份"的容器形态建模成第一类 K8s 资源——比 Deployment（无状态副本）和 StatefulSet（编号 Pod）都更精准。隔离机制完全委托给标准 K8s 原语（gvisor / kata-containers / network-policy），controller 只做生命周期编排。

详细架构见 [src-agent-sandbox-architecture](#)。

Sandbox 生命周期图



关键事实

- 仓库: [kubernetes-sigs/agent-sandbox](#)
- 当前核验: GitHub API 显示 main 最近 push 2026-06-11, 最新稳定 tag v0.4.6, 另有 v0.5.0rc1
- 本 wiki 架构分析版本: v0.4.5+11 (HEAD e1d8898, 2026-04-23 左右)
- 主要语言: Go 1.26.2, 模块 sigs.k8s.io/agent-sandbox
- 4 个 CRD: 核心 Sandbox (agents.x-k8s.io) + 扩展 SandboxClaim / SandboxTemplate / SandboxWarmPool (extensions.agents.x-k8s.io)
- 治理: K8s SIG Apps 孵化项目, 遵循 [Kubernetes Resource Model](#) 范式
- SDK: Go + Python (codegen 生成) + 标准 K8s clientset/informer/lister
- examples 目录当前超过 20 个: quickstart / hello-world / kata-gke / openclaw / hermes / langchain / jupyter / vscode / chrome / HPA / Kueue / Cilium/NetworkPolicy / MCP / Ray / Python SDK 等

与 [HiClaw](#) 的关系

互补不竞争——agent-sandbox 是基础设施层（提供"安全运行 1 个有状态 Agent 容器"原语），HiClaw 是应用层（提供"管 N 个 Agent 协作 + IM 平面 + 凭据网关"）。理论上 HiClaw 的 Worker runtime 完全可以跑在 agent-sandbox 之上。

与 [HiClaw](#) 的完整对比

维度	agent-sandbox	HiClaw
层级	Runtime substrate / CRD 原语	多 Agent 协作平台
核心对象	Sandbox、Claim、Template、WarmPool	Worker、Team、Human、Manager
协作模型	不管 IM / team / manager	Matrix 房间 + Manager/Worker/Human
凭据模型	由 PodTemplate / 用户环境决定	Gateway consumer key + 真凭据托管
隔离模型	委托 gVisor/Kata/NetworkPolicy/PodTemplate	依赖容器后端，可进一步叠 agent-sandbox
启动优化	WarmPool / Template / suspend-resume	Worker lifecycle + runtime package
典型用户	平台工程师、runtime 基座维护者	Agent 平台/协作产品维护者

GKE Kata walkthrough 速记

1. 准备支持 Kata/gVisor 的 GKE 或兼容集群。
2. 安装 agent-sandbox CRD/controller。
3. 创建带 runtimeClass / securityContext / volume / network policy 的 SandboxTemplate。
4. 可选创建 SandboxWarmPool，预热若干 Pod。
5. 用户创建 SandboxClaim，controller 从 WarmPool 领取或新建 Sandbox。
6. Sandbox Ready 后，通过 service / SDK / router 访问容器内 agent 或工具。
7. 空闲后 scale to zero / suspend，保留 PVC 状态，后续 resume。

这个流程说明它的关键不是“创建 Pod”，而是把单实例、有状态、可暂停、可领取的 Agent 容器变成可声明和可复用的资源。

v0.5 / beta 路线

roadmap 当前显示几条主线：

- API 与 runtime 解耦，走 portable backend / common proto；
- SandboxTemplate / WarmPool rolling update；
- router 成为 first-class 项目组件；

- auto suspend/resume、scale-to-zero、TFFI 延迟优化；
- 智能 WarmPool 选择和 claim latency 优化；
- alpha → beta API versioning；
- TypeScript SDK、MCP server、Ray / LangChain / CrewAI / kAgent 集成。

这意味着 v0.5 之后的重点会从“CRD 能跑”走向“多后端、低延迟、SDK、router、生产指标”。

examples 导读

类别	examples
入门	quickstart,hello-world-sandbox,python-sdk-quickstart
隔离 / runtime	kata-gke-sandbox,chrome-sandbox,sandboxed-tools,sandbox-ksa,policy
Agent / framework	openclaw-sandbox,hermes-agent,langchain,gemini-cu-sandbox,code-interpreter-agent-on-adk
扩缩 / 调度	hpa-swp-scaling,kueue-agent-sandbox>manual-pdb
网络策略	composing-sandbox-nw-policies,mcp-server-sandbox
工具型环境	jupyterlab,vscode-sandbox,aio-sandbox,analytics-tool,python-runtime-sandbox,ray-integration

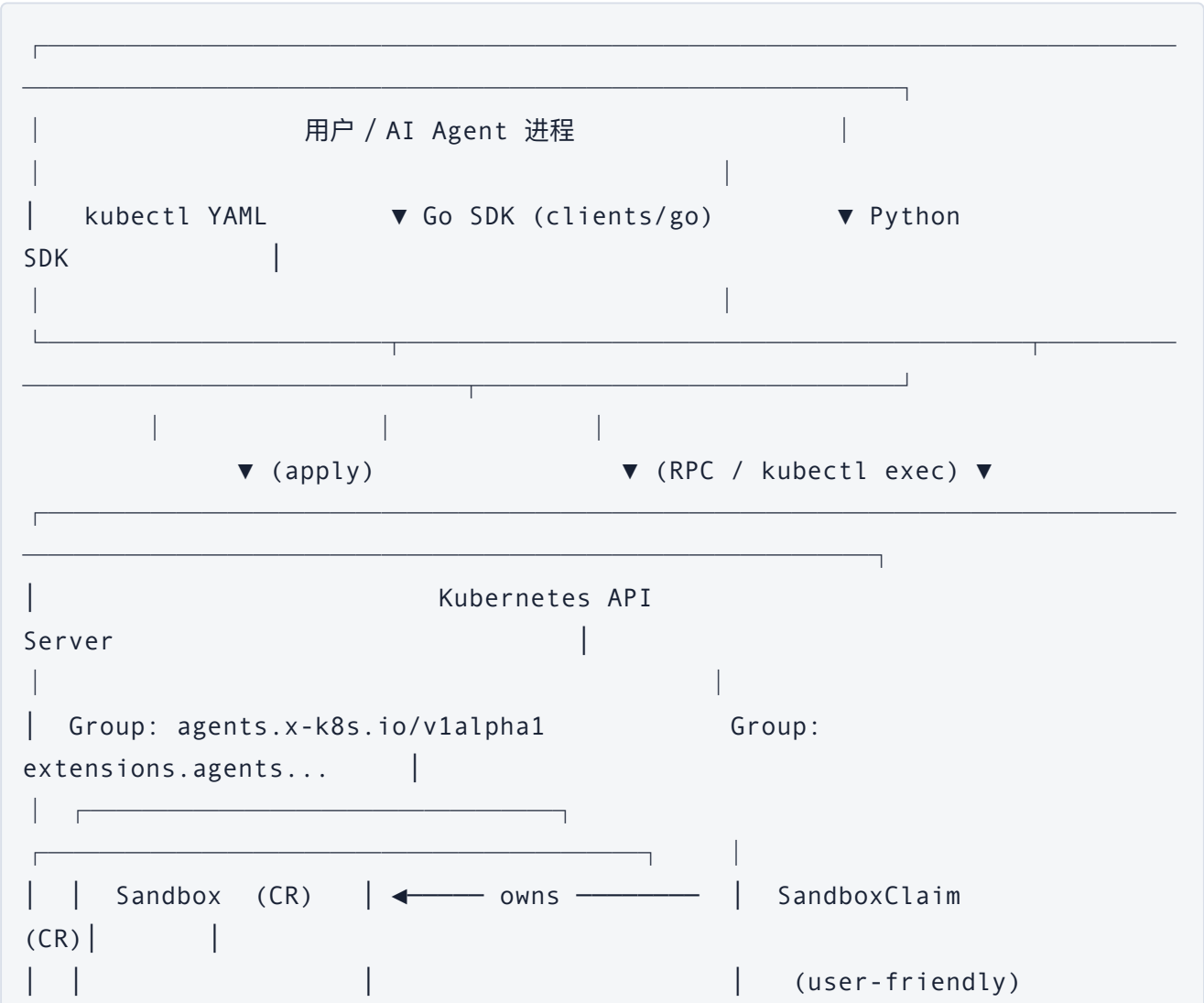
agent-sandbox 架构与设计思路分析

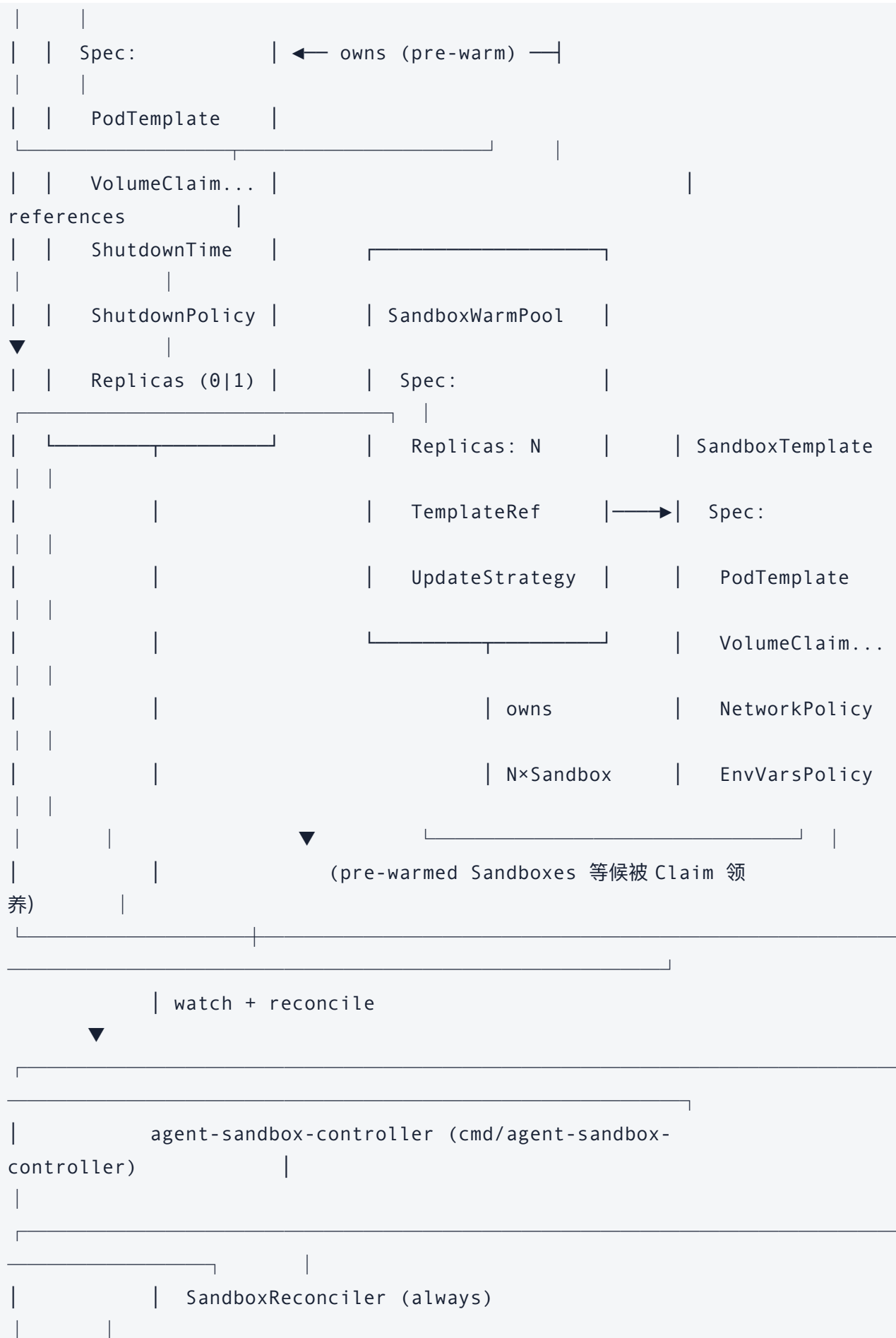
原文: raw/agent-sandbox-architecture-analysis.md · 仓库: [kubernetes-sigs/agent-sandbox](#) · 分析版本 v0.4.5+11 (HEAD e1d8898)

一句话定位

kubernetes-sigs/agent-sandbox 是 K8s SIG Apps 官方孵化的 Sandbox CRD 与 controller，把 AI Agent runtime 那种"长寿命、有状态、单实例、可暂停、有稳定身份"的容器形态建模成第一类 K8s 资源——比 Deployment（无状态副本）和 StatefulSet（编号 Pod）都更精准。核心手段：1 个 Sandbox CR = 1 个 Pod + 1 个 headless Service + 持久 PVC + 可选 Template/Claim/WarmPool 三件套，隔离机制（gVisor/Kata/NetworkPolicy）完全委托给标准 K8s 原语，controller 只做生命周期编排。

核心架构图







模块分层

层	职责
CRD 定义层	4 个 CRD 的 Go 类型 + kubebuilder 标注 → controller-gen 自动生成 YAML。Spec/Status 即 API 契约。
Reconciler 层	controller-runtime 风格的 4 个 reconciler。watch + owns + 幂等 reconcile。Claim/WarmPool/Template 通过 --extensions=true 开关启用。
运行时支持	绝对时间 + TTL 过期计算、Prometheus 指标、per-template-hash 的 FIFO 领养队列（O(1) WarmPool adopt）。
入口二进制	单二进制 agent-sandbox-controller，flags 控制 extensions 开关、leader election、并发度、OTel tracing、pprof。
Client SDK	Go / Python / 标准 K8s clientset。Go SDK 提供远程 exec、文件传输、gateway 隧道、追踪注入——给 AI Agent runtime 调用 Sandbox 用。
打包	Helm chart（CRDs / RBAC / Deployment / metrics Service）+ 单独的 k8s/crds/ 给手工 kubectl apply。
示例与集成	17 个 examples/: hello-world / kata-gke / openclaw / hermes / langchain / jupyter / vscode / chrome / HPA / Kueue / Cilium。"读完 examples 就会用"。

分层关键约束：api/ 与 extensions/api/ 是被 K8s API server 直接读的契约，只能改不能删；internal/ 严格私有；controllers/ 不允许 import extensions/（核心稳定、扩展演进的边界）。

关键数据流



```

| Reconcile(sandbox) - controllers/sandbox_controller.go |
|   ├── checkSandboxExpiry          (line 1003)         |
|   ├── reconcilePVCs              (line 853) → 创建 PVC |
|   ├── reconcilePod               (line 531)           |
|   |   ├── 如果有 SandboxPodNameAnnotation:           | |
|   |   |   ├── 走"warm pool 领养"分支, 复用现有 Pod   |
|   |   |   └── 否则 Create Pod {sandbox.Name}         |
|   ├── reconcileService           (line 398) → 创建   |
|   |   └── headless Service (ClusterIP=None)         |
|   ├── 算 Conditions (Ready/Finished)                 |
|   └── Update Status + requeueAfter(min(timeLeft, 2s)) |

```



```

| Pod 起来 → 容器内 Agent runtime (OpenClaw/Hermes/...) |
|
| AI Agent 进程 ←—— SDK (gateway/tunnel/exec) ——— | ← 外部 caller
| (在 gVisor / Kata 内运行, 宿主机内核被隔离)      |

```

| t = ShutdownTime (绝对时间)



```

| handleSandboxExpiry (line 922)                       |
|   ├── ShutdownPolicy=Retain (default): 删 Pod+Svc,   |
|   |   保留 Sandbox CR (status 显示 expired)         |
|   └── ShutdownPolicy=Delete: 整个 Sandbox 也删掉     |

```

SandboxClaim 快/慢路径

```
user kubectl apply SandboxClaim
|
▼
SandboxClaimReconciler.getOrCreateSandbox
(sandboxclaim_controller.go:320)
|
├─▶ 【快路径】adoptSandboxFromCandidates (line 719)
|   └─ query SimpleSandboxQueue[templateHash]
|   找到 1 个 WarmPool 已就绪的 Sandbox
|       └─ patch: 清掉 WarmPool 的 OwnerReference
|           └─ patch: 把 Claim 设为新 owner    ≈ 0 启动延迟
|
└─▶ 【慢路径】createSandbox (line 385)
    没找到 / WarmPoolPolicy=none / 自定义 env (跟 warmpool 不兼容)
    └─ 读 SandboxTemplate
    └─ 复制 PodTemplate + VolumeClaimTemplates
    └─ 注入 claim 的额外 env / labels
        └─ SetControllerReference(claim, sandbox)
        └─ Create Sandbox CR    ≈ Pod 冷启动延迟 (秒级)
```

设计决策与哲学

- 为什么不复用 Deployment / StatefulSet: AI Agent runtime 既不像 Deployment 那样无状态可副本 (Agent 有会话状态), 也不像 StatefulSet 那样需要编号有序 (每个 sandbox 是独立用户实例)。需要的是 "1 个 stable identity + 持久存储 + 可调度销毁 + 可暂停可恢复" 的单实例语义——Pet 概念在云原生的回归, 但这次明确为 AI Agent runtime 量身定做。replicas: 0|1 强制约束就是 "暂停 = scale to 0、恢复 = scale to 1" 语义的承载。
- 隔离机制全部委托标准 K8s 原语: controller 不强制任何隔离手段——gVisor / Kata / securityContext / serviceAccount / seccomp 都在用户填写的 PodTemplate.Spec 里, 原样透传。controller 唯一强势注入的是 (1) headless Service 强制走 DNS (多 sandbox 才能各自有稳定身份), (2) Template 级的默认 NetworkPolicy (deny RFC1918 内网 + deny 云元数据服务)。这种 "机制由 K8s 提供、策略由用户挑选" 的分工, 让它能兼容 GKE Kata、EKS Firecracker、本地 gVisor 任何隔离基座。这条与 AI Agent 外挂设计原则里 "隔离用现成机制、不重新发明" 完全吻合。

- **WarmPool + OwnerReference 转移实现 ~0 启动延迟**：单纯 Sandbox 冷启动慢（镜像拉取 + Kata/gVisor 初始化数秒到数十秒）。WarmPool 维护 N 个 pre-warmed Sandbox，Claim 来了不是新建 Pod，而是把 Ready 的 Pod 改归属者。SimpleSandboxQueue 是 per-template-hash 的线程安全 FIFO，O(1) 找候选。sandboxclaim_pod_exclusivity_test.go 这个测试名直白地说明 handoff 历史上踩过"1 个 Sandbox 对 2 个 Pod / 幽灵 Pod"的坑。
- **声明式 NetworkPolicy 而非隐式 firewalling**：Template 级别声明 Managed/Unmanaged。Managed 默认就是严格 deny——明确把"AI Agent 跑用户输入的代码必须默认禁掉内网访问 + 云元数据访问"做成系统约束，而不是文档里的最佳实践。
- **PVC 不被 Sandbox 标为 owned**：Sandbox 删除时 PVC 不级联回收，保留用户数据——"数据生命周期 ≠ 工作负载生命周期"做成了第一类设计。代价是孤儿 PVC 需要外部清理策略。
- **绝对时间 ShutdownTime，无心跳续约**：expiry 是声明式的，客户端要长寿命就主动 patch shutdownTime。controller 只做"现在 ≥ shutdownTime 就清理"的判断。避免 server 端维护"上次心跳时间"的状态，但代价是客户端要负责续期。
- **CRD validation 而非 webhook**：没有 admission webhook，全靠 OpenAPI v3 schema validation + controller 幂等 reconcile。少一个组件 = 少一个故障点——典型 SIG-style 的"简单优先"。

关键组件深入解读

SandboxReconciler 的三阶段子资源 reconcile

Reconcile() 主流程的工程亮点不在于复杂，而在于幂等三步永远按同一顺序：reconcilePVCs → reconcilePod → reconcileService。每一步内部都是 "Get → Create or Adopt or Validate ownership → Patch labels/annotations" 的三步式幂等。Service 一定是 headless (ClusterIP: None)，selector 用 agents.x-k8s.io/sandbox-name-hash=<hash> (hash 防止 sandbox 名过长触碰 K8s label 63 字符上限)。这导致每个 Sandbox 自动有 DNS A record: {sandbox}.{ns}.svc.{clusterDomain}——成为 Sandbox "稳定身份"承诺的实现基础。

requeueAfter = max(timeLeft, 2s) 是个被反复琢磨的小细节：既避免热循环打爆 apiserver，又保证即将过期的 Sandbox 准时被清理，不依赖 informer event。

SandboxClaimReconciler 的 adoption 协议

这是 extensions 子系统最有意思的代码，~60 行实现了 "WarmPool → Claim 的所有权热迁移"：

1. 从 SimpleSandboxQueue 拿到候选 Sandbox 名（已 Ready 的）；
2. 乐观锁 patch：用 OptimisticLockErrorMsg resourceVersion 保护，避免两个并发 Claim 抢同一个 Sandbox；
3. 清掉 WarmPool 标识标签；
4. OwnerReferences 置换：移除 WarmPool 的 controller ref，加 Claim 的 controller ref——后续删 WarmPool 不会再级联删这个 Sandbox；

5. Claim status 记录 AdoptedSandboxName，下次 reconcile 走 short-circuit。

关键不变量是 4 + 5 必须原子（同一个 patch 请求里）。这就是 sandboxclaim_pod_exclusivity_test.go 守护的不变量。

与同类对比

维度	agent-sandbox	HiClaw	普通 K8s Deployment
抽象层级	基础设施（Sandbox 原语）	应用层（Manager + Workers 协作）	基础设施（无状态副本）
单实例 + 稳定身份	✅（核心设计目标）	⚠️（Worker 之间通过 Matrix 房间）	❌
持久存储	✅	✅（MinIO + PVC）	❌
多 Agent 编排	❌（单 sandbox，多 sandbox 靠上层）	✅（Team/Manager CRD）	❌
WarmPool 预热	✅	❌	❌
隔离机制	委托 K8s（gVisor/Kata 任选）	Higress 网关 + 凭据隔离	取决于 runtimeClass
治理	K8s SIG Apps 官方孵化	阿里 Higress 团队开源	K8s core

定位结论：agent-sandbox 与 [HiClaw](#) 互补不竞争。前者提供"安全运行 1 个有状态 Agent 容器"的基础设施；后者提供"管 N 个 Agent 协作 + IM 平面 + 凭据网关"的应用层。理论上 HiClaw 的 Worker runtime 完全可以跑在 agent-sandbox 之上——一个用 Sandbox 做隔离底座，一个用 Worker CRD 做协作上层。这种分层正是云原生生态的健康样态。

相关页面

- [HiClaw](#) — 应用层多 Agent 协作平台（与本项目互补）
- k8s-operator — 本项目所遵循的 controller pattern
- declarative-agent-management — 用 K8s CRD 声明式管理 AI Agent 的方法论
- [agent-credential-isolation](#) — 凭据零暴露设计模式（HiClaw 用，本项目不直接处理）
- ai-agent-plugin-patterns — AI Agent 外挂的 9 条设计原则

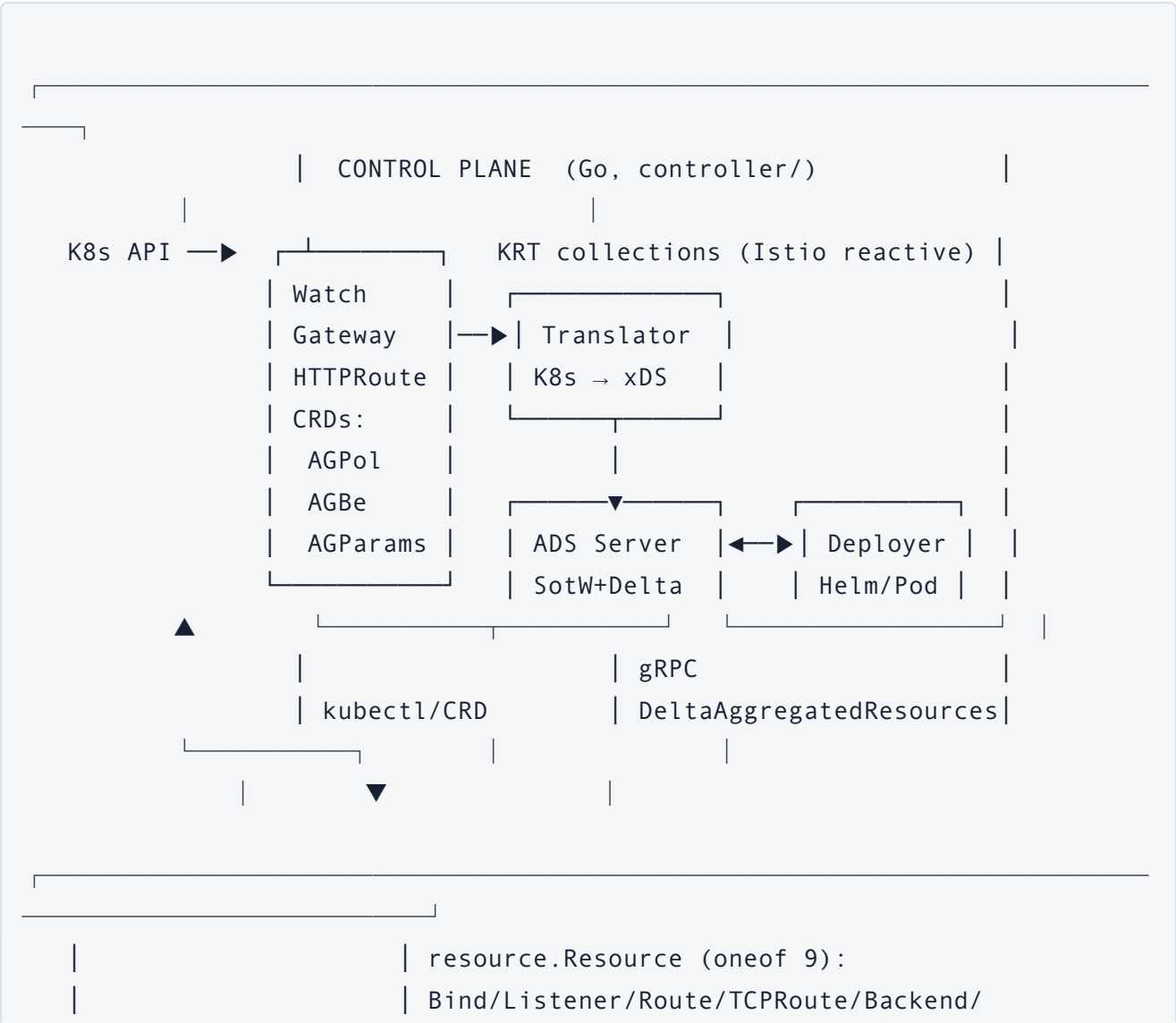
agentgateway 架构与设计思路分析

原文: raw/agentgateway-architecture-analysis.md · 仓库: [agentgateway/agentgateway](#) · 分析版本 v1.2.0-alpha.2+24 (HEAD 9ca3e04)

一句话定位

agentgateway 是面向 AI 流量的 L7 代理：把 LLM API（OpenAI / Anthropic / Gemini / Bedrock / Vertex / Azure / Copilot）、MCP 工具调用、A2A Agent 间通信 统一在一个 Rust 数据面里，用 Gateway API CRD + xDS 做声明式控制面分发。本质上是「Istio 的控制面骨架 + 为 AI 协议特化的 Rust 数据面」的组合——把 mesh 经过验证的反应式配置分发、KRT 反应式表、HBONE mTLS 隧道全套基建搬来做 AI Gateway。

核心架构图





模块分层

Workspace 顶层

模块	语言	角色
crates/agentgateway-app/	Rust	薄壳二进制（信号 / 日志 / panic hook）
crates/agentgateway/	Rust	数据面主库，三协议网关都在此
crates/xds/	Rust	xDS 客户端 + proto 绑定
crates/hbone/	Rust	Istio HBONE（mTLS over HTTP/2 CONNECT）传输
crates/celx/ + crates/cel-fork/	Rust	CEL 策略求值（fork 加 HTTP 集成）
controller/	Go	controller-runtime + ADS server + Helm deployer
api/	Go	resource.proto 的 Go 绑定
ui/	TypeScript	Next.js 管理 UI

数据面分层（crates/agentgateway/src/）

层	模块	职责
L1 配置	control/ config.rs	本地 yaml + xDS client + outbound client
L2 状态	state_manager.rs store/	反应式 Store 聚合 binds/discovery/ads
L3 代理	proxy/gateway.rs proxy/httpproxy.rs	TCP accept → TLS → route → upstream
L4 HTTP	http/	filter chain / retry / timeout / transform
L5 协议	llm/ mcp/ a2a/	三个 AI 协议网关
L6 横切	cel/ telemetry/ transport/	策略求值 / 可观测 / HBONE 隧道
L7 管理	management/ ui.rs	管理 HTTP / 健康 / UI 静态资源

自家 CRD

CRD	短名	用途
AgentgatewayPolicy	agpol	CEL 策略容器，挂在 Gateway / Listener / Route 上
AgentgatewayBackend	agbe	后端配置：AI / MCP / A2A / Static
AgentgatewayParameters	—	GatewayClass 级 Pod 模板参数

上游 Gateway API 直接复用：Gateway HTTPRoute GRPCRoute TCPRoute TLSRoute GatewayClass ListenerSet InferencePool。

关键数据流

一次 LLM 请求的端到端数据流





▼ 响应 (流式 SSE / chunked)

| provider.transform_response() |

| — 反向 schema 转换 → 统一 OpenAI 格式 |

| — guardrails (post-flight) |

| — telemetry: tokens_in / tokens_out / cost / latency |

| — access log + OTLP span |

▼

Client receives OpenAI-compatible SSE stream

控制面 → 数据面 配置分发

kubectl apply Gateway/HTTPRoute/agpol/agbe.yaml

▼

controller-runtime watch → KRT collection mutate

▼

Translator: K8s objs → resource.Resource oneof

├ Bind (TCP listen addr)

├ Listener (TLS / SNI / protocol)

├ Route (HTTP route + filters)

├ TCPRoute

├ Backend (AI / MCP / A2A / Service / Static)

├ Policy (CEL IR + transform + retry + ratelimit)

├ RouteGroup

└ Workload/Service (Istio Address 兼容)

▼

ADS Server (gRPC, controller/pkg/xds/server.go)

DeltaAggregatedDiscoveryService

▼ Delta xDS (增量)

agentgateway/control/AdsClient


```

    ▼ apply
state_manager + Stores { binds, discovery }
|
    ▼ 触发
proxy/gateway 重建 Listener / Route 索引

```

xDS 资源类型只有两种：

- `istio.workload.Address` —— 兼容 Istio ambient mesh 的 workload/service 发现
- `agentgateway.dev.resource.Resource` —— 主资源 oneof (9 个变体覆盖整个数据面状态)

只用 Delta ADS，单连接复用，不每种资源一个 stream。

一次 MCP 工具调用

```

MCP Client (Cursor / Claude Desktop / OpenAI Responses API)
| JSON-RPC over stdio | HTTP Streamable | SSE
▼
mcp/router.rs — 路由层根据 session id 找到对应的上游
|
▼
mcp/handler.rs — 工具列表 federation (merge_tools)
|   |
|   └─ 多个 upstream MCP server 的 tools/list 合并
|       + CEL RBAC 过滤 (mcp/rbac.rs)
|
▼ tools/call
mcp/upstream/{stdio.rs | streamablehttp.rs | sse.rs | openapi.rs}
|
▼
Upstream MCP server / OpenAPI endpoint

```

下游 transport: HTTP Streamable / SSE / stdio。上游 transport: 同三种 + `openapi.rs` 把任何 OpenAPI spec 自动桥接成 MCP tools (无需上游侧改造)。federation 通过 `handler.rs::merge_tools()` 合并多个 MCP server 的工具集，命名空间用 `_` 前缀避冲突，CEL RBAC 按 client identity 过滤可见性。

A2A (Agent-to-Agent)

```

Agent Client
| HTTP GET /.well-known/agent-card.json
▼
a2a/mod.rs — 拦截 Agent Card 发现请求

```

- | 一 透传上游，但 *重写响应里的 URL*
- | 把 upstream 真实地址换成 gateway 自身地址
- | 一 之后所有 Agent 调用都走向 gateway，自然拿到 policy/observability

A2A 是三协议里最轻的——核心就 URL 重写。Google A2A 协议把入口放在 Agent Card well-known endpoint，gateway 控制了发现，就控制了所有后续调用。

设计决策与哲学

- 沿用 Istio 的基建做 AI Gateway: Gateway API + KRT collections + xDS + HBONE 全套复用，控制面是 Istio 的影子结构。开发团队 (Solo.io / Istio 系) 选择不重新发明轮子，把 mesh 经过验证的反应式配置分发 + 反应式表建模 + mTLS 隧道搬来，只把数据面替换成「专为 AI 协议优化的 Rust 代理」。Day 1 即获得完整 mesh 兼容性 + 控制面/数据面可独立替换的解耦能力。
- 三协议共享同一条 pipeline: LLM / MCP / A2A 在数据面共享 Route / Policy / Backend / CEL 求值的同一条管道，差别只在 Backend.kind 决定走哪个 provider 适配器。带来单一策略语言 + 统一观测 + 统一 mTLS。
- CEL 作为策略 IR: 所有授权 / 转换 / 速率限制都被编译成 CEL 表达式存到 Policy.spec.expression，运行时由 Rust 数据面解释执行。controller 不预编译，避免 Go/Rust CEL 实现差异。用 cel-fork + celx 是因为原生 cel-rust 缺与 HTTP 请求对象的深度集成。
- 凭据托管 prompt injection: 与 [HiClaw](#) 的 Higress 凭据托管哲学一致——LLM provider 真凭据 (OpenAI key / AWS IAM / GCP service account) 放在 backend secret，gateway 在请求时注入，Agent / 业务端只面对 gateway 自己的认证 (JWT / API key / mTLS)。Agent 即使被注入也偷不到真凭据。详见 [agent-credential-isolation](#)。
- 控制面 / 数据面通过 proto 解耦: Go controller + Rust 数据面只用 resource.Resource proto 通信，可独立演进。未来能换掉任一端 (Crossplane 直产 xDS / envoy 实现 agentgateway proto 扩展)。
- 两个 tokio runtime 拆分热路径: main runtime 跑控制面 (admin HTTP / xDS / reload)，worker runtime 跑数据面 accept，避免控制面任务抢占 worker 影响 P99。
- OpenAI 兼容 API 做统一前门: 客户端代码零改造接 7 个 LLM provider; 非 OpenAI 特性 (Gemini 多模态、Bedrock 私有模型) 通过扩展字段暴露。

关键组件深入: LLM Provider 适配器

crates/agentgateway/src/llm/ 是数据面最厚的协议适配层。统一抽象骨架:

```
enum AIProvider {
    OpenAI, Anthropic, Gemini, Bedrock, Vertex, Azure, Copilot,
}

trait Provider {
    async fn transform_request(&self, req: OpenAIRequest) ->
    ProviderRequest;
```

```

    async fn transform_response(&self, resp: ProviderResponse) ->
OpenAIResponse;
    async fn stream_chunk(&self, chunk: Bytes) -> OpenAIChunk;
}

```

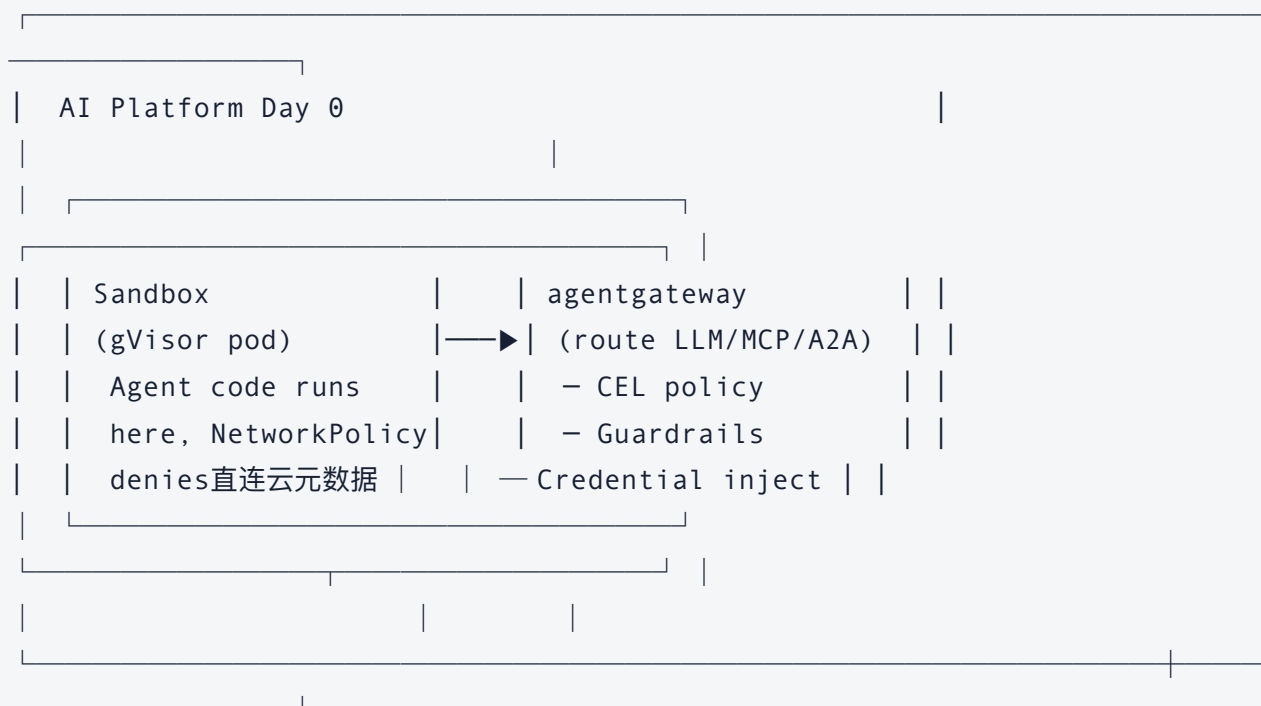
外部统一暴露 OpenAI 兼容 API (/v1/chat/completions / /v1/embeddings)，内部 dispatch 到 provider 双向翻译。

Guardrails —— LLM 流量特有的内容审查，pre-flight / post-flight 双向，支持 regex / OpenAI Moderation / AWS Bedrock Guardrails / Google Model Armor / Azure Content Safety / 自定义 webhook，每 backend 按声明顺序串成 chain。

负载均衡 —— 跨副本走 two-random-choice (tail-latency 比 round-robin 稳)；跨 model alias 走 weighted random。

与 [HiClaw](#) / [agent-sandbox](#) 的关系

- 跟 [HiClaw](#) 是不同层次互补：HiClaw 在应用层做 Agent 协作 (Matrix IM + Worker CRD)，agentgateway 在基础设施层做 AI 流量代理。HiClaw 的 Worker 跑用户代码访问 LLM / MCP 时，理论上可以让 agentgateway 当出口代理，由 agentgateway 接管凭据 / 审计 / guardrails。
- 跟 [agent-sandbox](#) 拼成 AI 平台底盘：agent-sandbox 做 Agent 运行时隔离 (Pod + gVisor / Kata + NetworkPolicy)，agentgateway 做 Agent 出口流量治理 (LLM / MCP / A2A)。两者并集 = AI 工作负载完整的"运行 + 通信"治理面。NetworkPolicy 限制 Sandbox Pod 出口只能走 agentgateway，强制所有 AI 流量过 policy + guardrails + 审计。



相关页面

- [agentgateway](#) —— 项目主页
- [mcp](#) —— MCP 协议（agentgateway MCP gateway 的协议基础）
- [gateway-api](#) —— K8s Gateway API（agentgateway 控制面 API 基础）
- [xds](#) —— xDS 协议（控制面 → 数据面分发）
- [hbone](#) —— Istio HBONE 隧道（agentgateway 跟 mesh 互操作）
- [cel](#) —— Common Expression Language（agentgateway 策略 IR）
- [istio](#) —— Istio（agentgateway 控制面骨架来源）
- [HiClaw](#) —— 应用层 Agent 协作平台（互补关系）
- [agent-sandbox](#) —— K8s SIG 的 AI Agent 运行时隔离（互补关系）
- [ai-agent-plugin-patterns](#) —— Agent 外挂设计原则
- [agent-credential-isolation](#) —— Agent 凭据零暴露模式

AI Infra Learning 中文 Star 项目清单整理

来源：GitHub Stars list [googs1025/lists/ai-infra-learning-中文](#)，描述为“中文 AI infra/LLM 教程与 awesome list”。本次通过 GitHub GraphQL API 于 2026-06-12 复核，列表共 32 个仓库。

这组项目不是单一技术栈，而是一条中文 AI Infra 学习路径：从 LLM 基础、AI 系统/基础设施、CUDA kernel、推理优化、模型部署、Agent 应用，到面试/项目材料。它更适合整理成学习路线与项目地图，而不是逐个都做架构深挖。

项目清单

项目	方向	stars	语言	当前判断
Infrasys-AI/AISystem	AI 系统全栈	16923	Jupyter Notebook	AI infra 底层体系核心教材，优先级高
Infrasys-AI/AIInfra	AI 基础设施	7319	Jupyter Notebook	AI Infra 中文主线材料，优先级高
cr7258/ai-infra-learning	学习会议/资料	478	-	中文学习社群材料，适合做入口索引
CalvinXKY/InfraTech	AI Infra 代码练习	2555	Jupyter Notebook	PyTorch/vLLM/SGLang/性能加速练习，适合实践线
liguodongiot/llm-action	LLM 工程化实战	24508	HTML	大模型工程化与应用落地中文材料
InftyAI/Awesome-LLMOps	LLMOps 工具清单	238	Python	LLMOps 工具索引，适合作补充清单
xlite-dev/Awesome-LLM-Inference	LLM/VLM 推理论文代码	5281	Python	推理优化论文与代码索引，关联 llm-inference
xlite-dev/LeetCUDA	CUDA kernel 练习	11241	Cuda	CUDA/Tensor Core/HGEMM/FA-2 实战，优先级高
Tony-Tan/CUDA_Freshman	CUDA 入门	2759	Cuda	CUDA 新手入门材料
a-hamdi/GPU	GPU kernel 100 days	601	Cuda	kernel 训练打卡，适合补充练习
			-	CUDA 官方编程指南中文翻译

项目	方向	stars	语言	当前判断
HeKun-NVIDIA/CUDA-Programming-Guide-in-Chinese	CUDA 中文指南	1985		
datawhalechina/happy-llm	LLM 基础	31123	Jupyter Notebook	从零开始构建大模型，适合基础入口
datawhalechina/self-llm	开源模型微调/部署	30859	Jupyter Notebook	中文开源模型食用指南，实践价值高
AiHubCN/Awesome-Chinese-LLM	中文 LLM 清单	22611	-	中文大模型生态索引
Hannibal046/Awesome-LLM	LLM 总清单	26922	-	通用 LLM awesome list
datawhalechina/hello-agents	Agent 教程	58595	Python	从零开始构建智能体，Agent 学习主线
panaversity/learn-agent-ai	Agent-native cloud	4220	Jupyter Notebook	Dapr/MCP/A2A/K8s/ Knowledge Graph 组合路线
luzhenqian/ai-coding-lab	AI 编程实战	137	TypeScript	Vibe Coding → Agent/RAG 的实践教程
libukai/awesome-agent-skills	Agent Skills	4677	Python	Agent Skills 生态中文入口，关联 agent-skills-plugin-system-map
RKiding/Awesome-finance-skills	Finance Agent Skills	2483	Python	垂直领域 Skills 示例
nanocoai/nanoclave	Containerized personal agent	29817	TypeScript	安全容器化 OpenClaw 替代项目，值得后续单独看
hhaAndroid/awesome-mm-chat	多模态 chat 清单	283	Python	多模态 chat 补充索引
WeThinkIn/AIGC-Interview-Book	AIGC/LLM/Agent 面试	3913	-	面试知识库
bcefgjhj/ai-agent-interview-guide	Agent 面试	1231	Python	Agent 面试/项目/简历材料
wdndev/llm_interview_note	LLM 面试笔记		HTML	LLM 算法/应用工程师面试题

项目	方向	stars	语言	当前判断
		14461		
antgroup/agent-ai-landscape	Agentic landscape	472	TypeScript	数据驱动 agentic landscape
academicpages/academicpages.github.io	个人主页模板	17130	SCSS	与 AI infra 主线弱相关，可忽略或放个人品牌
cr7258/ai-briefing	AI briefing	2	Dart	个人/小项目，暂不优先
LincanLi-X/Awesome-Data-Centric-Autonomous-Driving	自动驾驶数据中心清单	179	-	领域专题，非主线
LMD0311/Awesome-World-Model	World Model 清单	2106	-	具身/自动驾驶方向补充
mainpropath/AI-java	Java AI 示例	290	Java	Java 应用学习材料，非 AI infra 核心
mainpropath/AI-SmartFuse-Framework	Java AI framework	288	Java	Java 应用框架，非核心

初步分层

1. AI Infra 总论: AISystem、AIInfra、ai-infra-learning、InfraTech、llm-action。
2. CUDA / GPU kernel: LeetCUDA、CUDA_Freshman、GPU、CUDA-Programming-Guide-in-Chinese。
3. LLM 推理优化: Awesome-LLM-Inference，可与 [paged-attention](#)、[radix-attention](#)、[kv-cache-offload](#)、[llm-inference](#) 连接。
4. 模型基础 / 微调 / 部署: happy-llm、self-llm、Awesome-Chinese-LLM、Awesome-LLM。
5. Agent / Skills / 应用开发: hello-agents、learn-agent-ai、ai-coding-lab、awesome-agent-skills、Awesome-finance-skills、nanoclave。
6. 面试 / 职业化: AIGC-Interview-Book、ai-agent-interview-guide、llm_interview_note。
7. 领域扩展: 自动驾驶、world model、多模态 chat、Java AI 应用。

后续建议

优先把这份 list 作为“学习路径源材料”连接到 [ai-infra-learning-cn-map](#)。真正值得后续单独摄入源码/架构的项目是: Infrasys-AI/AISystem、Infrasys-AI/AIInfra、CalvinXKY/InfraTech、xlite-dev/LeetCUDA、xlite-dev/Awesome-LLM-Inference、datawhalechina/self-llm、datawhalechina/hello-agents、nanocoai/nanoclave。

AlBrix

AlBrix 是 vLLM 生态的可插拔 GenAI inference infrastructure，提供 LLM Gateway/Router、LoRA 管理、LLM app-tailored autoscaler、Unified AI Runtime、distributed inference、distributed KV cache、异构 GPU serving 和 GPU 硬件故障检测。

解决的问题

企业使用 vLLM fleet 时，真正的难点不只在 engine：还需要 adapter 管理、跨副本路由、KV 复用、动态扩缩、成本优化和 GPU 故障处理。AlBrix 把这些能力拆成可组合组件。

核心边界

AlBrix 更贴近 vLLM 生态和企业控制面，不是新的推理 engine，也不是 Kubernetes 通用模型 API。它的 KV offloading 可以作为独立组件使用，路由和 autoscaling 也可以按需组合。

适用场景

适合已有 vLLM 技术栈、希望逐步加入 LoRA、KV、Gateway、异构 GPU 和企业运维能力的团队。需要 Gateway API/InferencePool 标准化时比较 [llm-d](#)；需要完整 P/D/KV runtime 时比较 [dynamo](#)。

详见 [src-aibrix-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

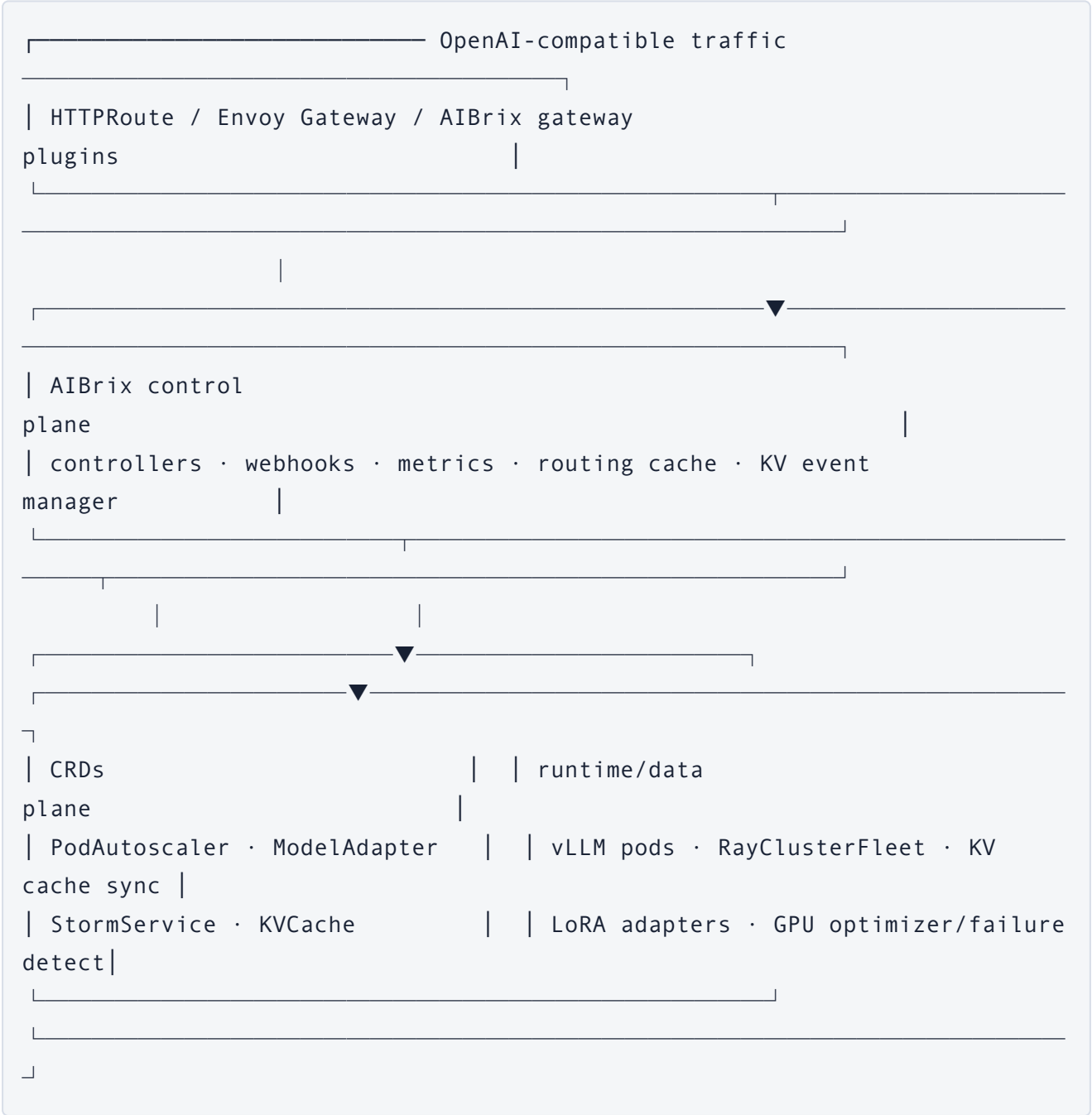
在 M4 模块地图中的位置

AlBrix 位于 Kubernetes 推理控制面层，覆盖 routing、autoscaling、模型与 adapter 生命周期、runtime，以及 KV 和多角色编排。职责边界见 [llm-inference-serving-project-map](#)，与 engine 及其他 serving 层的组合选择见 [llm-serving-engine-selection-map](#)。

AlBrix 架构与设计思路分析

vllm-project/aibrix 是 vLLM 生态的 Kubernetes GenAI inference infrastructure，而不是推理引擎。仓库包含 controller-manager、gateway plugins、KV cache watcher、CRD、webhook、metrics、cache、PodAutoscaler、ModelAdapter/RayCluster/Roleset 等模块，目标是把 vLLM 大规模部署、路由、扩缩容、LoRA、分布式推理和 KV cache 管理做成云原生控制面。

核心架构图



模块分层

层/目录	责任
cmd/controllers, pkg/controller/**	controller-runtime 控制面，PodAutoscaler 等 reconciler。
api/autoscaling, api/orchestration, api/model	CRD API: PodAutoscaler、StormService、RayClusterFleet、Roleset、KVCache、ModelAdapter。
pkg/plugins/gateway/**	gateway/routing 插件，包含 P/D disaggregation 路由逻辑。
pkg/cache, pkg/kvevent	KV event manager、cache snapshot、pod/model cache、ZMQ 事件。

关键数据流

1. 用户通过 Gateway 访问模型，gateway plugin 根据 model labels、pod metrics、KV/cache 状态选择后端。
2. controller 监听 AIBrix CRD 和 K8s workload，创建/调整 vLLM/RayCluster/RoleSet 等资源。
3. PodAutoscaler 支持 HPA/KPA/APA，不同 metric source 包括 pod/resource/custom/external/domain。

设计决策

- AIBrix 把 vLLM 从单 engine 运维提升到 K8s control plane。
- 它比 llm-d 更像一体化发行：gateway、autoscaler、LoRA、KV cache、GPU failure detection 都在一个 repo。
- CRD + controller-runtime 是主线，Gateway API/Envoy 是流量入口。

对比定位

和 [dynamo](#) 相比，AIBrix 更 Kubernetes/controller-first；Dynamo 更强调推理系统运行时和分离式 P/D/KV 数据路径。和 llm-d 相比，AIBrix 更一体化，llm-d 更生态拆分（router、kv-cache、guides）。和 [skypilot](#) 相比，AIBrix 管集群内 inference，不管多云资源采购。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

Batch Inference

Batch inference 指把大量推理请求作为离线或异步任务提交，系统负责排队、持久化、执行、重试、取消、进度追踪和结果归档。它和在线 serving 共用模型服务能力，但关注点不同：在线请求优化 tail latency，batch inference 优化吞吐、可靠性、成本和可审计输出。

和在线 serving 的区别

维度	Batch inference	在线 inference
入口	文件 / job / JSONL / Batch API	HTTP/SSE/gRPC 单请求
用户体验	异步提交，稍后查询结果	同步或 streaming 返回
状态	job status、progress、output/error file	request state、stream state
调度目标	吞吐、成本、重试、可恢复	延迟、排队、公平性、cache hit
代表项目	llm-d-batch-gateway	llm-d , gateway-api-inference-extension , ai-gateway

典型架构



选型提示

如果用户关心“一个请求尽快返回”，看 [inference-routing](#)、[llm-inference](#) 和 gateway；如果用户关心“几十万条请求稳定跑完、失败可追踪、结果可下载”，就需要 [batch-inference](#) 这一层。[llm-d-batch-gateway](#) 的价值正在于把 batch job control plane 和下游 [llm-d](#) serving fleet 解耦。

Container Device Interface

CDI (Container Device Interface) 用标准化 spec 描述容器需要的设备节点、mount、env、hooks 等 edits，由兼容的 container runtime 按选定的 CDI device ID 应用配置。它是设备注入接口，不负责 scheduler 选节点或 allocator 选设备。

在 M5-C Device / GPU 地图中的位置

设备工具链或 driver 生成/维护 CDI spec；[k8s-device-plugin](#) 可通过配置的 CDI strategy 返回设备标识，[dra-driver-nvidia-gpu](#) 可在 Prepare 后返回 CDI devices，kubelet 再把相应配置交给 runtime 消费。[gpu-operator](#) 管理受管 driver/toolkit 等组件的部署，hami 可在支持的分配路径交接设备与隔离配置，两者都不能把 CDI 当作独立分配决策器。

两条注入路径见 [k8s-gpu-device-stack](#)，Prepare/CDI/checkpoint 与共享 Claim 回收边界见 [kubernetes-dra-design-deep-dive](#)。CDI 注入成功也不证明设备健康或共享隔离策略已满足。

在 GPU 栈中的位置

[k8s-device-plugin](#) 支持通过 CDI annotations/device list strategy 把 NVIDIA 设备交给容器；[dra-driver-nvidia-gpu](#) 也在 Prepare 阶段生成或引用 CDI 配置。CDI 是传统 device plugin 与 DRA 都会复用的设备注入抽象。

选型提示

如果你在比较 envvar、volume-mounts、CDI annotations，优先理解 CDI：它更适合复杂设备、hooks 和 runtime edits 的标准化。

Disaggregated Serving

把 LLM 推理的 prefill（首 token 计算）与 decode（后续 token 流式生成）拆到独立的 GPU 池中分别扩缩的部署模式。[NVIDIA Dynamo](#) 将 P/D 分离作为一等的可选部署模式，同时支持聚合式 worker 和部署；具体拓扑需根据工作负载、SLO 与硬件条件选择。相关能力通过 [vllm](#)、[sglang](#) 和 TensorRT-LLM 各自的集成或 connector 接入，成熟度和功能覆盖应以所选 release 与 backend 为准。

在 M4 中的位置

P/D 分离同时改变 [llm-inference-serving-project-map](#) 的 D2 请求路径和 D4 KV 生命周期：Prefill 与 Decode 独立扩缩，交接时按明确的 backend contract 传递元数据与 KV。具体组合需要同时检查路由协调、engine 接口和传输后端。

为什么要分离

Prefill 和 decode 的负载特征完全不同：

阶段	计算密度	内存压力	并行度	延迟敏感点
Prefill	高 ($O(n^2)$ attention, n =输入 token 数)	中（一次性算完）	大 batch 友好	TTFT (time to first token)
Decode	低（每步 $O(n)$ ）	高（KV cache 持续累积）	小 batch 内存瓶颈	ITL (inter-token latency)

聚合模式（同一 GPU 同时跑 prefill 和 decode）会让两阶段互相干扰：

- 大 prefill 来一发，正在 decode 的请求被挤出，ITL 抖动
- 小 prefill 来很多发，GPU 算力没用满
- 显存按 max KV 预留，decode 用不到的 prefill 内存被浪费

分离模式让 prefill 池和 decode 池独立扩缩：prefill 用更多算力（H100 / B200），decode 用更大显存（H200 / GB200），各自 batch 策略也独立调优。

关键工程问题：KV 怎么传

Prefill 完算出的 KV 必须传到 decode 池才能续写：

[Prefill Worker]	[Decode Worker]
① 算 KV cache	
② 返回 disaggregated_params	



主流传输方案：

- NIXL (NVIDIA Inference Xfer Library)：统一抽象 GPUDirect RDMA / NVLink / NVMe / 对象存储
- GPUDirect-RDMA: GPU 之间直接 RDMA，绕过 CPU
- MooncakeConnector (vLLM 实现)：基于 Mooncake 项目的 P/D 解耦传输

聚合式与分离式时序

聚合式: Client → GPU Instance — Prefill — Decode — Stream

分离式: Client → Router → Prefill Pool — KV Transfer → Decode Pool → Stream

└── queue / health / SLO / autoscaling ──┘

KV Transfer 失败恢复

Prefill 完成 → 登记 KV metadata → 网络传输



Dynamo 中的实现

Dynamo 把分离式做成第一类公民：

1. PrefillRouter (独立组件) 选 prefill worker → 跑 prefill → 返回 disaggregated_params
2. PrefillRouter 选 decode worker → 把 KV 转移元数据传过去
3. KV blocks 经 NIXL 跨 worker 传输
4. Decode worker 开始流式生成

后端集成范围包括 [sglang](#)、TensorRT-LLM 与 [vllm](#) (其中 vLLM 可使用 MooncakeConnector 等 connector)；可用 connector、成熟度与功能覆盖需按目标 release/backend 验证。

收益与代价

收益：

- 各阶段独立扩缩，资源利用率更高
- 大 prefill 不再撞挂 decode (ITL 抖动消除)
- 异构硬件搭配 (prefill 用算力强卡，decode 用显存大卡)

代价：

- 多一跳网络传输 (KV blocks)，需要高速互联 (NVLink/RDMA) 才划算
- 路由复杂度上升 (要选两次 worker)
- 故障域增加 (prefill worker 死了，decode worker 收不到 KV)

相关页面

- 旗舰实现：[dynamo](#)、[src-dynamo-architecture](#)
- 兼容 backend：[vllm](#)、[sglang](#)
- 协同概念：[kv-cache-offload](#) (KV 在多级存储间流动)、[radix-attention](#) (KV-aware 路由配套)
- 上位概念：[llm-inference](#)

DRA Driver for NVIDIA GPUs

NVIDIA GPU 的 Kubernetes Dynamic Resource Allocation driver，围绕 ResourceClaim、ResourceSlice、NodePrepareResources、ComputeDomain/Multi-Node NVLink、动态 MIG/VFIO 配置展开。详见 [src-dra-driver-nvidia-gpu-architecture](#)。

证据说明：2026-09-27 通过 NVIDIA 仓库入口查询的官方默认分支快照为 [495bf4c59b94](#)，链接采用 GitHub 返回的规范仓库，commit 不代表 release。[src-dra-driver-nvidia-gpu-architecture](#) 保留 2026-06-12 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，阶段与生命周期见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

driver 提供 DeviceClass 等安装配置、发布 ResourceSlice 库存，并通过 NodePrepareResources/NodeUnprepareResources 执行设备配置与 [cdi](#) 交接；GPU 与 ComputeDomain 分别处理本机设备和 Multi-Node NVLink 相关生命周期。这不同于 [k8s-device-plugin](#) 的传统 extended-resource Allocate 路径，也不等于 node-feature-discovery 的节点能力发现。

厂商支持矩阵必须与 Kubernetes feature stage 分开：锚定仓库 README 对独立 GPU plugin 与 ComputeDomain 的支持说明，不能直接套用到 [gpu-operator](#) 26.7 所管理版本。基础 DRA stable 不保证所有动态 MIG/VFIO/sharing 功能同样稳定。

架构边界

它采用 Kubernetes DRA 设备资源 API，不是传统 device plugin API 的小补丁。与 [k8s-device-plugin](#) 相比，它把资源声明、配置和调度语义前移到 Claim 模型；hami 也有 DRA 接入，比较时应看厂商设备配置能力、分配路径与容器隔离，而非是否使用 DRA。

选型判断

适合需要 ResourceClaim/ResourceSlice、动态设备配置或 ComputeDomain 的平台团队；部署应按所选 Kubernetes/driver release 核对 API、feature gates 和硬件支持。

只需要 GPU extended resource 暴露时可评估 [k8s-device-plugin](#) 和 [gpu-operator](#)；选择 DRA 时须独立验证分配、节点准备与回收全链路。

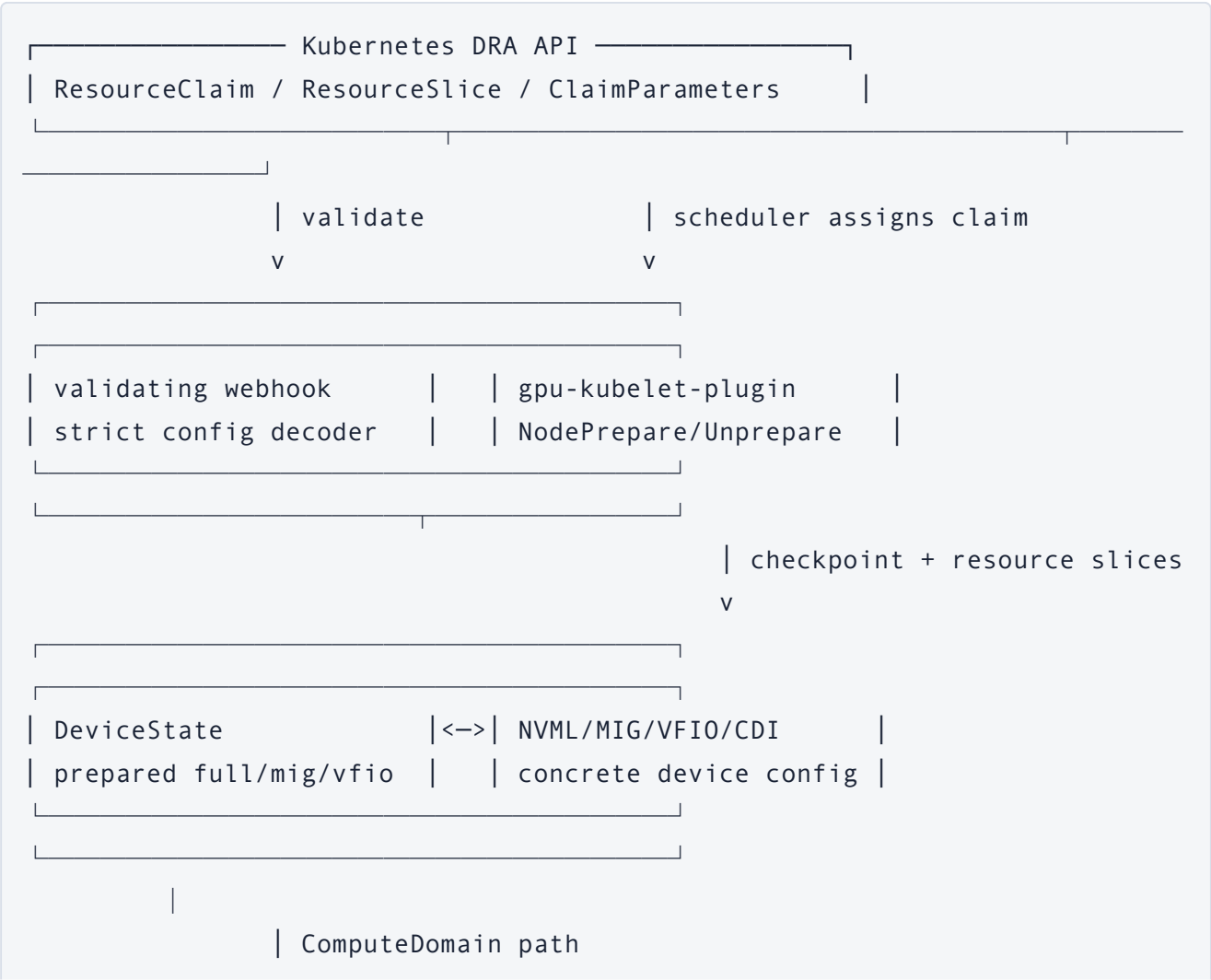
DRA Driver for NVIDIA GPUs 架构与设计思路分析

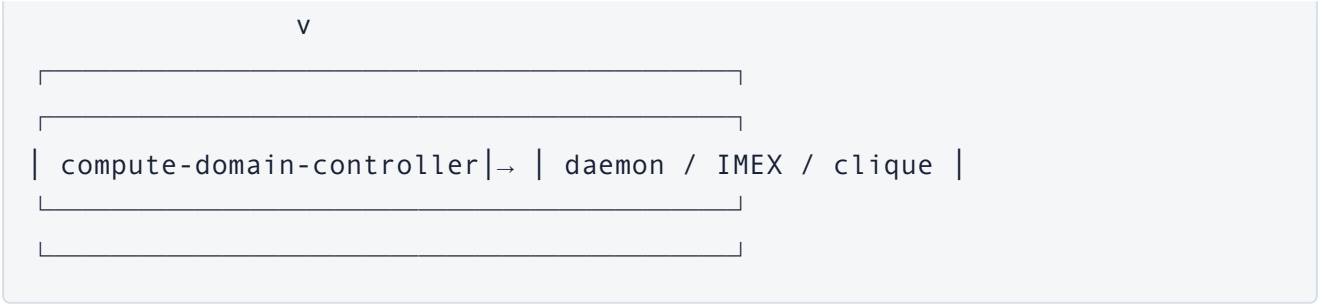
原文: raw/dra-driver-nvidia-gpu-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/dra-driver-nvidia-gpu> · 分析版本 HEAD 749a743 (2026-06-11)

一句话定位

NVIDIA GPU 的 Kubernetes Dynamic Resource Allocation driver，围绕 ResourceClaim/ResourceSlice、NodePrepareResources、ComputeDomain/Multi-Node NVLink 和动态 MIG/VFIO 配置展开。它代表 K8s 新一代设备分配模型，而不是传统 device plugin API 的增量补丁。这页和 [kubernetes dra cdi gpu-sharing](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图





模块分层

层 / 模块	职责
DRA kubelet plugins	实现 kubelet DRA plugin，发布 ResourceSlice，执行 Prepare/Unprepare 和 checkpoint 清理。
GPU state/config	抽象 full GPU、MIG、VFIO、sharing、checkpoint 与资源 slice。
ComputeDomain 控制面	管理 ComputeDomain/Clique 生命周期，动态渲染 daemonset 与 IMEX primitives。
API 与 webhook	定义 GpuConfig/MigDeviceConfig/ComputeDomain 等配置，并验证 ResourceClaim 参数。
部署与站点	Helm/demo/docs 展示 K8s 1.32+ DRA 使用方式。

关键数据流



设计决策与哲学

- 接受 DRA 的声明式 claim 模型：资源配置进入 ResourceClaim，而不是 Pod annotation；这把厂商调度逻辑接到 Kubernetes 1.32+ DRA 语义上。
- checkpoint 是动态设备生命周期的事实来源：NewDriver 在 DynamicMIG 下会清理未知 MIG devices，避免重启后静态/动态状态混乱。
- GPU 和 ComputeDomain 分成两个插件/控制面：ComputeDomain 面向 Multi-Node NVLink/IMEX，GPU 面向本机 device allocation，生命周期和风险边界不同。

与同类项目的架构差异

维度	DRA Driver for NVIDIA GPUs	同类 A	同类 B
定位	K8s DRA NVIDIA driver	k8s-device-plugin: beta device plugin API	HAMi: scheduler extender vGPU sharing
资源 API	ResourceClaim/ ResourceSlice	extended resource + kubelet plugin	Pod annotation + extender
高级能力	dynamic MIG/VFIO/ ComputeDomain	MIG/MPS/time-slicing/CDI	memory/core sharing + heterogeneous vendors

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [kubernetes](#)
- [dra](#)
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)

Dynamo

NVIDIA 开源的数据中心级 LLM 推理编排层。Dynamo 位于推理 engine 之上，负责入口、路由、P/D 协调、KV 传输、容量控制、Kubernetes 部署与故障连续性；不替代 SGLang、vLLM 或 TensorRT-LLM 的 GPU 执行内核。

架构边界

Frontend / Gateway EPP → KV-aware Router → backend workers 是请求路径；DGDR / Profiler / AIConfigurator → DGD / DCD → Operator 是部署路径；Planner → connector / Operator 是运行时容量路径。三条路径通过 Runtime 的 discovery、request 和 event planes 连接。

什么时候用

- 多 GPU、多节点或长上下文在线推理。
- 需要独立扩缩 prefill/decode、KV-aware routing 或高速 KV transfer。
- 需要按 TTFT/ITL/SLA 和 TCO 做容量规划与自动扩缩。
- 需要 Gateway API 入口、请求迁移、canary health check 或 topology-aware deployment。

什么时候不用

单模型、单 GPU、没有跨节点协调和 SLA 控制需求时，直接使用 [vllm](#) 或 [sglang](#) 更简单。若只需要标准 Kubernetes 模型生命周期而不需要 Dynamo 的 P/D、KV 和 Planner，应优先比较 kserve、ome 等 Operator。

与同类比较

项目	主要边界	与 Dynamo 的差异
vllm / sglang	推理 engine	更靠近单机/多 GPU 执行与 batching
llm-d	K8s inference stack、router、pool	更强调云原生入口与组件生态，Dynamo 更强调 engine 上方的端到端编排
gateway-api-inference-extension	Gateway API inference endpoint picking	可作为 Dynamo 的入口承载，不能替代完整 P/D/KV/Planner
kserve / ome	模型服务 CRD/operator	更偏声明式生命周期，Dynamo 提供更深的 serving runtime 协调

详细架构与业务问题见 [src-dynamo-architecture](#)。

在 M4 模块地图中的位置

Dynamo 围绕可插拔 engine 提供分布式 serving runtime，协调 frontend/routing、服务发现与事件、worker 角色、KV transfer 和容量规划。职责边界见 [llm-inference-serving-project-map](#)，与 engine、Kubernetes 组件的组合选择见 [llm-serving-engine-selection-map](#)。

Elastic KV Cache

Elastic KV cache 指把推理引擎可寻址的 KV 容量与当前真实占用的 GPU 物理显存解耦：引擎拥有稳定的逻辑槽位和 tensor 地址，物理页只在活跃请求使用时映射，并在 block/token 不再被运行请求或 prefix cache 引用后回收。

[kvcached](#) 是这一模式的直接实现；[vllm](#) 的 [paged-attention](#) 和 [sglang](#) 的 [radix-attention](#) 则分别提供上层 block/token 与 prefix reuse 语义。

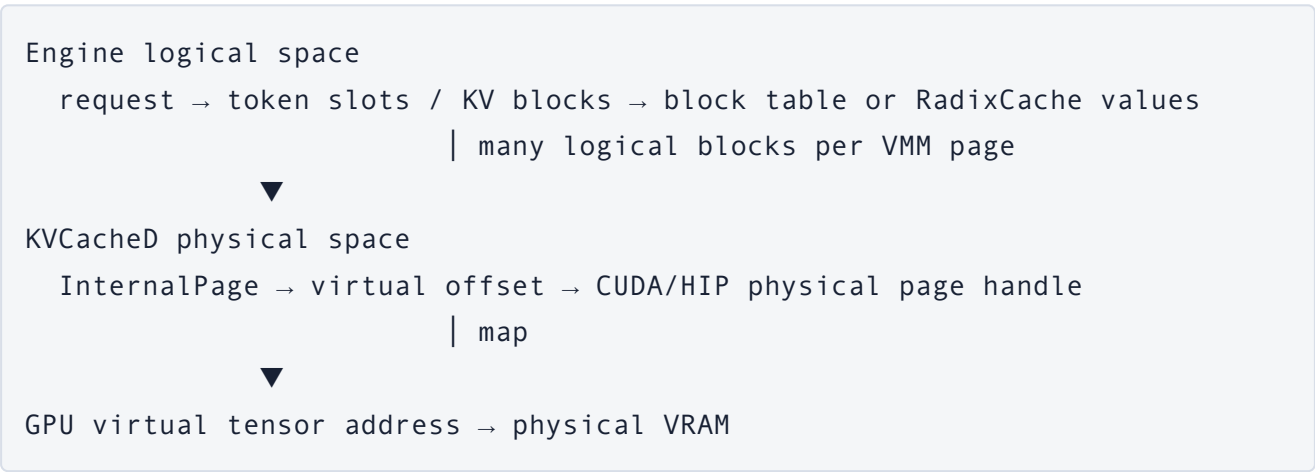
四类容量

理解弹性 KV 必须区分四个数字：

容量	含义	谁决定
Virtual capacity	engine 可以寻址的最大 KV slot/block 数	engine profile + KVCacheD tensor reservation
Logical free capacity	当前没有被 request/cache metadata 占用的 slot/block	BlockPool 或 token allocator + KVCacheManager ledger
Mapped physical capacity	当前已有真实 VRAM backing 的 VMM pages	PageAllocator/FTensorAllocator
Device physical headroom	同 GPU 所有进程竞争后的剩余 VRAM	GPU driver，全设备动态状态

`logical free > 0` 不代表下一次 allocation 一定成功，因为另一个进程可能在检查后拿走 device headroom。共享环境中的 capacity check 是 snapshot，不是 reservation。

两级分页模型



[paged-attention](#) 的“page/block”解决 sequence KV 的逻辑碎片和 block table 映射；KVCacheD 的 page 解决 GPU virtual address 与 physical allocation 的映射。它们是上下两层分页，不是重复实现。

为什么 tensor 指针可以稳定

GPU VMM 允许先 reserve 一段 virtual address，再把不同 physical allocation handle map 到其中的 page-aligned offset。KVCacheD 用该 VA 创建 torch tensor view；attention kernel 捕获和读取的是相同地址。物理页释放时重新映射 zero page，tensor 本身不销毁。

这使 engine 无需感知物理页生命周期，但引入严格约束：

- shape/stride 必须与 backend 的真实读写方式一致。
- unmap 前必须确保所有 GPU command 已结束。
- 同一 logical offset 在 TP/PP rank 上必须处于一致状态。
- block/page geometry 必须保证逻辑 block 不落入无法映射的跨页空洞。

prefix cache 与弹性的张力

完成请求后，prefix cache 继续保留 block/token index，就等价于继续引用对应 KV 数据。只要 page 上还有一个被缓存的 block，整个物理页都不能释放。因此更高 hit rate 与更强显存弹性天然冲突。

常见策略：

- unlimited：最接近原生 cache，弹性最弱。
- disabled/immediate eviction：最容易释放显存，但丢失跨请求复用。
- bounded cache：限制 cached tokens/blocks，并优先淘汰能清空完整 physical page 的 victim。

contiguous 与 per-layer layout

Layout	物理映射	优点	约束
Contiguous/ compound	一个 page 同时覆盖 多 layer/KV buffer	map 次数少、跨层 backing 一致	per-layer view 有 stride/interleave；部分 copy、PD、ROCm backend 不接受
Non- contiguous/ per-layer	每层独立 FTensor/ page	每层 region 连续，兼容 更多 backend/transfer	map handle更多，多层原子性和开销更 高

Layout 不是纯性能开关，而是 attention backend、KV copy、P/D connector 与 Mamba/hybrid state 的数据契约。

正确性模型

Elastic KV cache 需要同时维持三类不变量：

- 1. **Ledger invariant**：逻辑 block 不能重复分配或丢失，partial allocation 失败必须完整回滚。
- 2. **Mapping invariant**：一个 logical offset 的所有 layer/K/V target 必须全映射或全未映射，不能处于 partial mapping。
- 3. **Execution lifetime invariant**：GPU/worker 在最后一次读写完成前，physical page 不得释放。

为满足这些不变量，[kvcached](#) 使用 map rollback/quarantine、TP/PP transactional unmap、vLLM async worker barrier、shutdown drain 和 fail-pool 语义。

与相邻概念的区别

概念	主要解决的问题	与 Elastic KV 的关系
paged-attention	engine 内 KV block 管理和 attention addressing	上层逻辑索引
radix-attention	token-level prefix reuse	上层 cache metadata
kv-cache-offload	KV 数据跨 GPU/CPU/SSD/远端层级迁移	可组合，但数据路径和故障域不同
gpu-sharing	GPU 调度、分区、执行/显存隔离	更外层资源治理
sleep mode	释放 KV，甚至权重和 allocator state	粗粒度实例生命周期；Elastic KV 是请求级/页级

评估指标

- idle/steady/peak mapped KV bytes。
- logical blocks、mapped pages、reserved pages、page occupancy。
- allocation miss、scheduler retry/preemption、map/unmap latency。
- prefix hit rate 与 page-aware eviction 造成的命中损失。
- TTFT、ITL、throughput 和 OOM rate。
- TP/PP failure injection、shutdown 后 shared segment/socket cleanup。

相关页面

- 实现：[kvcached](#)、[src-kvcached-architecture](#)
- 关系地图：[kvcached-sglang-vllm-knowledge-system](#)
- 引擎：[vllm](#)、[sglang](#)
- 基础概念：[llm-inference](#)、[paged-attention](#)、[radix-attention](#)

Gateway API Inference Extension

Gateway API Inference Extension 是 Kubernetes Gateway API 的推理扩展，定义 InferencePool、Endpoint Picker interaction 和 inference endpoint picking 的协议/控制面边界。

解决的问题

通用 Service/负载均衡不理解模型身份、推理协议和 endpoint 级运行状态。Inference Extension 让 Gateway 在转发请求前询问模型感知的 Endpoint Picker，从而为 LLM serving 提供标准化选择接口。

核心边界

它是协议和 API 扩展，不是完整模型服务平台、推理 engine 或具体路由算法。[llm-d-router](#) 是其重要生产化实现之一；kserve、[kthema](#) 等平台可以在这条标准边界上集成。

适用场景

适合希望复用 Gateway API、把 endpoint picking 从具体 proxy 中解耦的 Kubernetes 平台。语义/成本/质量模型选择应放在它之前，由 [semantic-router](#) 或 routellm 完成。

详见 [src-gateway-api-inference-extension-architecture](#)、[src-k8s-gateway-routing-comparison](#)。

Gateway API Inference Extension 架构与设计思路分析

kubernetes-sigs/gateway-api-inference-extension 是 Kubernetes SIG 路线的推理流量标准化入口。它定义 InferencePool 等 API，提供 EPP/LWEPP、client-go、conformance、benchmarking 和 Gateway API 集成，是 llm-d-router 等项目对齐的标准层。

核心架构图



模块分层

层/目录	责任
api/v1, apix/**	InferencePool 和 EndpointPicker 相关 API。
cmd/epp, cmd/lwepp, pkg/epp, pkg/lwepp	Endpoint Picker 实现。
client-go/**	生成客户端。
conformance/**, benchmarking/**	一致性与性能验证。

关键数据流

1. HTTPRoute/Gateway 指向 InferencePool。
2. EPP/LWEPP 根据 endpoint 状态、端口和模型能力选择后端。
3. conformance/benchmark 确保实现符合 Gateway API 推理扩展语义。

设计决策

- 把推理路由变成 Gateway API extension，而不是每个 serving 系统自定义。
- 保留 EPP 参考实现，同时让 downstream 可替换。
- 最近支持 port-aware endpoint filtering，说明 data parallelism 已进入标准关注点。

对比定位

和 llm-d-router 相比，它是标准/API 和参考层；和 Envoy AI Gateway 相比，它更偏 endpoint picking，不管 provider auth/cost/prompt mutation。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

GitHub Stars P0–P2 当前状态核验

本页是 github-stars-ingest-candidates 的 GitHub 当前状态复核底稿。2026-06-12 通过 GitHub API 重新读取 P0-P2 项目的描述、stars、最近 push、主语言、license 和 topics，并把它们作为 [github-stars-backlog-implementation-map](#) 的事实输入。每个项目的原始核验结果已逐一保存到 raw/github-stars-p*-*.md。

[note] Snapshot 与架构分析的区别

raw/github-stars-p*-*.md 是 GitHub 元数据快照，只记录仓库当前状态，不是源码架构分析。正式 \$ingest-codebase 产物统一在 raw/<slug>-architecture-analysis.md、wiki/sources/src-<slug>-architecture.md 和 wiki/html/sources/src-<slug>-architecture.html。例如 raw/github-stars-p1-semantic-router.md 对应的正式分析是 raw/semantic-router-architecture-analysis.md 与 [src-semantic-router-architecture](#)。

它不是新的 backlog。它的作用是把“候选项目”变成可比较的当前项目剖面，供后续按专题理解架构边界和选型差异。

Raw 快照文件

P0

- raw/github-stars-p0-agent-substrate-substrate.md
- raw/github-stars-p0-agentscope-runtime.md
- raw/github-stars-p0-mem0.md
- raw/github-stars-p0-reme.md
- raw/github-stars-p0-openai-codex.md
- raw/github-stars-p0-pi.md
- raw/github-stars-p0-oh-my-pi.md
- raw/github-stars-p0-multica.md
- raw/github-stars-p0-open-cowork.md
- raw/github-stars-p0-aibrix.md
- raw/github-stars-p0-llm-d.md
- raw/github-stars-p0-llm-d-router.md
- raw/github-stars-p0-llm-d-kv-cache.md

P1

- raw/github-stars-p1-kagent.md

- raw/github-stars-p1-kubectll-ai.md
- raw/github-stars-p1-k8m.md
- raw/github-stars-p1-kubewall.md
- raw/github-stars-p1-gateway-api-inference-extension.md
- raw/github-stars-p1-envoy-ai-gateway.md
- raw/github-stars-p1-kgateway.md
- raw/github-stars-p1-higress.md
- raw/github-stars-p1-semantic-router.md
- raw/github-stars-p1-routellm.md
- raw/github-stars-p1-plano.md
- raw/github-stars-p1-gpustack.md
- raw/github-stars-p1-ome.md
- raw/github-stars-p1-kserve.md
- raw/github-stars-p1-kubeai.md
- raw/github-stars-p1-code-review-graph.md
- raw/github-stars-p1-gitnexus.md
- raw/github-stars-p1-deepwiki-open.md

P2

- raw/github-stars-p2-codex-plugin-cc.md
- raw/github-stars-p2-claude-tap.md
- raw/github-stars-p2-cc-connect.md
- raw/github-stars-p2-tokscale.md
- raw/github-stars-p2-hami.md
- raw/github-stars-p2-dra-driver-nvidia-gpu.md
- raw/github-stars-p2-gpu-operator.md
- raw/github-stars-p2-k8s-device-plugin.md

P0 项目

项目	stars	最近 push	语言	license	当前定位
agent-substrate/ substrate	517	2026-06-11	Go	Apache-2.0	Agent Substrate core system, 适合补 agent-runtime-sandbox-selection-map 的高密度 stateful agent substrate 层。
agentscope-ai/ agentscope-runtime	816	2026-06-04	Python	Apache-2.0	Production-ready agent runtime: secure tool sandboxing、Agent-as-a-Service API、K8s/serverless、observability。

项目	stars	最近 push	语言	license	当前定位
mem0ai/mem0	58393	2026-06-12	Python	Apache-2.0	Universal memory layer for AI agents，和 agent-memory-selection-matrix 中的应用级 memory layer 对比价值最高。
agentscope-ai/ReMe	3074	2026-06-10	Python	Apache-2.0	AgentScope 生态 Memory Management Kit，代表 framework 内建 memory kit 路线。
openai/codex	90546	2026-06-12	Rust	Apache-2.0	Lightweight terminal coding agent，是 coding-agent-selection-map 必须纳入的官方 coding loop 产品。
earendil-works/pi	61856	2026-06-11	TypeScript	MIT	AI agent toolkit: unified LLM API、agent loop、TUI、coding agent CLI。
can1357/oh-my-pi	11964	2026-06-12	TypeScript	MIT	Terminal AI coding agent: hash-anchored edits、LSP、browser、subagents、tool harness。
multica-ai/multica	36331	2026-06-12	Go	NOASSERTION	Managed agents platform，把 coding agents 产品化为可分配任务、追踪进度、复合 skills 的 teammate。
OpenCoworkAI/open-cowork	1590	2026-06-07	TypeScript	MIT	Desktop agent app，整合 Claude Code、MCP、Skills、sandbox、IM。
vllm-project/aibrix	4870	2026-06-11	Go	Apache-2.0	GenAI inference infrastructure components，补 llm-serving-engine-selection-map 的 serving control plane。
llm-d/llm-d	3346	2026-06-12	Shell	Apache-2.0	Kubernetes 上的 modern distributed inference stack，总入口。
llm-d/llm-d-router	220	2026-06-12	Go	Apache-2.0	LLM inference request intelligent entry point，进入 routing 专题。
llm-d/llm-d-kv-cache	155	2026-06-11	Go	Apache-2.0	Distributed KV cache scheduling and offloading libraries，直接连接 kv-cache-offload 。

P1 项目

项目	stars	最近 push	语言	license	当前定位
kagent-dev/kagent	2979	2026-06-12	Go	Apache-2.0	Cloud Native Agentic AI，把 ai-ops、K8s agent 和 MCP 接到一起。
GoogleCloudPlatform/kubectl-ai	7494	2026-05-08	Go	Apache-2.0	AI powered Kubernetes Assistant，kubectl 入口。
weibaohui/k8m	829	2026-06-04	Go	MIT	Mini Kubernetes AI Dashboard，含 MCP、操作权限、多集群、异常检测。
kubewall/kubewall	1903	2026-05-19	TypeScript	Apache-2.0	Single-binary Kubernetes dashboard + AI integration。
kubernetes-sigs/gateway-api-inference-extension	690	2026-06-11	Go	Apache-2.0	Gateway API Inference Extension，是 inference routing 进入 K8s Gateway API 的标准化入口。
envoyproxy/ai-gateway	1742	2026-06-11	Go	Apache-2.0	Envoy Gateway 上的 GenAI access/routing 管理。
kgateway-dev/kgateway	5559	2026-06-12	Go	Apache-2.0	Cloud-native API Gateway and AI Gateway。
higress-group/higress	8624	2026-06-07	Go	Apache-2.0	AI Native API Gateway，和 agent-credential-isolation 、HiClaw 凭据托管背景相关。
vllm-project/semantic-router	4321	2026-06-12	Go	Apache-2.0	System-level intelligent router for mixture-of-models，连接 AI Gateway 和模型路由。
lm-sys/RouteLLM	5011	2024-08-10	Python	Apache-2.0	LLM router serving/evaluation framework，偏成本/质量路由算法基线。
katanemo/plano	6585	2026-06-09	Rust	Apache-2.0	AI-native proxy and data plane，和 agentgateway 属于 gateway/data plane 层。
gpustack/gpustack	5145	2026-06-12	Python	Apache-2.0	GPU cluster manager，编排 vLLM/SGLang 推理引擎。
ome-projects/ome			Go		

项目	stars	最近 push	语言	license	当前定位
	464	2026-06-12		Apache-2.0	Open Model Engine, K8s operator for LLM serving、GPU scheduling、model lifecycle。
kserve/kserve	5563	2026-06-12	Go	Apache-2.0	Standardized distributed generative and predictive AI inference platform on Kubernetes。
kubeai-project/kubeai	1209	2026-06-10	Go	Apache-2.0	AI inference operator, 覆盖 LLM/VLM/embedding/speech-to-text。
tirth8205/code-review-graph	18405	2026-06-10	Python	MIT	Local-first code intelligence graph for MCP/CLI。
abhigyanpatwari/GitNexus	41980	2026-06-12	TypeScript	NOASSERTION	Browser-side repo knowledge graph + Graph RAG agent。
AsyncFuncAI/deepwiki-open	16855	2026-06-03	Python	MIT	Open-source DeepWiki, AI-powered wiki generator for repos。

P2 项目

项目	stars	最近 push	语言	license	当前定位
openai/codex-plugin-cc	20740	2026-04-18	JavaScript	Apache-2.0	Claude Code 调用 Codex 的 delegation plugin。
liaohch3/claude-tap	1671	2026-06-12	Python	MIT	Coding agent API traffic trace viewer, 覆盖 Claude Code、Codex、Gemini、Cursor、OpenCode、Pi、Hermes。
chenhg5/cc-connect	12197	2026-06-11	Go	NOASSERTION	把本地 coding agents 接到 Feishu/Lark、Slack、Telegram、Discord、企业微信等 IM。
junhoyeo/tokscale	3680	2026-06-11	Rust	MIT	多 coding agent token usage tracker + leaderboard/graph。

项目	stars	最近 push	语言	license	当前定位
Project-HAMi / HAMi	3558	2026-06-11	Go	Apache-2.0	Heterogeneous GPU sharing on Kubernetes。
kubernetes-sigs/dra-driver-nvidia-gpu	654	2026-06-11	Go	Apache-2.0	NVIDIA GPU DRA driver。
NVIDIA/gpu-operator	2732	2026-06-12	Go	Apache-2.0	Kubernetes GPU driver/runtime/operator 管理基座。
NVIDIA/k8s-device-plugin	3786	2026-06-11	Go	Apache-2.0	Kubernetes NVIDIA device plugin 核心项目。

核验结论

- P0 不应再只是待办：它们分别补齐 runtime substrate、memory layer、coding agent 执行面、personal agent productization 和 K8s inference control plane。
- P1 是专题补强：K8s AI assistant、AI Gateway/routing、model serving operator、code intelligence/wiki generator 都可以直接进入横向地图。
- P2 不是低价值，而是更像生态配套层：coding agent plugin/observability/IM bridge/token cost，以及 GPU/DRA/device plugin 基座。
- lm-sys/RouteLLM 最近 push 停在 2024-08-10，仍有算法基线价值，但不应按同等活跃度对待。

GPU Sharing

GPU sharing 指多个 workload 共享同一物理 GPU 或 MIG/MPS/time-slicing/vGPU 资源的调度与隔离方法。目标是在吞吐、隔离、成本和利用率之间取平衡。

在 M5-C Device / GPU 地图中的位置

共享方案要分别回答四件事：scheduler 如何选择设备并记账，allocator 如何交接设备，硬件是否分区，runtime 如何限制实际使用。node-feature-discovery 提供能力信号；[k8s-device-plugin](#)、[dra-driver-nvidia-gpu](#) 与 hami 采用的资源模型和隔离机制不同。[cdi](#) 只承载注入配置，不把记账额度自动变成隔离保证。完整分层见 [k8s-gpu-device-stack](#)，Claim 与节点准备的区别见 [kubernetes-dra-design-deep-dive](#)。

部署时须为同一物理设备和资源名明确分配所有权，尤其不能让 HAMi 与 NVIDIA 官方 plugin 在同一节点争用 `nvidia.com/gpu`。[gpu-operator](#) 可以管理软件组件，但需关闭冲突的 `allocation operand`；HAMi-DRA 改变分配路径，不等于改变 HAMi-core 的软件隔离边界。

主要路线

路线	代表	特点
MIG	k8s-device-plugin , gpu-operator	硬件分区；plugin 可暴露预配置实例，分区配置由相应管理组件完成，粒度受硬件/profile 限制
Time-slicing	k8s-device-plugin	复用执行时间，不等于独立显存或硬件故障域隔离
MPS	k8s-device-plugin	通过 MPS 服务共享执行与配置资源限制；支持范围依版本，不等价于 MIG 硬件分区
HAMi-core memory/core sharing	hami	调度/记账配合容器内调用拦截，隔离效果取决于注入与兼容调用路径
DRA device configuration	dra-driver-nvidia-gpu	声明式 Claim 分配和设备配置；基础 API 稳定不意味着全部动态分区/共享扩展已稳定
Application-aware elastic KV	kvcached	只让接入的 vLLM/SGLang KV physical pages按需占用VRAM；不提供通用CUDA显存quota、执行隔离或K8s调度

KVCacheD 在 GPU sharing 中的位置

[kvcached](#) 属于应用内、语义感知的内存复用：它知道哪些分配是KV block/token，因此可以在不停止模型的情况下按request生命周期map/unmap physical pages。MIG/MPS/time-slicing/HAMi则从设备或runtime层管理更广泛的GPU workload。

两者可以组合，但不能混为一谈：KVCacheD提高同卡多模型的KV利用率，外层方案仍需负责设备分配、记账、执行共享与租户隔离。详见 [elastic-kv-cache](#) 和 [kvcached-sglang-vllm-knowledge-system](#)。

和 GPU Operator 的关系

[gpu-operator](#) 管软件栈生命周期；GPU sharing 管 workload 如何共享设备。两者层级不同，但生产环境经常同时出现。

GPU 编程学习

GPU 编程学习是从“会调用模型/框架”走向“能理解推理性能、显存瓶颈和 kernel 行为”的底层路径。当前 wiki 已有 [src-ai-infra-learning-cn-stars](#) 和 [ai-infra-learning-cn-map](#) 作为 AI Infra 学习路线，[musa-learning-notes](#) 则补上一个具体的 CUDA / MUSA 动手系列入口。

分层

层	目标	代表材料
执行模型	理解 SIMT、warp、block、grid、同步和错误暴露	musa-learning-notes Week 1/3
Runtime API	会写 host/device 内存管理、kernel launch、错误检查和计时	musa-learning-notes Week 1/2
异步与吞吐	理解 pinned memory、Stream、Event、Graph 和 launch overhead	musa-learning-notes Week 2
访存与算子	理解 coalescing、shared memory、bank conflict、GEMM tiling	musa-learning-notes Week 4/5
迁移与生态	理解 CUDA→MUSA 的 API 高相似度和硬件/工具链差异	src-musa-learning-notes-architecture
系统连接	把 kernel 行为连接到 llm-inference 、 paged-attention 、 radix-attention 、 k8s-gpu-device-stack	wiki serving / GPU pages

为什么要放进 AI Infra 路线

LLM serving 的瓶颈经常落在 GPU 侧：KV cache 布局、attention kernel、batching、P/D 分离、显存复用、launch overhead、通信和拓扑都会影响系统设计。只看 [model-serving-operator](#) 或 [inference-routing](#) 会理解控制面，但难以判断底层性能假设是否成立。

[musa-learning-notes](#) 的价值在于把学习问题缩小到可运行小实验：先跑 Runtime 骨架，再跑 Stream/Graph，再看 reduce、访存和 GEMM。它同时提醒 CUDA 经验不能直接等价迁移到 MUSA，尤其是 warp=128、mcc 工具链、muBLAS/MCCL 和 MUSA Graph 当前实现成熟度。

后续可扩展方向

- 为 [gpu-programming-learning](#) 继续摄入 CUDA_Freshman、LeetCUDA、LeetGPU 或 a-hamdi / GPU，形成 CUDA 与 MUSA 对照路线。

- 把 Week 3-5 的 Reduce、transpose、GEMM 实测补成性能表，再和 [llm-inference](#) 中的 kernel/attention 优化关联。
- 把 [k8s-gpu-device-stack](#) 的设备管理层与本页的 kernel 学习层区分清楚：前者回答“GPU 怎么分配给 workload”，后者回答“workload 在 GPU 上为什么快或慢”。

GPUStack

GPUStack 是 GPU cluster manager 与 Model-as-a-Service 平台，能够跨本地服务器、Kubernetes 和云环境管理 GPU，配置 vLLM、SGLang、TensorRT-LLM 或自定义 inference engine，并提供模型服务和 GPU 运维能力。

解决的问题

GPU 资源、模型部署、engine 参数、认证、监控和计量通常分散在不同系统。GPUStack 把 GPU 调度、模型服务、engine 选择、故障恢复、访问控制、token/API 计量和多集群可见性整合起来。

核心边界

GPUStack 更像一体化 GPU/MaaS 平台，而不是单纯 K8s CRD 库或专用 P/D router。它的价值在资源与服务整体运营；如果组织已有成熟 K8s/Gateway/Operator，只想补一个模型生命周期层，kserve 或 ome 可能更贴合。

适用场景

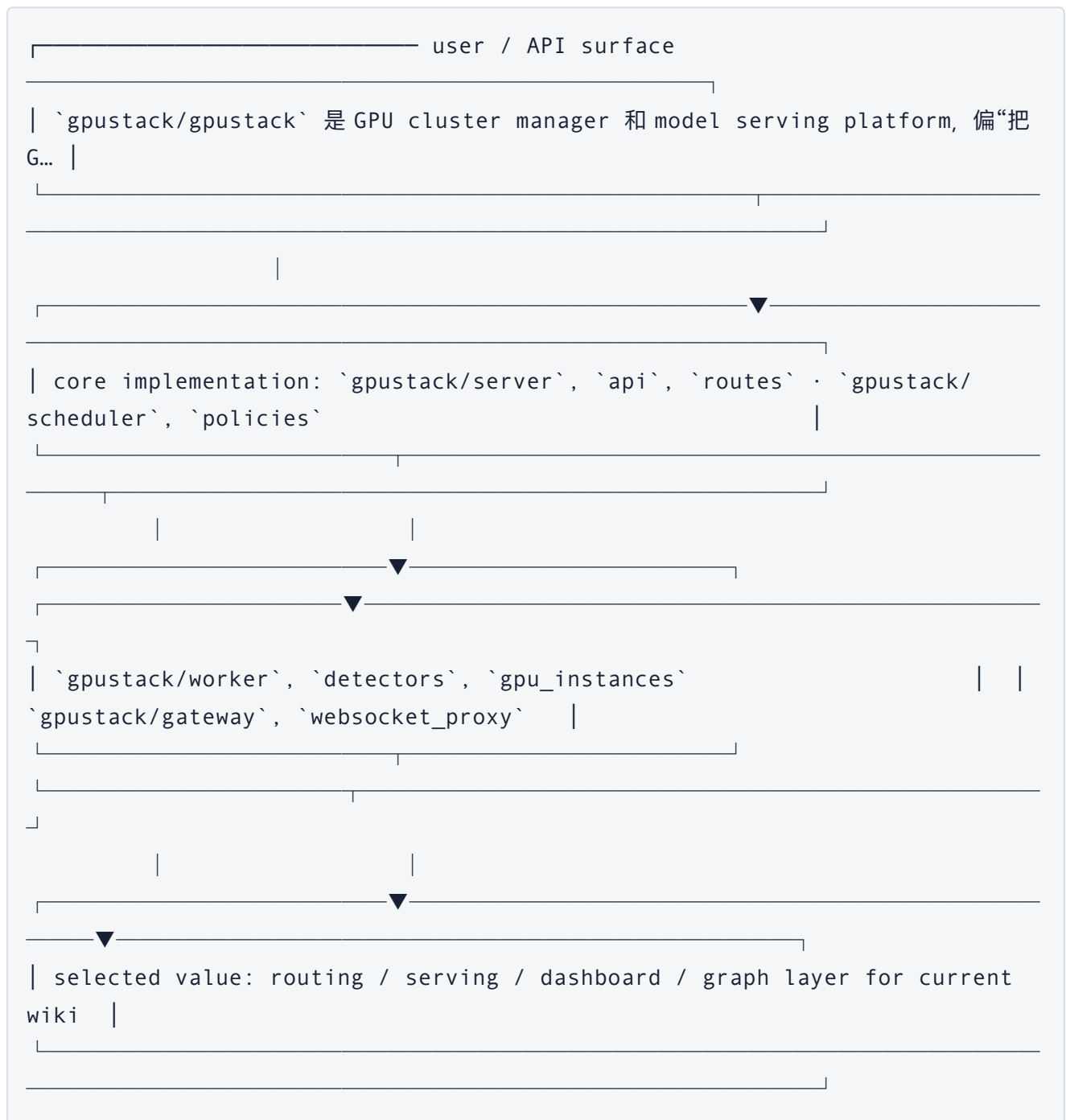
适合 GPU 云、私有 GPU 集群、跨环境 Model-as-a-Service、需要多种加速器和统一计量运维的团队。需要 Gateway API/InferencePool 标准化看 [llm-d](#)；需要深度 P/D/KV 编排看 [dynamo](#) 或 [kthena](#)。

详见 [src-gpustack-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

GPUStack 架构与设计思路分析

gpustack/gpustack 是 GPU cluster manager 和 model serving platform，偏“把 GPU/模型服务统一管理起来”，而不是单一 inference engine。源码分为 API/server、scheduler、worker、gateway、GPU detectors、cloud providers、K8s integration、migrations 和 charts。

核心架构图



模块分层

层/目录	责任
gpustack/server, api, routes	服务端 API 和路由。
gpustack/scheduler, policies	模型部署和 GPU 调度策略。
gpustack/worker, detectors, gpu_instances	节点/worker 和 GPU 检测。
gpustack/gateway, websocket_proxy	请求入口和代理。

关键数据流

1. 用户注册 workers/GPU 和模型。
2. server/scheduler 选择 worker/GPU 并启动 vLLM/SGLang/Ollama 等服务。
3. gateway 暴露统一 endpoint 并转发请求。

设计决策

- 把 GPU 发现、调度、模型生命周期和 endpoint 放到一个平台。
- Python 实现利于快速集成模型生态。
- 适合中小团队自管 GPU serving，不是 Gateway API 标准路线。

对比定位

和 KServe/OME/KubeAI 相比，GPUStack 更产品化 cluster manager；和 AIBrix/llm-d 相比，它更偏平台管理，不是专门研究高级 KV/P-D 路由。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

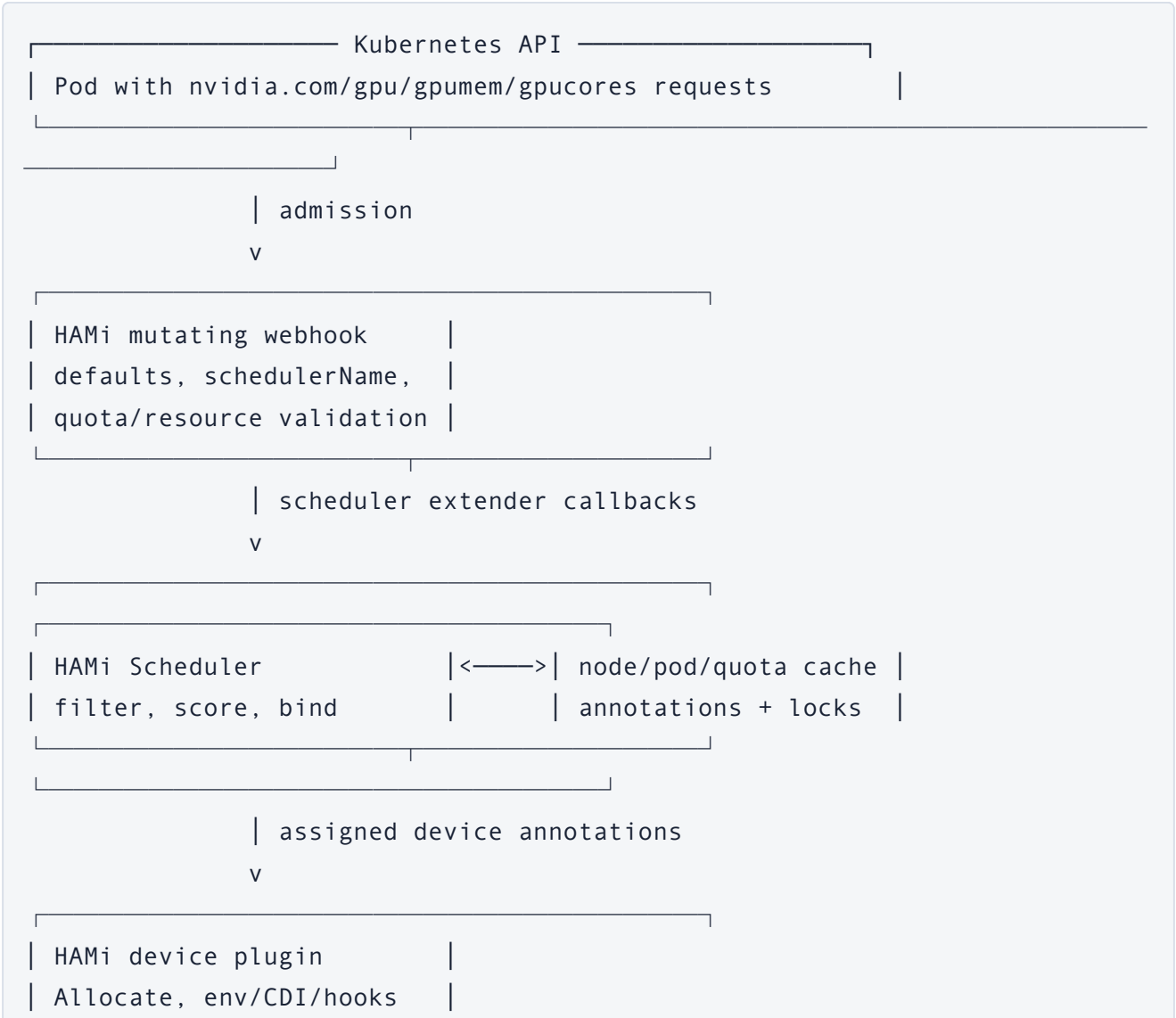
HAMi 架构与设计思路分析

原文: raw/hami-architecture-analysis.md · 仓库: <https://github.com/Project-HAMi/HAMi>
HAMi · 分析版本 HEAD 5dca58e (2026-06-11)

一句话定位

Kubernetes 异构 GPU sharing / vGPU 项目，通过 mutating webhook、scheduler extender、device plugin 和设备厂商后端，把 GPU memory/core/count 等细粒度配额写入 Pod 注解并在 Allocate 阶段兑现。它更像“共享调度与隔离层”，不是 NVIDIA 官方基础 device plugin 的简单替代。这页和 [kubernetes dra cdi gpu-sharing](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图

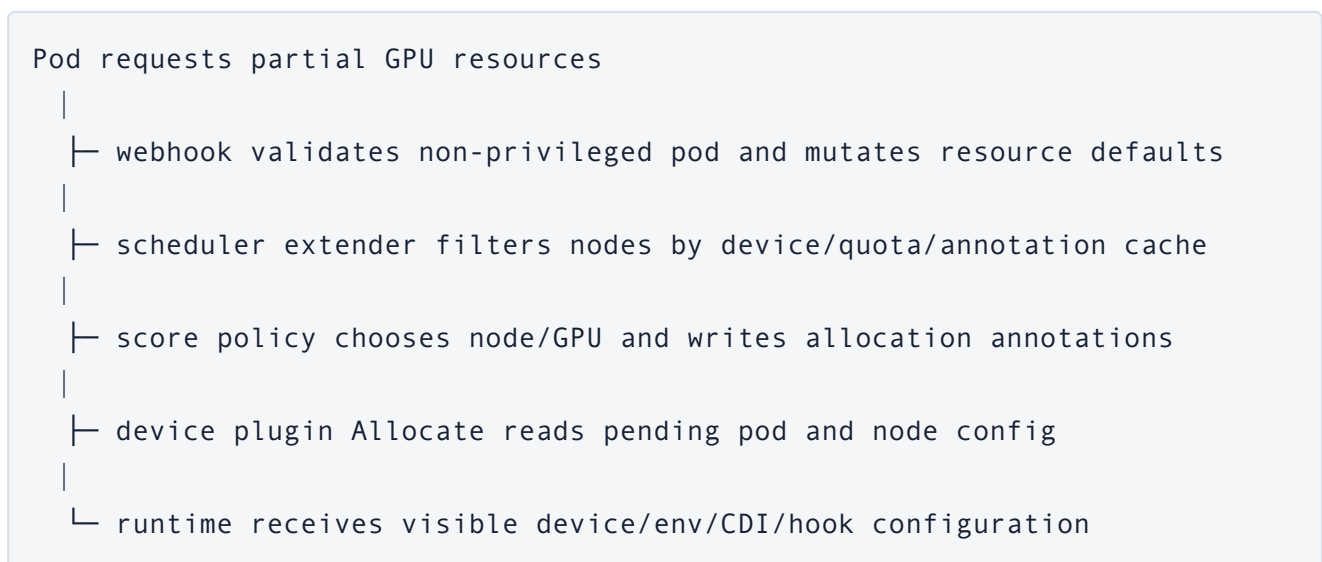




模块分层

层 / 模块	职责
Webhook	拦截 Pod admission，注入 schedulerName、默认 GPU memory/core/count、quota 检查。
Scheduler extender	维护 node/pod/quota cache，提供 /filter /bind /score 路由和 binpack/spread 策略。
设备抽象	多厂商设备注册、资源名、quota、annotation 编解码和打分逻辑。
Device plugin	对接 kubelet device plugin API，读取 node config，注册虚拟化资源，生成 env/CDI/挂载。
Monitor/metrics	反馈容器内 GPU 使用、Prometheus 指标和调度可观测性。

关键数据流



设计决策与哲学

- 使用 scheduler extender 而不是只靠 device plugin: GPU memory/core 共享需要调度时全局决策, 单纯 kubelet Allocate 已经太晚。
- annotation 是调度到分配的契约: Scheduler 选择设备后通过 Pod/Node annotation 传递给 device plugin, 降低对 kube-scheduler 内部扩展的侵入。
- 多厂商后端共享同一 device abstraction: pkg/device 提供 NVIDIA/AMD/Ascend/Cambricon 等扩展点, HAMi 定位异构设备共享而非单厂商工具。

与同类项目的架构差异

维度	HAMi	同类 A	同类 B
定位	vGPU/GPU sharing 调度与隔离	k8s-device-plugin: 官方 GPU 暴露	GPU Operator: 软件栈生命周期管理
调度参与	强, webhook + extender + bind	弱, 主要 kubelet plugin	间接, 部署组件
资源粒度	memory/core/count/厂商自定义	GPU/MIG/MPS/time-slicing	driver/runtime/plugin/DCGM 组件

选型提示

如果你在快速理解或选型, 先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”, 还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题: 比如 trace viewer 不能替代 remote control bridge, GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [kubernetes](#)
- [dra](#)
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)

HiClaw

阿里 Higress 团队 (agentscope-ai) 出品的开源 K8s 原生多 Agent 协作平台。把"管 AI Agent 大军"问题完全套进 K8s 范式: 4 个 CRD (Worker / Team / Human / Manager) 声明 Agent, controller 把每个 Agent 落成"容器 + Matrix IM 用户 + Higress consumer"三位一体, 人类与 Agent 在同一组 Matrix 房间里协作, 真实 LLM 凭据托管在 Higress 网关, Worker 只持 consumer key 实现 [凭据零暴露](#)。

详细架构与设计哲学见 [src-hiclaw-architecture](#)。

关键事实

- 仓库: [agentscope-ai/HiClaw](#)
- 当前核验: GitHub API 显示 main 最近 push 2026-06-10, 最新 tag v1.1.2
- 本 wiki 架构分析版本: v1.1.0 (2026-04-24, HEAD e21ac83)
- 主要语言: Go (controller, 153 文件) + Python (worker runtimes: QwenPaw / Hermes)
- 同栈: Higress AI Gateway · Tuwunel (Matrix 服务端) · MinIO · Element Web · k3s-io/kine (嵌入式 etcd)
- Worker runtime 三选一: OpenClaw (claude-code 系) / QwenPaw (基于 Qwen Code) / Hermes (autonomous coding agent) ; 当前默认 QwenPaw

从 0 到一个 Worker 跑起来

HiClaw 的最小闭环不是“启动一个 Python Agent”, 而是创建一套协作身份和凭据边界:

1. 安装 HiClaw: 本地 Docker 模式走 installer, K8s 模式走 Helm chart。
2. 准备 LLM 配置: `credentials.llmProvider`、`credentials.defaultModel`、`credentials.llmApiKey`、`credentials.llmBaseUrl`。
3. 启动控制面: controller 创建/管理 CRD, 并连接 Matrix、gateway、storage。
4. 创建 Human / 管理员身份, 进入 Element Web 或其他 Matrix client。
5. Manager Agent 通过 Matrix 收到“创建 Worker”的请求。
6. Manager 调 `hiclaw worker create ...`, controller 写入 Worker CR。
7. Worker reconciler 创建 Matrix user/room、gateway consumer key、对象存储权限和 Worker 容器。
8. Worker 容器启动后进入 Matrix 房间, 后续人类、Manager、Worker 都在 IM 协作平面里交流。

这条路径体现 declarative-agent-management: 用户声明 Worker, controller 负责把容器、IM 身份、网关凭据和存储权限收敛到期望状态。

Helm 组件配置

当前 Helm chart 的主要组件来自 `helm/hiclaw/values.yaml` 与 `templates`:

组件	values 路径	作用
controller	controller.*	CRD reconciler、worker lifecycle、gateway/credential/storage 编排；可配 replica、resources、workerBackend、uninstall hook
manager	manager.*	是否创建 Manager CR、Manager runtime/model/image/resources
matrix	matrix.*	Tuwunel 或 Synapse；managed/existing；控制 Matrix server、持久化和服务端口
gateway	gateway.* / higress.*	managed Higress 或 existing AI Gateway；配置公网 URL、APIG 信息、Higress subchart
storage	storage.*	managed MinIO 或 existing OSS；bucket、endpoint、MinIO 资源和持久化
credentialProvider	credentialProvider.*	云上 APIG/OSS 场景下签发 STS / RAM role token
secrets	credentials.*	Matrix admin、LLM provider、默认模型和 base URL

本地 kind/minikube 默认更偏 managed Tuwunel + managed Higress + managed MinIO；云上 ACK/ACS 场景可以把 gateway/storage 切到 existing provider。

与 claude-mem 的区别

维度	HiClaw	claude-mem
目标	管理多个 Agent 的协作、身份、容器、房间、凭据	给 Claude Code 增加长期记忆
状态单位	Worker/Team/Human/Manager CR	用户 prompt、tool log、memory fact
控制面	Kubernetes controller + Matrix + gateway	Claude Code hooks + memory server/storage
安全重点	Agent 只持 consumer key，凭据在 gateway	记忆采集、压缩、检索和上下文注入
适合问题	多 Agent 运维、人在回路、协作拓扑	单 Agent 跨会话记忆

二者可组合：HiClaw 负责“Agent 如何被创建、协作和持权”，记忆系统负责“Agent 如何记住长期事实”。

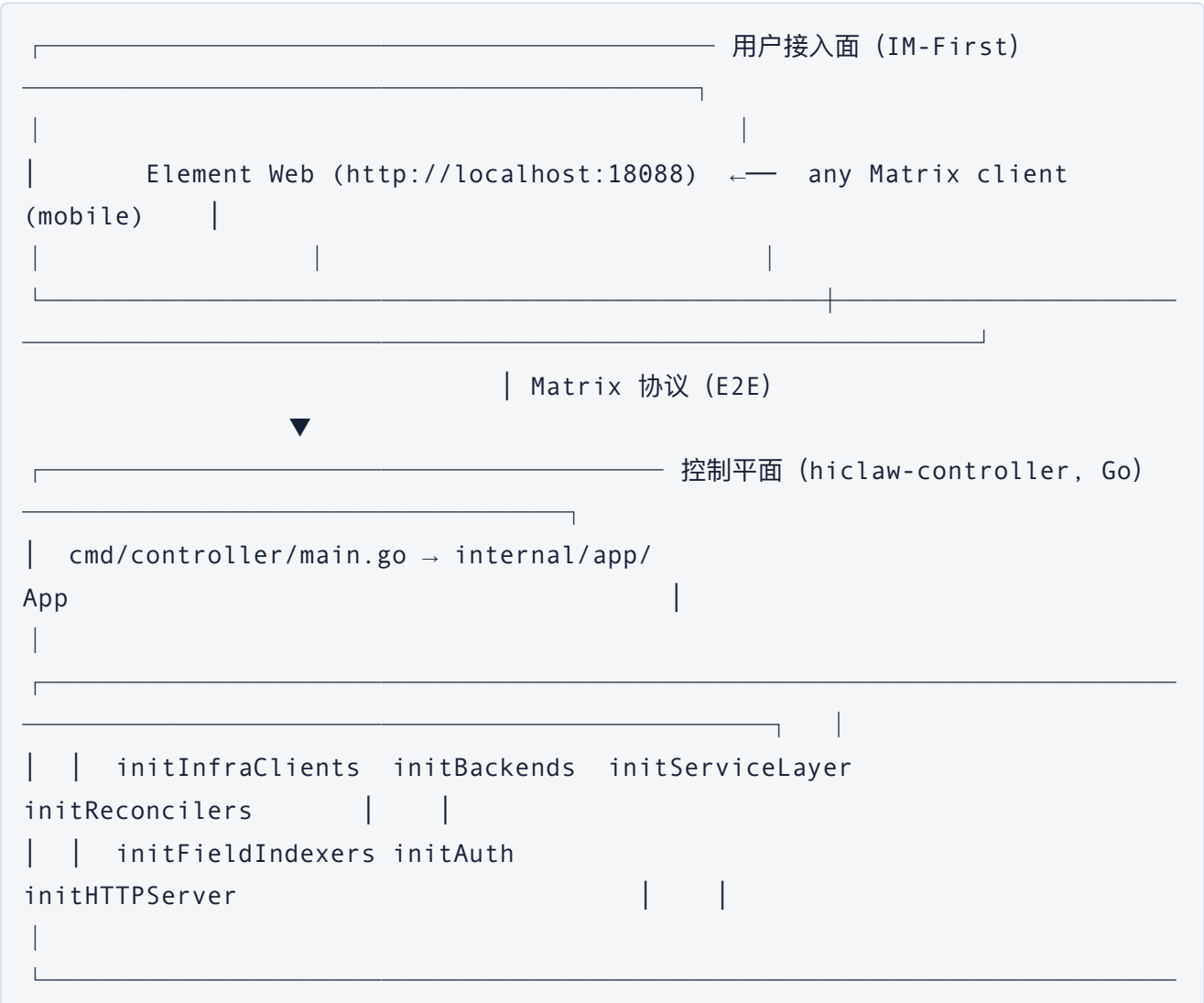
HiClaw 架构与设计思路分析

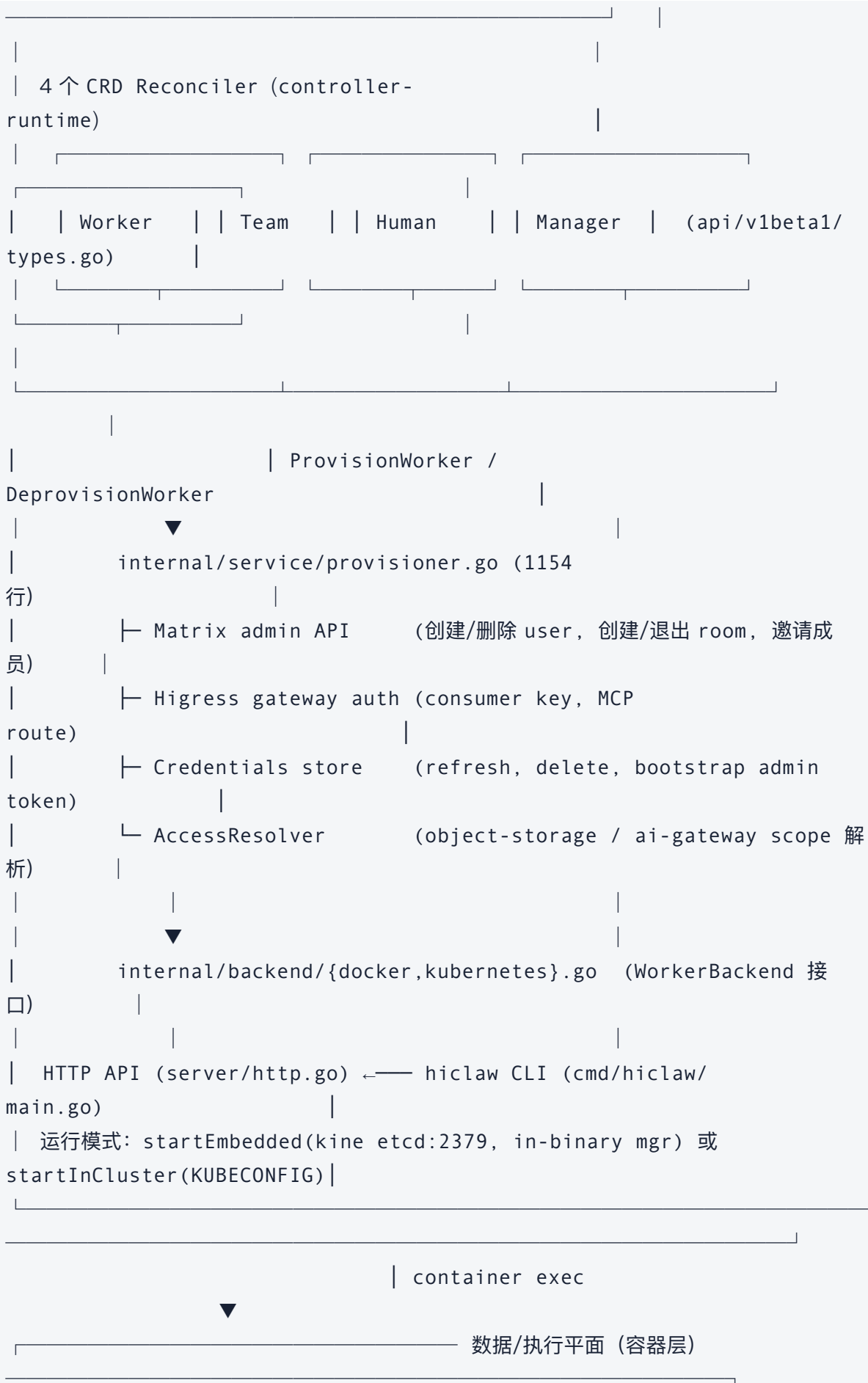
原文: raw/hiclaw-architecture-analysis.md · 仓库: [agentscope-ai/HiClaw](#) · 分析版本 v1.1.0 (HEAD e21ac83)

一句话定位

[HiClaw](#) 是把 k8s-operator 模式套到 AI Agent 运维上的多 Agent 协作平台——用 4 个 CRD (Worker / Team / Human / Manager) 声明 Agent, controller 把每个 Agent 落成"容器 + Matrix IM 用户 + Higress consumer"三位一体; 人类、Manager、Workers 全在同一组 Matrix 房间里协作; 真实 LLM API key 永远不落到 Worker, 只发 consumer key ([凭据零暴露](#))。一行 curl | bash 装到 Docker, 或 helm install 到 K8s, 二者形态对齐。

核心架构图







Helm chart (helm/hiclaw/) 在 K8s 上把上面 4 个组件一并部署
install/hiclaw-install.sh / .ps1 在 Docker 单机上等价部署

四个 CRD

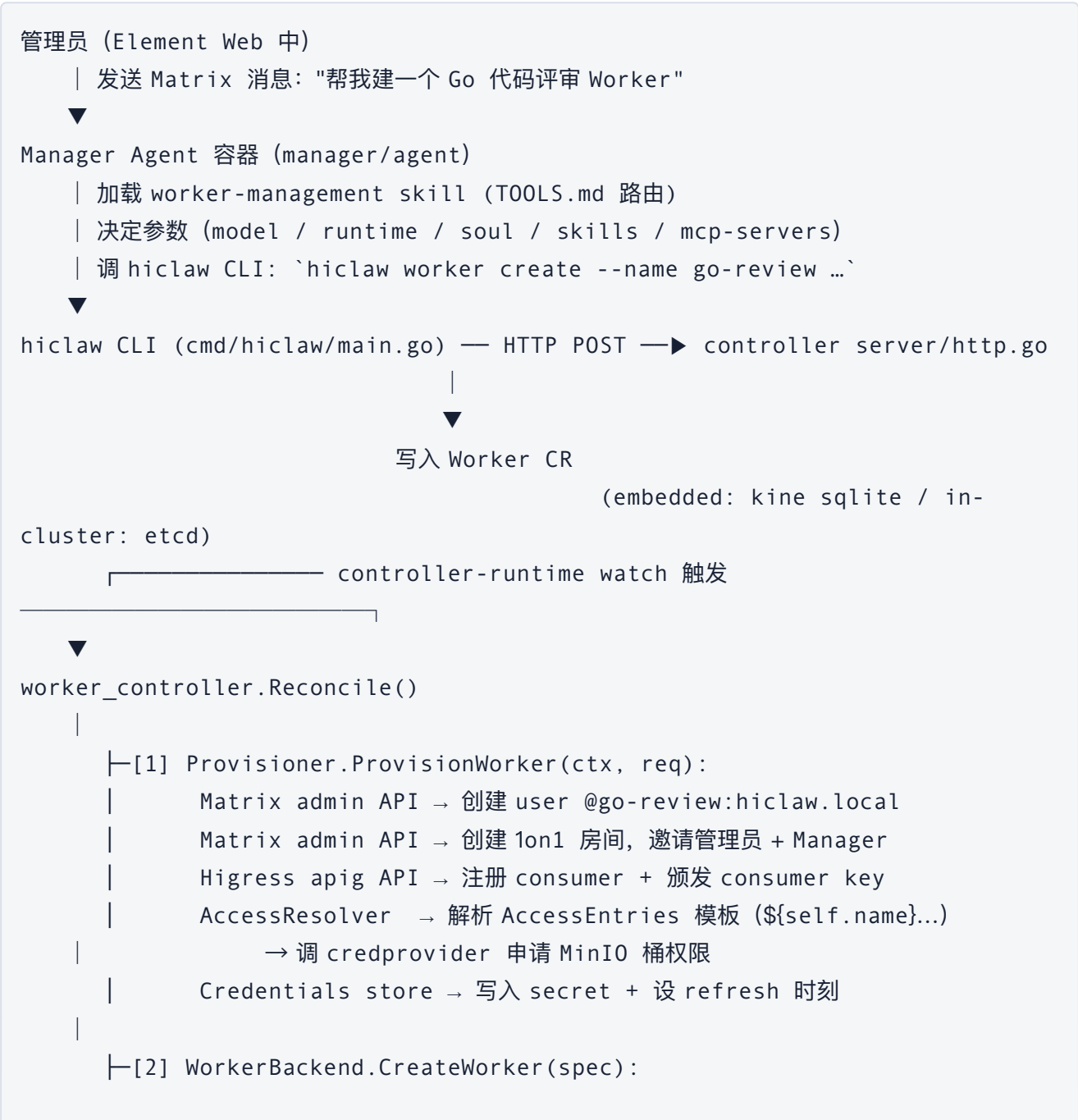
CRD	表达什么	关键 Spec 字段
Worker	单个 AI Agent	model · runtime (openclaw/copaw/hermes) · image · soul · skills · mcpServers · package (file:///http(s):///nacos://) · expose · accessEntries · state (Running/Sleeping/Stopped)
Team	一组 Worker + Leader + 房间策略	leader · workers · peerMentions · channelPolicy · admin
Manager	系统唯一的管家 Agent	model / runtime / soul / skills (含 12 个管理 skill)
Human	管理员的 Matrix 身份	matrixUserID · channel scope

模块分层（简版）

层	职责
CRD 定义	api/v1beta1/types.go, 仅依赖 k8s.io/apimachinery, 不引用任何 internal 包
应用骨架	internal/app/App 用 7-step init 串行装配, 扩展只需 append {name, initFn}
CR Reconciler	Worker / Team / Human / Manager 4 套, 仅做编排, 绝不直接调外部 API
服务编排	internal/service/provisioner.go (1154 行, 脏活集中地, reconcile 的大半在这里)
容器后端	WorkerBackend 接口 + docker.go / kubernetes.go 两个实现, 按 config 选
基础设施集成	internal/{matrix,gateway,oss,credprovider,accessresolver}/ 分别封装 Tuwunel / Higress / MinIO / 阿里云凭据 / 模板解析
运行模式	startEmbedded (k3s-io/kine SQLite-backed etcd 在 127.0.0.1:2379 + 内嵌 mgr) 或 startInCluster (标准 KUBECONFIG)

层	职责
Manager Agent	manager/agent/ 容器化 orchestrator, 12 个管理 skill + workers-registry.json, 通过 hiclaw CLI 操作 controller
Worker runtime	OpenClaw (claude-code 系, Dockerfile only) · QwenPaw (Python, copaw/) · Hermes (Python, autonomous coding)
部署	helm/hiclaw/ (K8s) / install/hiclaw-install.{sh,ps1} (Docker 单机), 二者部署形态对齐

关键数据流：建一个 Worker 端到端



```

|         docker.go 或 kubernetes.go (依 config)
|     → 把 PodTemplate + 环境变量 + soul/identity/skills 卷挂上
|     → 启动 Worker 容器
|
|     └─[3] member_reconcile.go:
|         把 Worker Matrix user 加入相关 Team 房间 (如有)
|
|     └─[4] 更新 Worker.Status:
|         Phase = Running
|         MatrixUserID = "@go-review:hiclaw.local"
|         RoomID       = "!abc...:hiclaw.local"
|         ContainerState = "running"
|         ExposedPorts  = [...]    ← 若 Spec.Expose 有端口

```



Worker 容器启动后:

- Python runtime (copaw/hermes): bridge.py 用 consumer key 登录 Matrix, 订阅自己的房间
- OpenClaw runtime: 宿主 Claude Code 自带 hook 完成同等动作



管理员立刻在 Element Web 看到 @go-review 出现并打招呼; 后续的代码评审请求可直接 @

稳态通信信道

```

Worker → LLM:      Worker 容器 —consumer key→ Higress AI Gateway —真实 API
key→ LLM provider
Worker ↔ Worker:  双方都加入同一 Matrix 房间, 消息走 Twunel; 大文件落 MinIO (节省
token)
Worker ↔ Human:   Matrix 房间 (管理员是房间成员, 可以随时介入)
Worker → 工具:     Worker 容器 —consumer key→ Higress —route→ MCP server
(external 或 in-cluster)
Manager → 控制器:  Manager 容器内的 hiclaw CLI —HTTP→ controller —CR write→
etcd/kine

```

设计决策与哲学

- 声明式 Agent 运维: Agent 是 CR 不是对象。免费拿到 K8s 的 declarative / reconcile / self-healing / RBAC——这是 HiClaw 与 langgraph / autogen / crewai 这类「代码中编排 Agent」框架的根本不同。

- IM 协议作为协作平面 *lingua franca*：选 Matrix 而非自研 Web UI/事件总线——每个 Agent 是 IM 用户，每场协作是 IM 房间，人类天然就是房间成员，可观测性与人在回路同时拿到，避免两个大坑。代价：所有 Agent 都要做 Matrix 适配。
- **凭据零暴露**：Worker 永远拿不到真实 API key / GitHub PAT / OSS AK，只持 Higress consumer key。结果：MCP 工具市场和 skills 市场可以大胆装载社区作品——即便 Worker 被 prompt injection 攻陷，攻击者拿到的也只是可被即时 revoke 的 consumer key。
- 运行时三选一可插：WorkerSpec.Runtime 字段切换 OpenClaw (claude-code 系) / QwenPaw (Qwen Code 系，省钱) / Hermes (autonomous coding agent, 自主)。最近 commit d3e33e8 把默认 runtime 从 OpenClaw 切到 QwenPaw，价格敏感倾向明显。
- 容器后端双轨：WorkerBackend 接口 + docker.go / kubernetes.go 两实现，让"单机 Docker Desktop 跑 5 个 Worker"与"K8s 集群跑 500 个 Worker"共用同一 controller 二进制。
- 嵌入式 K8s (kine + 内嵌 controller-manager)：单机模式跑 kine (SQLite-backed etcd) + 同进程的 controller-manager，于 127.0.0.1:2379。用户感觉装了 Docker，骨子里是 K8s API——CRD / Reconcile / Helm 在无真 K8s 环境也成立。
- schema-less 模板字段：AccessEntry.Scope 用 *apiextensionsv1.JSON 而非严格 struct，支持 \${self.name} / \${self.kind} / \${self.namespace} 运行期变量。CRD 不绑定特定 cloud provider；解析延迟到 accessresolver。
- hiclaw CLI 注入 Manager / Worker 容器：Manager Agent 不引用 Go SDK，通过 fork CLI 操作 controller HTTP API——让 skill (markdown 文档) 可以直接生成 hiclaw worker create ... 这种命令字符串。
- MCP via Nacos 包市场：Worker 能力通过 WorkerSpec.Package 字段 (nacos:/// http(s):/// file:///) 按需加载。Manager 侧的 hiclaw-find-worker skill 就是做 Nacos 搜索 + 导入。Worker 模板像 docker 镜像一样可分发。

关键组件深入：Provisioner

internal/service/provisioner.go (1154 行 / 29 公开方法) 是 controller 的脏活集中地，可分三组：

1. Matrix 编排 (ProvisionWorker / ProvisionTeamRooms / EnsureRoomMember / ReconcileRoomMembership ...) ——通过 Tuvunel admin API 完成 user / room / membership / alias 全生命周期；MatrixUserID(name) 与 roomAliasLocalpart(kind, name) 是命名规则集中点，保证幂等。
2. Higress 凭据 (EnsureManagerGatewayAuth / EnsureWorkerGatewayAuth / RefreshCredentials / DeleteCredentials) ——用 alibabacloud-go/apig-20240327/v6 在 Higress 上为每个 Agent 注册 consumer + key。

3. Manager 引导 (IsManagerLLMAuthReady / IsManagerJoinedDM / SendManagerWelcomeMessage) ——容器首启时检查 Manager 是否就绪 + 投递欢迎语 (按 language / timezone 渲染)。

横向拆分时机已到：按 Matrix / Higress / Credentials / Manager-bootstrap 四个域拆到独立文件，Provisioner 退化为外观。

与同类对比

维度	HiClaw	autogen	langgraph	crewai
形态	K8s operator + 容器化	Python library	Python library	Python library
Agent 表达	CR (YAML)	Python class	StateGraph node	Crew/Agent class
部署单元	Pod / Container	进程	进程	进程
协作面	Matrix 房间	In-process bus	StateGraph 边	Crew 内调度
人在回路	默认 (房间成员)	需自接 UI	需自接 UI	需自接 UI
凭据模型	网关托管	Agent 持真 key	Agent 持真 key	Agent 持真 key
跨 Agent 文件	MinIO 桶	自实现	自实现	自实现

最大差异：HiClaw 把 Agent 当 K8s 一等公民资源，所以是"可被 reconcile 自愈 / 可被 helm 上线 / 可被 RBAC 限定"的东西。其余框架本质是"在你进程里跑一段 Python"，做企业级运维要重头补。代价：学习曲线高、每 Worker 一个容器有 RAM / 镜像开销。

演进信号

- 近 30 天最活跃: install/hiclaw-install.sh (20) · internal/service/provisioner.go (12) · api/v1beta1/types.go (12) · internal/config/config.go (11) → 主线是"装上 / 升级体验" + "controller 核心模型稳定"
- 关键 PR: #780 默认 runtime → QwenPaw · #749 --keep-all 升级 · #783 helm uninstall 残留清理 · #785 manager 容器 API 对齐 controller /api/v1 · #798 error() 多行安全
- v1.0 → v1.1 阶段，下一步看点: Hermes runtime 主线化、多租户 RBAC、kine → 真 etcd 平滑迁移工具

相关页面

- [HiClaw](#)
- claude-code
- higress · matrix-protocol · mcp · k8s-operator
- declarative-agent-management
- [agent-credential-isolation](#)

- autogen · langgraph · crewai

Inference Routing

Inference routing 指 LLM/模型服务请求进入 serving fleet 后，如何选择 provider、模型、endpoint、pod、prefill/decode worker 或 KV cache 命中路径。

在 M4 中的位置

Routing 位于流量入口与 engine 执行之间。[llm-inference-serving-project-map](#) 的 D1/D2 分别展示职责边界与请求路径，区分 Gateway、Proxy/EPP/Router 和 Model Server。路由负责选择请求去向，engine 负责执行推理；选型时不能把两者当作替代品。

四类路由

类型	代表	决策信号
Gateway/API endpoint picking	gateway-api-inference-extension , llm-d-router	endpoint health、metrics、InferencePool、目标模型
语义/模型能力路由	semantic-router , RouteLLM	prompt semantic、成本/质量、guard、模型能力
KV/cache aware routing	llm-d-kv-cache , dynamo , kthena , aibrix	prefix overlap、KV block location、worker load、LoRA/cache affinity
Batch-to-serving dispatch	llm-d-batch-gateway	batch job、per-model plan、processor concurrency、下游 endpoint capacity
Edge/provider governance	envoy-ai-gateway, ai-gateway	auth、provider、quota、rate limit、audit、fallback

选型提示

不要把所有 router 混为一类。[llm-d-router](#) / Gateway API router 更靠近 Kubernetes endpoint；semantic router 更靠近 prompt/model selection；KV-aware router 更靠近 serving runtime 和 cache locality，[llm-d-kv-cache](#) 则把 KV event/index/scoring 这层单独拆出来。[dynamo](#) 的补充是：selection host 可以是 Dynamo Frontend、GAIE EPP 或 standalone router，但 KV overlap 与 active load 的选择模型保持一致。

[llm-d-inference-sim](#) 对 routing 选型很有用：它不会给出真实 GPU 性能，但能在无 GPU 环境中模拟 OpenAI/vLLM API、KV block/cache events、TTFT/ITL 和 fake metrics，从而验证 endpoint picking、KV-aware scoring 和 autoscaling 闭环。[llm-d-batch-gateway](#) 则说明 batch workload 的“路由”更多发生在 processor 的 per-model plan 和并发控制层，不应和在线低延迟 request routing 混在一起评估。

KV-aware 路由决策树

```
请求 → 协议/凭证/配额通过? — 否 → 拒绝或 fallback
      ↓ 是
      模型与能力匹配? ——— 否 → model router / cascade
      ↓ 是
      endpoint 健康? ———— 否 → 排除
      ↓ 是
      prefix/KV locality? → 计算 cache credit
      ↓
      综合 queue + KV + SLO + cost
      ↓
      选择 endpoint / prefill-decode pair
      ↓
      结果与 KV events 反馈回路路由
```

网关层与 Endpoint Router 的组合

```
Client
  | OpenAI / Anthropic request
  ▼
Envoy AI Gateway (可选)
  | auth · provider · quota · global rate limit
  ▼
Semantic Router / RouteLLM (可选)
  | model · capability · quality/cost · cascade
  ▼
Gateway API + Inference Extension
  | InferencePool / Endpoint Picker protocol
  ▼
llm-d Router EPP
  | KV · queue · priority · health · pod placement
  ▼
vLLM / SGLang / TRT-LLM endpoint
```

这几层可以组合，但不是必须全部部署。envoy-ai-gateway 解决 provider 与边缘治理，[semantic-router](#)/routellm 解决模型/能力选择，[gateway-api-inference-extension](#) 定义标准协议，[llm-d-router](#) 负责 Kubernetes serving endpoint 选择。

inference-perf

GenAI inference performance benchmarking tool，用于对 OpenAI-compatible/serving endpoint 做负载、延迟和吞吐测量。详见 [src-inference-perf-architecture](#)。

架构边界

和 llm-d-benchmark 相比，inference-perf 更像单个 benchmark harness；llm-d-benchmark 负责更完整的实验生命周期。

什么时候用

场景	判断
需要 GenAI benchmark 能力	适合，inference-perf 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-d-benchmark , llm-inference , llm-d-inference-sim 组合。

核心组件

- CLI/config: benchmark 参数与 endpoint 配置
- Load generator: 并发、请求分布、payload 模板
- Metrics collector: latency/throughput/error/token stats
- Reports: 结果输出供 llm-d-benchmark / serving 选型使用

选型提示

把 inference-perf 放在 GenAI benchmark 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

inference-perf 架构与设计思路分析

原文: raw/inference-perf-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/inference-perf> · 优先级 P1

一句话定位

GenAI inference performance benchmarking tool，用于对 OpenAI-compatible/serving endpoint 做负载、延迟和吞吐测量。

核心架构图

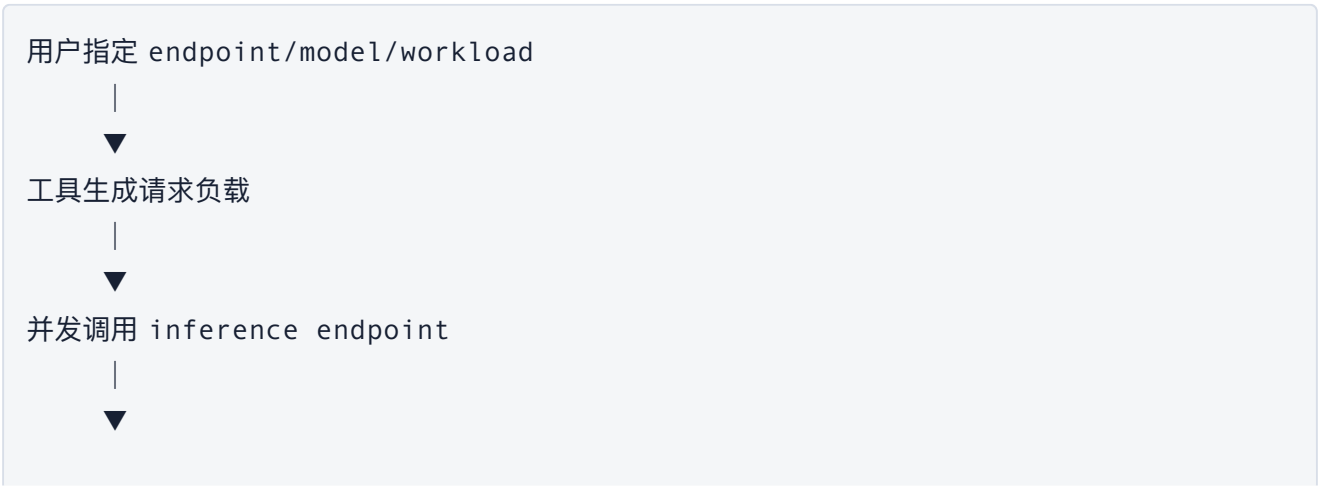




模块分层

层 / 模块	职责
CLI/config	benchmark 参数与 endpoint 配置
Load generator	并发、请求分布、payload 模板
Metrics collector	latency/throughput/error/token stats
Reports	结果输出供 llm-d-benchmark / serving 选型使用

关键数据流



收集 TTFT/ITL/latency/throughput

|



输出 benchmark report

设计决策与哲学

- 补齐 GenAI benchmark 维度: inference-perf 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: 和 llm-d-benchmark 相比, inference-perf 更像单个 benchmark harness; llm-d-benchmark 负责更完整的实验生命周期。
- 选型价值: 它应和 [llm-d-benchmark](#), [llm-inference](#), [llm-d-inference-sim](#) 一起看, 而不是孤立评估。

相关页面

- [inference-perf](#)
- [kubernetes](#)
- [llm-d-benchmark](#)
- [llm-inference](#)
- [llm-d-inference-sim](#)

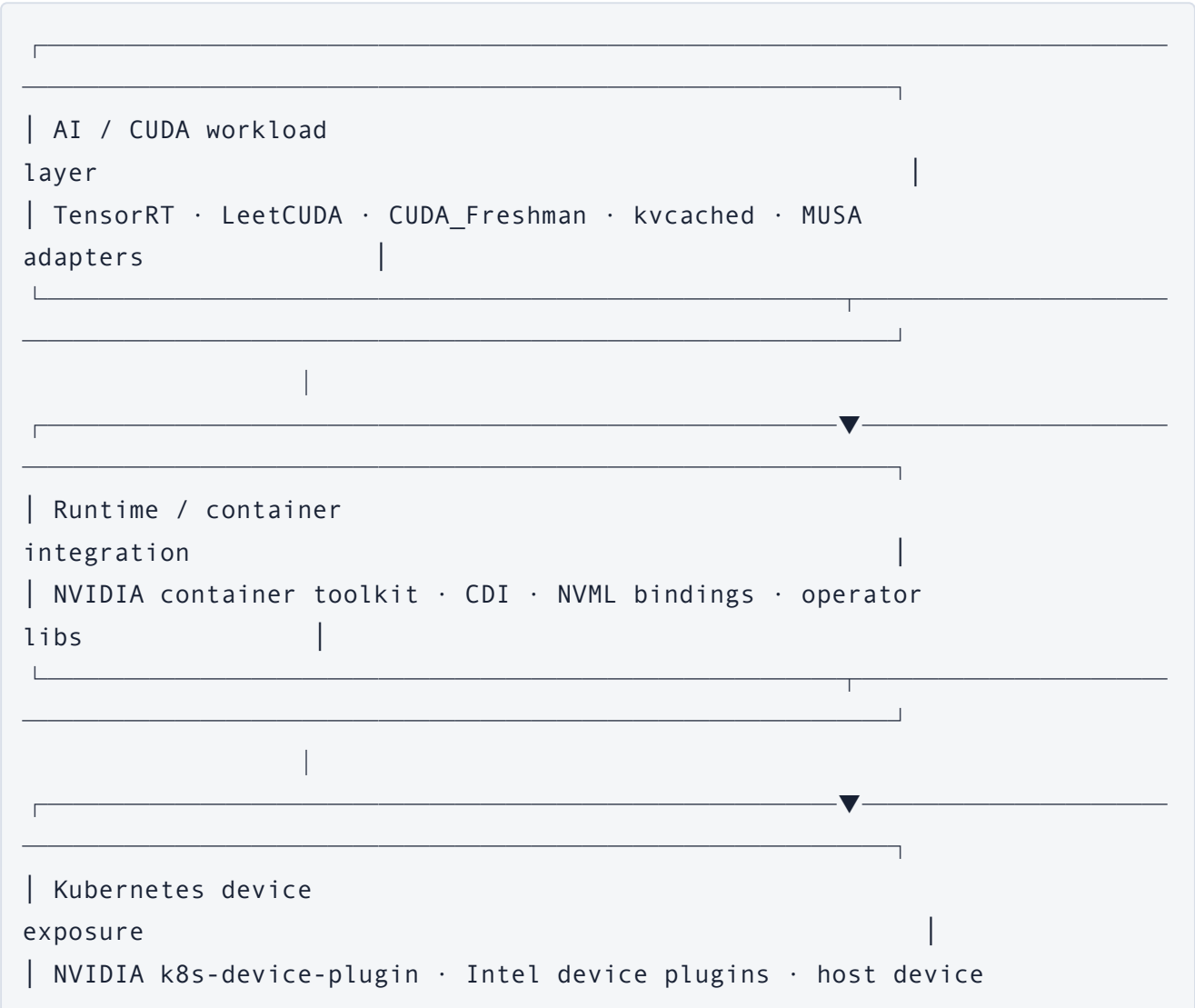
K8s GPU & Device Plugins Star 项目清单整理

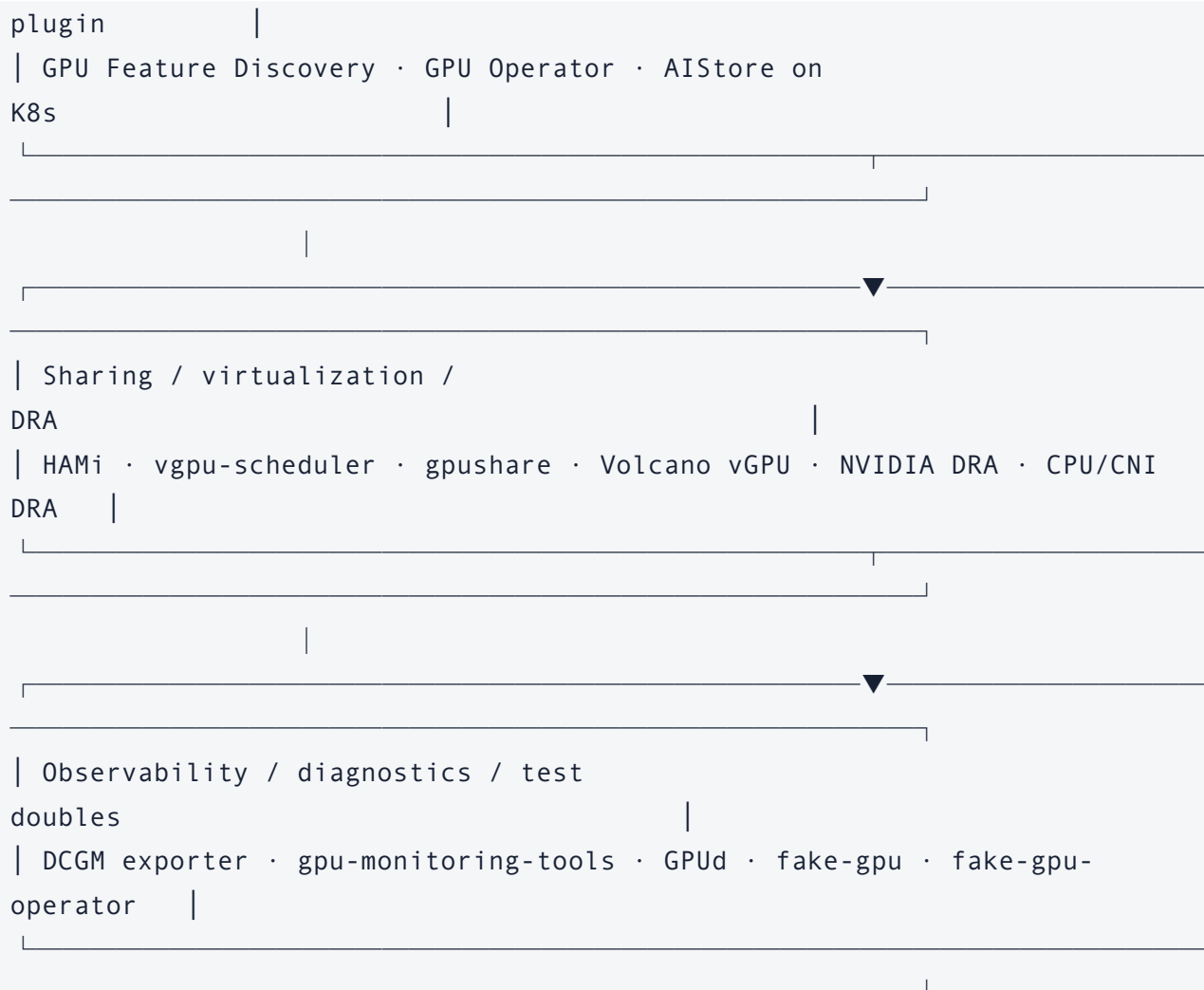
原文: raw/k8s-gpu-device-plugins-stars-list.md · 来源: [googs1025 的 K8s GPU & Device Plugins Stars list](#) · 抓取日期: 2026-06-04 · 仓库数: 36

一句话定位

这个 Star list 聚焦 kubernetes 异构设备资源层: 从最传统的 device plugin、GPU Operator、container toolkit、DCGM/NVML 监控, 到 vGPU / GPU sharing、Dynamic Resource Allocation、CDI、fake GPU 测试环境, 再延伸到 CUDA 学习、TensorRT、KV cache 虚拟化这类 GPU workload 能力。

分层地图





观察

- Kubernetes GPU 资源层已经从“device plugin 把卡暴露出来”扩展成完整栈：driver/runtime/operator、feature discovery、metrics、diagnostics、admission、scheduler、sharing 和 DRA 都是独立问题。
- vGPU/GPU sharing 仍有多条历史路线并存：gpushare、TKE gpu-manager、4paradigm vgpu-scheduler、HAMi、Volcano vGPU；其中 HAMi 是当前更值得深挖的社区化代表。
- DRA 是下一代设备资源抽象的主线：NVIDIA GPU DRA、CPU DRA、CNI DRA、DRA example driver 说明 Kubernetes 正在把“设备分配”从 device plugin 扩展成更声明式、更可组合的资源 API。
- GPU 可观测不只是 metrics：DCGM exporter/NVML 是数据源，GPUd/fake-gpu/fake-gpu-operator/knavigator 面向诊断、测试和调度系统开发。
- [llm-inference](#) 把 GPU 资源层进一步复杂化：kvcached 把 KV cache 也推向可虚拟化资源，和 GPU sharing、[kv-cache-offload](#)、DRA 之间有明显交叉。

优先深挖候选

优先级	项目	原因
1	NVIDIA/k8s-device-plugin	Kubernetes GPU device plugin 标准参照。
2	NVIDIA/gpu-operator	生产 GPU 集群运维主入口。
3	Project-HAMi/HAMi	当前 GPU sharing / vGPU 方向最值得系统分析的项目。
4	kubernetes-sigs/dra-driver-nvidia-gpu	Kubernetes DRA + GPU 的下一代资源 API 样板。
5	cncf-tags/container-device-interface	CDI 是 container runtime 侧设备声明标准，和 DRA/device plugin 相邻。
6	NVIDIA/dcgm-exporter	GPU metrics 和 Prometheus 集成标配。
7	leptonai/gpud	GPU 诊断从 metrics 走向自动化 health/issue detection。
8	ovg-project/kvcached	KV cache 资源化和 GPU sharing/LLM serving 的交叉点。

相关页面

- [kubernetes](#)
- [llm-inference](#)
- [kv-cache-offload](#)
- [vllm](#)
- [sglang](#)
- [dynamo](#)
- [disaggregated-serving](#)

Kthena

Kthena 是 Volcano 社区的 Kubernetes-native AI serving platform，覆盖模型生命周期、请求路由、P/D 分离、扩缩、限流、canary、拓扑感知和 gang scheduling。

核心抽象

- **ModelBooster**：一站式部署 API，级联创建下游 serving 资源。
- **ModelServing**：以 ServingGroup × Role 表达推理 workload，支持 Prefill/Decode、恢复、滚动更新、拓扑和 gang 调度。
- **ModelServer**：通过 workload selector 发现并暴露推理 Pod，承载重试、超时和连接策略。
- **ModelRoute**：按模型、LoRA、路径或 header 匹配，支持权重流量、canary、token rate limit。
- **AutoScalingPolicy**：指标驱动、panic/stabilization window、异构资源和 P/D role-level 扩缩。

架构边界

Kthena 的显著设计是 control plane 与 data plane 解耦：workload controller 和 router 可分别安装，router 也能接入 Gateway API。它比纯 model operator 更靠近完整 LLM serving platform；比 RBG 更直接提供模型/路由 API；与 [llm-d](#) 的差异在于更强调 Volcano 拓扑/gang 和自身的 ModelRoute/ModelServing 资源模型。

适用与限制

适合希望在 Kubernetes 内获得一体化 LLM serving、P/D role autoscaling、KV/LoRA-aware routing 和 Volcano 调度的团队。若组织已经标准化 Gateway API/InferencePool 并希望最大化复用该生态，应同时比较 [llm-d](#)；若主要需求是跨多云 GPU 管理和 MaaS，应比较 [gpustack](#)。

KubeAI

KubeAI 是轻量 Kubernetes AI inference operator，使用 Model CRD、OpenAI-compatible model proxy、model loader 和 autoscaler 管理 LLM、VLM、embedding、speech-to-text 等服务。

解决的问题

把“模型文件 + 推理 engine”快速转换成 Kubernetes 内可调用的 OpenAI-compatible endpoint，并在模型闲置或流量变化时执行加载、卸载和扩缩，降低自托管 AI 服务的上手成本。

核心边界

KubeAI 偏应用友好和快速部署，不负责重新实现 vLLM/SGLang kernel，也不是以多角色 P/D、复杂 KV locality 或 GPU topology 为中心的编排层。复杂场景通常需要额外的 Gateway、调度或 serving stack。

适用场景

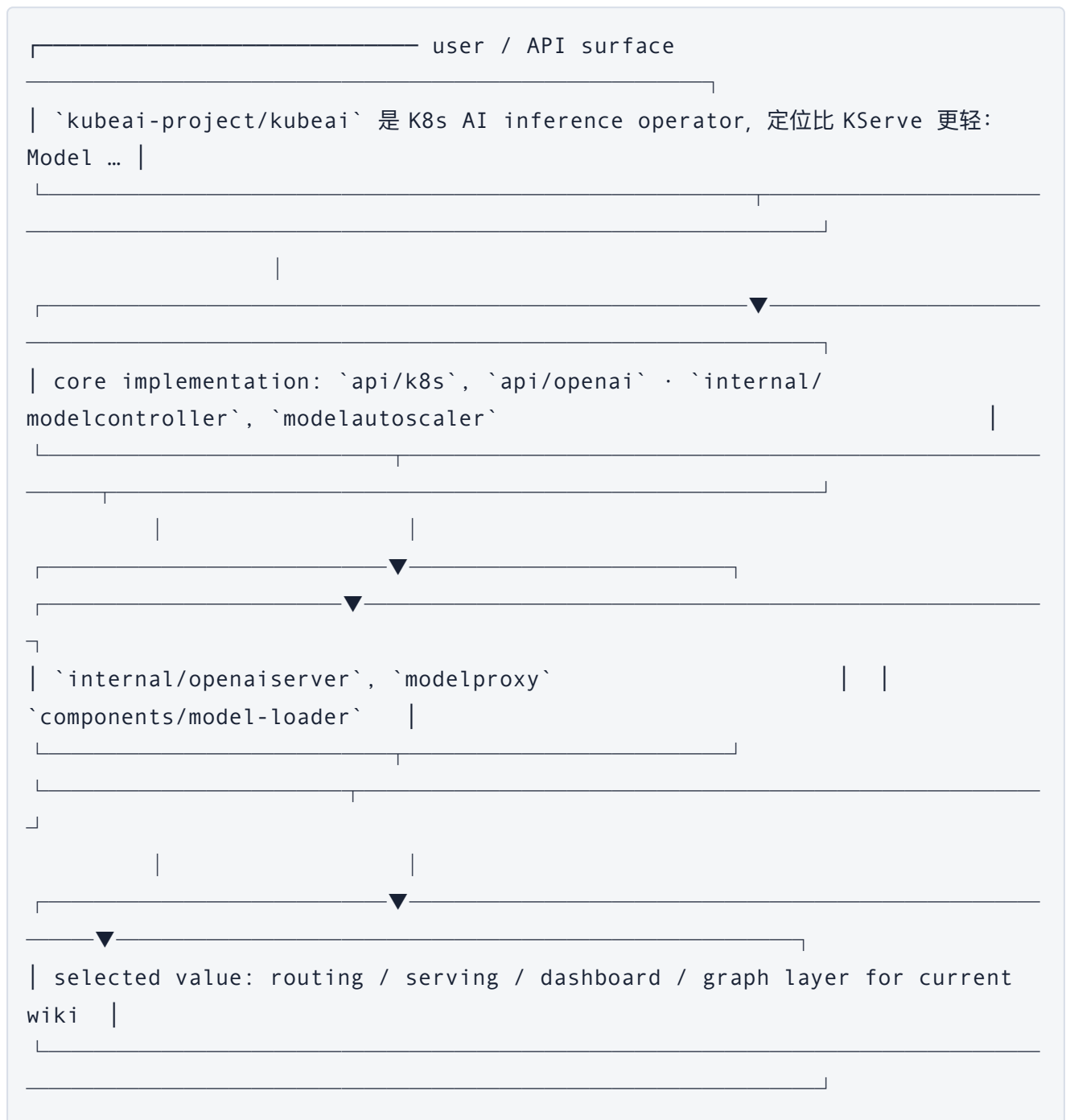
适合中小规模自托管、模型种类较多、希望 scale-from-zero 和统一 API 的团队。需要 predictive + GenAI 统一平台看 kserve；需要 enterprise vLLM 组件看 [aibrix](#)；需要深度 P/D/KV 看 [dynamo](#) 或 [kthena](#)。

详见 [src-kubeai-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

KubeAI 架构与设计思路分析

kubeai-project/kubeai 是 K8s AI inference operator，定位比 KServe 更轻：Model CRD、OpenAI-compatible server、model proxy、autoscaler、model loader、vLLM client、Helm charts 和内置模型 manifests。

核心架构图



模块分层

层/目录	责任
api/k8s, api/openai	K8s API 和 OpenAI API 类型。
internal/modelcontroller, modelautoscaler	Model controller 和扩缩容。
internal/openaiserver, modelproxy	OpenAI-compatible endpoint 和代理。
components/model-loader	模型加载组件。

关键数据流

1. 用户创建 Model CRD 或安装 charts/models。
2. controller 创建 pod/deployment 并注入 model labels。
3. OpenAI server/model proxy 统一暴露 endpoint。

设计决策

- 以 Model 为核心资源，简化用户心智。
- 内置模型 charts/manifests 适合快速上手。
- 能力覆盖广但高级 routing/KV 优化弱于 llm-d/AIBrix。

对比定位

和 KServe 相比更轻；和 GPUStack 相比更 Kubernetes operator；和 OME 相比路线更直接。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

Kubernetes Device Plugin

Kubernetes Device Plugin 是 kubelet 扩展机制，让厂商把 GPU、FPGA、RDMA 等特殊硬件注册为节点资源，并在 kubelet 调用 Allocate 时返回设备与容器配置。

在 M5-C Device / GPU 地图中的位置

Device Plugin 向 kubelet 注册资源名与 endpoint，通过 ListAndWatch 发布设备/health，再由 kubelet 更新节点 extended-resource capacity/allocatable。scheduler 按节点资源选择位置，节点侧 Allocate 返回 env、mount 或 cdi 交接信息；完整流程见 [k8s-gpu-device-stack](#) 的 D2。node-feature-discovery 提供能力标签，[gpu-operator](#) 管受管组件，均不替代本次设备分配。

[dra-driver-nvidia-gpu](#) 的 Claim/库存/Prepare 路径见 [kubernetes-dra-design-deep-dive](#)。两种 API 不应让同一设备被重复分配；例如 hami 与 NVIDIA 官方 plugin 不能在同一节点争用 `nvidia.com/gpu`，共存须明确资源所有权与安装模式。

核心流程

设备发现后建立 plugin Unix socket，向 kubelet 注册并通过 ListAndWatch 报告库存/健康；Pod 请求资源并绑定节点后，kubelet 选择设备 ID、调用 Allocate，再经容器配置将设备交给 runtime。

代表项目

[k8s-device-plugin](#) 是 NVIDIA 官方实现，覆盖 NVML/CUDA/Tegra/VFIO discovery、MIG/MPS/time-slicing、health check、env/volume/CDI device list strategy。

与 DRA 的关系

DRA 不是简单替代所有 device plugin，而是为更复杂的声明式资源配置提供新 API。理解 device plugin 仍是理解 GPU on Kubernetes 的基础。

Kubernetes Dynamic Resource Allocation

Kubernetes DRA 是新一代特殊设备资源分配模型，用 ResourceClaim、ResourceSlice 和 driver plugin 让 GPU、DPU、RDMA 等设备的配置与调度更声明式、更可扩展。

在 M5-C Device / GPU 地图中的位置

基础 DRA 自 Kubernetes v1.35 stable；binding conditions、consumable capacity、partitionable devices、optional node operations 等扩展独立分阶段，见 [kubernetes-dra-design-deep-dive](#) 的「当前上游核验（2026-09-27）」表及其官方证据。ResourceClaim allocation 是控制面分配，NodePrepareResources 是节点准备，[cdi](#) 是 runtime 注入交接，不能将任一步成功当作全链路已就绪。

[k8s-gpu-device-stack](#) 对比 [k8s-device-plugin](#) 与 [dra-driver-nvidia-gpu](#) 的路径：node-feature-discovery 的节点信号和 [gpu-operator](#) 的组件管理不取代 Claim 分配，hami 的 DRA 接入也不自动改变容器隔离强度。

和传统 device plugin 的区别

传统 [device-plugin](#) 主要把节点设备暴露成 extended resources，Pod 请求后由 kubelet Allocate。DRA 把资源 claim、参数、slice 和第三方 driver 放到调度语义里，让设备厂商能表达更复杂的配置，例如 dynamic MIG、VFIO、ComputeDomain。

代表项目

[dra-driver-nvidia-gpu](#) 是 NVIDIA GPU 的 DRA driver 实现，覆盖 ResourceClaim validation、NodePrepare/Unprepare、ResourceSlice、dynamic MIG/VFIO 和 ComputeDomain。

Upstream 进展

2026-07-15 复核 kubernetes/enhancements 后，DRA 的近期重点已经从 ResourceSlice / ResourceClaim 基础模型，扩展到 workload 级 claim、可消费容量、共享容量、设备兼容组、ResourceClaim device status、DRA attributes Downward API 和标准 numaNode 属性。详见 k8s-v1.37-scheduling-node-dra-progress 和 [kubernetes-dra-design-deep-dive](#)。

对 GPU/AI 平台来说，最值得跟踪的是 5729-resourceclaim-support-for-workloads、5075-dra-consumable-capacity、4817-resource-claim-device-status、5304-dra-attributes-downward-api 和 6072-dra-standard-numanode，因为它们把设备容量、状态和拓扑从 driver 私有逻辑推进到 Kubernetes 可推理 API。

选型提示

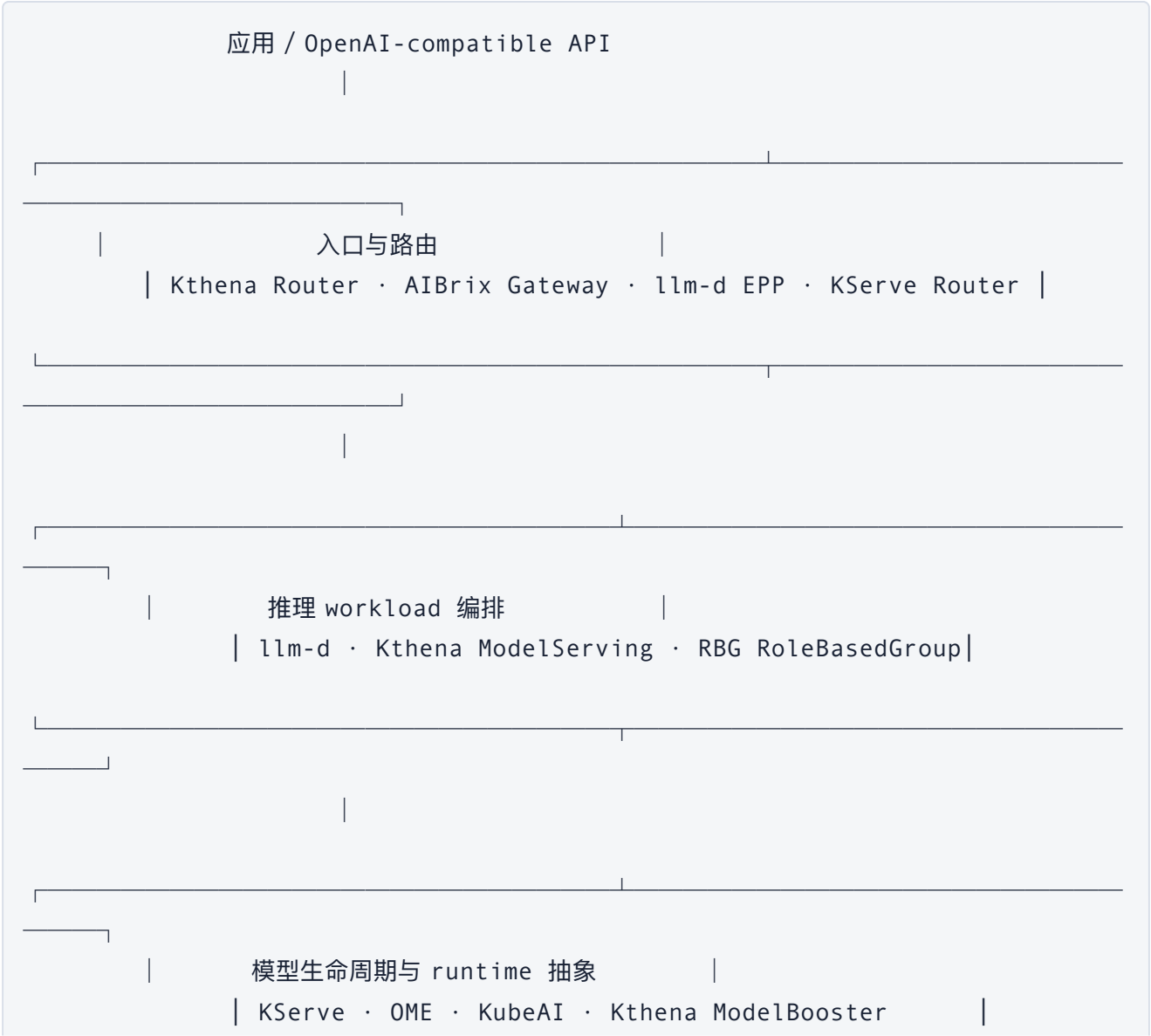
选型应比较目标 release 的设备需求、driver 支持和运维路径：简单 extended-resource 场景可用 [k8s-device-plugin](#)；需要 Claim、属性选择或跨 Pod 生命周期时评估 DRA。基础 API 已稳定，但动态 MIG、共享和其他扩展仍须逐项核对 Kubernetes 与厂商支持范围，不能从基础 DRA 的阶段推导整套部署成熟度。

Kubernetes Serving Stack 对比：llm-d、AIBrix、KServe、KubeAI、OME、GPUStack、RBG、Kthena

原文: raw/k8s-serving-stack-comparison-2026-09-13.md · 资料范围: 各项目当前 README 与官方文档

核心判断

这八个项目不在同一个抽象层：llm-d、AIBrix、Kthena 偏分布式推理 serving；KServe、OME、KubeAI 偏模型服务 control plane；GPUStack 偏 GPU/MaaS 平台；RBG 偏多角色、有状态 workload 原语。

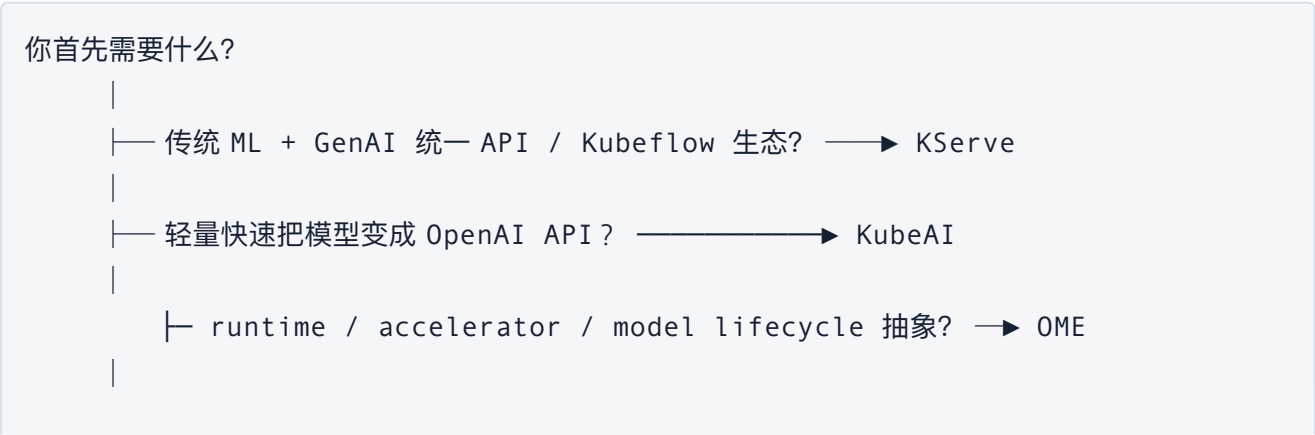


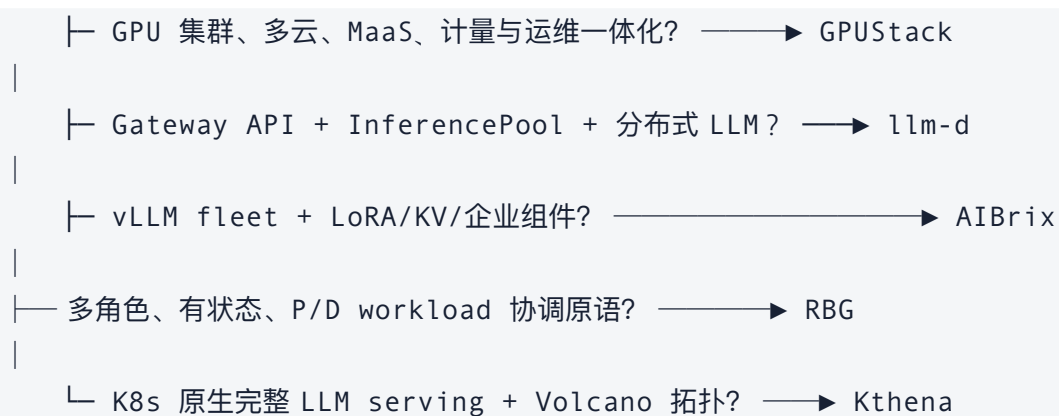


对比结论

项目	最适合	不要误解成
llm-d	Gateway API + InferencePool + 分布式 LLM	一个新的推理 engine
aibrix	vLLM fleet、LoRA、KV、企业 serving 组件	通用模型 API 标准
kserve	predictive + generative AI 的统一 K8s 平台	只针对 LLM 的专用 router
kubecai	快速部署 OpenAI-compatible 模型服务	复杂 P/D topology controller
ome	model/runtime/accelerator 生命周期抽象	KV-aware traffic plane
gpustack	GPU 集群、多云、MaaS、计量运维	轻量 K8s CRD 库
rbg	多角色、有状态、协调式 inference workload	完整 API gateway
kthenea	K8s 原生 LLM serving、路由、P/D、Volcano	只负责模型生命周期的 operator

选型路径





这组比较补充了 [llm-inference](#) 的 K8s serving stack 层，也把 [model-serving-operator](#) 与 [inference-routing](#) 的边界拆开：有的项目管模型生命周期，有的管在线请求选择，有的管 GPU 资源，有的提供多角色 workload 原语。

KV Cache Offload (KV 多级缓存)

把 LLM KV 缓存按访问频率分布在 GPU 显存 → CPU pinned → 本地 NVMe → 远端对象存储 多级存储层上，自动升降级，扩展 effective context length 超出单 GPU 显存上限的方法论。

在 M4 中的位置

KV offload 是 [llm-inference-serving-project-map](#) 中 D4 KV 生命周期的一条分支。Engine 管理本地 KV 布局；索引、感知 KV locality 的路由、跨 worker 传输和远端存储层可由外围 serving 层提供。评估时要明确这些能力分别由哪个组件承担。

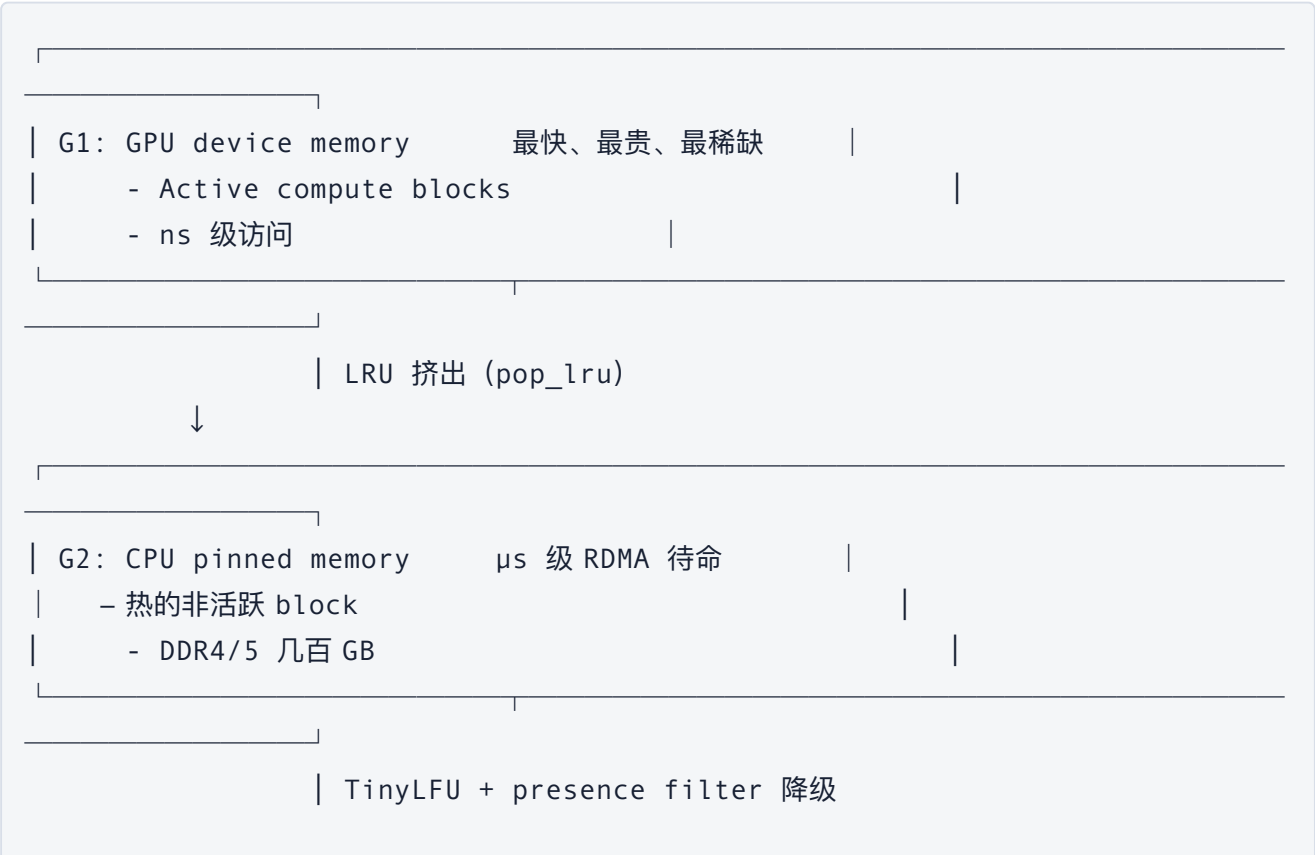
为什么需要多级

GPU 显存有限 (A100 80GB / H200 141GB / B200 192GB)，但生产 LLM serving 面临：

- 长 context 需求：128K / 1M token 的 system prompt + 多轮对话累积
- 多副本共享：同一份 system prompt 被几千个请求复用，但 KV 重新算太贵
- 代价不对称：算 KV $\approx$ 数百 ms (prefill)，从 CPU 读回来 $\approx \mu s$ ，从 SSD $\approx$ ms

把不活跃的 KV blocks 从 GPU 挤到便宜的存储层，等需要时再拉回来，比每次重新 prefill 划算得多。

四级层次 ([Dynamo KVBM](#) 实现为参考)





KV 查找、提升与重新计算流程

请求需要 prefix KV → GPU 命中？

```

├─ 是 → 直接 attention
└─ 否 → CPU 命中？
    ├─ 是 → async load → promote GPU
    └─ 否 → NVMe/远端命中？
        ├─ 是 → transfer → promote
        └─ 否 → 重新 prefill
  
```

GPU pressure → 选择 refcount=0 的 block → 写入下层 tier
→ 更新 block identity/locality → 发布 cache event

关键机制

- 块身份必须全局唯一：KV block 在所有层、所有 worker 中需要有同一个名字。Dynamo 用 128-bit SequenceHash (XXH3 + LoRA id)，让块在 GPU/CPU/SSD/S3 之间认得出。
- 升降级策略：高频用 LRU 控制 G1→G2，冷热分明用 LFU 控制 G2→G3。Dynamo KVBM 用 TinyLFU + presence filter（跳过已在目的层的 block）。
- 零拷贝传输：跨层传输用 GPUDirect RDMA / GDS (GPU Direct Storage) / 对象存储 SDK 减少 CPU 参与。NIXL 把这些统一成一个抽象。

- 去重 / 归并：多副本独立产出的同一 prefix 应该只存一份。Dynamo 用 consolidator 按 SequenceHash 合并跨进程事件。
- 弱引用避免悬空：offload pipeline 持有 weak ref, evict 后自然失效，不会"已经移走了还以为在原地"。

与 [PagedAttention](#) / [RadixAttention](#) 的关系

- PagedAttention 解决"单 GPU 内 KV 怎么按 block 管"
- RadixAttention 解决"多请求间 prefix 怎么免重算"
- KV cache offload 解决"KV 装不下时怎么挤出去 / 找回来"

三者正交、互补：现代推理栈通常同时部署。

与 [Disaggregated Serving](#) 的协同

分离式 P/D 把 KV 从 prefill worker 传到 decode worker 时，本质上也是一种 "offload"——只不过目的地是另一个 GPU 而不是本地 CPU/SSD。Dynamo 把这两件事都包装成 NIXL MemType，传输路径同形。

与 [llm-d-kv-cache](#) 的关系

[llm-d-kv-cache](#) 更偏 KV locality index，而不是完整 offload manager。它订阅 vLLM/SGLang KV events，维护 block 到 pod/tier 的命中信息，并把 score 提供给 [llm-d-router](#)。因此它回答“请求打到哪里 KV 命中更高”；[dynamo](#) KVBM / [kv-cache-offload](#) 回答“KV 应该放在哪一层、如何迁移、如何读回”。

收益与代价

收益：

- 长 context 不再被 GPU 显存上限卡死
- 多请求的 system prompt KV 跨集群共享（router 配合）
- 显存压力下降 → 更大 batch / 更多并发

代价：

- 多一层调度逻辑（什么时候升 / 什么时候降）
- 不命中时延迟跳到 μs / ms / 秒级
- 跨层传输需要高速互联或对应硬件（GDS / RDMA）

相关页面

- 旗舰实现：[dynamo](#) KVBM ([src-dynamo-architecture](#) "KVBM 四级层次"节)
- 路由索引：[llm-d-kv-cache](#) ([src-llm-d-kv-cache-architecture](#))
- 协同概念：[paged-attention](#)、[radix-attention](#)、[disaggregated-serving](#)
- 上位概念：[llm-inference](#)

KVCacheD

本地源码分析: [src-kvcached-architecture](#), HEAD 884108704f44; 三项目关系知识地图: [kvcached-sglang-vllm-knowledge-system](#); 逐函数调用链: [kvcached-source-code-deep-dive](#)。

一句话定位

KVCacheD 是面向 [vllm](#) 与 [sglang](#) 的 GPU KV cache 虚拟内存插件: 给引擎提供地址稳定、容量较大的虚拟 KV tensor, 再按请求真实使用的 block/token 动态映射 CUDA/HIP 物理页, 使同一 GPU 上的多个模型实例弹性共享 VRAM。

它在系统中的边界

KVCacheD 管理:

- KV tensor 的 GPU virtual address reservation 与 physical page backing。
- engine logical block/token 到 VMM page 的映射、释放、回滚和 resize。
- SGLang/vLLM allocator、pool、tensor allocation 和 capacity profiling 的适配。
- TP/PP worker map/unmap 一致性、shared-memory limit 和可观测性。

KVCacheD 不管理:

- 请求排队、batching、preemption policy 和模型 forward。
- attention kernel、block table/RadixCache 的核心算法。
- 跨实例共享同一份 prefix KV 内容。
- 通用 GPU 调度、硬件分区或 hostile multi-tenant 隔离。
- 模型权重本身; 可选 controller 只是调用引擎已有 sleep/release API。

架构边界

```
vLLM / SGLang
  scheduler · prefix metadata · block/token IDs · attention kernels
                                | allocator/tensor contract
                                ▼
KVCacheD engine adapters
                                | logical block operations
                                ▼
KVCacheManager → PageAllocator → FTensorAllocator
                                | VA map/unmap
```

最重要的三个抽象不要混淆：engine block/token page 是调度和 attention 的逻辑单位；KVCacheD page 是 GPU VMM 物理映射单位；attention kernel block 是 backend 执行单位。三者大小可以不同。

与 vLLM 的关系

[vllm](#) 的 scheduler-side BlockPool/KV coordinator 与 worker-side KV tensor 常分属不同进程。KVCacheD 因此替换 BlockPool 为 ElasticBlockPool，在 GPUModelRunner 创建 VMM-backed tensor，并让 EngineCore 通过 Unix socket 把相同 offset 的 map/unmap 广播到所有 TP/PP worker。queued/async execution 下，物理页释放前还有 worker collective barrier。

保留的 vLLM 语义包括：block table、request hash、APC、null block、ref count、scheduler preemption、attention backend 和 bind_kv_cache。

与 SGLang 的关系

[sglang](#) 的 Scheduler、token allocator、KV pool 和 [radix-attention](#) 围绕 token index 工作。KVCacheD 将 token/paged allocator 和 MHA/MLA/Mamba/hybrid memory pool 替换为 elastic 版本，但保留原生 RadixCache、ScheduleBatch、ModelRunner 和 attention backend。当前每个 TP worker 本地拥有并驱动自己的 KVCacheD pool。

何时使用

- 同卡常驻多个模型，权重可以共存，但各模型 KV 峰值错开。
- compound AI 或 multi-agent 流程中，多个专用模型负载间歇出现。
- 推理与 fine-tuning、diffusion 等任务共置，需要 KV cache 主动让出 VRAM。
- 能固定 engine/PyTorch/backend 版本并维护 GPU correctness regression。

何时不使用

- 单模型已经长期吃满 GPU，几乎没有可复用的 idle KV reservation。
- 需要跨节点 KV transfer/offload、prefix 内容共享或 KV-aware routing；应看 [kv-cache-offload](#) 和 distributed serving 层。
- 需要强租户隔离或通用 CUDA memory quota；应看 [gpu-sharing](#) 的 MIG/MPS/vGPU 路线。
- 目标 engine/backend/layout 超出 adapter 支持范围，且无法锁版本和回归。
- 不能接受 version-aware monkey patch 对上游内部 API 的维护成本。

同类与相邻方案

方案	与 KVCacheD 的关系
paged-attention	vLLM 的逻辑 block/KV indexing; KVCacheD 可作为其下层 physical backing
radix-attention	SGLang 的 prefix metadata/token-index reuse; KVCacheD 管其引用的物理 backing
kv-cache-offload	把 KV 内容迁到 CPU/SSD/远端; 与同卡 GPU VMM 弹性互补而非等价
gpu-sharing	调度、分区和通用 runtime 隔离; KVCacheD 只优化接入引擎的 KV memory

入口

- 完整源码分析: [src-kvcached-architecture](#)
- 三项目知识体系: [kvcached-sglang-vllm-knowledge-system](#)
- 源码深潜: [kvcached-source-code-deep-dive](#)
- 集成与实验: [kvcached-integration-implementation-guide](#)
- 核心概念: [elastic-kv-cache](#)
- 引擎背景: [vllm](#)、[sglang](#)

KVCacheD 与 SGLang、vLLM 架构关系及源码实现

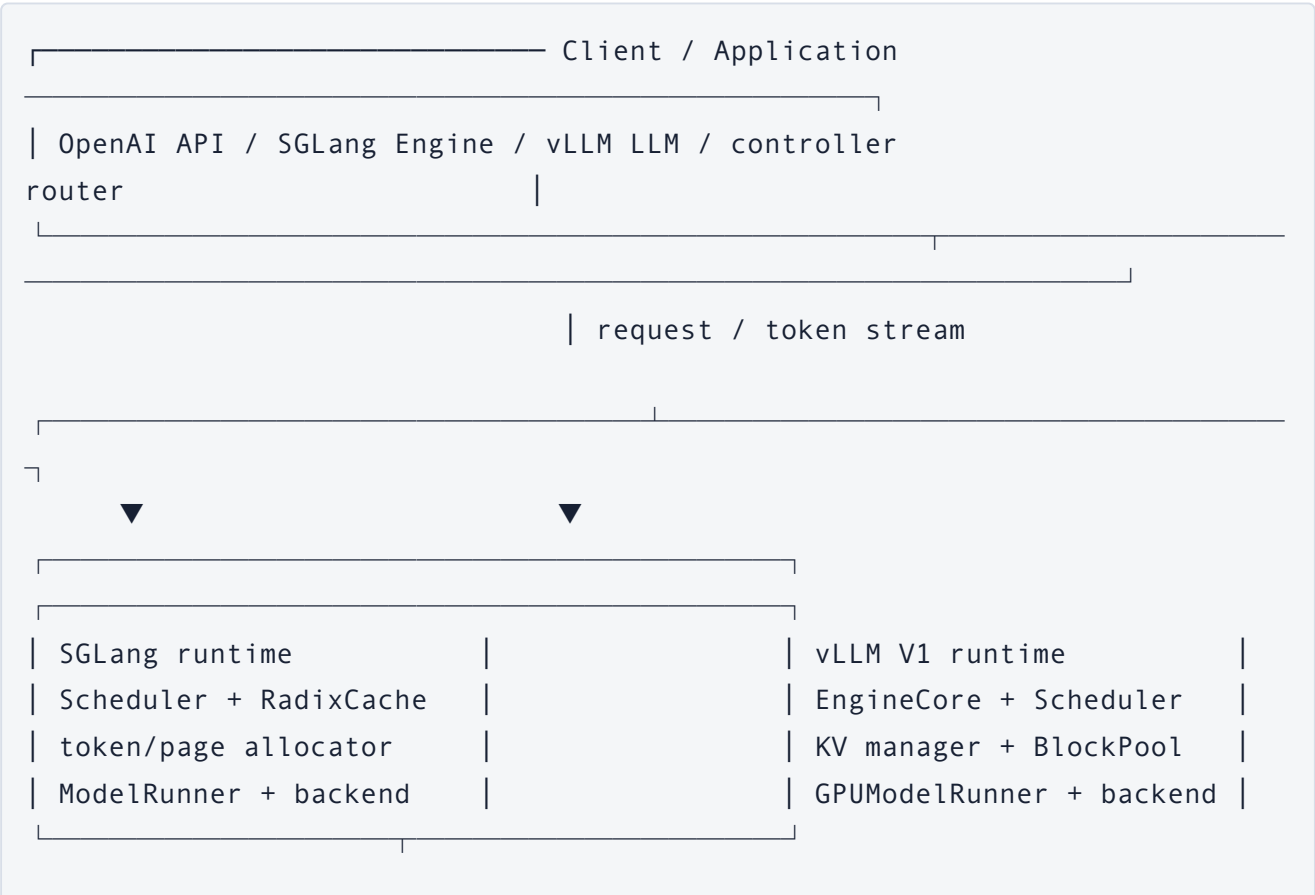
原文: raw/kvcached-architecture-analysis.md · 主仓库: /Users/zhenyu.jiang/kvcached · 对照仓库: /Users/zhenyu.jiang/sglang、/Users/zhenyu.jiang/vllm · 分析版本 KVCacheD 884108704f44 / SGLang 44ef8fecfe69 / vLLM dc36fcce902a

一句话定位

[kvcached](#) 不是新的推理引擎，而是插入 [sglang](#) 与 [vllm](#) KV 分配路径的 GPU 虚拟内存层：引擎仍管理 scheduler、prefix metadata、block/token index 和 attention kernel，KVCacheD 则让稳定的虚拟 KV tensor 按需映射/释放真实 VRAM page，使多个引擎实例在同一 GPU 上弹性共享物理显存。

这是一种 [elastic-kv-cache](#)：它不共享跨进程 prefix 内容，也不等同 [kv-cache-offload](#)、MIG、MPS 或 vGPU。当前采用 .pth + import hook + version-aware monkey patch 接入，两套上游引擎源码不直接依赖 KVCacheD。

核心架构图



| patched allocation/free/tensor creation |



KVCached integration layer

| version-aware patches · SGLang/vLLM adapters · layout/geometry validation |

| Elastic token/page allocator · ElasticBlockPool · prefix-cache bound |



KVCached Python control layer

| KVCacheManager: block ledger · page affinity · reserve/resize/limit · rollback |

| TP/PP IPC: worker listeners · transactional map/unmap · lifecycle cleanup |

| observability: /dev/shm records · kvctl/kvtop · pool snapshots |



C++ / GPU VMM data plane

| FTensorAllocator → stable virtual address → torch Tensor view |

| PageAllocator → 2 MiB+ logical pages → physical CUDA/HIP allocation handles |

| cuMemAddressReserve/cuMemMap/cuMemUnmap or HIP equivalents |



GPU virtual address space ⇔ physical VRAM

模块分层

层 / 模块	职责
自动注入与版本适配	安装 .pth、注册 import hook、按版本应用 SGLang/vLLM patch
引擎语义适配	将 SGLang token/page pool 或 vLLM BlockPool 接到 KVCacheManager，并构造原生 backend 期望的 tensor view
Python 逻辑分配	维护 block ledger、page affinity、回滚、resize、limit、prefix bound 和延迟释放
多进程一致性	经 Unix socket 向 TP/PP worker 广播映射，以 prepare/commit/abort 保证 unmap 一致性
C++ VMM 数据面	预留 VA、创建 stable tensor、按 offset 映射 CUDA/HIP physical page、维护 reserve/quarantine 状态
外部控制	/dev/shm 使用量和 limit、kvctl/kvtop、可选多模型 router/sleep controller

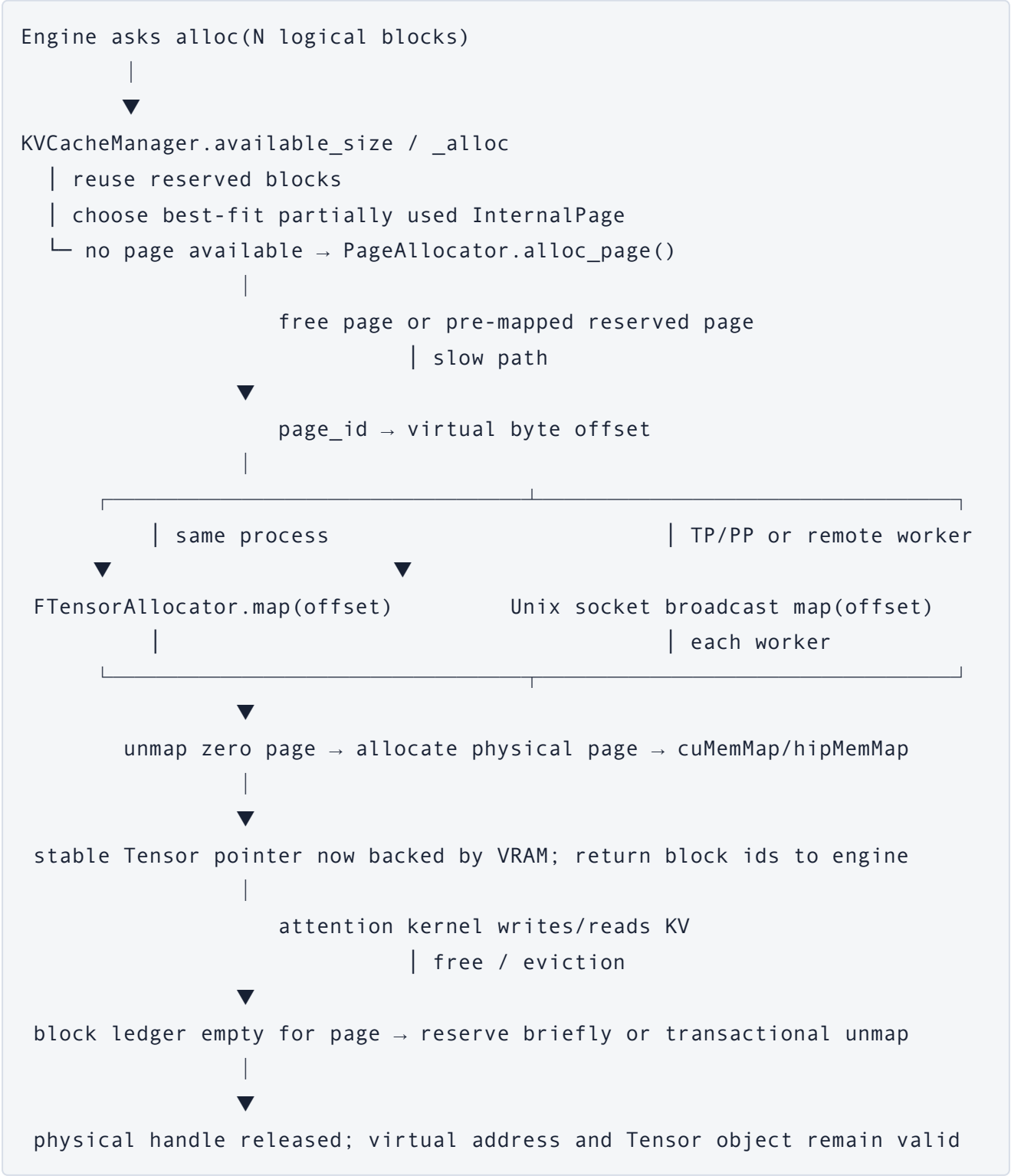
关键数据流

启动与自动注入



ENABLE_KVCACHED=true 时走 elastic path ; 否则保留原引擎行为

逻辑 block 映射为物理 VRAM



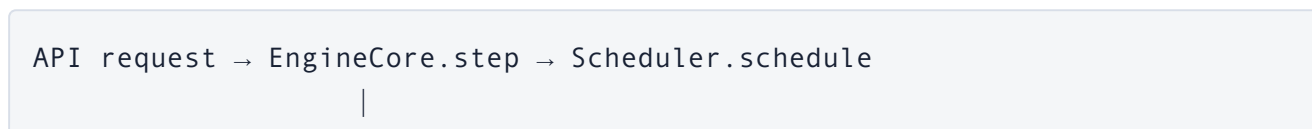
FTensor 先预留虚拟地址并用 zero page 初始化，然后通过 `torch::stable::from_blob` 暴露 tensor。请求触碰新 page 时，在相同 VA 上换入真实 physical page；释放后重新映回 zero page，所以 tensor 指针不变，原生 attention kernel 不需要了解 KVCacheD。

SGLang 集成流



[radix-attention](#) 和 Scheduler 保持原生。KVCacheD 主要替换 token/page allocator、MHA/MLA/Mamba/hybrid pool 的 buffer 创建，并给 RadixCache 加物理内存上限。当前每个 SGLang TP worker 本地拥有自己的 pool；真实 TP width 只用于 listener/rank 信息，manager 不把本地分配重复广播给 peer。

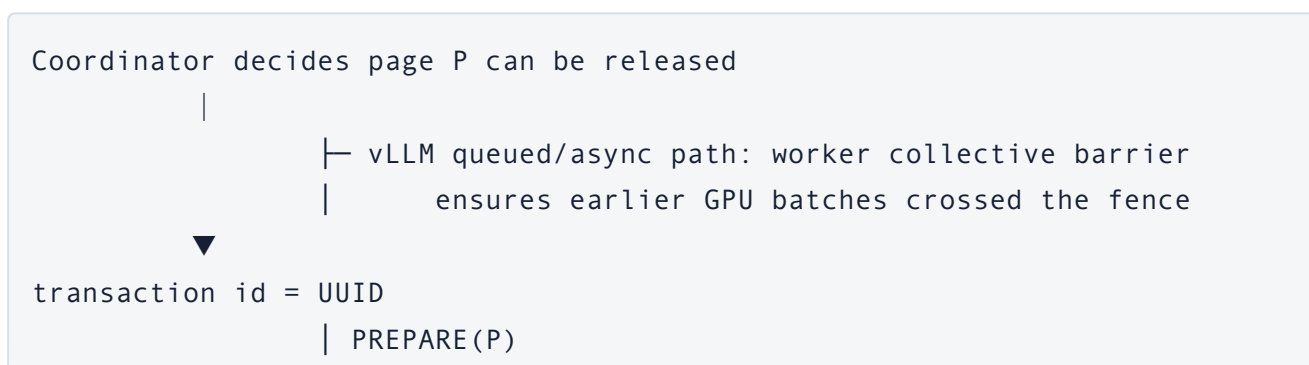
vLLM 集成流





[paged-attention](#)、block table、request hash、scheduler 和 backend 仍是原生逻辑。ElasticBlockPool 复制 null block、ref count、APC full-block cache、duplicate hash 和 LRU 契约，但 block id 的物理 backing 由 KVCachedD 提供。共享池竞争造成的 transient miss 被翻译为 `allocate_slots() -> None`，由 vLLM scheduler preempt/retry。

TP/PP 事务性 unmap





Map 失败可回滚新增映射；unmap 若只有部分 rank 成功，会让 block table 在各 GPU 上含义不同，因此使用两阶段事务。commit 结果不明时只能幂等重试 commit；无法确认一致性就 fail pool，而不是继续产生 silent corruption。

可选多模型控制器



控制器解决进程拉起、路由和 idle sleep；核心 VMM 则解决活跃实例的 KV 物理显存弹性。它不是 Kubernetes operator，也不在 request 热路径里决定 page 分配。

设计决策与哲学

- 稳定 VA、弹性 backing：engine 始终持有稳定 tensor，真实 VRAM 按活跃 block/token 映射。
- 适配 storage，不替换 engine：[vllm](#) 与 [sglang](#) 仍拥有调度、cache metadata、模型执行和 kernel。
- 物理 page 与 engine block 解耦：默认 VMM page 2 MiB，一个 page 可装多个 engine block；只有整页空闲才能归还 VRAM。
- 把共享容量视为动态状态：available_size 不是 reservation；并发实例可使后续 alloc 失败，失败必须进入 scheduler retry 而非 engine crash。
- prefix cache 有界：无限 APC/RadixCache 会 pin 住 physical page，使弹性退化；默认以 cached-token budget 协调命中率与回收能力。
- 一致性优先：map rollback、quarantine、unmap 2PC 和 queued-batch barrier 都选择 fail-loud，避免 KV silent corruption。
- 零上游改动换来版本耦合：plugin 安装简单，但上游内部类、签名、layout、runner 变化都要更新 compatibility patch。

核心组件深入

FTensor 与 KVCacheManager

FTensor 把“地址空间”与“物理 page handle”分开；PageAllocator 管 free/reserved/in-use/quarantined page；KVCacheManager 再把 engine block id 分组到 page。manager 使用 page-affine best-fit，避免长请求散落后由少量 prefix block pin 住很多 page，并在物理竞争中对部分分配做来源感知回滚。

contiguous layout 用一次 compound mapping 覆盖所有 layer/K/V，映射少但 per-layer view 有 stride；non-contiguous 每层独立，适合要求独立 region 的 PD/NIXL 或 ROCm backend。CUDA 默认前者，ROCm 默认后者。

两套引擎为什么不能共用同一适配器

SGLang 的 Scheduler、token allocator 和 KV pool 共同围绕 token index 工作，替换 allocator/pool 即可覆盖主路径。vLLM V1 的 scheduler-side BlockPool 与 worker-side tensor 分进程，EngineCore 分配 block id 后必须通知所有 GPU worker 映射同一 offset，还要在异步批完成后才能 unmap。因此 vLLM patch 覆盖 EngineCore、coordinator、BlockPool、GPUWorker、GPUModelRunner 和 shutdown。

项目关系与边界

问题	结论
是否替代推理引擎	否；是两套 engine 的 KV memory plugin

问题	结论
是否替代 PagedAttention/ RadixAttention	否；保留 block table/radix tree，只改变 backing allocator
是否共享跨实例 KV 内容	否；共享同卡 physical capacity，不共享 cache metadata/data
是否等同 KV offload	否；核心是 GPU VA ↔ GPU VRAM page，P/D/NIXL只是额外兼容
是否提供显存安全隔离	否；只管理接入的 KV path，不是 MIG/MPS/vGPU sandbox
是否管理模型权重	核心不管理；可选 controller 通过 engine sleep API释放更多资源

集成与验证重点

最小接入需要先安装匹配 backend 的 PyTorch 和 KVCacheD，再同时设置 `ENABLE_KVCACHED=true`、`KVCACHED_AUTOPATCH=1`。日志必须出现 patch 成功信息，并通过 `kvctl list/kvtop`、`nvidia-smi` 和固定 seed 请求验证确实进入 elastic path。

生产化应锁定 KVCacheD/engine/PyTorch/GPU backend/attention backend/model 组合，依次验证：单实例正确性、双实例竞争、prefix cache bound、limit 收缩、abort/shutdown、TP、PP、async scheduling、目标 layout/hybrid/quantized path。README 的 tested-up-to 与 HEAD 中新增的更高版本 adapter 不是同一强度的兼容承诺。

不适合采用的情况包括：无法固定和回归 engine 版本；目标 backend/layout 不在 adapter 支持集合；必须使用 KV cache events；需要跨节点 KV 内容共享；需要 hostile multi-tenant 隔离；不能接受 monkey patch 维护成本。

知识体系结论

KVCacheD 的价值不是让单次 attention 计算更快，而是把原来“启动时静态占有”的 KV cache 变成“地址稳定、物理按需、跨实例可控”的资源。理解它的关键不是只记住 GPU VMM，而是同时掌握三个 contract：引擎的 block/token/cache 语义、KVCacheD 的 block-to-page ledger，以及多 rank GPU 执行结束前不可 unmap 的生命周期约束。

相关页面

- [kvcached](#)
- [kvcached-source-code-deep-dive](#)
- [sglang](#)
- [vllm](#)
- [elastic-kv-cache](#)
- [gpu-sharing](#)

- [llm-inference](#)
- [paged-attention](#)
- [radix-attention](#)
- [kv-cache-offload](#)

LeaderWorkerSet

LeaderWorkerSet 用一组 leader/worker Pods 表达一个复制单元，适合 LLM inference、分布式 serving 和需要稳定 group 语义的 workload。详见 [src-lws-architecture](#)。

证据说明：2026-09-25 核验的官方默认分支快照为 [d4f1525f15a4](#)，该 commit 不代表 release。[src-lws-architecture](#) 保留 2026-06-14 的 raw-backed Source 快照；当前跨项目职责见 [kubernetes-workload-gang-scheduling-design](#)，调度器流程见 [kubernetes-scheduler-core-design](#)。

在 M5-B Workload / Scheduling 地图中的位置

在上述快照中，`leaderworkerset.x-k8s.io/v1` 管理同类模型服务的 leader/worker 组及其恢复、扩缩与发布；`disaggregatedset.x-k8s.io/v1` 进一步以角色和 slices 组合子 LWS，协调分离推理的多角色生命周期。它们与 `jobset` 的批作业集合职责不同，也不管理 quota admission：准入通过 `kueue` 集成，最终 placement 由 `kube-scheduler` 及配置的 `scheduler-plugins` 完成。DS 的跨角色编排不等于整组原子准入。

架构边界

JobSet 面向作业集合；LWS 面向长期运行的 leader/worker 服务复制单元。

什么时候用

场景	判断
需要 分布式 workload API 能力	适合，LeaderWorkerSet 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-inference , batch-inference , <code>kueue</code> 组合。

核心组件

- API: LeaderWorkerSet CRD
- Controller: replica group rollout/status
- Pod template: leader/worker roles
- Integrations: serving/HPC/AI workload

选型提示

把 LeaderWorkerSet 放在 分布式 workload API 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

LeaderWorkerSet 架构与设计思路分析

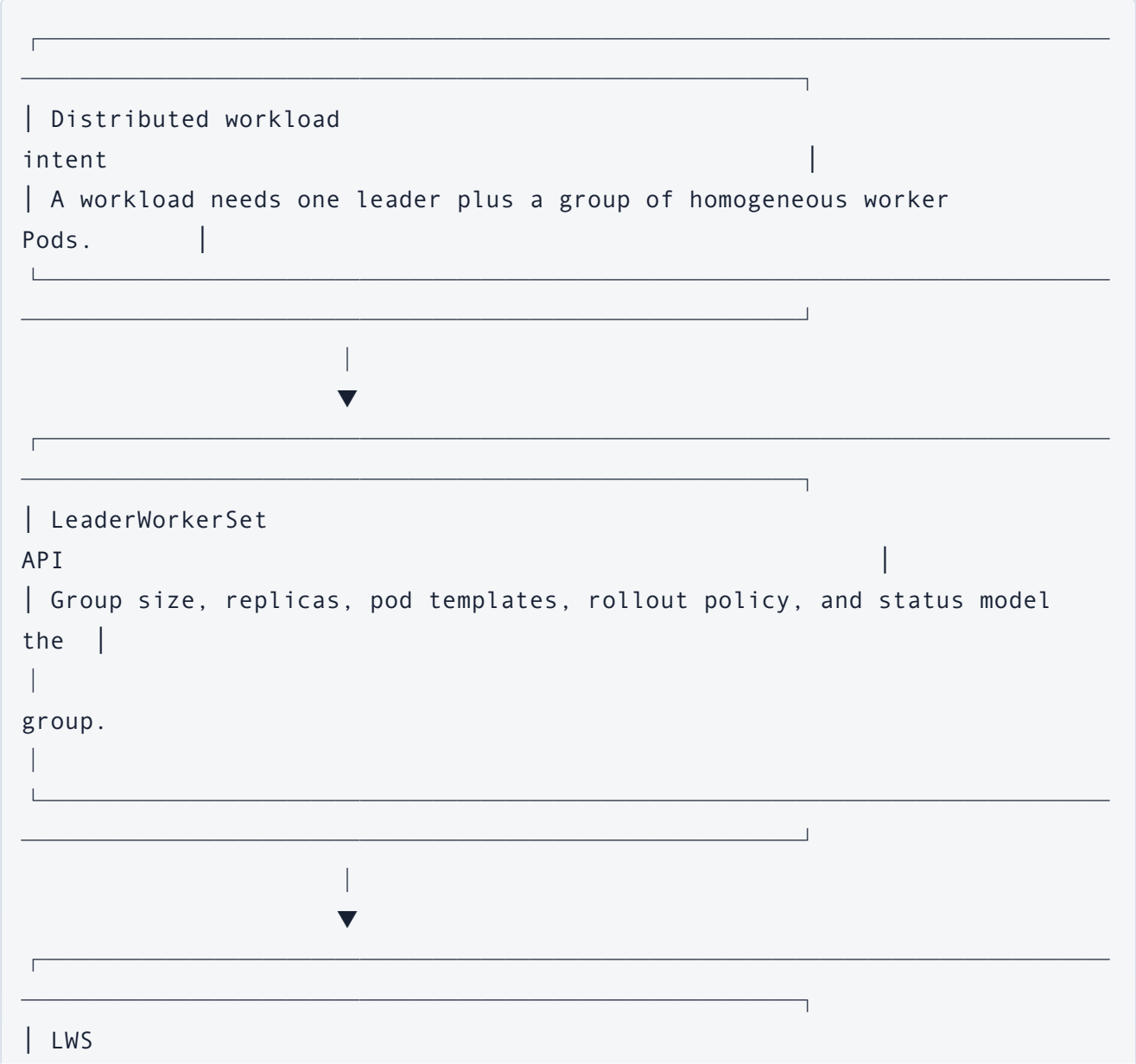
原文: raw/lws-architecture-analysis.md · 仓库: <https://github.com/kubernetes-sigs/lws>

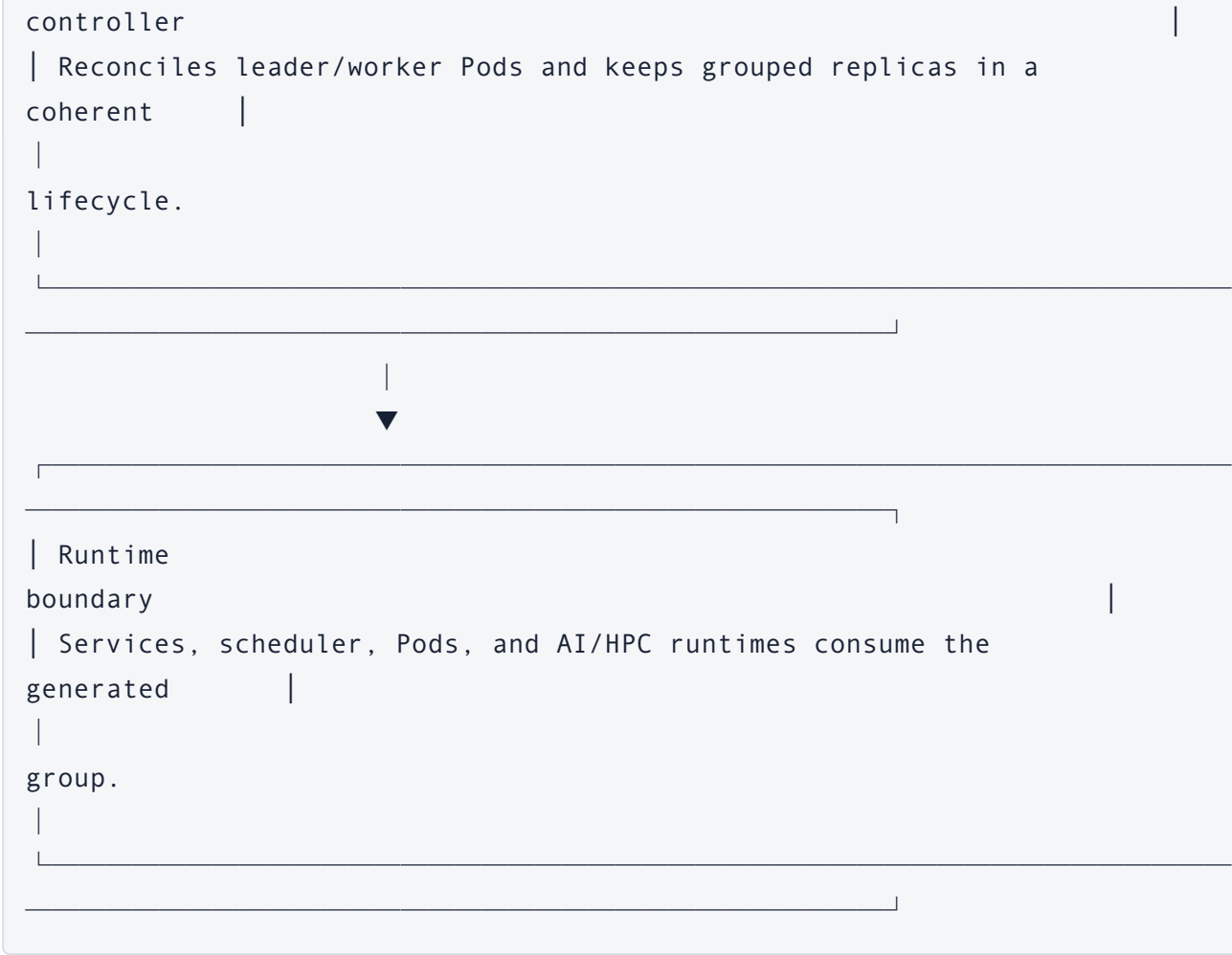
lws · 优先级 P1

一句话定位

LeaderWorkerSet 用一组 leader/worker Pods 表达一个复制单元，适合 LLM inference、分布式 serving 和需要稳定 group 语义的 workload。

核心架构图

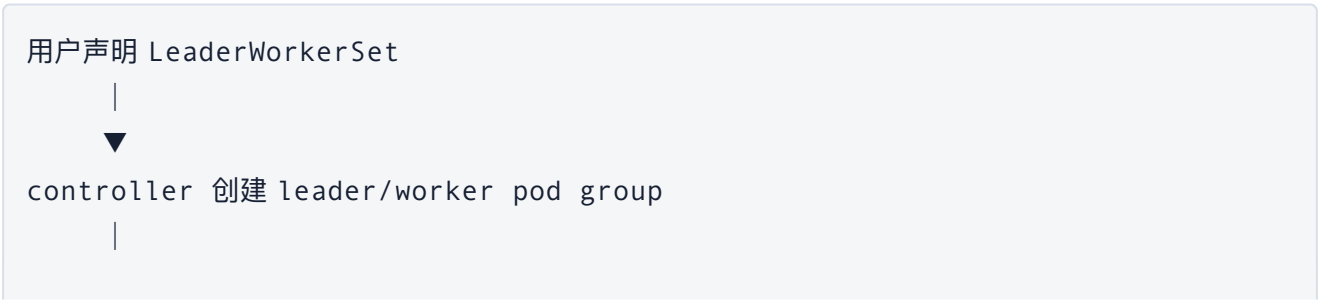


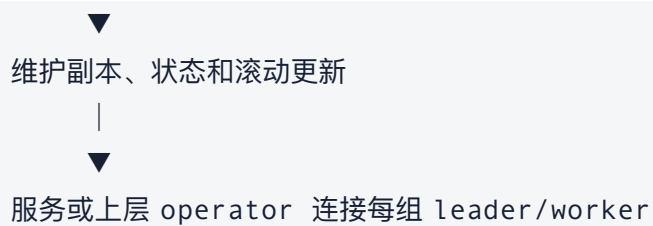


模块分层

层 / 模块	职责
API	LeaderWorkerSet CRD
Controller	replica group rollout/status
Pod template	leader/worker roles
Integrations	serving/HPC/AI workload

关键数据流





设计决策与哲学

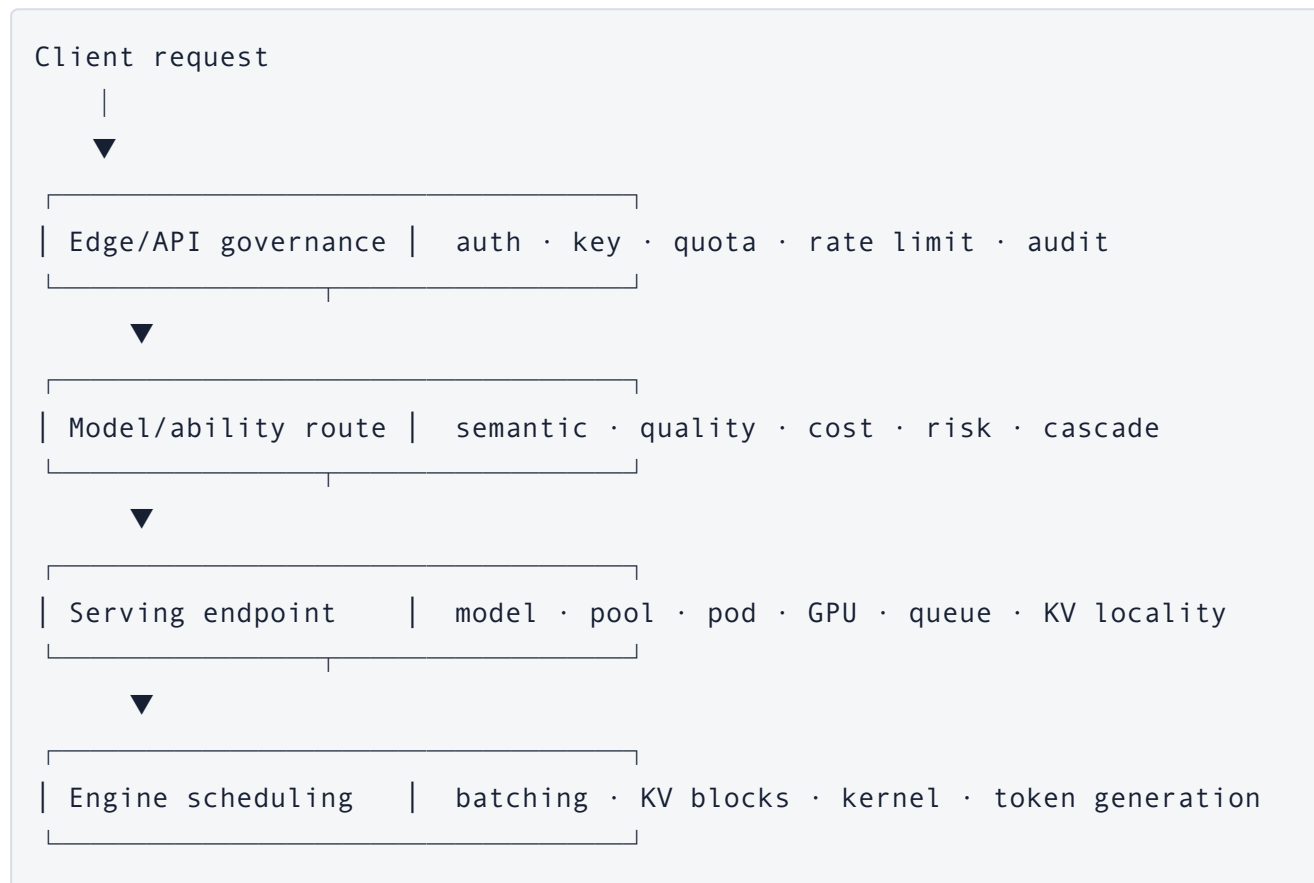
- 补齐 分布式 workload API 维度：LeaderWorkerSet 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：JobSet 面向作业集合；LWS 面向长期运行的 leader/worker 服务复制单元。
- 选型价值：它应和 [llm-inference](#), [batch-inference](#), kueue 一起看，而不是孤立评估。

相关页面

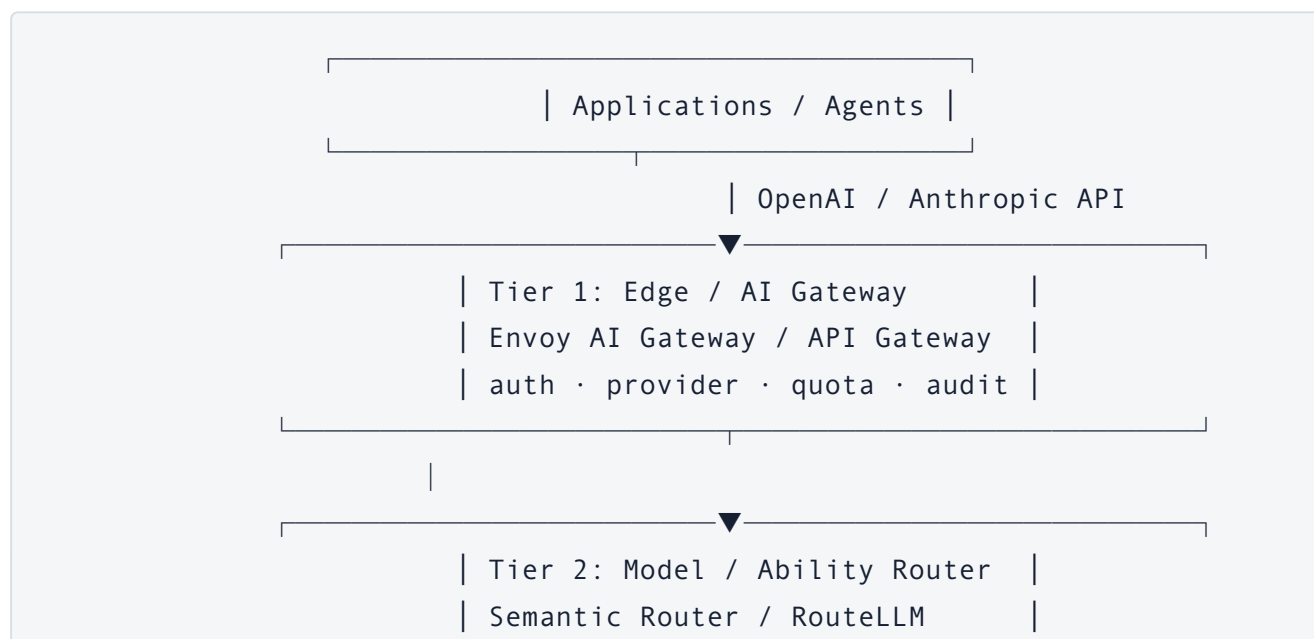
- [lws](#)
- kubernetes
- [llm-inference](#)
- [batch-inference](#)
- kueue

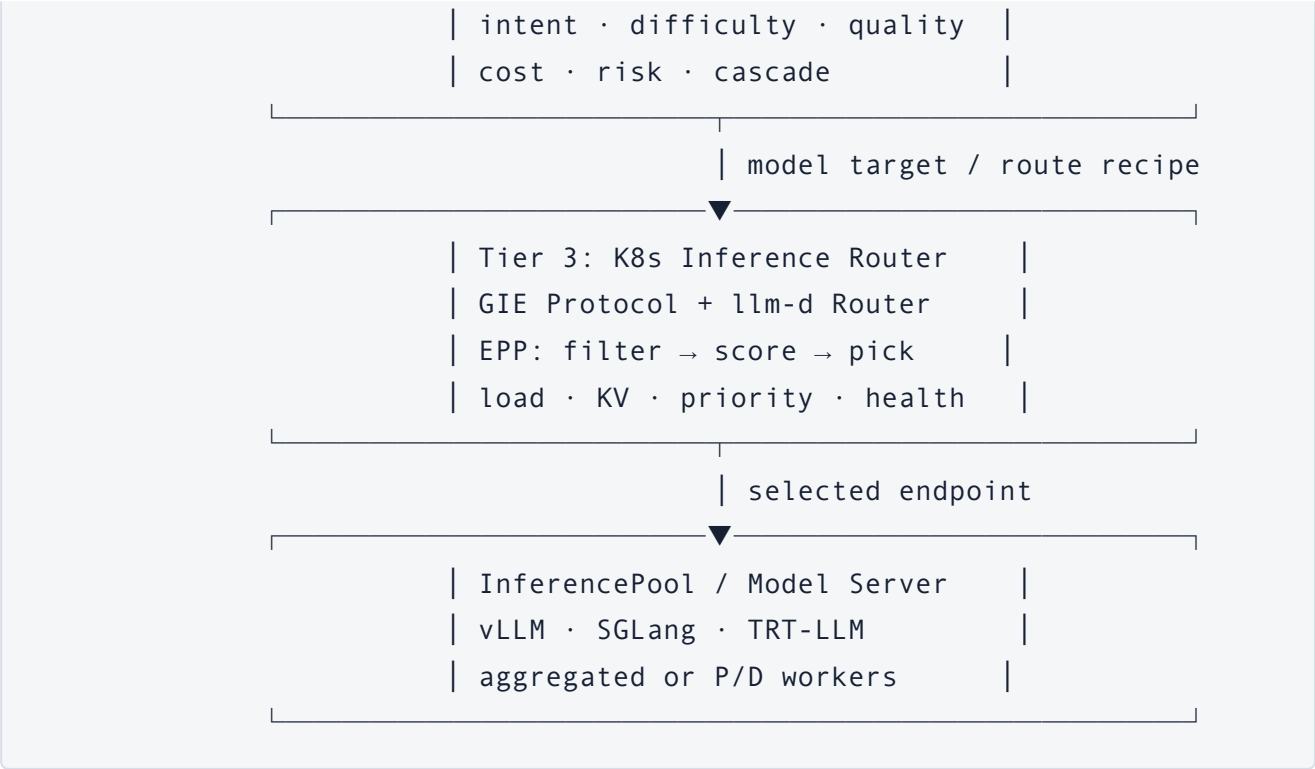
LLM 网关与路由层对比

先分清四类决策



统一架构图



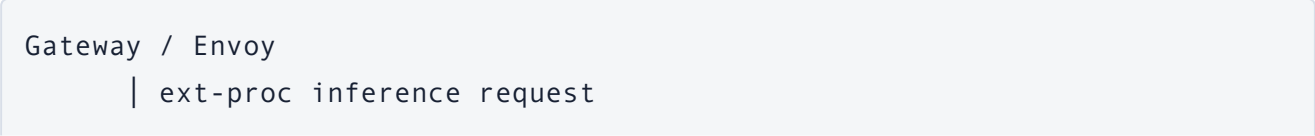


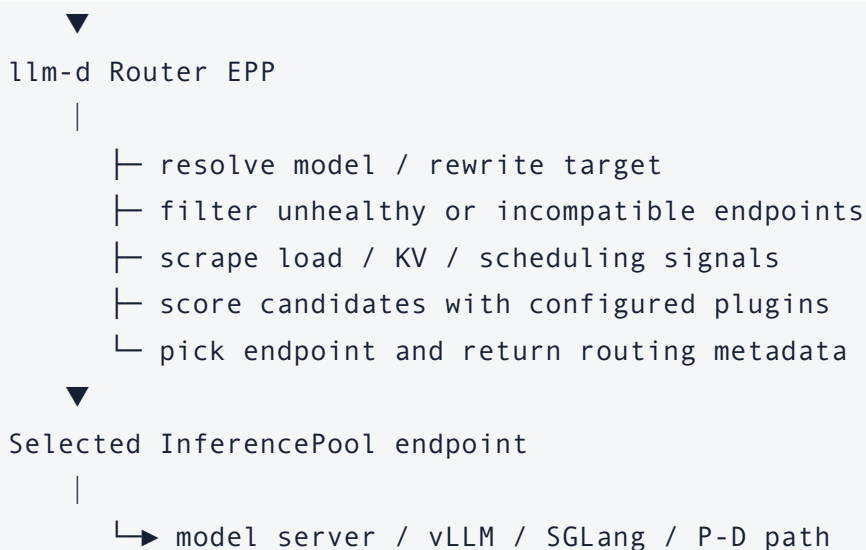
项目定位

项目	决策层	核心输出	Kubernetes 边界
llm-d-router	endpoint/pod/ worker	selected endpoint + routing metadata	EPP/ext-proc、Gateway API、 InferencePool
semantic-router	model/ability/ recipe	model、cascade、 capability path	可部署在 K8s，但不是 GIE 协 议本身
routellm	strong/weak model	cheaper 或 stronger model	SDK/OpenAI-compatible server，不是 operator
gateway-api-inference- extension	inference endpoint protocol	inference endpoint pick	Gateway API 的推理扩展
envoy-ai-gateway	edge/provider governance	provider/model upstream	Envoy Gateway + two-tier gateway

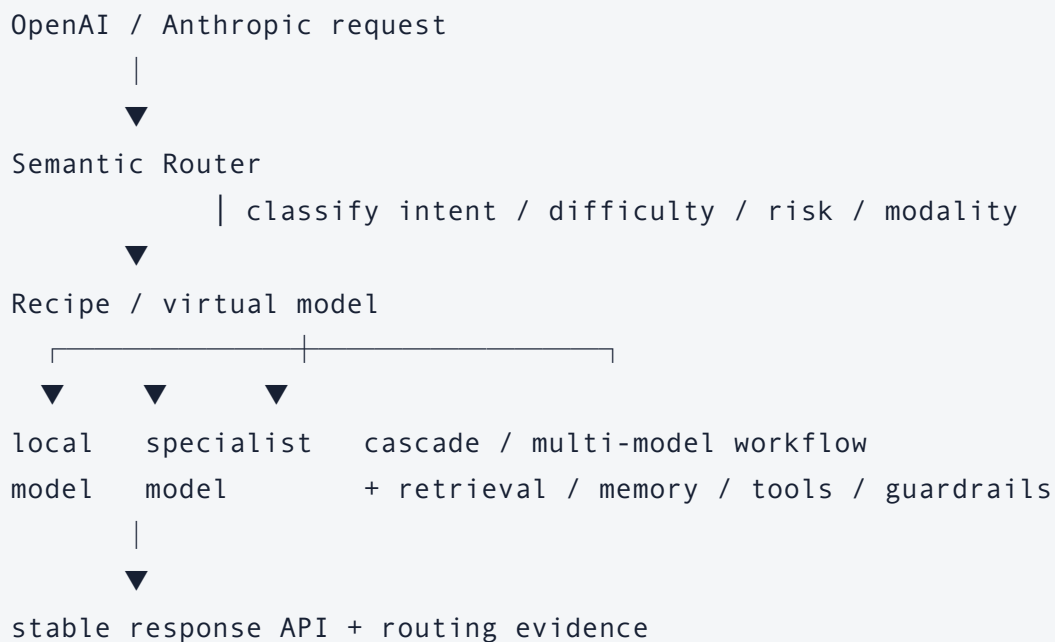
五条请求流程图

llm-d-router

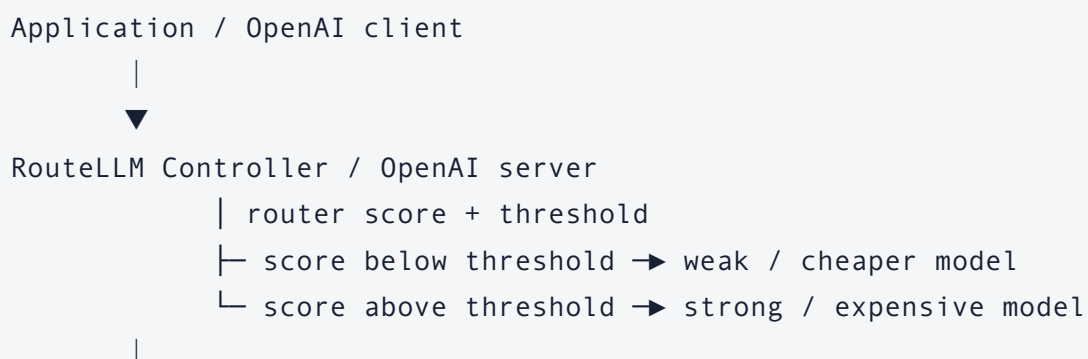


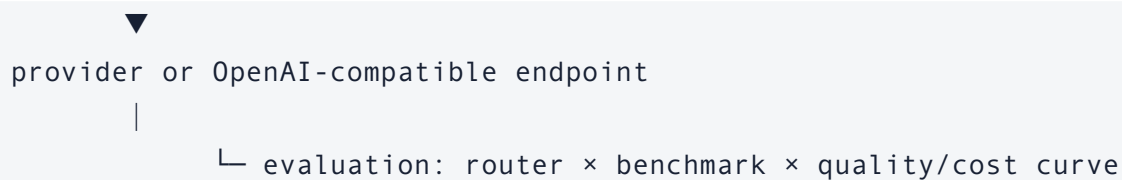


Semantic Router

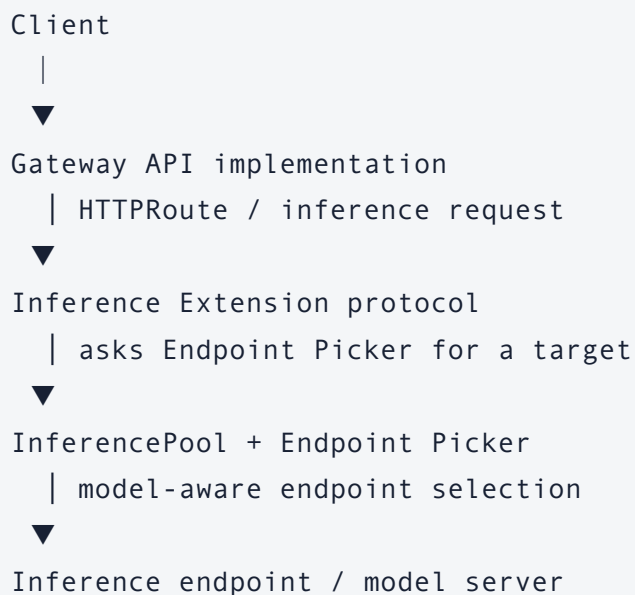


RouteLLM





Gateway API Inference Extension



Envoy AI Gateway



选型结论

- provider、key、quota、限流和审计：envoy-ai-gateway。
- 语义、风险、能力和质量-成本模型选择：[semantic-router](#) / routellm。

- KV、队列、GPU、健康和 InferencePool endpoint 选择: [gateway-api-inference-extension](#) / [llm-d-router](#)。
- 多层组合时, 必须记录每一层的 route decision、模型名、endpoint、策略版本和失败原因。

相关页面

- [inference-routing](#)
- [ai-gateway](#)
- [llm-inference](#)
- [src-k8s-gateway-routing-comparison](#)

llm-d

llm-d 是 Kubernetes-native distributed inference stack，围绕 Gateway API/Inference Extension、EPP、InferencePool 和 model server 组织，并扩展 KV cache、P/D、autoscaling、benchmark、batch 与 simulator。

解决的问题

它把单机 vLLM/SGLang engine 提升为可在 Kubernetes 上运行的分布式 LLM 服务：请求要按 endpoint 状态、KV locality 和推理负载选择，P/D worker 要能协同，容量还需依据可观测指标扩缩。

核心边界

llm-d 是 serving ecosystem，不是新的推理 engine，也不是单一 operator。Gateway/EPP 负责入口和 endpoint picking，InferencePool 表达后端集合，KV/P/D 组件补充性能关键路径。[llm-d-workload-variant-autoscaler](#) 已是 deprecated 的历史设计；新部署应按当前官方指南采用 [EPP metrics → KEDA Prometheus scaler → HPA](#)。

适用场景

适合已经采用 Gateway API、希望标准化多模型/多 endpoint 入口，并需要分布式推理性能的 Kubernetes 平台。若只需单模型快速部署，[kubecai](#) 更轻；若需要更深的 engine runtime/P-D/KV 一体化，可比较 [dynamo](#)；若需要 GPU 集群 MaaS，可比较 [gpustack](#)。

详见 [src-llm-d-architecture](#) 与 [src-k8s-serving-stack-comparison](#)。

在 M4 模块地图中的位置

llm-d 位于 Kubernetes routing/serving stack 层，通过 Proxy/EPP 和 InferencePool 将 Gateway 流量、路由信号与 Model Server Pod 连接起来。职责边界见 [llm-inference-serving-project-map](#)，与 engine 及外围组件的组合选择见 [llm-serving-engine-selection-map](#)。

llm-d Batch Gateway

llm-d Batch Gateway 是 [llm-d](#) 生态里的 OpenAI Batch API 实现，把 `/v1/files`、`/v1/batches`、JSONL 输入输出、任务状态、取消、重试和归档做成一套离线推理控制面。详见 [src-llm-d-batch-gateway-architecture](#)。

架构边界

它不是推理引擎，也不是在线 [inference-routing](#) 组件。它位于用户 batch API 与下游 llm-d Router/model endpoint 之间，负责 [batch-inference](#) 的 job lifecycle、对象存储、PostgreSQL 元数据、Redis/Valkey queue、processor 执行和 GC。

什么时候用

场景	判断
需要 OpenAI-compatible Batch API	适合，用 <code>/v1/files</code> + <code>/v1/batches</code> 承接长时任务。
大量 JSONL 请求需要异步执行	适合，processor 支持 per-model execution plan 和受控并发。
想复用 llm-d Router / serving endpoint	适合，它把 batch workload 接到已有 serving fleet。
只需要在线低延迟请求	不适合，直接看 llm-d Router 、 gateway-api-inference-extension 或 AI Gateway。

同类对比

维度	llm-d-batch-gateway	llm-d Router	kserve / kubeflow
抽象对象	batch job / file / output	request / InferencePool / endpoint	model / inference service
主要状态	PostgreSQL + Redis + object store	endpoint metrics / routing config	Kubernetes API status
关注点	离线 job 可恢复、输出归档、进度	在线 endpoint picking	模型部署生命周期

选型提示

如果要比较 serving stack，先把在线请求和离线 batch 分开。[llm-d-batch-gateway](#) 解决的是“很多请求如何异步、可恢复、可审计地跑完”；[inference-routing](#) 解决的是“一个在线请求应该打到哪个 endpoint”。

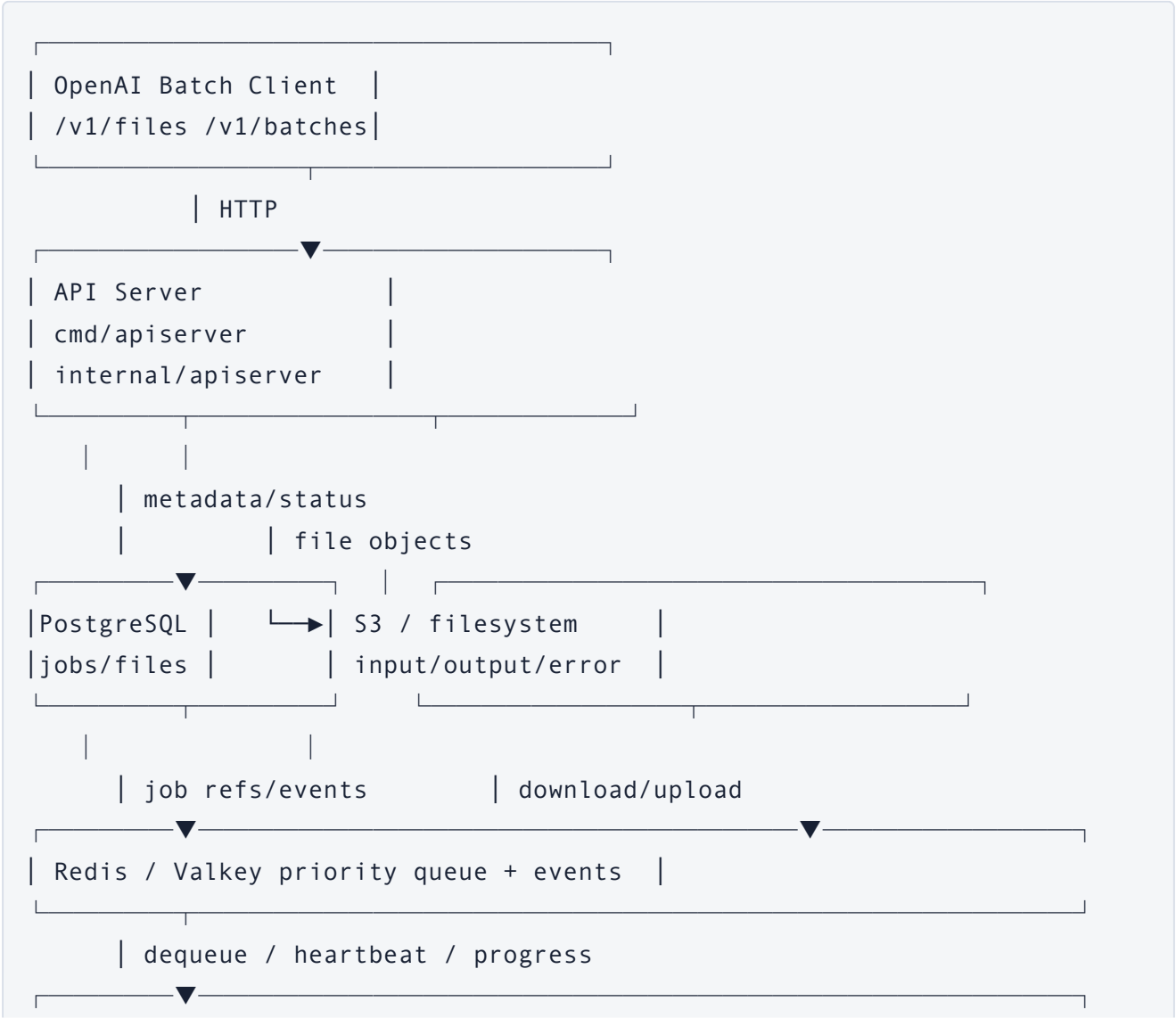
llm-d Batch Gateway 架构与设计思路分析

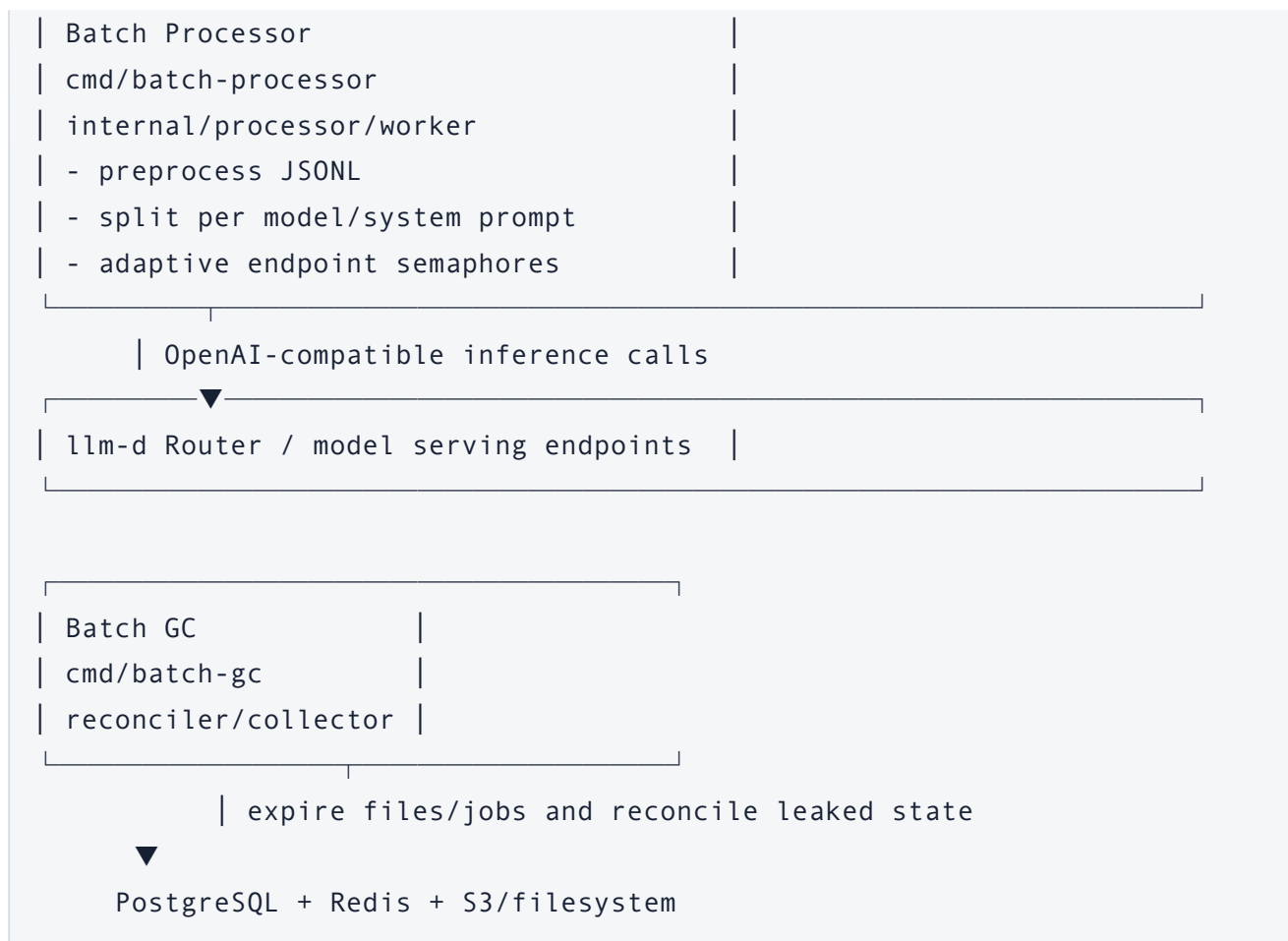
原文: raw/llm-d-batch-gateway-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-batch-gateway> · 分析版本 HEAD 66fae7e

一句话定位

[llm-d-batch-gateway](#) 是 [llm-d](#) 生态里实现 OpenAI Batch API 的离线推理入口，把 `/v1/batches` 和 `/v1/files` 这类长时任务拆成 API server、持久化数据层、Redis/Valkey 队列、batch processor 和 GC。它不替代 [inference-routing](#) 或推理引擎，而是在前面补上 [batch-inference](#) 的 job 生命周期、文件存储、重试、取消、进度和输出归档。

核心架构图





模块分层

层 / 模块	职责
API 入口	提供 OpenAI-compatible batch/file API，校验请求，写入 job/file 元数据，提交队列事件。
任务状态与队列	PostgreSQL 保存 job/file 元数据，Redis/Valkey 保存优先级队列、事件、heartbeat 和进度。
文件存储	管理 input/output/error JSONL 对象，支持本地 filesystem 和 S3/Object storage。
Processor 控制循环	拉取 job、预处理输入、执行推理、写结果、处理取消/过期/停机恢复。
推理客户端	对下游 llm-d Router / model endpoint 发 OpenAI-compatible 推理请求。
垃圾回收	清理过期 job/file，修复 DB/queue/object store 状态漂移。

关键数据流



设计决策与哲学

- Batch job 是持久化控制面，不是一次 HTTP proxy：API server、processor、GC 都以 PostgreSQL/Redis/Object Store 为边界，目标是长时可恢复任务。
- 按 model/system prompt 预处理而不是盲目逐行转发：输入 JSONL 会被解析并生成 per-model 执行计划，让 batch workload 更接近 [llm-d](#) 的 model-aware serving。
- processor 内部用多层并发阀门保护下游：global semaphore、worker token 和 per-endpoint adaptive semaphore 让 batch throughput 受控，而不是无限 fan-out。
- 文件、状态、队列分离：大对象走 S3/filesystem，元数据走 PostgreSQL，调度和事件走 Redis/Valkey。

相关页面

- [llm-d-batch-gateway](#)
- [batch-inference](#)
- [llm-d](#)
- [llm-inference](#)
- [model-serving-operator](#)

llm-d Benchmark

llm-d Benchmark 是 [llm-d](#) 生态的 Kubernetes 实验编排和评测工具，用 llmdbenchmark CLI 把 standup、smoketest、run、result collection、analysis、teardown 串成可复现流程。详见 [src-llm-d-benchmark-architecture](#)。

架构边界

它不是单一压测 engine。真正的负载生成可以来自 inference-perf、GuideLLM、vLLM benchmark 等 harness；llm-d Benchmark 负责渲染 scenario/specification、部署 stack、发现 endpoint、运行 harness、收集结果和保留 workspace。

什么时候用

场景	判断
需要复现实验配置和 Kubernetes manifests	适合，workspace 保留 rendered config、manifest、log、result。
需要比较多套 llm-d stack 或多组参数	适合，global step 顺序执行，per-stack step 可并行。
需要无 GPU 先验证控制面	可与 llm-d-inference-sim 配合。
只想测单个本地 vLLM server 的原始吞吐	可能直接用 harness 更轻。

同类对比

维度	llm-d-benchmark	inference-perf / GuideLLM	vLLM benchmark
主职责	实验生命周期编排	负载生成和测量	engine-specific benchmark
事实来源	workspace + rendered spec	harness config/result	脚本参数/result
适合问题	stack 对比、参数 sweep、部署到结果闭环	单次性能测量	vLLM 基线性能

选型提示

如果目标是“快速理解 [llm-d](#) 部署参数如何影响结果”，优先看 llm-d Benchmark；如果目标是“某个 engine 的 kernel/throughput 极限”，它只是外围编排器，不能替代 engine 自带 benchmark。

llm-d Benchmark 架构与设计思路分析

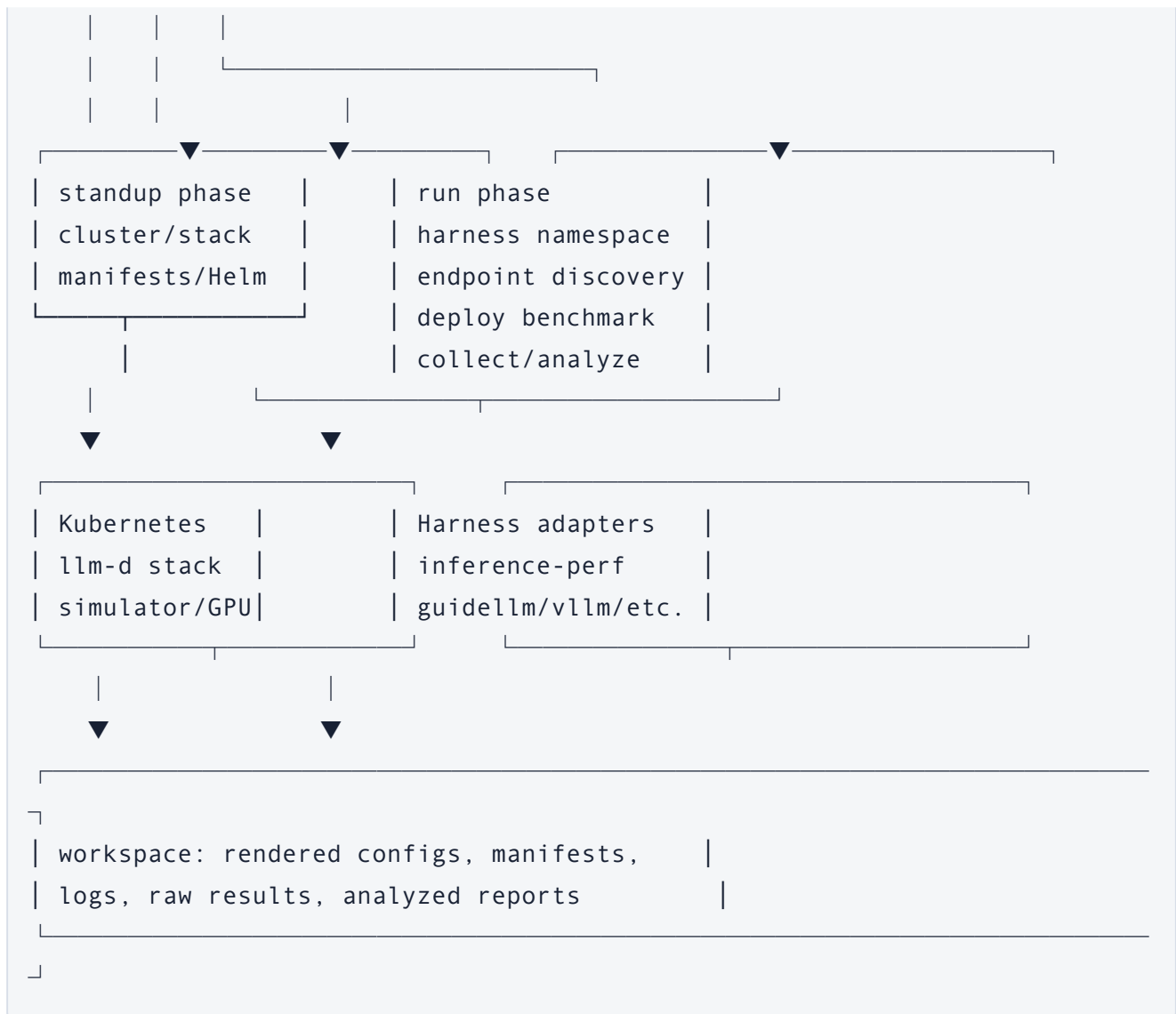
原文: raw/llm-d-benchmark-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-benchmark> · 分析版本 HEAD bd8dc5e

一句话定位

[llm-d-benchmark](#) 是 [llm-d](#) 生态的实验编排和评测工作台，用一个 llmdbenchmark CLI 把 `standup`、`smoketest`、`run`、`result collection`、`analysis`、`teardown` 串成可复现流程。它不是单一压测引擎，而是把 `inference-perf`、`GuideLLM`、`vLLM benchmark` 等工具纳入同一套 Kubernetes 实验生命周期。

核心架构图





模块分层

层 / 模块	职责
CLI 调度	解析命令，初始化 workspace，串起 spec rendering、plan rendering 和 phase execution。
配置模型	defaults → scenario → CLI overrides 的配置合并和实验规格定义。
模板渲染	渲染 Jinja spec、Kubernetes manifests、Helm values 和 stack plans。
执行器	把 global steps 与 per-stack steps 分开，支持多 stack 并行执行和 --stack 过滤。
生命周期阶段	standup / run / teardown / smoketest 的 step 定义。
harness 适配	适配 inference-perf、GuideLLM、vLLM benchmark、nop、inferencemax、aiperf 等压测工具。

关键数据流



设计决策与哲学

- 先渲染再执行：spec、plan 和 manifests 先进入 workspace，再执行 Kubernetes 动作，降低不可复现风险。
- benchmark 编排器不绑定单一压测工具：harness 适配层让 inference-perf、GuideLLM、vLLM benchmark 可以替换。
- global step 顺序执行，per-stack step 并行：适合一次对比多套 [llm-d](#) stack 或多个参数组合。
- workspace 是实验事实来源：rendered configs、manifests、logs 和结果集中保存，适合复现实验和审查差异。

相关页面

- [llm-d-benchmark](#)
- [llm-d-inference-sim](#)
- [llm-d](#)
- [llm-inference](#)
- [model-serving-operator](#)

llm-d Inference Sim

llm-d Inference Sim 是 [llm-d](#) 生态的轻量 vLLM 行为模拟器，用 OpenAI-compatible HTTP、vLLM-like gRPC/API、KV cache events、LoRA lifecycle、延迟模型和 metrics 模拟推理服务。详见 [src-llm-d-inference-sim-architecture](#)。

架构边界

它不做真实模型推理，也不代表真实 GPU 性能。它的定位是控制面和系统策略验证：让 [inference-routing](#)、[llm-d-benchmark](#)、autoscaling、P/D、KV-aware routing 和 failure injection 可以在无 GPU 或低成本环境中先跑通。

什么时候用

场景	判断
验证 router / Gateway / benchmark 流程	适合，协议和 metrics 尽量贴近 vLLM。
测试 KV-aware routing / prefix cache 信号	适合，能生成 block key、PrefixCacheStats 和事件。
需要生产性能数据	不适合，必须用真实 vllm / sglang / GPU。
做 CI 或 Kind 环境 smoke test	适合，比真实模型更便宜稳定。

同类对比

维度	llm-d-inference-sim	vllm	mock HTTP server
推理行为	行为模拟	真实推理	少量 API stub
KV/cache 语义	模拟 block/cache/events	真实 KV cache	通常没有
适合问题	控制面、路由、autoscaling、benchmark 流程	性能和生产执行	客户端单元测试

选型提示

把它当成“系统测试替身”，不要当成性能基线。要理解 [llm-d](#) 的控制面闭环，它和 [llm-d-benchmark](#) 放在一起看最有价值。

llm-d Inference Sim 架构与设计思路分析

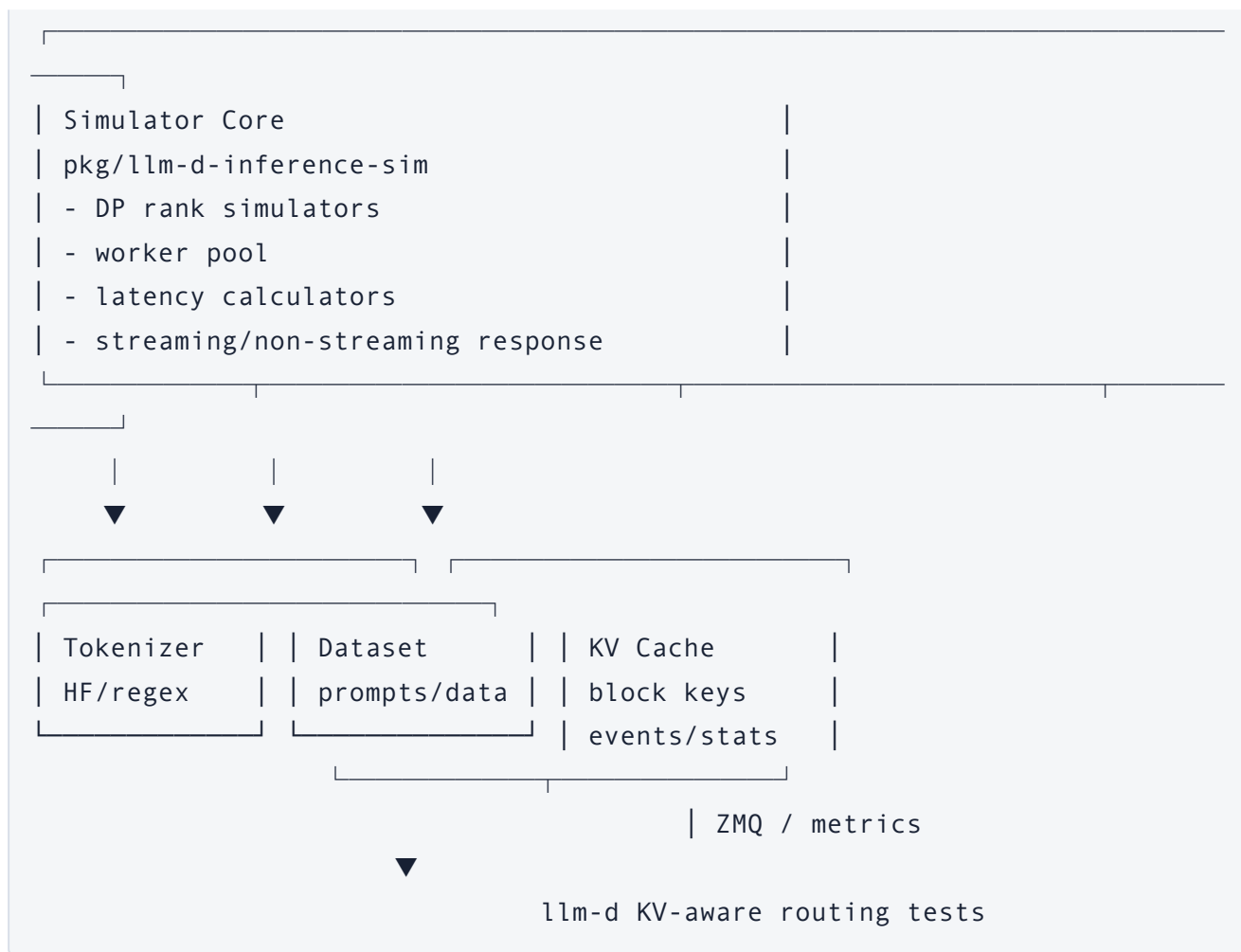
原文: raw/llm-d-inference-sim-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-inference-sim> · 分析版本 HEAD 6fb66f3

一句话定位

[llm-d-inference-sim](#) 是一个轻量 vLLM 行为模拟器，用 Go 实现 OpenAI-compatible HTTP API、vLLM-compatible gRPC/API、KV cache 事件、LoRA 生命周期、延迟模型和 metrics，而不需要 GPU 或真实大模型。它让 [inference-routing](#)、scheduler、[llm-d-benchmark](#)、autoscaler 和 P/D/KV cache 策略能在便宜、可控、可注入故障的环境中被验证。

核心架构图





模块分层

层 / 模块	职责
入口命令	读取配置、初始化 simulator、启动 HTTP/gRPC 通信层和数据集工具。
通信层	同端口 cmux HTTP/gRPC, OpenAI/vLLM/admin/metrics/health routes, graceful drain。
API 适配	OpenAI-compatible completion/chat/responses/embeddings 和 vLLM-compatible 协议对象。
模拟核心	创建 DP rank simulator、worker pool、处理请求、模拟 TTFT/ITL/streaming。
Token / Dataset	支持 HuggingFace tokenizer、regex/simulated tokenizer、dataset-backed response。
KV cache	计算 llm-d kv block key, 维护 block cache, 生成 PrefixCacheStats 和事件。

关键数据流



设计决策与哲学

- 协议兼容优先于真实模型计算：它实现 OpenAI HTTP、vLLM/gRPC 和多种 admin/metrics endpoint，让上游系统无需知道自己面对的是 simulator。
- 延迟是可配置模型，不是固定 sleep：TTFT、inter-token、load factor、remote prefill/KV transfer 被拆开建模。
- KV cache 行为是一等模拟对象：它计算 llm-d KV block key、cached prompt tokens 和 PrefixCacheStats，可测试 KV-aware routing。
- 无 GPU 可运行降低系统实验门槛：[llm-d-benchmark](#)、router、WVA、Gateway API 相关测试能在 Kind 或普通集群中先跑通。

相关页面

- [llm-d-inference-sim](#)
- [llm-d-benchmark](#)
- [llm-d](#)
- [inference-routing](#)

- [llm-inference](#)

llm-d KV Cache

llm-d KV Cache 是 [llm-d](#) 的 KV cache locality index / scoring library。它从 vLLM/SGLang KV events 获取 block create/evict 信号，把 prompt/token block 映射到 pod 或 tier 的命中情况，再给 [llm-d-router](#) 的 scorer 返回 cache-hit 分数。详见 [src-llm-d-kv-cache-architecture](#)。

架构边界

它不负责模型执行，也不等同于 [kv-cache-offload](#)。它更像一个 router 可查询的 KV locality control plane：把 cache event、tokenization、block key、backend index 和 scoring 串起来。

核心抽象

抽象	作用
<code>kvcache.Indexer</code>	根据 prompt/tokens 生成 block keys，并对候选 pods 评分。
<code>kvblock.Index</code>	KV block 索引，支持 in-memory、Redis/Valkey 等 backend。
<code>kvevents</code>	订阅 vLLM/SGLang KV events 并标准化为 index 更新。
tokenizer service	提供 Tokenize / RenderChatTemplate / RenderCompletion 等接口。
PVC/FS connectors	为不同 KV 存储或文件路径接入提供示例。

什么时候看它

场景	判断
想做 KV-aware endpoint routing	适合，核心就是把 KV locality 变成 scorer 输入。
想实现多级 KV offload	先看 dynamo / kv-cache-offload ；llm-d KV Cache 更偏索引和评分。
想验证无 GPU router 行为	配合 llm-d-inference-sim 使用。
只关心单机 engine 内 KV 管理	先看 vllm / sglang 。

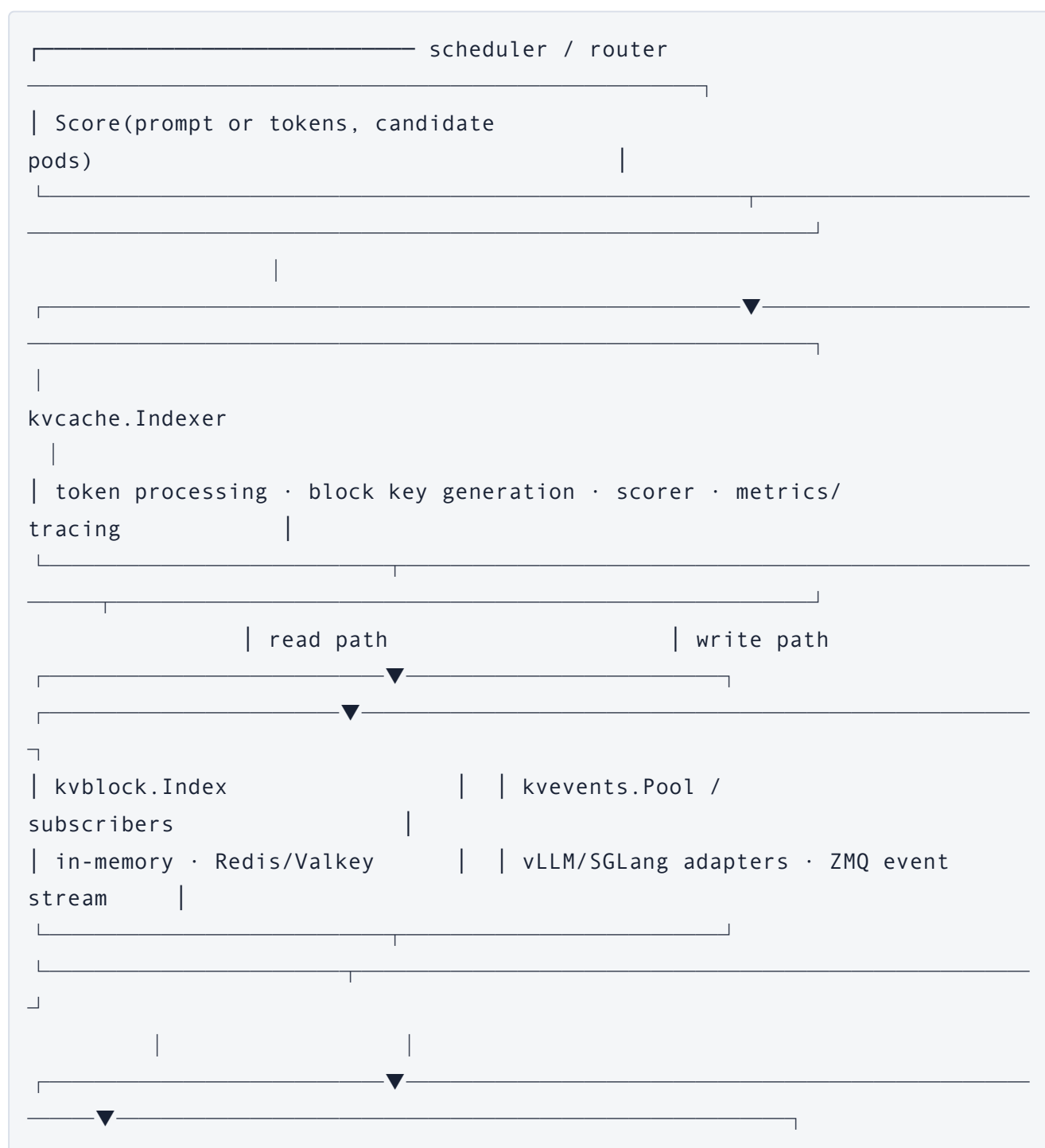
选型提示

[llm-d-kv-cache](#) 说明 KV cache 在分布式 serving 中已经从 engine 内部细节变成路由信号。选型时要分清三层：engine 内 block 管理、跨 worker KV transfer/offload、router 查询 KV locality。

llm-d KV Cache 架构与设计思路分析

llm-d/llm-d-kv-cache 是 llm-d 的 KV cache 智能路由/索引库。它不直接跑模型，而是从 vLLM/ SGLang KV events 获取 block locality，把 prompt/token block 映射到 pod/tier 命中情况，给 router/scorer 返回 cache-hit 分数；同时包含 tokenizer gRPC、KV index service examples、Valkey/Redis/in-memory backend、PVC/FS connectors。

核心架构图



```
| vLLM/SGLang pods: KV block create/evict events, GPU/CPU/FS/object
tiers      |
└──────────────────────────────────────────────────────────────────────────────────┘
```

模块分层

层/目录	责任
pkg/kvcache/**	Indexer、backend、score types、metrics collector。
pkg/kvcache/kvblock/**	核心索引：in-memory、Redis/Valkey、cost-aware memory、HMA、token processor。
pkg/kvevents/**	事件订阅池、ZMQ subscriber、vLLM/SGLang adapter、event realignment。
pkg/tokenization/**, api/tokenizerpb/**	tokenizer pool/service，提供 Tokenize/RenderChatTemplate/RenderCompletion 等接口。

关键数据流

1. 写路径：vLLM pod 产生 KVEvents，subscriber pool 消费后经 engine adapter 标准化，更新 kvblock index。
2. 读路径：router/scorer 传入 prompt/tokens 和候选 pods，Indexer 生成 block keys，查询哪些 pods/tier 命中。
3. score 返回每个 pod 的 KV hit 分数，供 Router 与 load/SLO 等其他 scorers 融合。

设计决策

- 把 KV cache locality 做成独立库/服务，是 llm-d 拆分式架构的关键。
- 事件驱动比主动轮询更接近实时，但依赖 vLLM/SGLang event 格式稳定。
- 索引 backend 可本地/Redis/Valkey，方便从单机实验到分布式 router。

对比定位

和 [dynamo](#) 的 KVBM/KV offload 相比，llm-d-kv-cache 更像 router 可用的 KV locality index；和 AIBrix 的 pkg/cache 相比，它独立成库并提供 tokenizer/proto/examples；和 semantic router 相比，它按 token block/cache 命中路由，不按语义意图路由。

相关链接

- GitHub 当前状态底稿: [src-github-stars-backlog-current-state](#)
- 选型地图: [github-stars-backlog-implementation-map](#)

llm-d Latency Predictor

llm-d Latency Predictor 是给 llm-d inference scheduler 的 ML-based latency scoring service，用预测延迟信号增强 endpoint picking。详见 [src-llm-d-latency-predictor-architecture](#)。

架构边界

它不是 router 本体，而是 router/scorer 的外部预测信号。

什么时候用

场景	判断
需要 latency predictor 能力	适合，llm-d Latency Predictor 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-d , inference-routing 组合。

核心组件

- Service API: latency scoring endpoint
- Model/features: 请求、模型、endpoint 或历史指标特征
- Integration: llm-d scheduler/router scoring
- Training/evaluation utilities

选型提示

把 llm-d Latency Predictor 放在 latency predictor 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

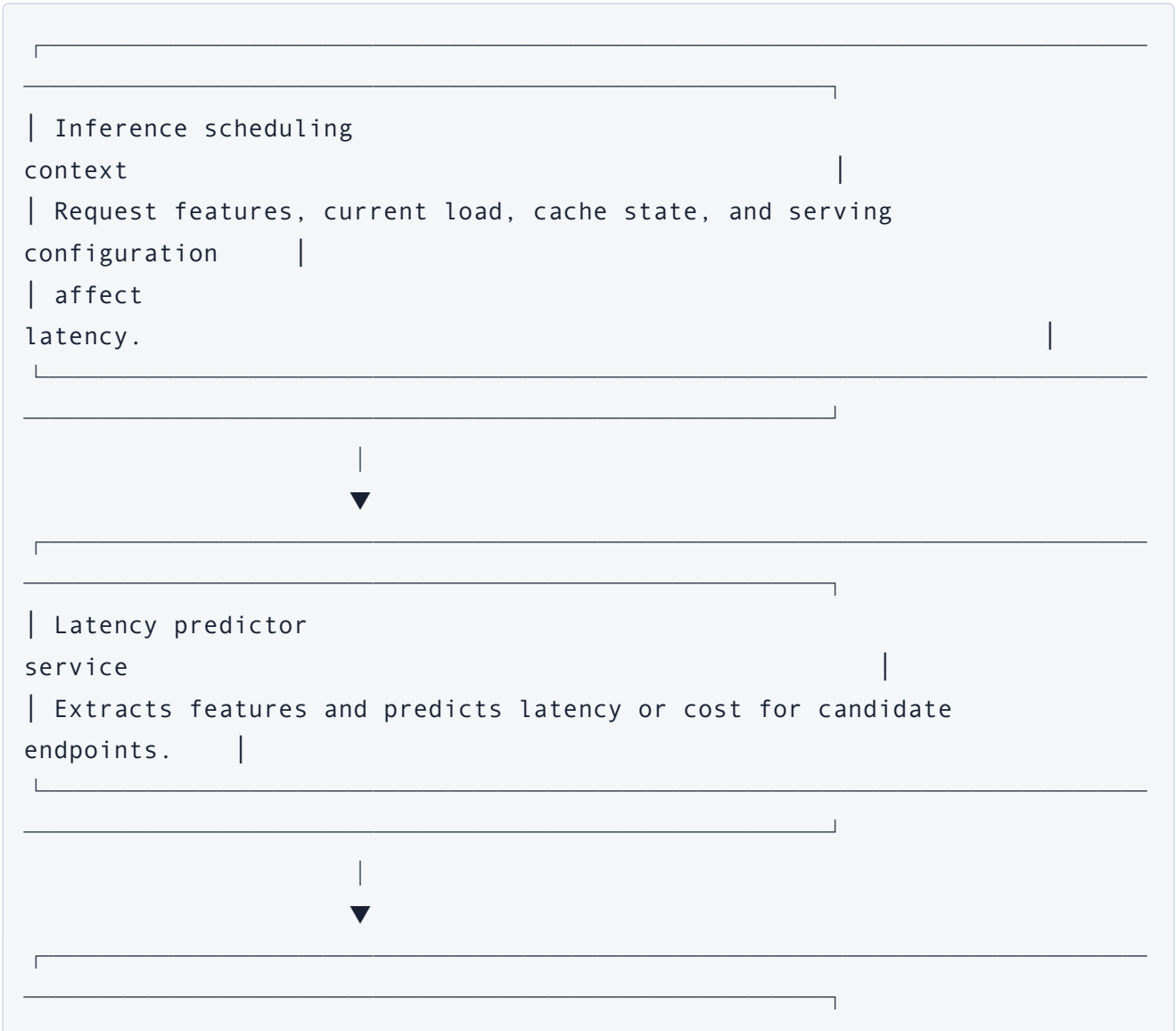
llm-d Latency Predictor 架构与设计思路分析

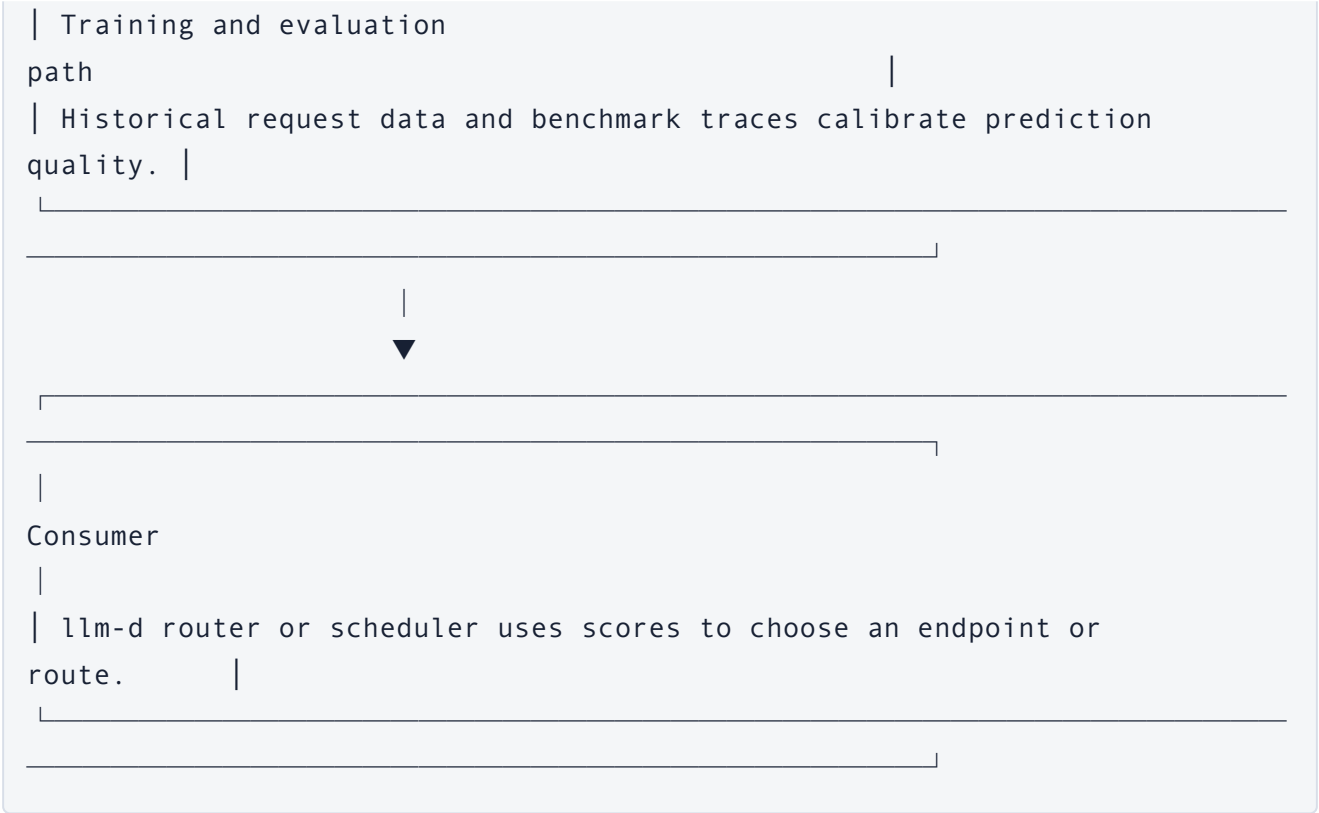
原文: raw/llm-d-latency-predictor-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-latency-predictor> · 优先级 P1

一句话定位

llm-d Latency Predictor 是给 llm-d inference scheduler 的 ML-based latency scoring service，用预测延迟信号增强 endpoint picking。

核心架构图

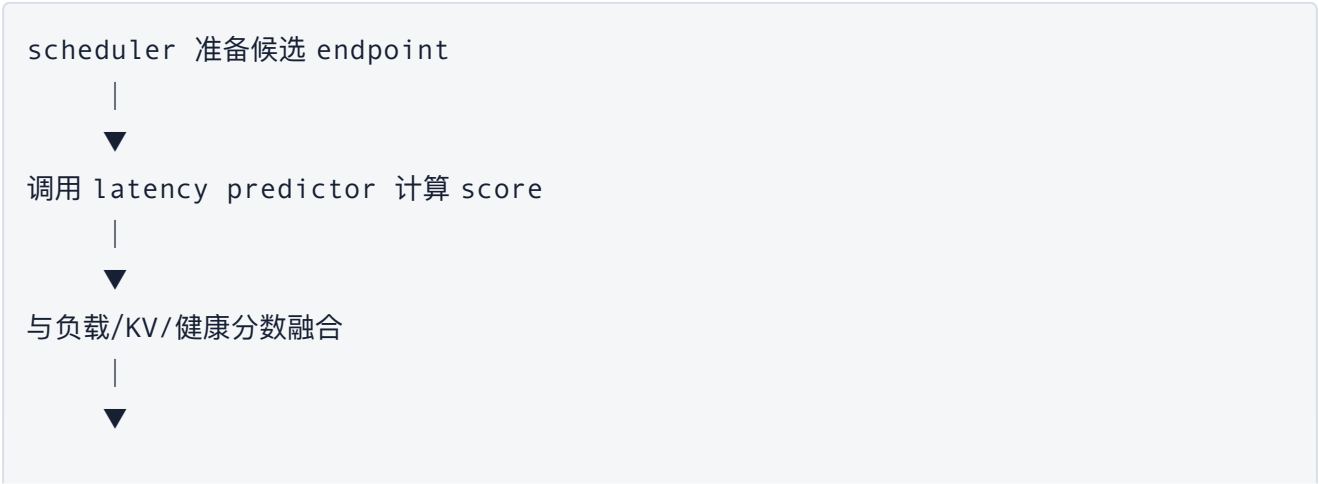




模块分层

层 / 模块	职责
Service API	latency scoring endpoint
Model/features	请求、模型、endpoint 或历史指标特征
Integration	llm-d scheduler/router scoring
Training/evaluation utilities	Training/evaluation utilities

关键数据流



选择 endpoint

|



真实延迟回流用于校准

设计决策与哲学

- 补齐 latency predictor 维度: llm-d Latency Predictor 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：它不是 router 本体，而是 router/scorer 的外部预测信号。
- 选型价值：它应和 [llm-d](#), [inference-routing](#) 一起看，而不是孤立评估。

相关页面

- [llm-d-latency-predictor](#)
- [kubernetes](#)
- [llm-d](#)
- [inference-routing](#)

llm-d P/D Utils

llm-d P/D Utils 是面向 Prefill/Decode 分离部署的 skills/scripts 工具集，用于 preflight、GPU topology、RDMA/NCCL/network/NIXL 等诊断。详见 [src-llm-d-pd-utils-architecture](#)。

架构边界

它不是 serving controller，而是 P/D 部署前后的诊断工具箱。

什么时候用

场景	判断
需要 P/D diagnostics 能力	适合，llm-d P/D Utils 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-d , disaggregated-serving 组合。

核心组件

- Preflight scripts: cluster and runtime checks
- GPU topology checks
- Network/RDMA/NCCL diagnostics
- Agentic skills/workflows for P/D deployment

选型提示

把 llm-d P/D Utils 放在 P/D diagnostics 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

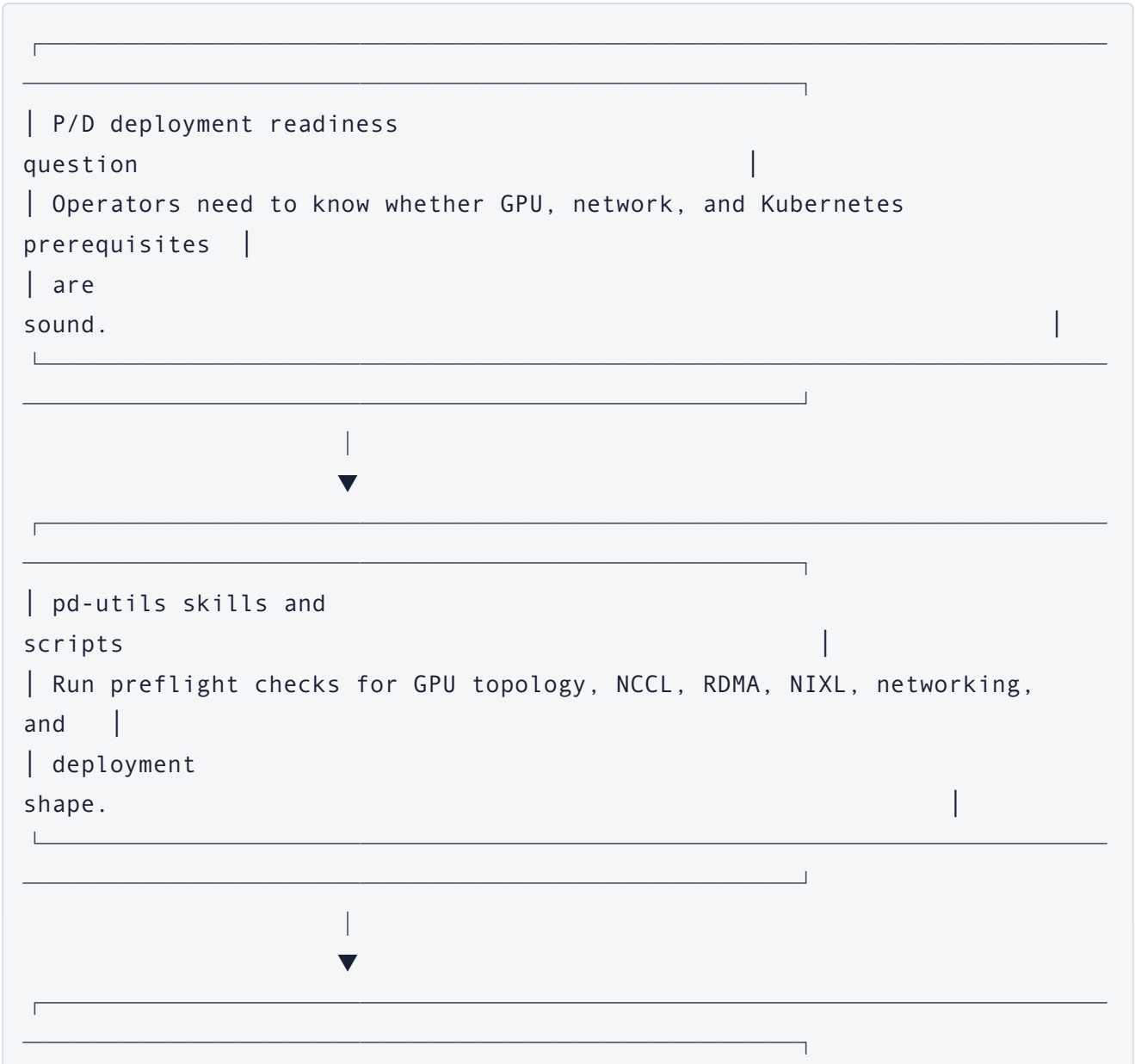
llm-d P/D Utils 架构与设计思路分析

原文: raw/llm-d-pd-utils-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-pd-utils> · 优先级 P1

一句话定位

llm-d P/D Utils 是面向 Prefill/Decode 分离部署的 skills/scripts 工具集，用于 preflight、GPU topology、RDMA/NCCL/network/NIXL 等诊断。

核心架构图

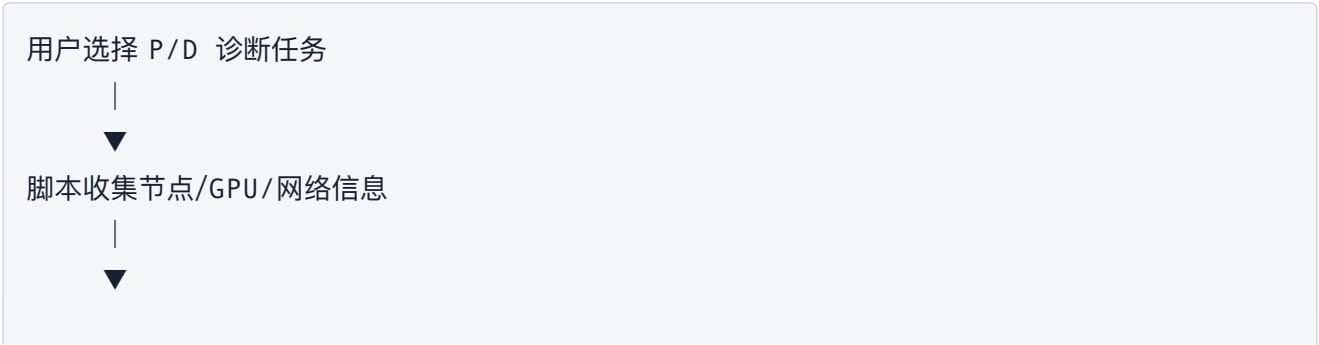




模块分层

层 / 模块	职责
Preflight scripts	cluster and runtime checks
GPU topology checks	GPU topology checks
Network/RDMA/NCCL diagnostics	Network/RDMA/NCCL diagnostics
Agentic skills/workflows for P/D deployment	Agentic skills/workflows for P/D deployment

关键数据流



执行连通性和通信测试



输出失败项和建议



部署前修复基础设施问题

设计决策与哲学

- 补齐 P/D diagnostics 维度: llm-d P/D Utils 让当前 wiki 不只停留在 serving engine 或单个 operator，而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断：它不是 serving controller，而是 P/D 部署前后的诊断工具箱。
- 选型价值：它应和 [llm-d](#), [disaggregated-serving](#) 一起看，而不是孤立评估。

相关页面

- [llm-d-pd-utils](#)
- [kubernetes](#)
- [llm-d](#)
- [disaggregated-serving](#)

llm-d Planner

[llm-d-planner](#) 是 llm-d-incubation 中的部署规划与推荐平台，把业务需求转换成模型/GPU/SLO/成本决策和 Kubernetes YAML。它处于 serving runtime 上游，不在在线 Router 请求路径中。

解决的问题

应用团队通常知道用户数、场景和预算，却不知道应该选什么模型、GPU、上下文长度、replica 数和 TTFT/ITL 目标。Planner 将这些决策从人工 trial-and-error 变成可解释、可编辑、可部署的流程。

架构图



方法与集成

Planner 使用可替换 LLM provider 做 intent extraction，把 use case 映射到 traffic profile 和 TTFT/ITL/E2E SLO；Capacity Planner 估算 weights、KV、activation、GPU 数和并发；GPU Recommender 估算不同硬件的性能；Recommendation Service 按 quality、cost、latency 生成 Best Quality、Lowest Cost、Lowest Latency、Balanced 视图；Configuration Service 生成 KServe/vLLM/HPA/ServiceMonitor YAML。它还集成 vLLM simulator 支持无 GPU 开发。

当前定位偏 POC/平台入口，生产化仍需关注配置安全、认证、多租户、数据版本和实际部署 telemetry 回流。

llm-d Prism

llm-d Prism 是分布式推理性能分析 dashboard，把 benchmark 和运行数据做交互式分析，用于理解 P/D、路由和资源配置的效果。详见 [src-llm-d-prism-architecture](#)。

架构边界

llm-d-benchmark 负责跑实验；Prism 负责看懂实验结果。

什么时候用

场景	判断
需要 performance analysis 能力	适合，llm-d Prism 正是这一层的代表项目。
需要和 Kubernetes API / controller / runtime 集成	适合，它的主要价值来自 Kubernetes-native 工作流。
需要替代相邻层全部职责	不适合，应和 llm-d-benchmark , llm-inference 组合。

核心组件

- Frontend/dashboard: experiment visualization
- Data ingestion: benchmark result files or APIs
- Analysis views: latency/throughput/token/error breakdown
- Comparison workflow: stack/config/run dimensions

选型提示

把 llm-d Prism 放在 performance analysis 维度评估：先看它输入什么对象、输出什么对象，再看它是否会进入请求路径、调度路径、节点路径或 CI/实验路径。这个边界比 star 数更能决定它是否适合当前平台。

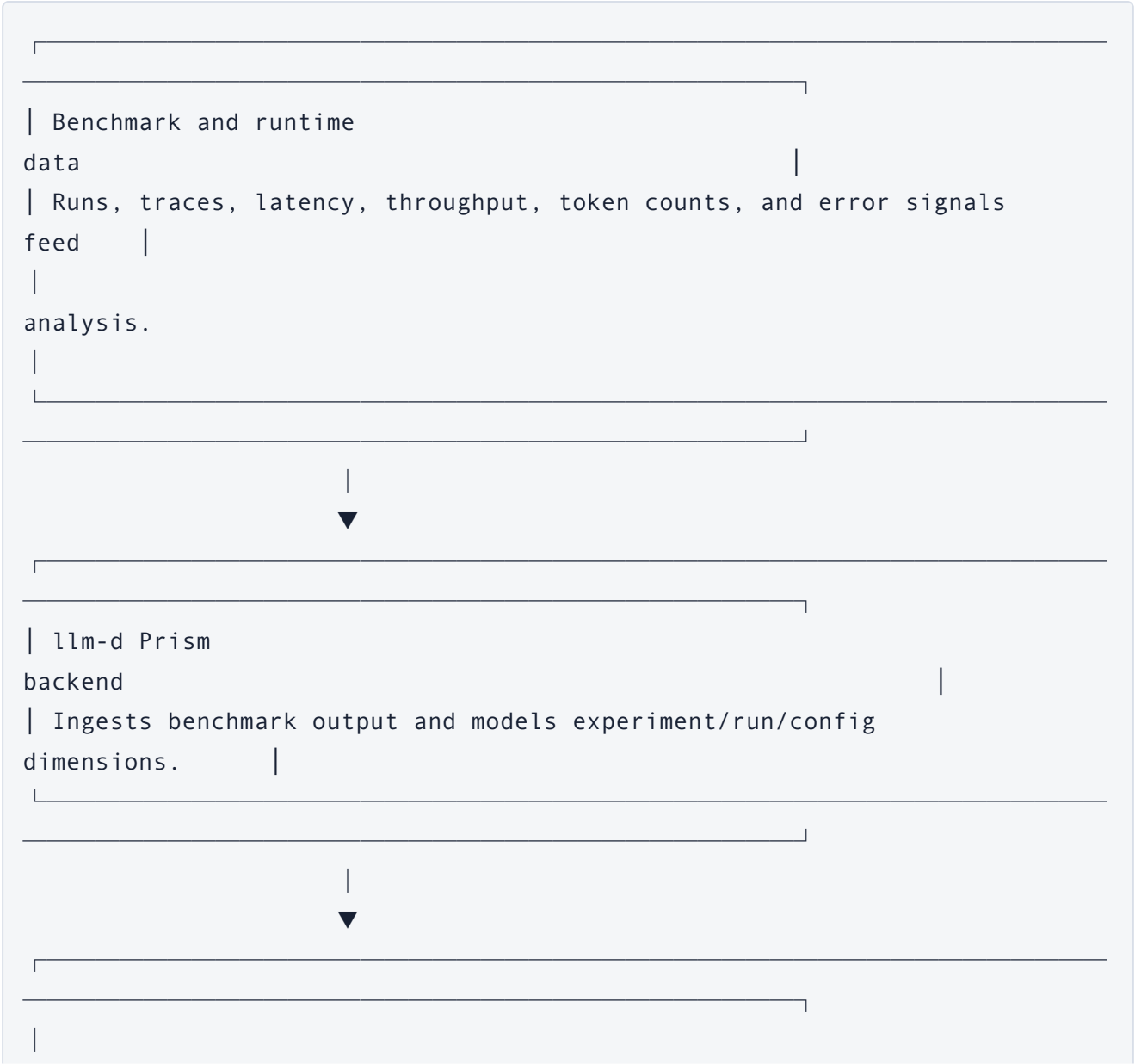
llm-d Prism 架构与设计思路分析

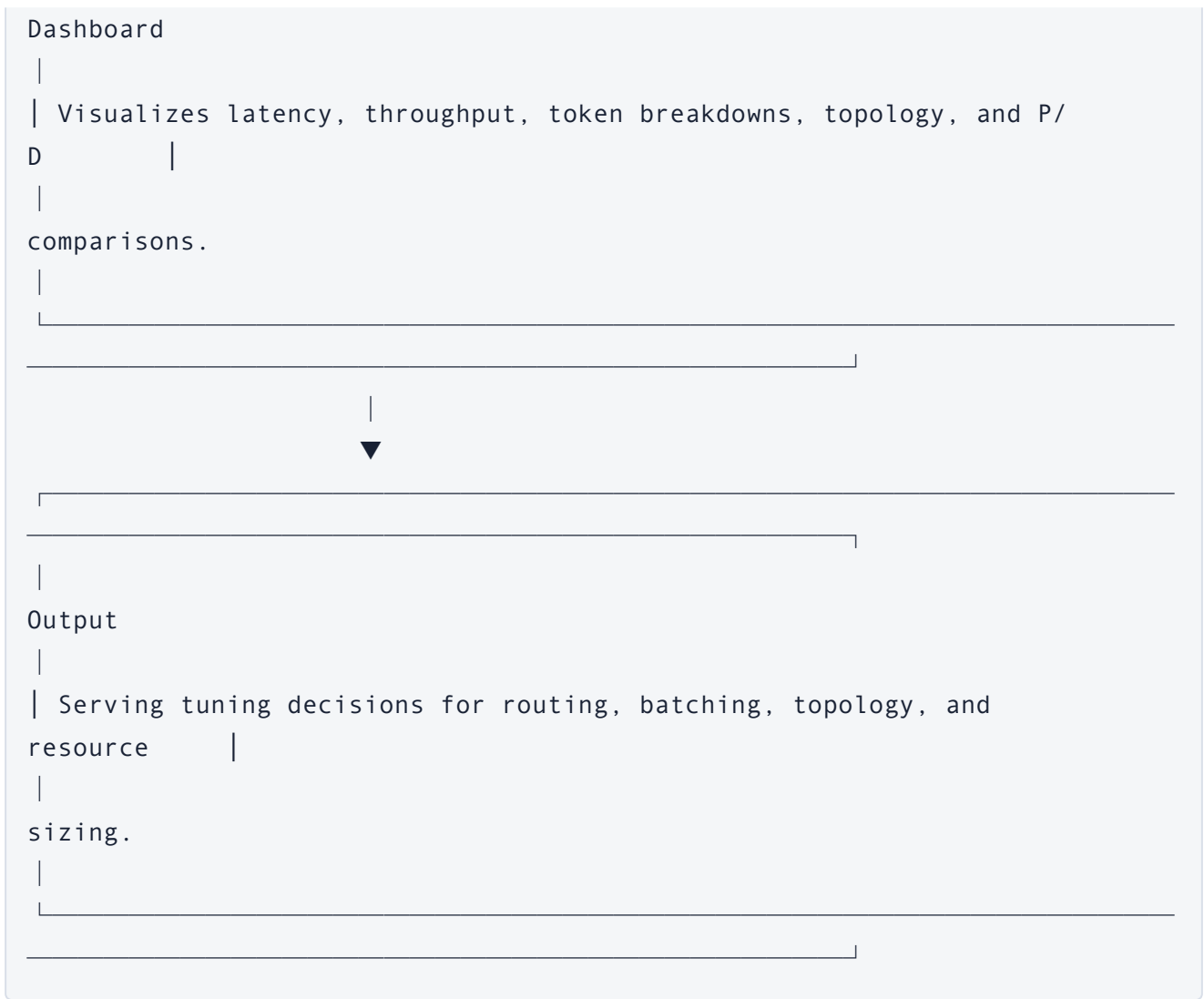
原文: raw/llm-d-prism-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-prism> · 优先级 P1

一句话定位

llm-d Prism 是分布式推理性能分析 dashboard，把 benchmark 和运行数据做交互式分析，用于理解 P/D、路由和资源配置的效果。

核心架构图

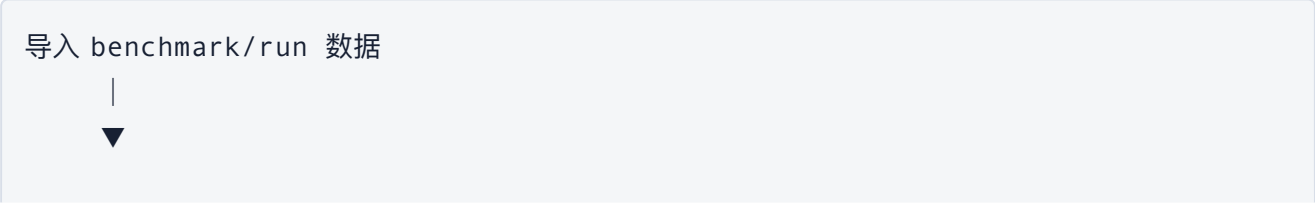




模块分层

层 / 模块	职责
Frontend/dashboard	experiment visualization
Data ingestion	benchmark result files or APIs
Analysis views	latency/throughput/token/error breakdown
Comparison workflow	stack/config/run dimensions

关键数据流



解析实验维度和指标



可视化 latency/throughput/token stats



对比不同 serving 配置



输出调参判断

设计决策与哲学

- 补齐 performance analysis 维度: llm-d Prism 让当前 wiki 不只停留在 serving engine 或单个 operator, 而能解释 Kubernetes 平台里的相邻控制面。
- 边界判断: llm-d-benchmark 负责跑实验; Prism 负责看懂实验结果。
- 选型价值: 它应和 [llm-d-benchmark](#), [llm-inference](#) 一起看, 而不是孤立评估。

相关页面

- [llm-d-prism](#)
- [kubernetes](#)
- [llm-d-benchmark](#)
- [llm-inference](#)

llm-d Router

llm-d Router 是面向 LLM serving 的 intelligent entry point，由 Envoy/ext-proc 或 Gateway API 接入，并通过 Endpoint Picker 对 InferencePool endpoint 执行 filter → score → pick。

解决的问题

普通负载均衡只知道连接和健康状态，不知道 KV locality、推理负载、请求优先级、模型重写和 P/D 目标。llm-d Router 把这些 runtime signal 注入 Gateway 数据面，减少错误 endpoint 选择和 KV 重算。

核心抽象

EPP 是决策引擎；filters、scorers、scrapers 是插件化信号处理框架；InferenceObjective 和 InferenceModelRewrite 把请求目标、优先级、模型重写与 canary/A-B 策略显式化。

边界与选型

它选择 Kubernetes inference endpoint，不负责 provider 凭据和全局 quota；这些属于 envoy-ai-gateway。它也不负责语义模型选择，后者看 [semantic-router](#) / routellm。适合 Gateway API、InferencePool、KV-aware 和分布式 serving 场景。

详见 [src-llm-d-router-architecture](#)、[src-k8s-gateway-routing-comparison](#)。

llm-d Router 架构与设计思路分析

llm-d/llm-d-router 是 llm-d 的智能入口。它通过 Envoy/ext-proc 或 Gateway API Inference Extension 接入流量，核心 EPP 用 filters、scorers、scrapers 和 scheduling profiles 对 InferencePool pods 做过滤/打分/选择；同时包含 disaggregation sidecar，协调 encode/prefill/decode 多阶段推理。

核心架构图



模块分层

层/目录	责任
cmd/epp, pkg/epp/server/**	EPP server 和 controller manager。
pkg/epp/config, pkg/epp/framework/ plugins/**	EndpointPickerConfig、plugins、scheduling profiles。
pkg/epp/datalayer/**	K8s binding、InferencePool endpoint graph、runtime state。
pkg/common/envoy/**	Envoy request/response/header/metadata/chunking 辅助。

关键数据流

1. proxy 收到请求后把 headers/body stream 给 EPP ext-proc。
2. EPP 解析请求模型/priority/profile，加载 EndpointPickerConfig，按 filters 排除不合适 endpoints。
3. scorers 访问 scrapers/datastore 中的 load、KV、metrics 等信号，按权重打分；picker 选择最高分 pod。

设计决策

- plugin 化 filters/scorers/scrapers 是 Router 的核心：新增路由策略不改主循环。
- EPP 通过 Gateway API/GIE 与 K8s 标准对齐，避免自定义所有流量原语。
- InferenceObjective 和 ModelRewrite 让“请求目标”进入控制面，而不是硬编码在路由器配置里。

对比定位

和 AIBrix gateway plugins 相比，llm-d-router 更聚焦标准 Router/EPP；和 agentgateway 相比，它面向 model server pod placement，而不是 MCP/A2A/LLM access governance；和 RouteLLM/semantic-router 相比，它更 K8s/runtime metrics 驱动。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

llm-d Workload Variant Autoscaler

llm-d Workload Variant Autoscaler (WVA) 是面向分布式 [llm-inference](#) 的 Kubernetes variant autoscaler，用 `VariantAutoscaling` CRD 把同一模型/`InferencePool` 下不同硬件、角色、成本或配置的 `serving variant` 纳入一个全局扩缩决策。详见 [src-llm-d-workload-variant-autoscaler-architecture](#)。

[!warning] Conflict
截至 2026-09-22，llm-d 当前 [dev 架构文档](#) 已将 Workload Variant Autoscaler 标为 deprecated，并描述 EPP metrics → KEDA Prometheus scaler → HPA 的扩缩路径。本 Entity 及 [src-llm-d-workload-variant-autoscaler-architecture](#) 保留其所记录版本的历史架构，不构成当前采用建议；当前采用前需重新核验。参见 [llm-inference-serving-project-map](#)。

架构边界

WVA 不直接替代 HPA/KEDA。它通过 Prometheus、GPU inventory、[gateway-api-inference-extension](#) `InferencePool`、`scale target` 和 `capacity model` 计算 `desired allocation`，然后把 `desired/current replica` 指标暴露给 HPA/KEDA 这类 autoscaler 执行。

什么时候用

场景	判断
同一模型有多个 <code>serving variant</code>	适合，例如不同 GPU、 <code>prefill/decode</code> 、 <code>batch/interactive</code> 或成本档位。
希望 <code>autoscaling</code> 理解 <code>InferencePool/modelID</code>	适合，比通用 HPA 更有 <code>serving</code> 语义。
只需要单 <code>Deployment CPU/GPU utilization</code> 扩缩	可能 HPA/KEDA 直接够用。
没有 Prometheus/custom metrics 基础设施	需要先补可观测和 metrics API。

同类对比

维度	llm-d-workload-variant-autoscaler	HPA / KEDA	kserve / Knative autoscaling
决策对象	多 <code>variant allocation</code>	单 <code>workload</code> 或 <code>event source</code>	<code>service/revision/model service</code>

维度	llm-d-workload-variant-autoscaler	HPA / KEDA	kserve / Knative autoscaling
推理语义	modelID / InferencePool / variant cost	通用指标	model serving 生命周期
执行方式	发指标给 HPA/KEDA	scale subresource	controller/autoscaler

选型提示

当 serving fleet 开始出现 prefill/decode 分离、不同 GPU 型号、不同成本档位或不同 SLO 池时，WVA 才有明显价值。单模型单部署阶段先把 [model-serving-operator](#) 和普通 autoscaling 跑稳更重要。

llm-d Workload Variant Autoscaler 架构与设计思路分析

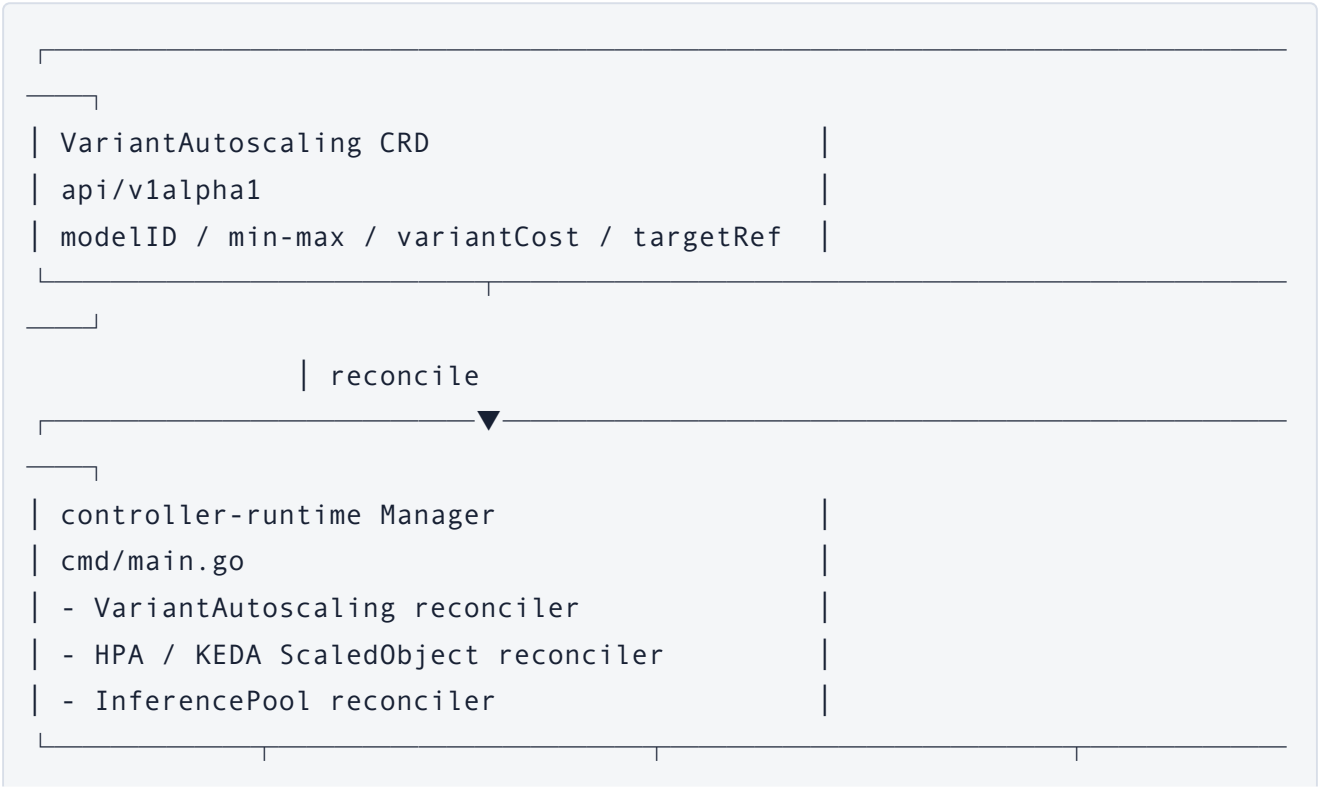
原文: raw/llm-d-workload-variant-autoscaler-architecture-analysis.md · 仓库: <https://github.com/llm-d/llm-d-workload-variant-autoscaler> · 分析版本 HEAD 526ce85

一句话定位

[llm-d-workload-variant-autoscaler](#) 是面向分布式 [llm-inference](#) 的 Kubernetes 全局 autoscaler，核心对象是同一模型/InferencePool 下的多个 serving variant。它通过 Prometheus、[gateway-api-inference-extension](#)、KEDA/HPA、GPU inventory 和 scale target 状态，把“不同硬件/角色/成本的变体应该各扩多少副本”转成外部指标和 status。

[!warning] Conflict
截至 2026-09-22，llm-d 当前 [dev 架构文档](#) 已将 Workload Variant Autoscaler 标为 deprecated，并描述 EPP metrics → KEDA Prometheus scaler → HPA 的扩缩路径。本页保留其所记录版本的历史设计与实现；当前采用前需重新核验。参见 [llm-inference-serving-project-map](#)。

核心架构图





模块分层

层 / 模块	职责
API 层	定义 VariantAutoscaling spec/status/conditions。
Manager 启动层	注册 K8s、Prometheus Operator、Gateway API Inference Extension、KEDA、LWS 等 scheme，组装 reconciler 和 engines。
Reconcile 层	解析 scale target、更新 conditions、读取 decision cache、patch status、watch workload 和 InferencePool。
Metrics 收集	采集 request rate、replica、queue/cache/capacity 相关指标。
优化引擎	维护 capacity store、GPU inventory、queueing/saturation analyzer、cost-aware optimizer。
执行动作	发出 desired/current replica 等指标供 HPA/KEDA 消费。

关键数据流





设计决策与哲学

- Variant 是 autoscaling 的一等对象：它不只看单个 Deployment 的 CPU/GPU 指标，而是把同一模型下不同硬件、角色或配置变体纳入一个优化问题。
- 不直接抢 HPA/KEDA 的职责：WVA 主要发指标，实际 scale 仍由 HPA/KEDA 完成。
- Gateway API Inference Extension 是 serving 语义入口：它需要理解 InferencePool 和 endpoint pool，而不是只做通用 workload autoscaling。
- 优化引擎需要容量知识而不只是当前利用率：capacity store、GPU inventory、queueing analyzer 和 optimizer 共同估计 serving capacity curve。

相关页面

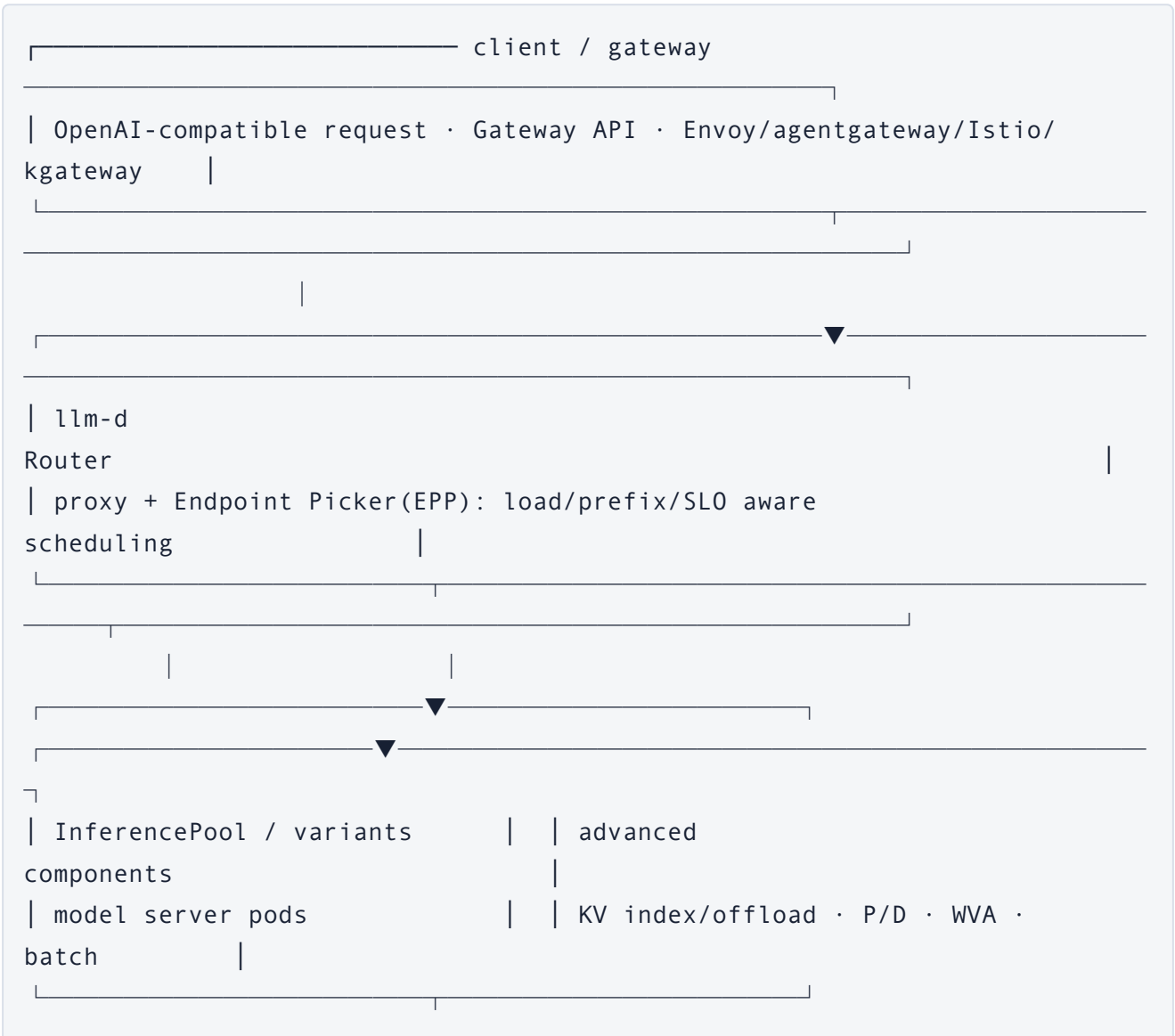
- [llm-d-workload-variant-autoscaler](#)
- [llm-d](#)
- [model-serving-operator](#)
- [gateway-api-inference-extension](#)
- [llm-inference](#)

llm-d 架构与设计思路分析

llm-d/llm-d 是 Kubernetes 上的分布式 LLM serving stack 总入口。它本身更像 docs/guides/recipes/architecture hub，把 llm-d-router、Gateway API Inference Extension 的 InferencePool、vLLM/ SGLang model server、KV cache management、P/D disaggregation、latency predictor、batch gateway、autoscaling 等组件组织成 well-lit paths。

[!warning] Conflict
截至 2026-09-22，llm-d 当前 [dev 架构文档](#) 已将 Workload Variant Autoscaler 标为 deprecated，并描述 EPP metrics → KEDA Prometheus scaler → HPA 的扩缩路径。本页保留其所记录版本的历史设计与实现；当前采用前需重新核验。参见 [llm-inference-serving-project-map](#)。

核心架构图



对比定位

和 AIBrix 相比，llm-d 更标准化/生态化，组件分仓；和 [dynamo](#) 相比，它更 Kubernetes/Gateway API first；和 [vllm](#)/[sglang](#) 相比，它不是 engine，而是 engine 之上的 serving system。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

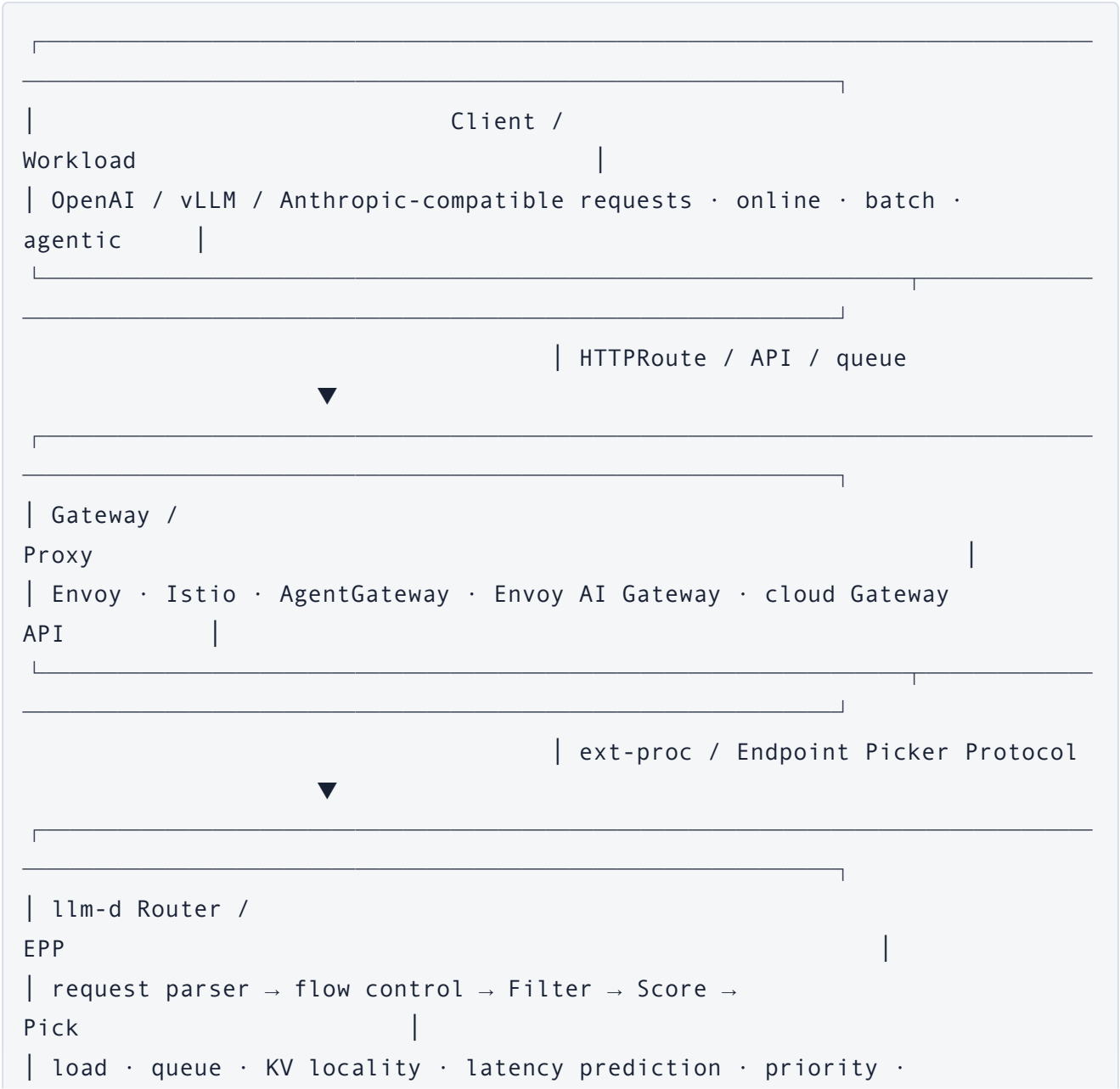
llm-d 核心项目群架构再调研

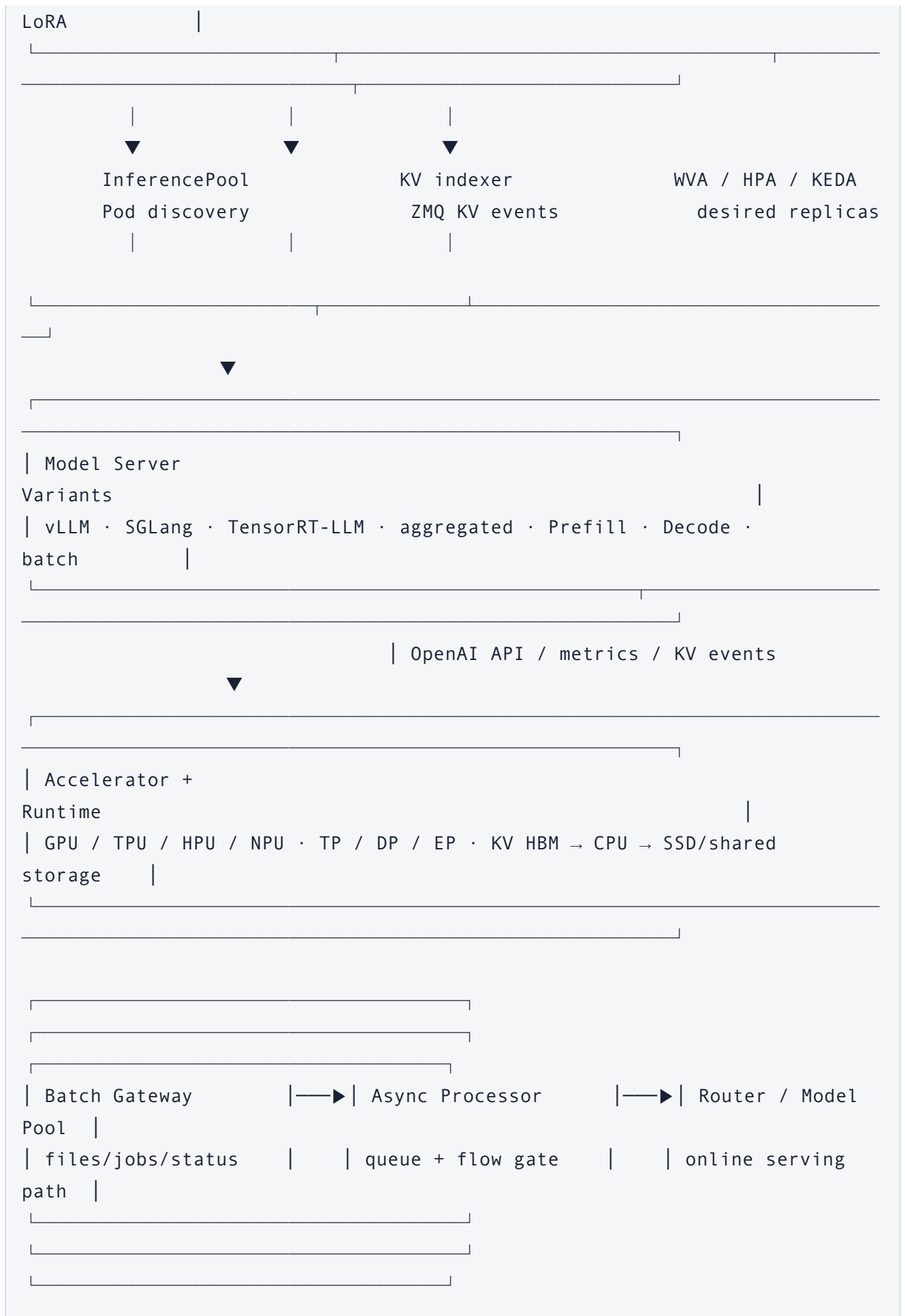
原文: raw/llm-d-core-projects-research-2026-09-13.md · 调研日期: 2026-09-13

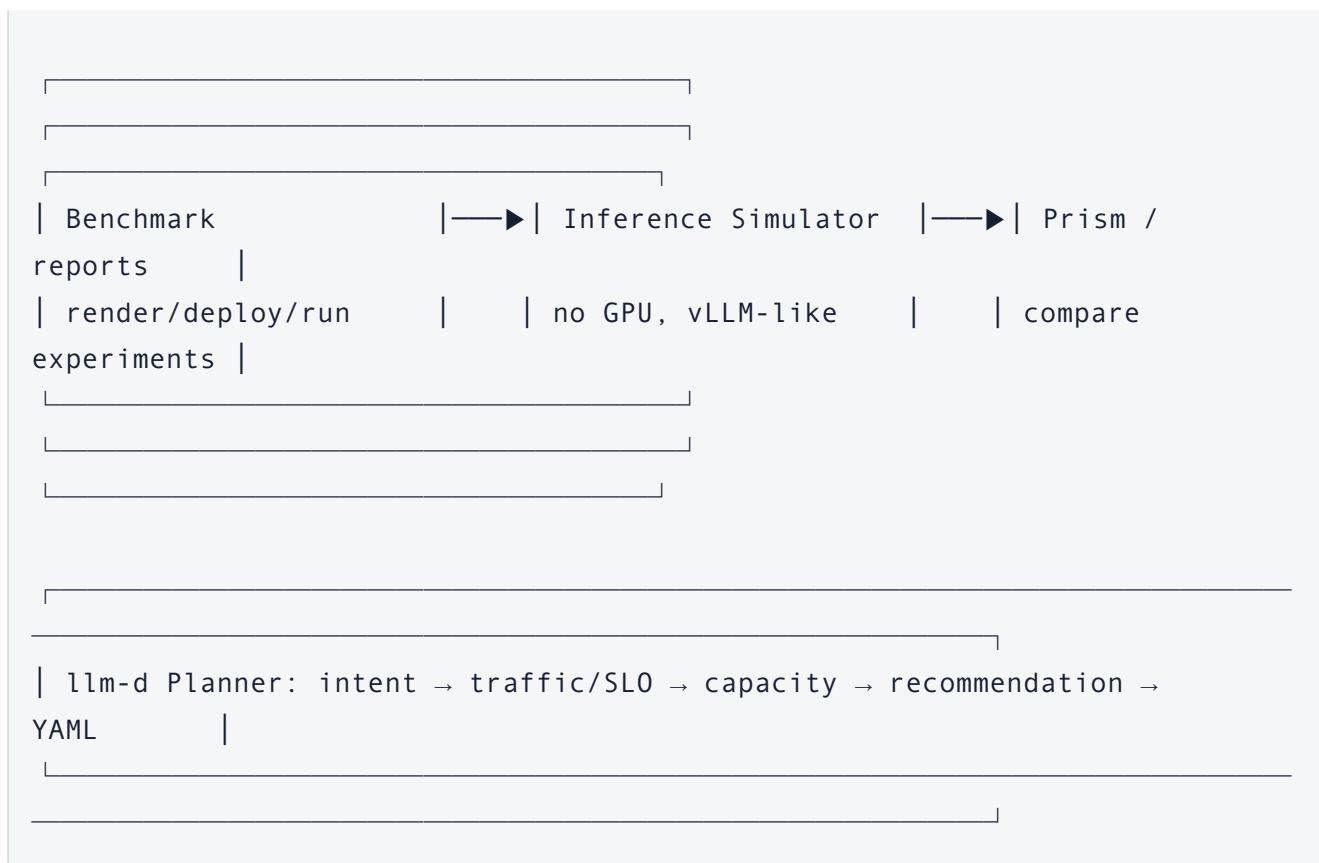
一句话定位

llm-d 是位于 [vllm](#) / [sglang](#) 之上的 Kubernetes-native 分布式推理控制与优化栈；核心项目围绕路由、KV cache、P/D、扩缩容、Batch、实验验证和部署规划形成完整链路。

总体架构图







核心项目职责

项目	作用	解决的业务问题	主要集成
llm-d	主架构、部署 recipe、API 和 well-lit paths	把模型 server 组合成生产 serving stack	Kubernetes、Gateway API、vLLM、SGLang、Prometheus
llm-d-router	Proxy + EPP 智能入口	endpoint picking、排队、公平性、P/D 编排	Envoy ext-proc、InferencePool、InferenceObjective
llm-d-kv-cache	KV event index/scorer library	重复 prefix 重算、缓存局部性丢失	vLLM/SGLang KV events、ZMQ、Router EPP
llm-d-workload-variant-autoscaler	variant 全局扩缩	异构 GPU、P/D、成本与 SLO 的联合分配	Prometheus、HPA/KEDA、InferencePool
llm-d-batch-gateway	OpenAI-compatible Batch API	大量离线请求的持久化、重试和限流	PostgreSQL、Redis/Valkey、object store、Router
llm-d-benchmark	实验生命周期编排	多环境、多配置实验不可复现	Jinja2、Kustomize/Helm、GuideLLM、inference-perf
llm-d-inference-sim	vLLM 行为模拟器		

项目	作用	解决的业务问题	主要集成
		无 GPU 验证 routing/ autoscaling/CI	OpenAI API、vLLM gRPC、 ZMQ、Prometheus
llm-d-planner	需求到部署规划	不懂 GPU/模型/SLO 的 团队难以落地	FastAPI、SQLite、LLM provider、KServe/vLLM YAML

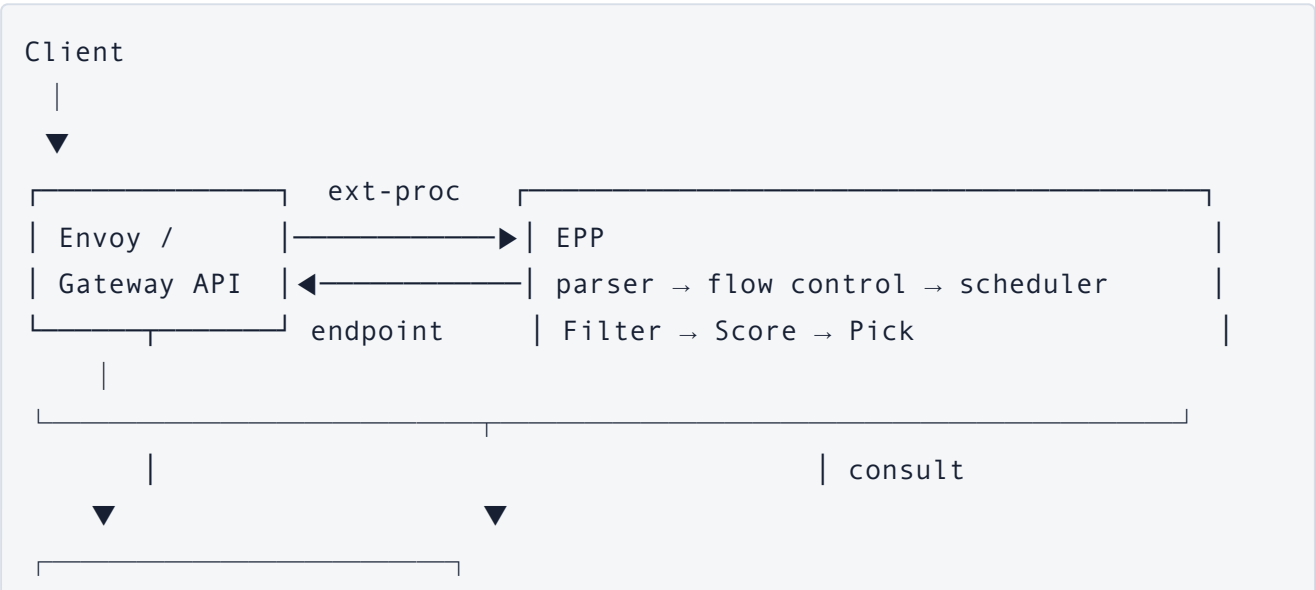
Router / EPP 请求路径

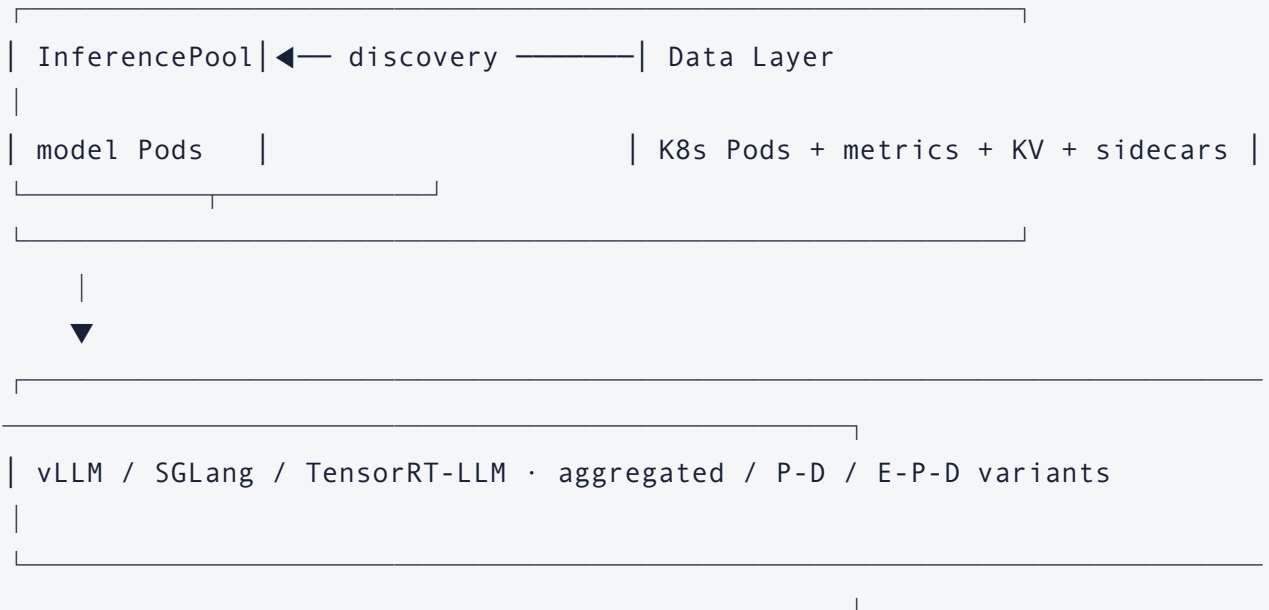
```
HTTPRoute → Gateway/Proxy
    → ext-proc 请求
    → EPP 解析 request
    → Flow Control admission / priority / fairness
    → Filter 候选端点
    → Score load / KV / latency / LoRA
    → Pick endpoint
    → Proxy 转发到 Model Server
    → vLLM/SGLang 执行 prefill + decode
    → metrics / KV events 反哺 EPP
```

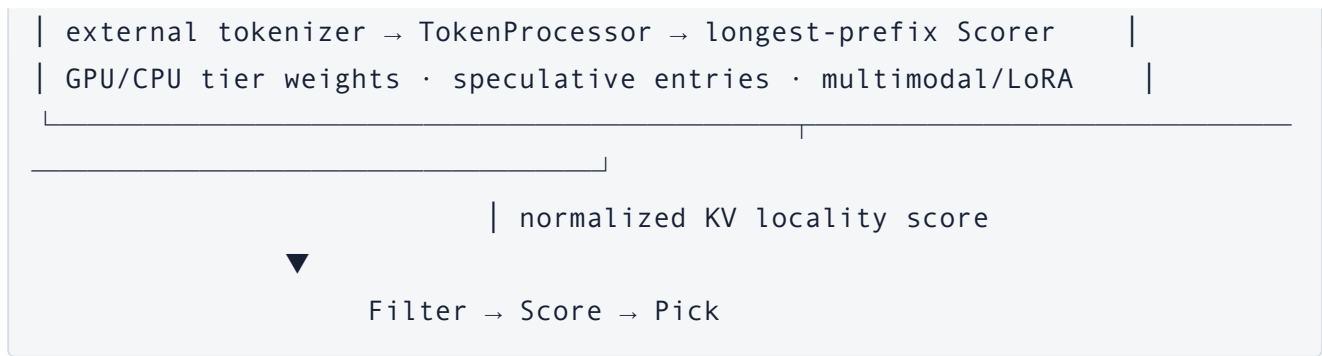
[llm-d-router](#) 的方法是插件化 Filter → Score → Pick，而不是把某个 routing heuristic 固化在 Proxy 中。EPP 还能通过 Flow Control 处理 priority band、saturation、fairness 和 ordering；Proxy 继续承担连接、TLS 和转发。

KV Cache 路径

Router 架构图







[llm-d-kv-cache](#) 的关键不是“把 KV 存到哪里”，而是把 engine 的 KV state 转成 Router 可查询的 locality signal。它支持 vLLM/SGLang event adapter、最长连续 prefix、tier 权重、multimodal/LoRA extra keys 和 speculative indexing；tokenization 正在从库内 pool 迁移到外部 host/sidecar。

Autoscaling、Batch 与实验闭环

Autoscaling

WVA 读取 queue、running、KV、latency、pending Pod 和 accelerator inventory，计算 **desired replicas**，再让 HPA/KEDA 执行。单一同构 Deployment 适合 KEDA + EPP metrics；多 variant、P/D、异构硬件和强 SLO 才需要 WVA。

Batch

Batch Gateway 负责 job/file/output 的 durable lifecycle；Async Processor 负责从 Redis 或 Pub/Sub 取单请求，结合系统信号做 flow control，再调用 Router。它和在线 Router 的区别是：前者优化 job 可恢复性和吞吐，后者优化单请求延迟与端点选择。

Benchmark 与 Simulator

Benchmark 用 declarative scenario、Jinja render、Kubernetes lifecycle、harness 和 workspace 实现可复现实验；Inference Sim 用 OpenAI/vLLM API、延迟公式、KV blocks、metrics 和 failure injection 让这些实验可以在无 GPU 环境做控制面 smoke test。Simulator 不是生产性能基线。

Planner 的上游入口

[llm-d-planner](#) 把自然语言或结构化业务意图转换成 traffic profile、TTFT/ITL/E2E SLO、quality/cost/latency 权重，再用 benchmark、capacity planner 和 GPU recommender 生成部署建议，最后渲染 KServe/vLLM/HPA/ServiceMonitor YAML。它是 adoption/planning 层，不在 Router 的在线请求路径内。

设计判断与边界

- [llm-d](#) 负责组合和标准化，不替代推理 engine。
- [llm-d-router](#) 负责 endpoint-level intelligence，不是通用 provider gateway。

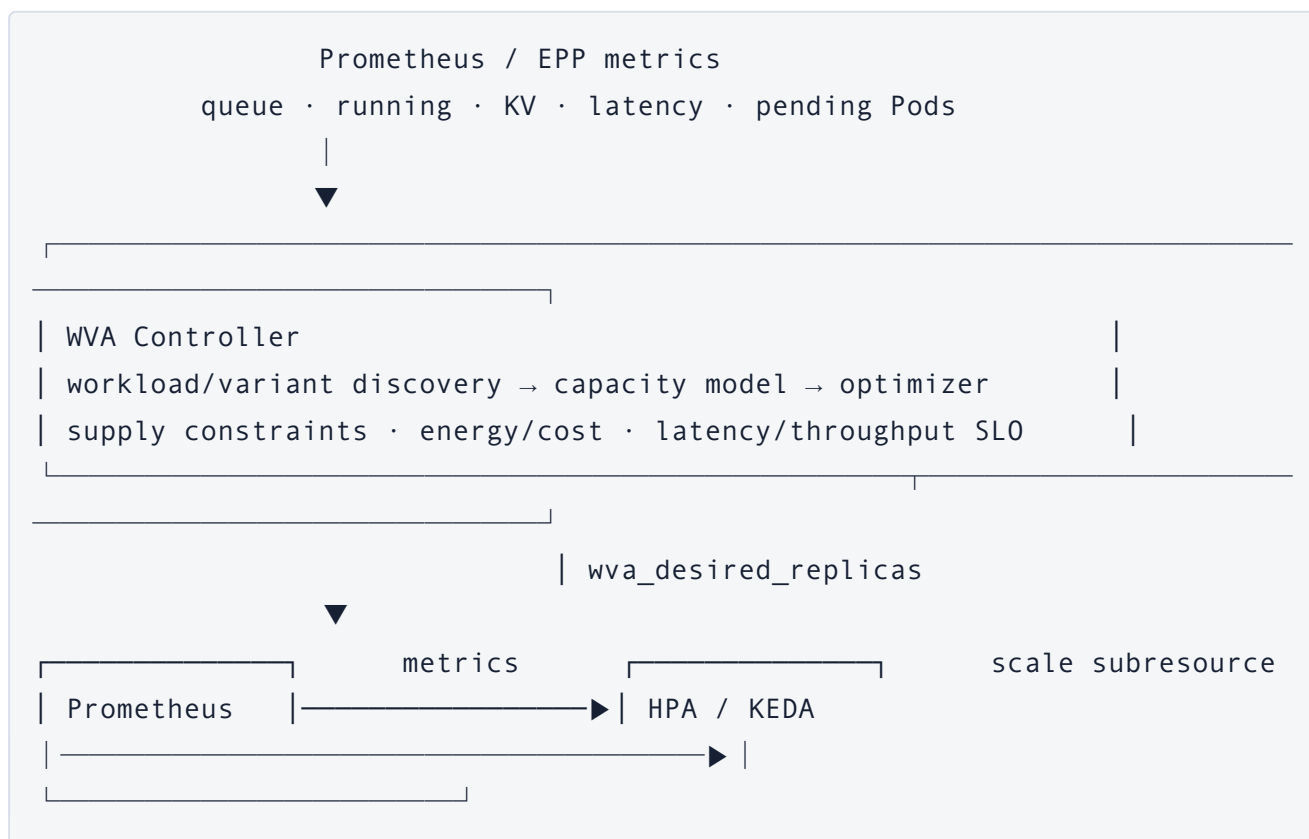
- [llm-d-kv-cache](#) 负责 locality index/scoring, 不等同于 KV offload 或 KV transfer。
- WVA 通过 HPA/KEDA 执行扩缩, 不直接取代 Kubernetes autoscaler。
- Batch Gateway 解决异步 job, 不应把 batch 请求直接当 interactive request。
- Benchmark 与 Simulator 负责验证和实验, 不能把模拟延迟当真实 GPU 指标。
- Planner 解决部署规划, 当前仍要区分生产愿景和 POC 实现。

各项目架构图补充

核心请求数据流

- HTTPRoute → Gateway/Proxy
 - ext-proc 请求
 - EPP 解析 request
 - Flow Control admission / priority / fairness
 - Filter 候选端点
 - Score load / KV / latency / LoRA
 - Pick endpoint
 - Proxy 转发到 Model Server
 - vLLM/SGLang 执行 prefill + decode
 - metrics / KV events 反哺 EPP

WVA

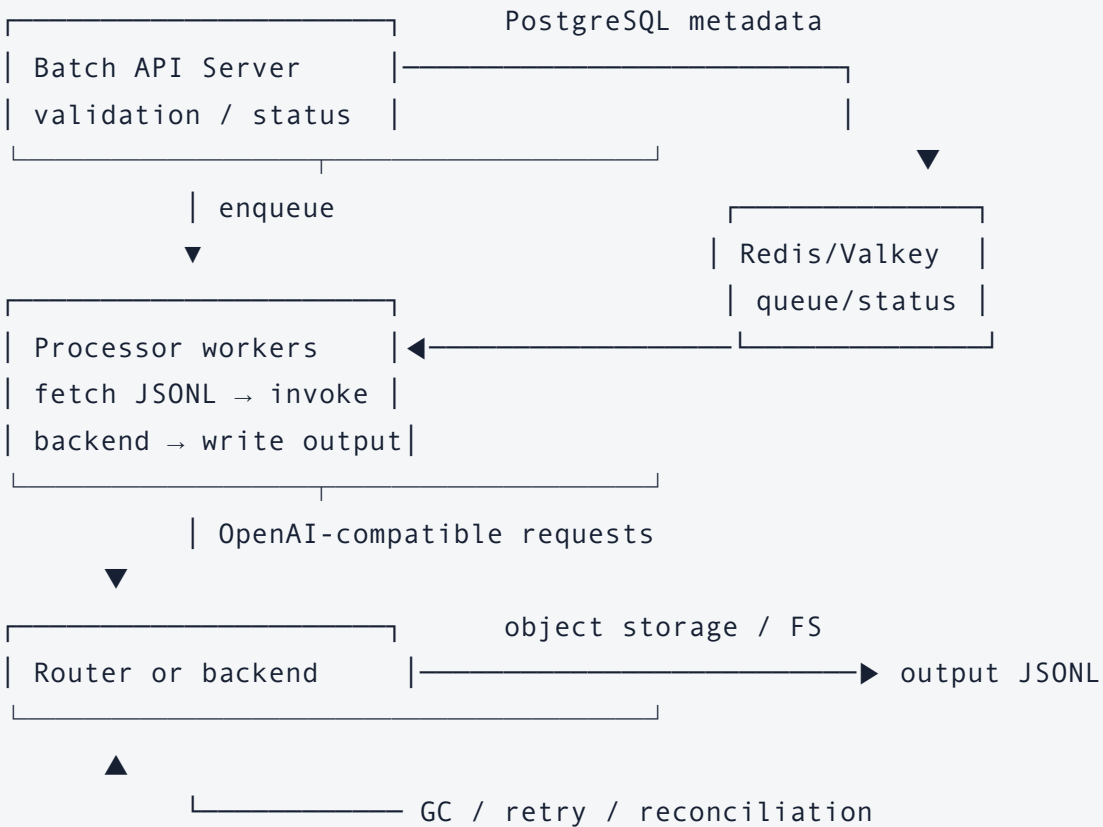


Model-server Deployments / variants

Batch Gateway

Client

| POST /v1/files + POST /v1/batches



Benchmark

Scenario YAML + cluster config + CLI overrides





Inference Sim



Planner



FastAPI / UI / CLI	
Intent Extraction → Specification Editor	
▼	
Knowledge Base	
model catalog · GPU catalog · SLO templates · benchmark DB	
Arena/Artificial Analysis quality · empirical/estimated data	
▼	
Recommendation	
capacity planner → config finder → quality/cost/latency scorer	
→ ranked views / what-if trade-offs	
▼	
Configuration / Deployment	
Jinja2 → KServe/vLLM/HPA/ServiceMonitor YAML → Kubernetes	

相关页面

- [inference-routing](#)
- [kv-cache-offload](#)
- [disaggregated-serving](#)
- [model-serving-operator](#)
- [gateway-api-inference-extension](#)
- [vllm](#)
- [sglang](#)

Model Serving Operator

Model serving operator 是在 Kubernetes 上声明式管理模型、runtime、endpoint、autoscaling、GPU 资源和推理服务生命周期的控制面。

代表项目

项目	重点
kserve	标准化 InferenceService / LLMISvc / LocalModel 平台
ome	Open Model Engine, CRD/controller + runtime selector + accelerator configs
kubeflow	Model CRD + OpenAI-compatible proxy + autoscaler + model loader
gpustack	GPU cluster manager + vLLM/SGLang 编排 + observability
llm-d-workload-variant-autoscaler	Deprecated 的历史 variant autoscaling 设计, 仅供迁移/设计参考
llm-d-batch-gateway	不管理模型部署, 但把 batch job lifecycle 接到 model serving endpoint
rbg	用 RoleBasedGroup / RoleInstance / CoordinatedPolicy 表达多角色、有状态、跨角色协调的 inference workload
kthena	用 ModelBooster / ModelServing / ModelServer / ModelRoute 组合完整 K8s LLM serving control/data plane
openkruise-kruise	不直接管理模型服务, 但提供 workload enhancement、image preheat、workload spread 等可借鉴的 K8s workload automation 模式

和 engine 的区别

[vllm](#) / [sglang](#) 是推理 engine; model serving operator 管的是 engine 在集群里的部署、扩缩、路由、模型生命周期和 API 暴露。

和 llm-d 外围组件的关系

[llm-d-batch-gateway](#)、[llm-d-benchmark](#)、[llm-d-inference-sim](#), 以及 deprecated/历史的 [llm-d-workload-variant-autoscaler](#), 说明 model serving operator 周围还有一圈工程能力:

能力	代表	和 operator 的边界
	llm-d-batch-gateway	

能力	代表	和 operator 的边界
Batch job control plane		不创建模型服务，复用已有 endpoint 执行异步 JSONL job。
Variant autoscaling (历史)	llm-d-workload-variant-autoscaler	Deprecated 设计曾为多个 variant 计算 desired allocation；现仅供迁移/设计参考。
Benchmark lifecycle	llm-d-benchmark	不是生产 controller，而是部署、运行、收集和分析实验。
Simulator	llm-d-inference-sim	不是真实模型服务，用于验证 controller/router/benchmark 闭环。

选型时不要把这些都归类成“serving operator”。operator 管声明式生命周期；batch、benchmark 和 simulator 分别补异步任务、评测和测试替身。对新部署，llm-d 当前路径是 [EPP metrics → KEDA Prometheus scaler → HPA](#)；[llm-d-workload-variant-autoscaler](#) 的 Entity Conflict 注记及历史设计仅用于迁移/设计参考。

和 Kubernetes 调度/指标外围的关系

模型服务 operator 只管理模型服务生命周期还不够。[dynamo](#) 展示了另一条边界：DGDR/Profiler/AIConfigurator 负责部署前配置搜索，DGD/DCD/Operator 负责资源物化，Planner 负责运行时 P/D 容量控制，而 Router 留在请求热路径。kueue / jobset / [lws](#) 决定 batch 和分布式 workload 怎么表达与入队；karpenter 决定节点容量怎样按 pending pods 扩出；metrics-server / prometheus-adapter 决定 autoscaling 能看到哪些指标。kubernetes-workload-automation 提供更上位的整体视角。

MUSA Learning Notes

MUSA Learning Notes 是 [googs1025/musa-learning-notes](#) 的 GPU 编程学习日志，用 6 周路线和 38 个 MUSA 示例串起 MUSA SDK、CUDA→MUSA 迁移、Stream/Graph、访存、GEMM、调试、多卡和 torch_musa。详见 [src-musa-learning-notes-architecture](#)。

架构边界

它不是生产 SDK、推理引擎或 benchmark 平台，而是 [gpu-programming-learning](#) 系列中的实践入口。仓库把学习路线放在 docs/roadmap.md，把概念和迁移规则放在 docs/concepts.md / docs/cuda-vs-musa.md，把可运行实验放在 code/week1 到 code/week6，再用 notes 记录 MUSA SDK 实测差异。

适合 / 不适合

场景	判断
想从 CUDA 背景迁移到 MUSA	适合，先看 CUDA→MUSA 映射，再按 week 跑示例
想补 GPU kernel 基础	适合，Runtime → Stream/Graph → Reduce/访存/GEMM 的路径清晰
想选型生产 LLM serving 平台	不适合，应看 llm-inference , vllm , sglang , dynamo , llm-d
想管理 Kubernetes GPU 设备	不适合，应看 k8s-gpu-device-stack , gpu-operator , k8s-device-plugin , gpu-sharing

同类位置

与 CUDA_Freshman 相比，MUSA Learning Notes 更关注 MUSA SDK 和摩尔线程差异；与 LeetCode / LeetCodeGPU 相比，它更像学习路径和迁移笔记，不只是题库。它可以作为 [src-ai-infra-learning-cn-stars](#) 中 GPU / CUDA / kernel 学习路线的本地实践补充。

MUSA Learning Notes 架构与设计思路分析

原文: raw/musa-learning-notes-architecture-analysis.md · 仓库: <https://github.com/googs1025/musa-learning-notes> · 分析版本 HEAD b4042c3

一句话定位

[musa-learning-notes](#) 是面向 [gpu-programming-learning](#) 和 CUDA 到 MUSA 迁移的公开学习日志，不是通用库或生产框架。它把 MUSA SDK 官方编程指南拆成 6 周课程，用 38 个 .mu / C++ / Python 小示例覆盖 Runtime API、Stream/Event/Graph、执行模型、访存优化、GEMM、调试、多卡和 torch_musa。它适合作为 wiki 中 GPU / CUDA / MUSA 学习系列入口，连接底层 kernel 训练、[llm-inference](#) 性能理解和 [k8s-gpu-device-stack](#)。

核心架构图

```
|
|-----|
| MUSA Learning
Notes
| Public learning log, not a reusable runtime or
library
|
|-----|
|
| docs/
|
| ├── roadmap.md           6-week curriculum and official-guide
mapping
| ├── concepts.md          SIMT, hardware, memory, sync, error
model
| ├── cuda-vs-musa.md      CUDA Runtime/API/tool/library migration
map
| ├── setup.md             local/AutoDL/MUSA SDK
setup
| └── articles/            long-form learning
notes
|
|-----|
```

```

|
code/
|
| ├── include/musa_common.h  MUSA_CHECK, MUSA_CHECK_KERNEL, CpuTimer,
GpuTimer |
| ├── CMakeLists.txt        mcc-based full-course build
harness      |
| ├── week1/                hello, thread index, device info, memory,
errors |
| ├── week2/                vectorAdd, pinned memory, timer, stream,
graph |
| ├── week3/                warp divergence, reduce, shfl, dynamic
parallel |
| ├── week4/                coalescing, offset, AoS/SoA,
transpose      |
| ├── week5/                shared, constant, tiled GEMM,
muBLAS          |
| ├── week6/                MUSA GDB, error dump, MCCL,
torch_musa      |
| └─ leetgpu/easy/          LeetGPU Easy kernels ported to
MUSA            |
|

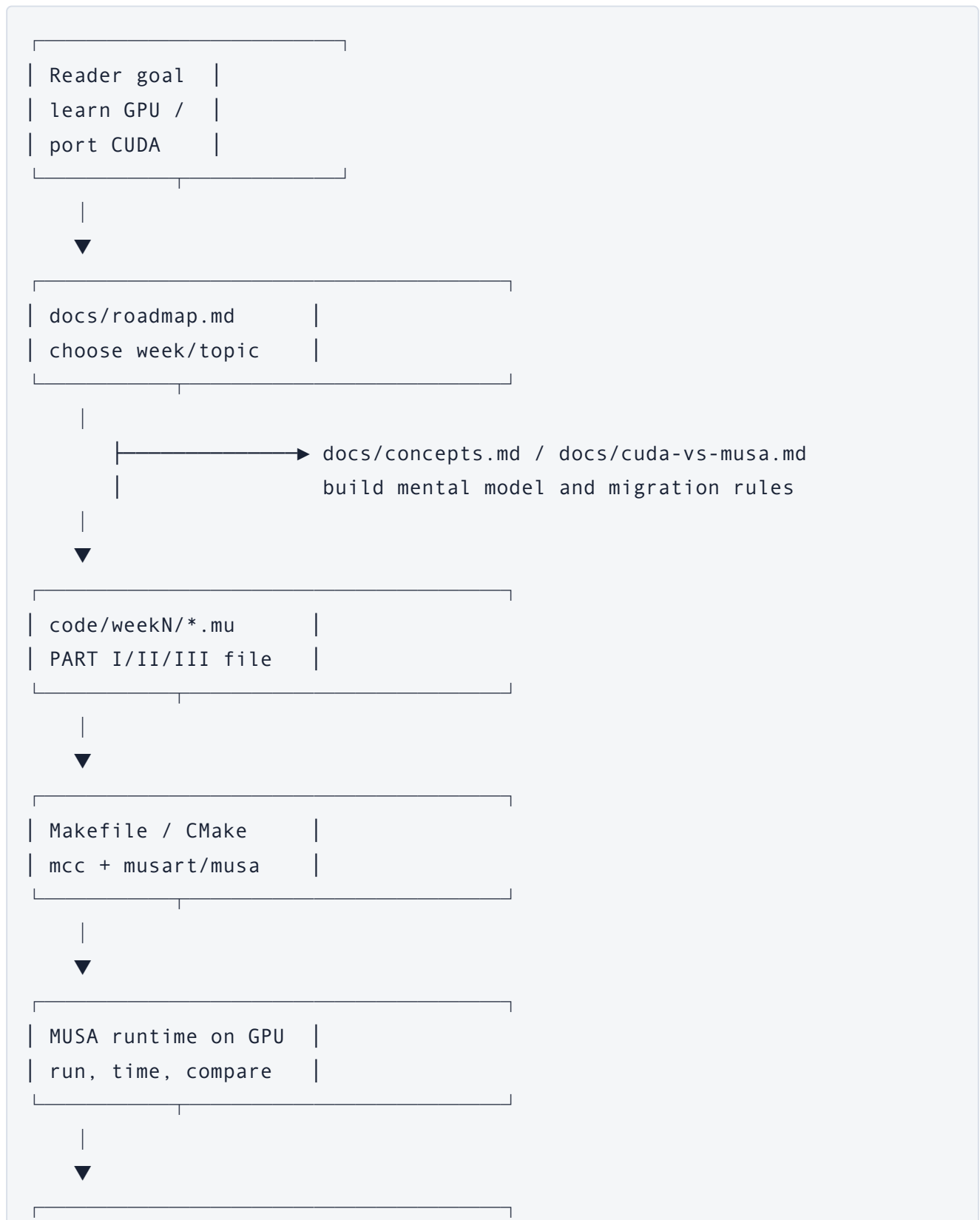
```

模块分层

层 / 模块	职责
学习路线与概念层	定义 6 周路线、基础概念、术语和每周学习目标
CUDA→MUSA 迁移层	建立 CUDA Runtime 到 MUSA Runtime 的命名映射，并突出 warp size、工具链、库名和 Driver API 差异
构建与公共工具层	用 mcc 编译 .mu，统一错误检查、计时和远程运行体验
渐进示例层	每个示例只讲一个点，按 Runtime → Stream/Graph → Reduce/访存/GEMM → 调试/多卡/框架递进
练习题与迁移训练层	把 LeetGPU/CUDA kernel 练习迁移成 MUSA 版本，形成可刷题的 kernel 训练路径
实测与复盘层	记录 AutoDL / MUSA SDK 实测数字、环境差异和故障排查

这个仓库的边界很清楚：它不封装一个可被外部调用的 SDK，也不提供生产级 benchmark harness。它的价值在课程结构和示例粒度：同一个概念通常对应一个最小 .mu 文件，读者可以把文档、代码、实测结果和习题串起来。

关键数据流



notes + exercises	
retain measurements	
and next questions	

设计决策与哲学

- 公开学习日志而非教程产品：仓库按 MUSA SDK 官方编程指南一周一章推进，保留学习过程和实测复盘，适合接入 [gpu-programming-learning](#)。
- 每个示例只演示一个点：roadmap 把 timer、数据布局、unroll、Stream、Graph 等变化拆成独立小例子，避免一个大 demo 同时承担太多概念。
- CUDA 迁移先靠高相似度，再停在硬件差异处：文档给出 Runtime API、Stream/Event、Graph、库名和工具链映射，但特别强调 MUSA warp=128、架构号、PTX/IR、拓扑和 Driver API 前缀不能无脑替换。
- 构建系统服务学习体验：顶层 CMake 在 project() 前设置 mcc，再给 mcc 加 -x musa，让 IDE 索引和整库构建围绕 .mu 示例工作。
- 把错误检查和计时前置为公共肌肉记忆：musa_common.h 抽出 MUSA_CHECK、MUSA_CHECK_KERNEL、CpuTimer、GpuTimer，把同步错误、异步错误和 GPU event 计时变成后续实验共同底座。
- 记录反直觉实测，而不是只复述 CUDA 经验：Week 2 记录 MUSA 3.1.0 上 unified memory prefetch 不支持、4-stream 只加速 1.14x、Graph 比 direct launch 慢 1.65x，保留 MUSA 当前实现成熟度的现实边界。

关键组件

CUDA→MUSA 对照文档

docs/cuda-vs-musa.md 是整个学习系列的迁移索引。它把 CUDA Runtime / Stream / Graph / 库名映射成 MUSA 对应物，但真正有价值的是指出 warp size: 128 vs 32、-arch=sm_xx 不可复用、PTX/IR 不同、Driver API 使用 mu* 前缀这些迁移边界。对已有 CUDA 背景的人来说，这页是防止迁移误判的检查表。

Week 2 Stream / Graph 示例

Week 2 是最像系统性能实验的一组示例。05_multi_stream.mu 展示“真异步”同时需要 pinned host memory、Async API 和 non-default stream；07_musa_graph.mu 展示 musaStreamBeginCapture → musaStreamEndCapture → musaGraphInstantiate → musaGraphLaunch 的最短路径，同时保留 MUSA 3.1.0 上 Graph 反而更慢的实测结论。这一组内容可以和 [llm-inference](#) 中逐 token 小 kernel、launch overhead 和静态 DAG 优化关联起来。

相关页面

- [musa-learning-notes](#)
- [gpu-programming-learning](#)
- [src-ai-infra-learning-cn-stars](#)
- [llm-inference](#)
- [k8s-gpu-device-stack](#)

NVIDIA Dynamo 架构与业务问题（文档重构版）

原文: raw/dynamo-architecture-analysis.md · 仓库: <https://github.com/ai-dynamo/dynamo> · 分析版本 HEAD ed64b7d
本页按当前 README 与官方文档重构; “文档明确”与“架构推导”已尽量区分。

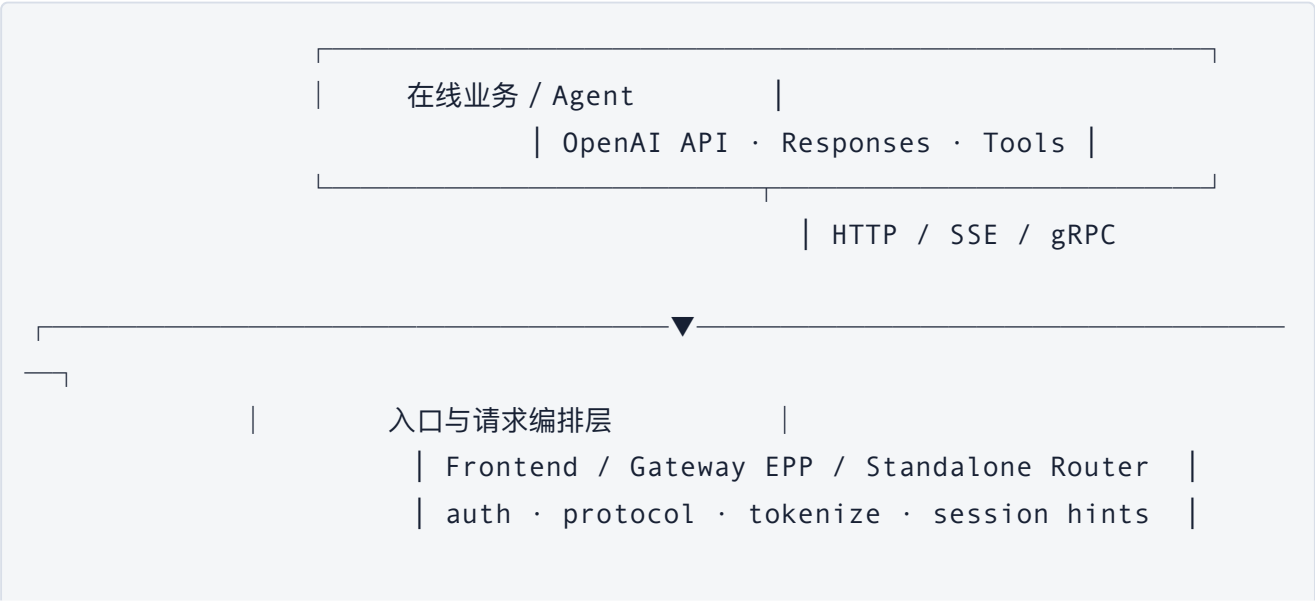
一句话定位

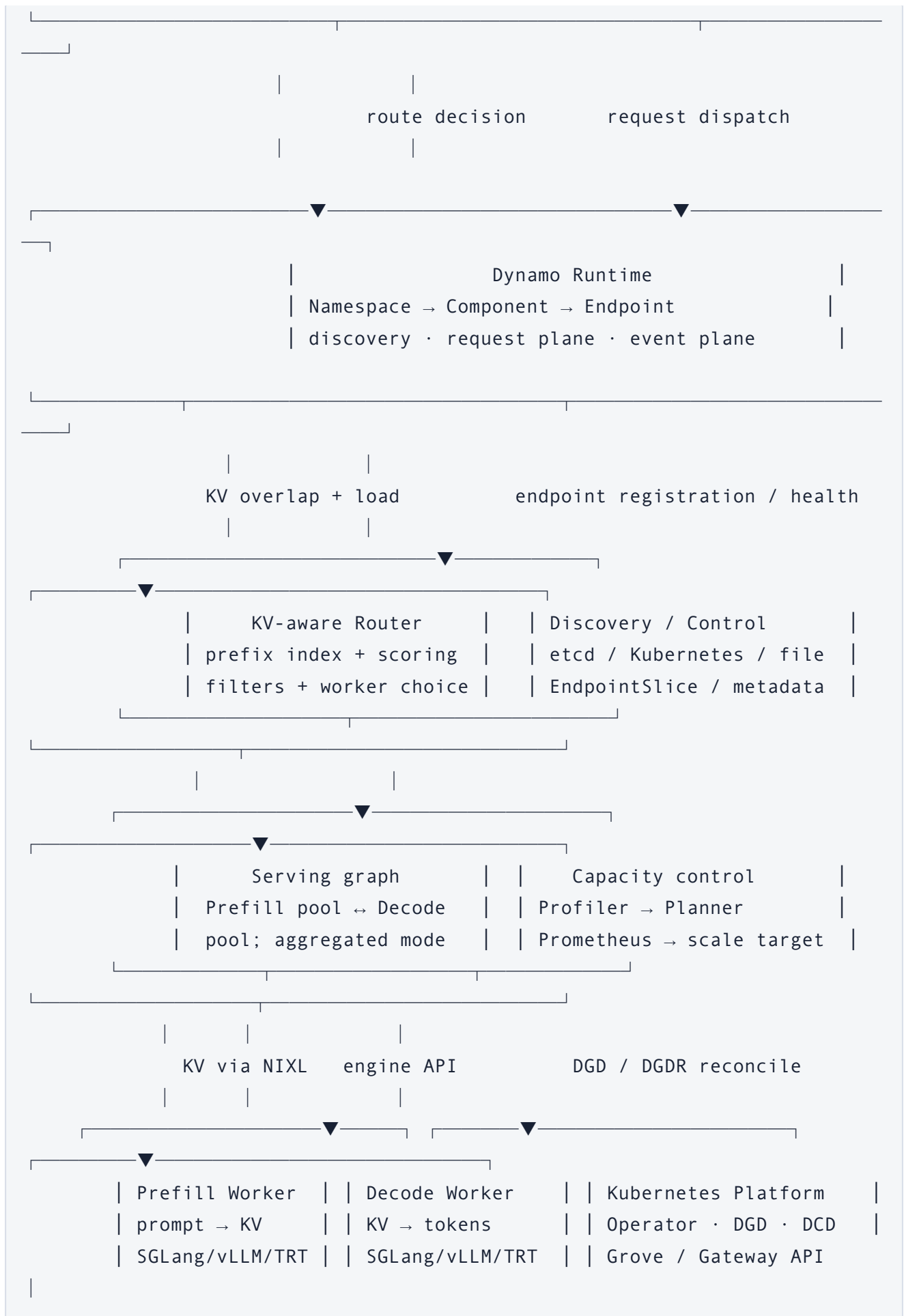
[dynamo](#) 是位于 [SGLang](#)、[vLLM](#)、TensorRT-LLM 之上的数据中心级 [LLM 推理编排层](#)。它解决跨节点、P/D 分离、KV 局部性、SLA 扩缩和故障迁移问题，而不是替代推理引擎本身。

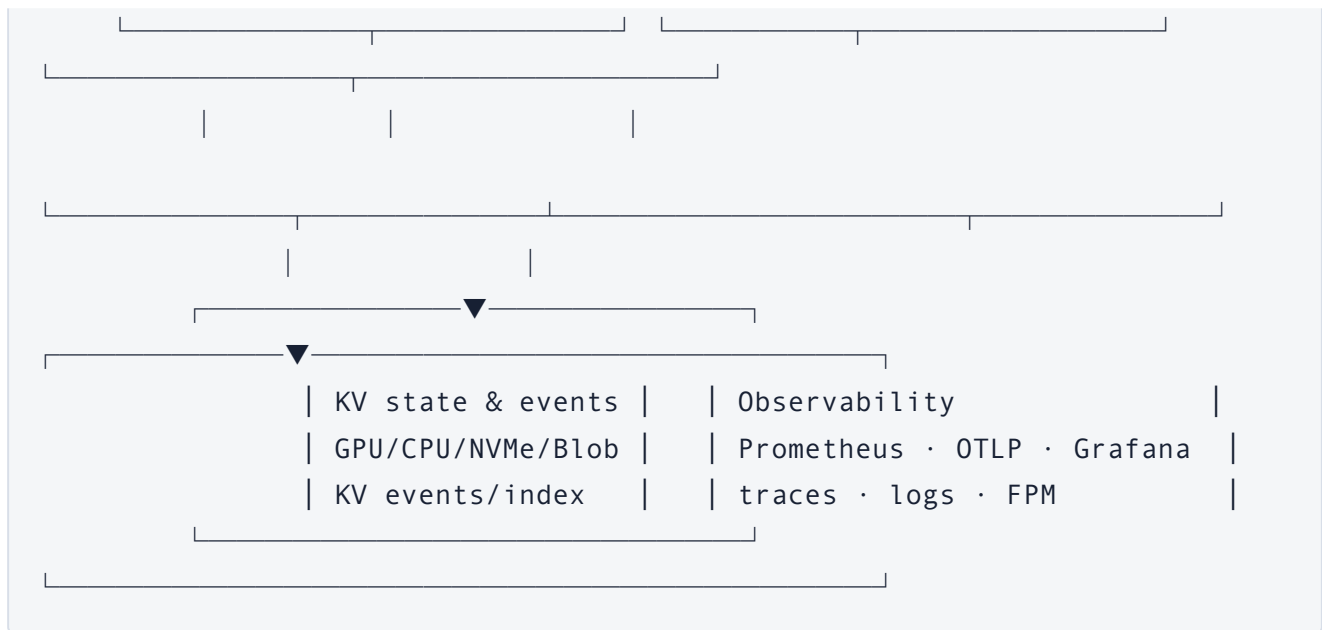
它想解决的业务问题

- 长 prompt 的 prefill 会阻塞 decode: 用独立 prefill/decode 池和 NIXL KV transfer 做资源隔离。
- 传统负载均衡不知道缓存: 用 KV overlap + 活动负载做成本路由, 减少重复 prefill。
- GPU 容量靠人工试错: 用 DGDR 描述意图, Profiler/AIConfigurator 生成 DGD, Planner 在线调整副本。
- worker 故障直接暴露给用户: 用 migration、worker inhibition、canary 和 graceful shutdown 保持服务连续性。
- 平台入口各有标准: 可选 Dynamo Frontend-native 路由, 或 Gateway API + GAIE EPP, 把入口治理和 serving selection 解耦。

核心架构图





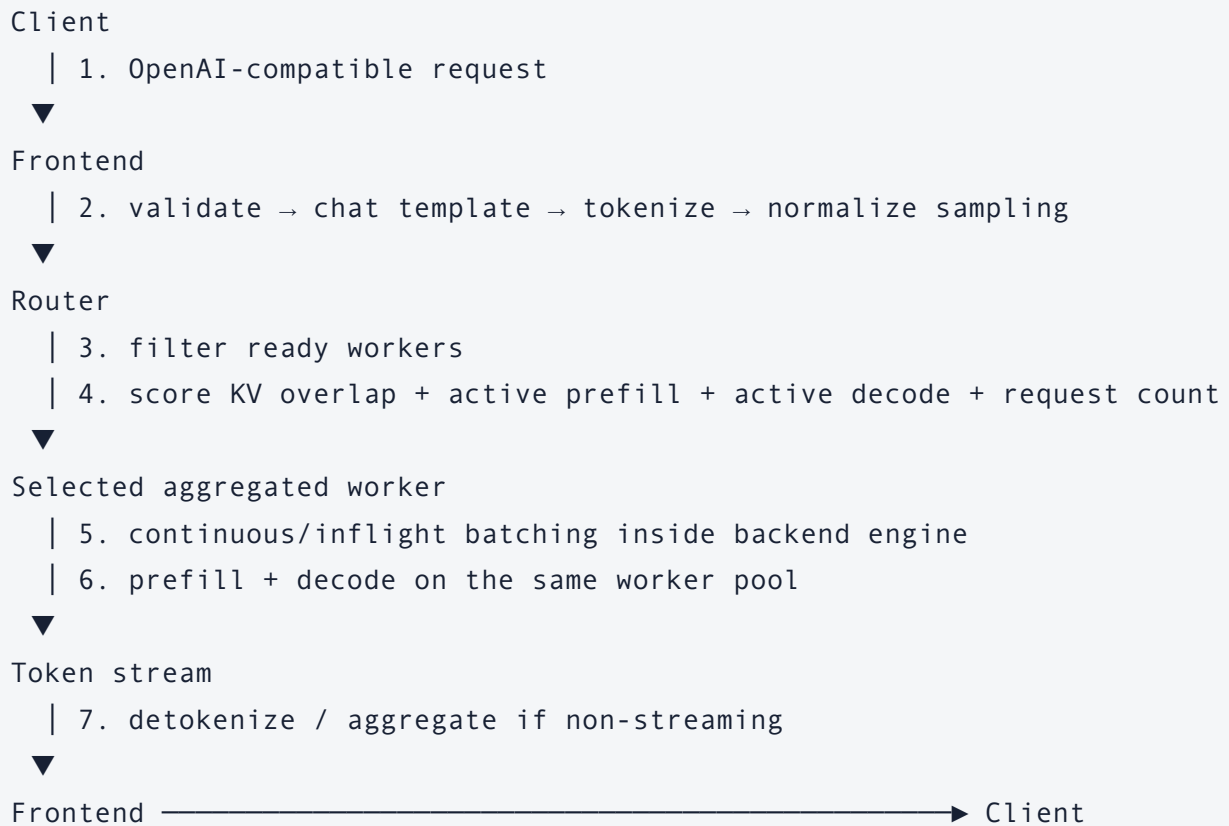


模块分层与边界

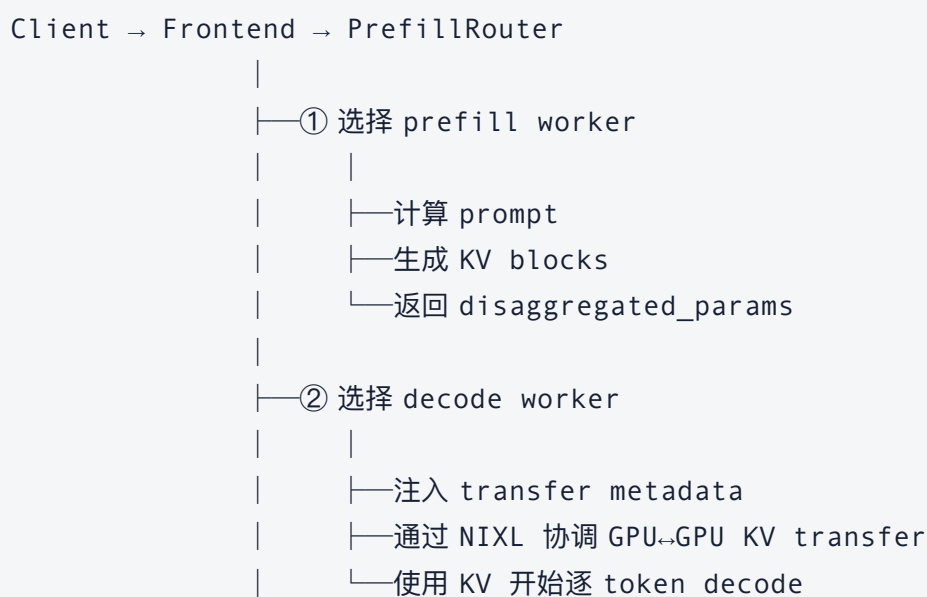
层	功能	边界
入口	Frontend、Gateway EPP、Standalone Router	协议、tokenize、入口治理；不实现 GPU kernel
决策	KV-aware Router、PrefillRouter	选择 worker；不拥有全局 Kubernetes 状态
Runtime	Namespace、Component、Endpoint、discovery/request/event planes	连接与生命周期；不替代 Operator
执行	Prefill/Decode/Aggregated Workers + backend engine	batching、sampling、KV、token generation
KV/传输	NIXL、KV index/events、KVBM/offload	KV transfer 与分层存储；不等价于强一致数据库
控制	DGDR、Profiler、AIConfigurator、Planner、DGD/DCD、Operator、Grove	配置搜索、扩缩、放置；不进入 token 热路径
横切	Prometheus、OTLP、canary、migration	观测与可靠性；不能阻塞推理热路径

请求与控制流

聚合请求

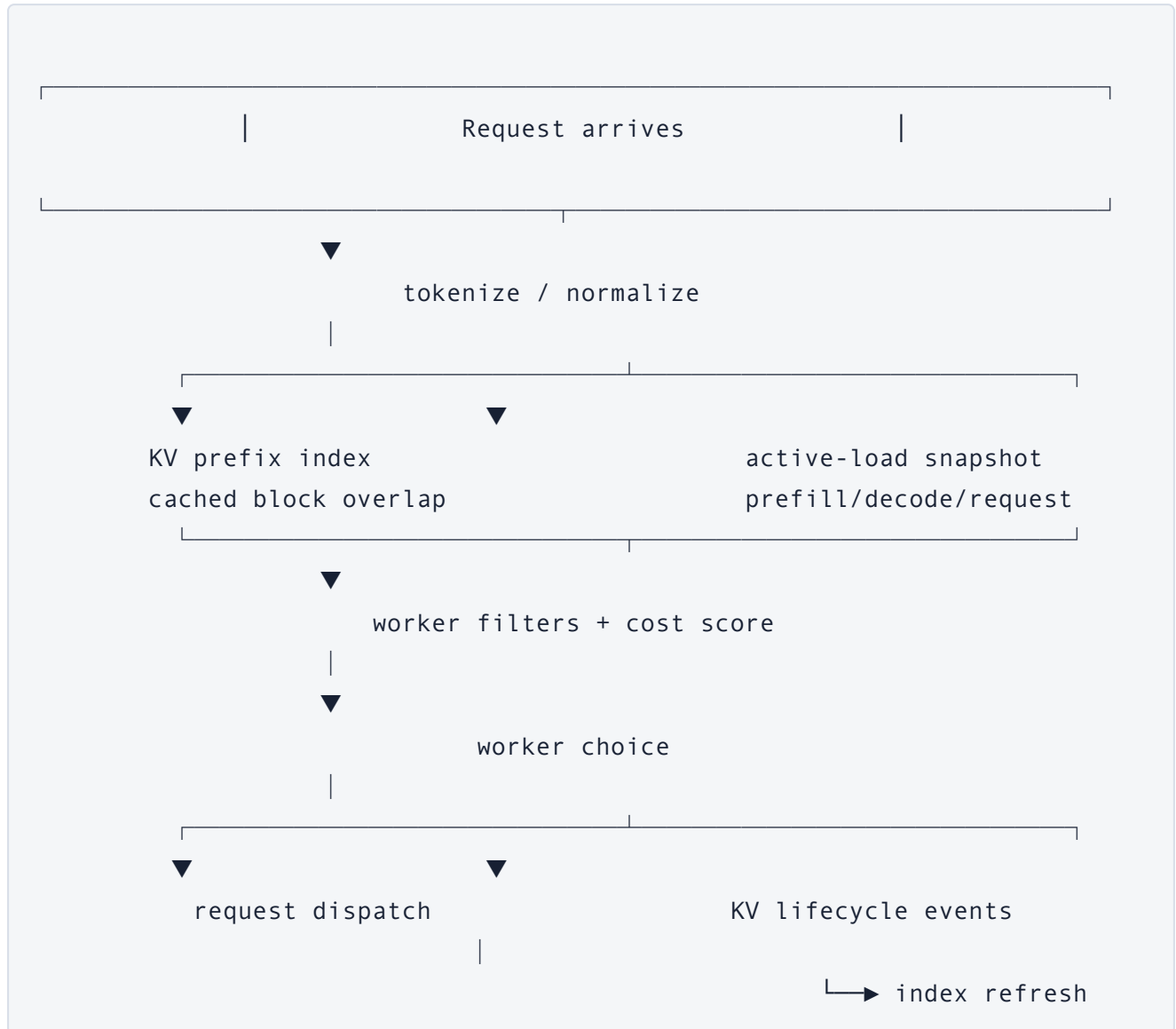


P/D 分离请求



└─③ stream tokens → Frontend → Client

KV-aware 路由反馈环



路由本质是把 cache locality 转成成本折扣，同时惩罚已分配的 prefill/decode 工作： $\text{adjusted_prefill} = \max(\text{raw_prefill} - \text{overlap_credit}, 0)$ ，再与 projected decode blocks、active request penalty 合并。它不是“永远选择 cache 命中最多的 worker”。详见 [inference-routing](#) 与 [radix-attention](#)。

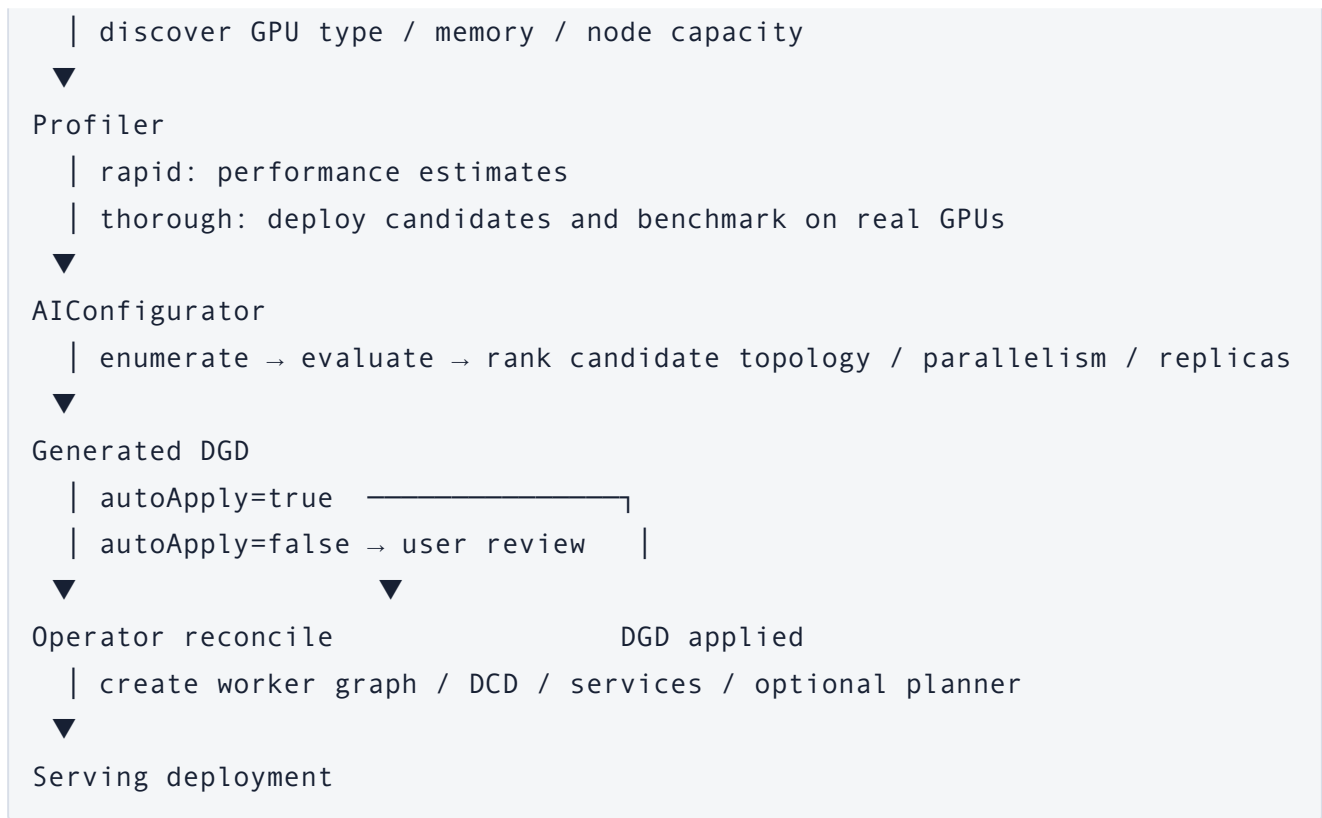
DGDR → DGD → Serving

用户意图

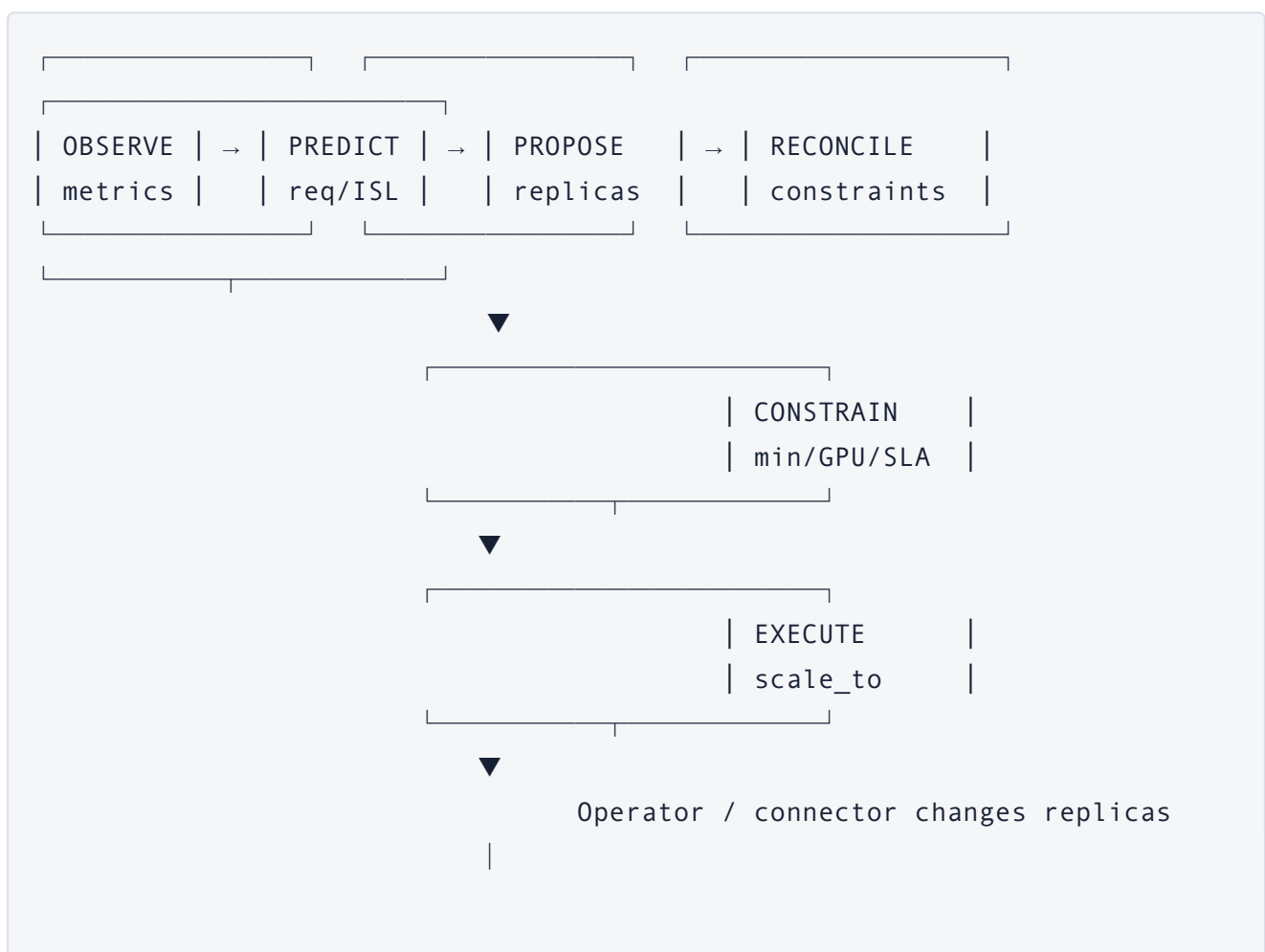
| model + backend + hardware + workload + SLA + optional planner



DGDR



Planner 扩缩闭环



OBSERVE

feedback

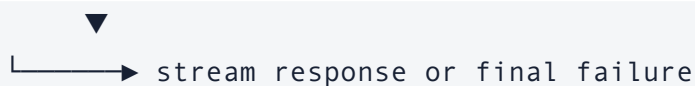
throughput loop 预测持续需求并提供下界，load loop 快速纠正 SLA 压力。它们共享 worker inventory、性能模型、KV hit rate、speculative accept length 与 GPU budget；因此 Planner 不是普通 CPU utilization HPA。

Gateway API 路由

```
Client
|
▼
Kubernetes Gateway / HTTPRoute
| 入口策略、认证、限流、边缘观测
▼
GAIE Endpoint Picker Plugin (EPP)
| 复用 Dynamo KV-aware selection
▼
Selected worker Frontend sidecar
| router-mode=direct
▼
Worker / engine
```

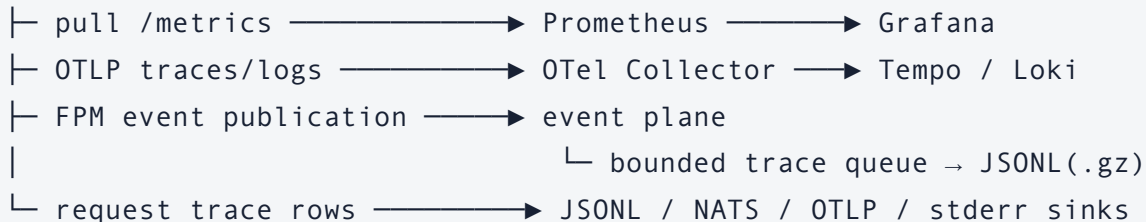
故障迁移

```
request in flight
|
▼
worker error / timeout / disconnect
|
├─不可迁移错误 → reject / return error
|
└─可迁移错误
    | cache token/request state
    | inhibit failed worker locally
    ▼
    choose healthy endpoint
    | migration_limit not exceeded
    └─┬──────────────────────────┬──
        ▼                       |
        replay request           | limit reached
```



可观测性

Dynamo processes



关键设计判断

- Engine 与 orchestration 分离：复用 backend 的 kernel、batching、sampling 专业能力。
- KV 是集群级资源：既影响路由，也影响 P/D transfer 和分层存储。
- 三平面解耦：request、discovery、event 具有不同延迟与一致性目标。
- Profiler 与 Planner 分工：部署前找配置，部署后调容量。
- Operator 与 Router 分工：最终一致地物化资源，毫秒级做在线选择。
- 入口可替换：Frontend-native 与 Gateway API/EPP 共用 serving selection。
- 旁路可观测：观测落后时丢 trace，不阻塞推理热路径。

选型与限制

适合长上下文、多 GPU/多节点、P/D 分离、需要 KV-aware routing、SLA autoscaling 或快速扩容的生产推理。单模型、单 GPU、没有跨节点协调需求时，直接使用 [vllm](#) 或 [sglang](#) 更简单。

P/D 分离会增加 KV transfer；Planner 依赖指标与性能模型；migration 需要请求幂等；远端 KV 需要额外的数据驻留和租户隔离治理。README 中的 2x TTFT、7x 启动等数字是特定 benchmark，不应视为通用保证；本次没有运行 GPU benchmark。

相关页面

- [dynamo](#)
- [llm-inference](#)
- [inference-routing](#)
- [disaggregated-serving](#)
- [kv-cache-offload](#)
- [model-serving-operator](#)
- [ai-gateway](#)

NVIDIA GPU Operator

NVIDIA GPU 软件栈的 Kubernetes Operator，传统 ClusterPolicy/NVIDIADriver 路径管理 driver、container-toolkit、device-plugin、DCGM、MIG manager、sandbox/vGPU 等组件，当前 GPUCluster 提供独立的 DRA 管理路径。历史架构详见 [src-gpu-operator-architecture](#)。

证据说明：2026-09-27 官方默认分支快照为 [60526e35efee](#)，commit 不代表 release。[src-gpu-operator-architecture](#) 保留 2026-06-12 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，DRA 分配与节点准备见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

Operator reconcile NVIDIA 节点软件及选定 operands，不选择单次 Pod 获得哪块 GPU。按 [26.7 文档](#)，ClusterPolicy 对应 [k8s-device-plugin](#) 路径，GPUCluster 对应 [dra-driver-nvidia-gpu](#) 管理路径，两者不能在同一集群并存，也不支持直接原地迁移。GPUCluster 需独立 NVIDIADriver/预装 driver 和兼容 [cdi](#) 的 runtime；这些条件不能由 node-feature-discovery 标签或 Operator Ready 替代。

架构边界

GPU Operator 管“节点软件栈怎么安装、升级、保持健康”，不是单次 Pod GPU allocation 算法。它通常部署或管理 [k8s-device-plugin](#)，并与 hami、[dra-driver-nvidia-gpu](#) 处在不同层。

选型判断

- 集群管理员管理 NVIDIA 软件栈：GPU Operator。
- kubelet 层暴露 GPU：[k8s-device-plugin](#)。
- 新 DRA API 分配：[dra-driver-nvidia-gpu](#)。
- 共享/虚拟化调度：hami。

NVIDIA GPU Operator 架构与设计思路分析

原文: raw/gpu-operator-architecture-analysis.md · 仓库: <https://github.com/NVIDIA/gpu-operator> · 分析版本 HEAD 0219120 (2026-06-11)

一句话定位

NVIDIA GPU 软件栈的 Kubernetes Operator，用 ClusterPolicy/NVIDIADriver CRD 驱动 driver、container-toolkit、device-plugin、DCGM、MIG manager、sandbox/vGPU 等组件的声明式部署和升级。它管的是 GPU node 软件生命周期，而不是单次 Pod GPU 分配算法。这页和 kubernetes k8s-operator [cdi gpu-sharing](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图



模块分层

层 / 模块	职责
Controller manager	启动 controller-runtime manager、scheme、webhook、leader election、ClusterPolicy 和 upgrade controller。
ClusterPolicy controller	单例 ClusterPolicy reconcile，按状态机逐步部署 operands 并更新状态/metrics。
State/resources renderer	按 CR spec 和 cluster info 渲染 DaemonSet/Service/ConfigMap/CRD 等对象并 create/update。
CRD/API	定义 ClusterPolicy、NVIDIADriver 等声明式 spec/status。
升级与验证	管理 driver upgrade、operator validator、组件就绪检查。

关键数据流

```
Admin applies ClusterPolicy
|
├─ controller-runtime manager enqueues ClusterPolicy reconcile
|
├─ controller initializes singleton, cluster info, node labels
|
├─ state machine renders each operand manifest from assets/state-*
|
├─ create/update objects and check DaemonSet/Deployment readiness
|
└─ conditions/metrics expose Ready/NotReady and driver upgrade state
```

设计决策与哲学

- Operator 管组件生命周期，不直接做 GPU allocation：它部署 device plugin、driver、runtime、DCGM 等 operands，真正 Allocate 仍由 device plugin/kubelet 路径完成。
- 状态机比单次 reconcile 更贴合 GPU 软件栈：ClusterPolicyController.step() 逐状态推进，便于表达 driver/toolkit/plugin/monitoring 的依赖顺序。

- 渲染资产与 cluster info 解耦: internal/state/driver.go 根据 OpenShift、precompiled、kernel、proxy 等信息生成实际 DaemonSet。

与同类项目的架构差异

维度	NVIDIA GPU Operator	同类 A	同类 B
定位	GPU 软件栈 Operator	k8s-device-plugin: kubelet plugin	DRA driver: 新 API 分配
核心对象	ClusterPolicy/NVIDIADriver	DaemonSet + config	ResourceClaim/ResourceSlice
主要用户	集群管理员	平台/节点管理员	DRA workload/platform

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

相关页面

- [kubernetes](#)
- [k8s-operator](#)
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)

NVIDIA k8s-device-plugin

NVIDIA 官方 Kubernetes device plugin，把 GPU/MIG/vGPU 发现为 kubelet extended resources，并在 Allocate 阶段通过 env、volume-mounts 或 CDI annotations 把设备传给容器。详见 [src-k8s-device-plugin-architecture](#)。

证据说明：2026-09-27 官方默认分支快照为 [86142cf1a93f](#)，commit 不代表 release。[src-k8s-device-plugin-architecture](#) 保留 2026-06-12 的 raw-backed Source；当前分层见 [k8s-gpu-device-stack](#)，DRA 对照路径见 [kubernetes-dra-design-deep-dive](#)。

在 M5-C Device / GPU 地图中的位置

插件向 kubelet 注册 extended resource，通过 ListAndWatch 报告设备与健康，在 Allocate 中返回设备交接配置；它没有 [kubernetes-dra](#) 的丰富 Claim/库存/条件模型。node-feature-discovery 的发现信号和 [gpu-operator](#) 的组件部署是相邻职责；其与 [dra-driver-nvidia-gpu](#) 的路径选择应按所需 API、driver 支持与资源所有权评估。

架构边界

它是 GPU 暴露基础层，不负责安装 driver，也不负责全局调度策略。GPU Operator 常用来部署和管理它；HAMi 经典路径使用自己的 device plugin，不能与官方插件在同一节点争用 `nvidia.com/gpu`；DRA driver 则采用 ResourceClaim/ResourceSlice 分配模型。

选型判断

适合需要理解 Kubernetes GPU allocation 基础机制、MIG/MPS/time-slicing/CDI 策略的人。做 AI serving 平台时，它是 [gpu-operator](#)、hami、[kubernetes-dra](#) 前必须理解的底层节点。

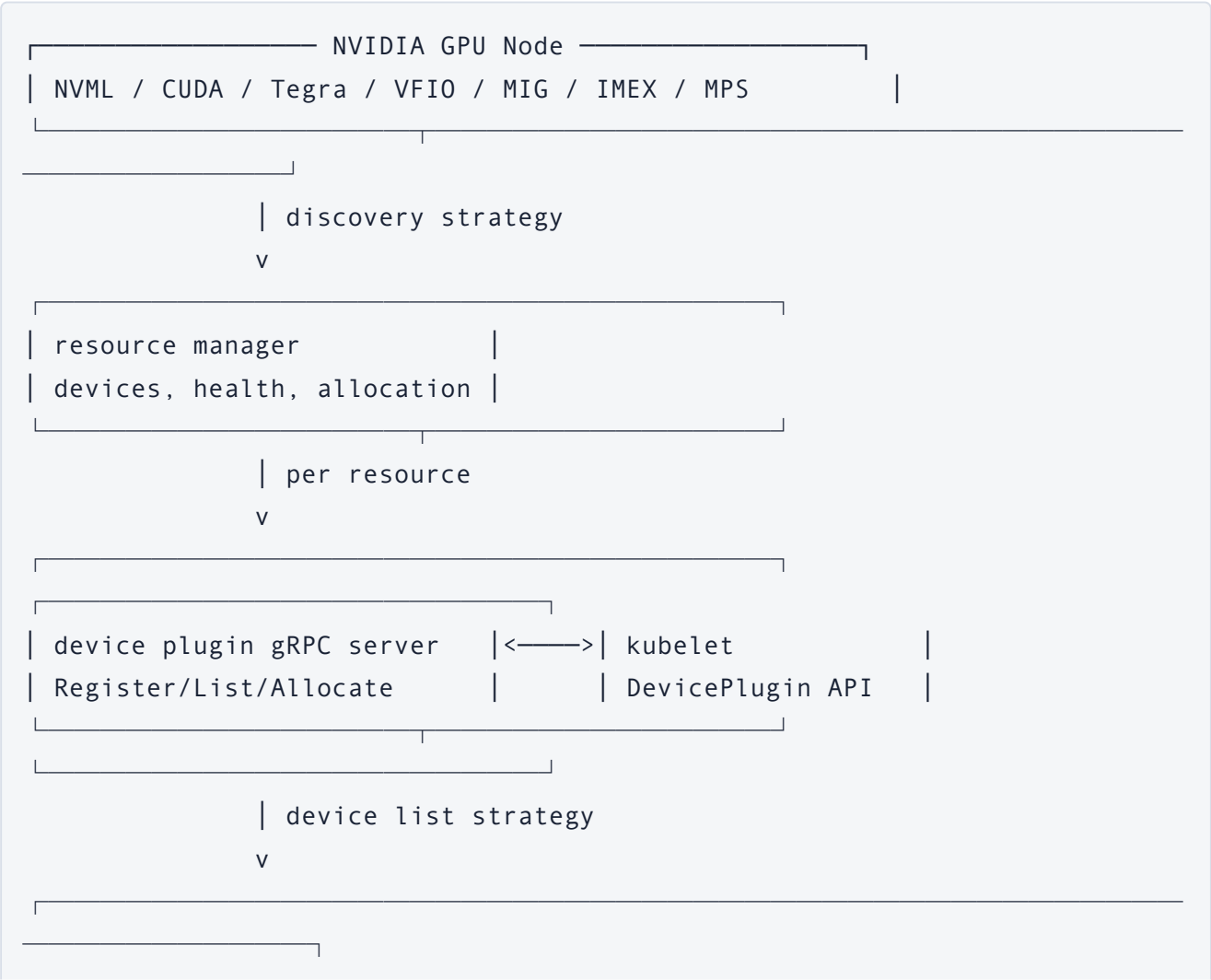
NVIDIA k8s-device-plugin 架构与设计思路分析

原文: raw/k8s-device-plugin-architecture-analysis.md · 仓库: <https://github.com/NVIDIA/k8s-device-plugin> · 分析版本 HEAD 8688949 (2026-06-10)

一句话定位

NVIDIA 官方 Kubernetes device plugin，把节点上的 GPU/MIG/vGPU 发现为 kubelet extended resources，并在 Allocate 阶段通过 env、volume-mounts 或 CDI annotations 把设备传给容器。它是 GPU 暴露的基础层，上层 GPU Operator/HAMi/DRA 都会和它形成互补或替代关系。这页和 [kubernetes cdi gpu-sharing k8s-gpu-device-stack](#) 形成对比，重点帮助理解项目边界和选型位置。

核心架构图



```
| container envvar / volume-mounts / CDI annotations |  
|  
|  
|
```

模块分层

层 / 模块	职责
CLI/config	解析 MIG strategy、device-list strategy、CDI/MPS/IMEX/driver root 等配置。
Plugin manager	解析 discovery strategy，创建 CDI handler、resource manager 和 per-resource plugin。
Resource manager	基于 NVML/CUDA/Tegra/VFIO 发现设备，处理 MIG/resource mapping、preferred allocation、health。
Device plugin server	实现 kubelet device plugin gRPC: Serve/Register/ListAndWatch/Allocate/PreStart。
Feature discovery/ MPS	提供 GPU label、MPS daemon 和 node feature discovery 集成。

关键数据流

```
Node starts nvidia-device-plugin DaemonSet  
|  
├─ main parses config/env flags and selected discovery strategy  
|  
├─ plugin-manager creates CDI spec and resource managers  
|  
├─ per-resource plugin registers Unix socket with kubelet  
|  
├─ ListAndWatch reports healthy GPU/MIG resources  
|  
└─ Allocate validates request and returns env/mount/CDI device edits
```

设计决策与哲学

- 保持 kubelet device plugin API 的基础语义：项目核心是发现、健康检查和 Allocate，不承担全局调度或多租户策略。
- 多种 device list strategy 兼容不同 runtime：envvar、volume-mounts、CDI annotations 并存，让老 runtime 和新 CDI 工作流都能接入。

- `resource manager` 隔离硬件发现差异: `internal/resource/factory.go` 按 NVML/Tegra/VFIO 选择 manager, plugin server 不直接关心硬件细节。

与同类项目的架构差异

维度	NVIDIA k8s-device-plugin	同类 A	同类 B
定位	官方 GPU device plugin 基础层	GPU Operator: 部署管理它	HAMi: 共享调度/虚拟化扩展
API	Kubelet DevicePlugin v1beta1	Operator CRD	scheduler extender + annotations
粒度	GPU/MIG/MPS/time-slicing/CDI	软件组件栈	memory/core/count 共享

选型提示

如果你在快速理解或选型，先判断它是“执行任务的 agent/runtime”、“观察与计量工具”、“消息/插件 bridge”，还是“Kubernetes/GPU 控制面”。同一条 star 列表里的项目经常解决的是相邻但不同的问题：比如 trace viewer 不能替代 remote control bridge，GPU Operator 也不能替代 DRA driver 或 device plugin 的分配路径。

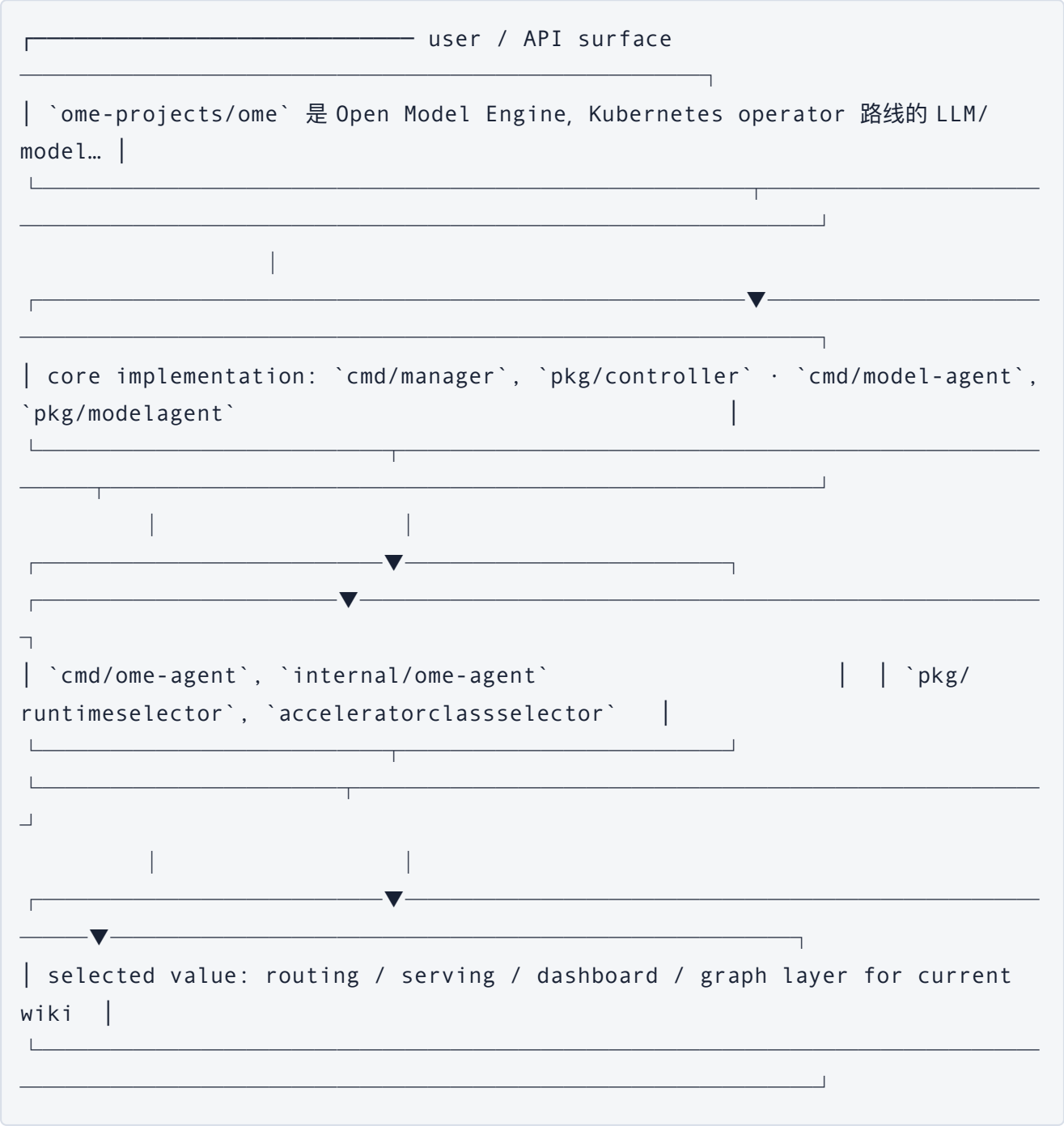
相关页面

- [kubernetes](#)
- [cdi](#)
- [gpu-sharing](#)
- [k8s-gpu-device-stack](#)
- [dra](#)

OME 架构与设计思路分析

ome-projects/ome 是 Open Model Engine, Kubernetes operator 路线的 LLM/model serving 控制面。仓库包含 manager、model-agent、ome-agent、CRD/config/webhook、runtime selector、accelerator class selector、storage/model download、OEP proposals。

核心架构图



模块分层

层/目录	责任
cmd/manager, pkg/controller	operator manager 和 controllers。
cmd/model-agent, pkg/modelagent	模型拉取/管理 agent。
cmd/ome-agent, internal/ome-agent	运行时 sidecar/agent。
pkg/runtimeselector, acceleratorclassselector	runtime/accelerator 选择。

关键数据流

1. 用户提交 OME model/workload CR。
2. controller 选择 runtime/accelerator/storage 并创建 K8s workload。
3. model-agent/ome-agent 处理模型下载、探测和运行时协作。

设计决策

- operator-first，强调模型生命周期和硬件适配。
- accelerator configs 的 data parallel override 是活跃问题。
- OEP 展示路线图比当前实现更宽。

对比定位

和 KServe 相比，OME 更 LLM/accelerator-aware；和 KubeAI 相比更偏 operator 基座和扩展提案。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

PowerMem

OceanBase 团队开源的、面向 LLM agents/apps 的持久化记忆中间件。Apache 2.0, Python 3.11+, PyPI 包名 powermem, 最新版本 v1.1.1（2026-05-13）。

一句话定位

把对话流水线压缩成可被语义检索的长期记忆。四路混合检索（向量 + 全文 + 稀疏向量 + 图谱，OceanBase 原生支持）+ LLM 驱动的事实抽取/更新 + **ebbinghaus-forgetting-curve** 衰减。SDK / CLI（pmem）/ FastAPI 服务 + Dashboard / mcp 服务 / VS Code 扩展 / claude-code-plugin 共用同一份 .env。

关键能力

维度	能力
存储后端	oceanbase（默认，含嵌入式 SeekDB）/ pgvector / SQLite
向量索引	HNSW / HNSW_SQ / IVFFLAT / IVFSQ / IVFPQ
全文检索	jieba / ngram / ngram2 / ik / beng / space 6 种 parser
稀疏向量	qwen_sparse（仅 OceanBase 后端支持）
图谱	OceanBase Graph Store, 3-hop 遍历 + 早停 + 防环
混合融合	hybrid-search-rrf + 自适应权重归一化
LLM 提供商	12 种: openai / anthropic / qwen / gemini / deepseek / ollama / vllm / langchain / azure / siliconflow / zai / qwen_asr
Embedder 提供商	15 种 + sparse
Reranker 提供商	4 种: qwen / jina / zai / generic
多 Agent	5 种 scope（private/agent_group/user_group/public/restricted）+ 4 种 Controller
多模态	text / image / audio（独立 audio_llm 配置）

接入形态

- Python SDK: `from powermem import Memory, auto_config; memory = Memory(config=auto_config())`

- CLI: `pmem memory add/search/list`, `pmem shell REPL`, `pmem config init`
- HTTP API + Dashboard: `powermem-server --port 8000`, Dashboard 自动挂载 `/dashboard/`
- mcp Server: `uvx powermem-mcp sse (sse / stdio / streamable-http)`
- VS Code 扩展: 把记忆接入 Cursor / Claude / Codex / Copilot / Windsurf
- `claude-code-plugin`: Go 二进制 hook + `UserPromptSubmit` 自动检索注入 `additionalContext`
- OpenClaw 集成: 通过 `memory-powermem` 插件

设计哲学（与同类记忆框架对照）

- 跨形态统一中间件 vs `claude-mem` 单一插件: 一个 `.env` 同时供 7 种客户端使用, 不复制配置/状态。
- 认知科学抽象 + 数学公式解耦存储: `working / short / long` 三层 + `ebbinghaus-forgetting-curve` $R = e^{(-t/S)}$; Intelligence 不直接改库, 只在 `metadata` 写分数; `MemoryOptimizer` 离线扫描决定晋升/清理 —— 避免每次写都跑昂贵 LLM。
- OceanBase 优先但绝不绑死: `VectorStoreFactory` + `StorageAdapter` 关键解耦点; v1.1.0 引入嵌入式 `SeekDB (pyseekdb)` 让用户零部署用上 OceanBase 语义。
- 后台线程池: 模块级 3-worker `ThreadPoolExecutor`, 前台只阻塞 LLM 抽取/重要性评估, `telemetry/audit/optimization` 异步。
- Provider 矩阵 + `pydantic-settings`: 每个 provider 一文件一 `config` 类; 切换只改 `.env` 里的 `LLM_PROVIDER` —— 跑通"Qwen + DashScope + OceanBase 全栈国产化"路径。

Benchmark 数字（LOCOMO 长对话）

指标	PowerMem	"塞满上下文" baseline	倍数
准确率	78.70%	52.9%	+25.8 pp
Retrieval p95 latency	1.44s	17.12s	11.9× faster
Tokens / query	~0.9k	~26k	28.9× less

版本演进

版本	日期	关键变更
0.1.0	2025-11-14	核心 memory + 混合检索; LLM 抽取; 遗忘曲线; 多 agent; OceanBase/PG/SQLite
0.2.0	2025-12-16	高级 profiles; 多模态
0.3.0	2026-01-09	生产级 HTTP API Server; Docker
0.4.0	2026-01-20	稀疏向量; profile-based query rewrite; schema 迁移工具
0.5.0	2026-02-06	pydantic-settings 统一配置; OceanBase 原生混合搜索

版本	日期	关键变更
1.0.0	2026-03-16	CLI <code>pmem</code> + Web Dashboard
1.1.0	2026-04-02	嵌入式 SeekDB; IDE 集成 (VS Code + Claude Code)
1.1.1	2026-04-?	bug fix release (GraphStoreFactory / sparse_embedder / custom prompt env / i18n)

相关页面

- 架构详解: [src-powermem-architecture](#)
- 默认存储后端: oceanbase
- 核心算法: ebbinghaus-forgetting-curve、hybrid-search-rrf
- 同类对照: claude-mem (单一 Claude Code 插件 vs PowerMem 通用中间件)、agent-memory (领域综述)
- 接入: claude-code-plugin、mcp

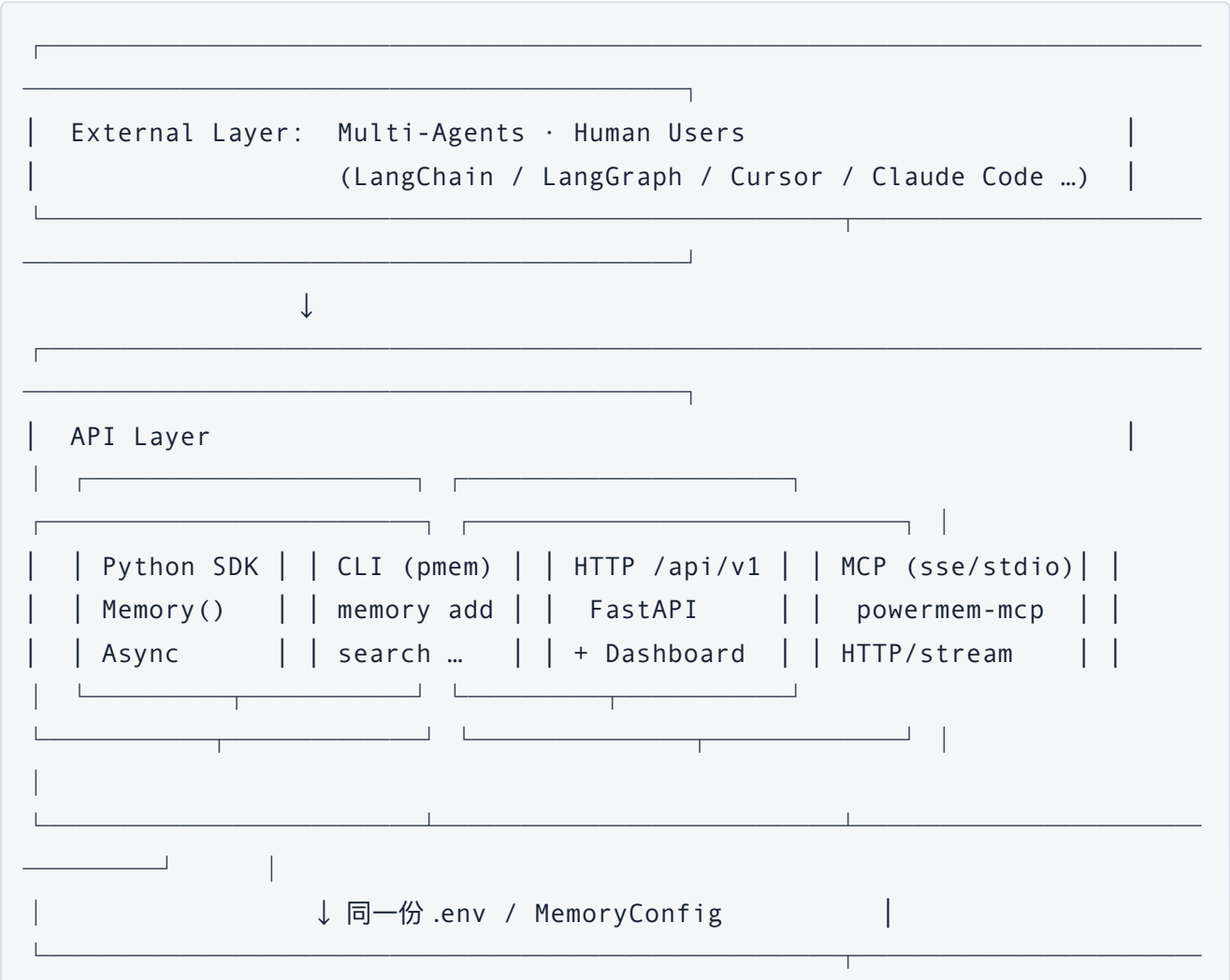
PowerMem 架构与设计思路分析

原文: raw/powermem-architecture-analysis.md · 仓库: <https://github.com/oceanbase/powermem> · 分析版本 v1.1.1 (commit 2c83b77, 2026-05-13)

一句话定位

[powermem](#) 是 oceanbase 团队开源的、面向 LLM agents/apps 的持久化记忆中间件：把对话流水线压缩成可被语义检索的长期记忆。关键手段是 向量 + 全文 + 稀疏向量 + 图谱 四路混合检索（OceanBase 原生支持） + LLM 驱动的事实抽取/更新 + [ebbinghaus-forgetting-curve](#) 衰减。SDK / CLI (pmem) / FastAPI 服务 + Dashboard / mcp 服务 / VS Code 扩展 / claude-code-plugin 共用同一份 .env；默认后端是 OceanBase，也支持 pgvector / SQLite / 嵌入式 SeekDB。在 LOCOMO 长对话评测上比"塞满上下文"方案准确率 78.7% vs 52.9%、p95 1.44s vs 17.12s、token 0.9k vs 26k。

核心架构图



↓

Core Layer (src/powermem/core/)

Memory / AsyncMemory (memory.py 2178 行, CRUD 入口)

└ StorageAdapter / SubStorageAdapter → routing

└ IntelligenceManager → IntelligentMemoryManager

└ ImportanceEvaluator (LLM 打分 0.0–1.0)

└ EbbinghausAlgorithm $R = e^{(-t/S)}$

└ TelemetryManager · AuditLogger

└ MemoryOptimizer (后台衰减/晋升/清理)

↓

Agent Layer (src/powermem/agent/) – scope / permission / privacy

User-Memory (src/powermem/user_memory/) – profile + query_rewrite

↓

Model Layer (src/powermem/integrations/)

LLMFactory

EmbedderFactory

RerankFactory

openai/anthro

openai/qwen/gemini

qwen/jina/zai

pic/qwen/gemini

huggingface/ollama

generic

deepseek/zai/

azure_openai/bedr

siliconflow/vll

ock/lmstudio/sili

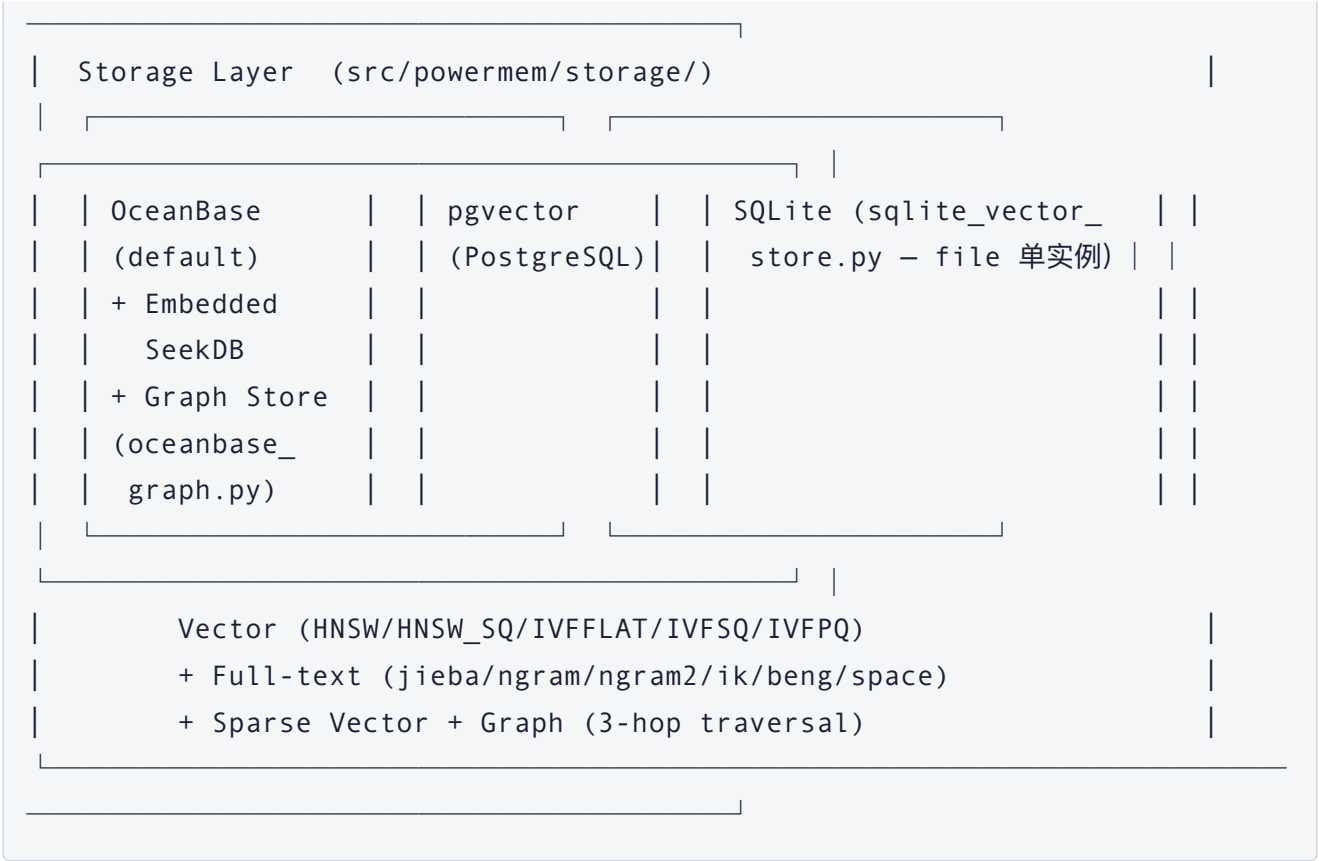
+ SparseEmb

m/ollama/lang...

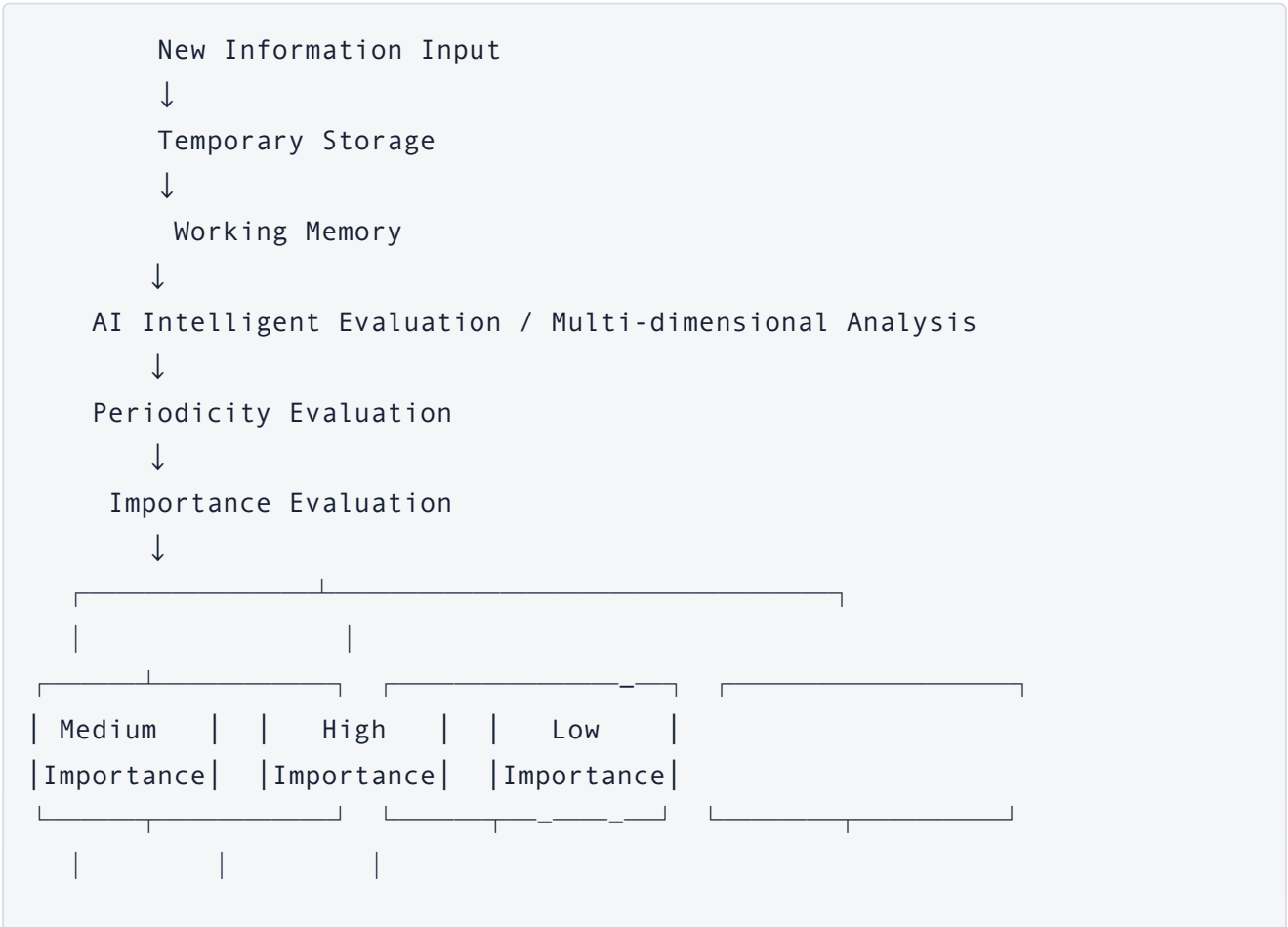
conflow/together...

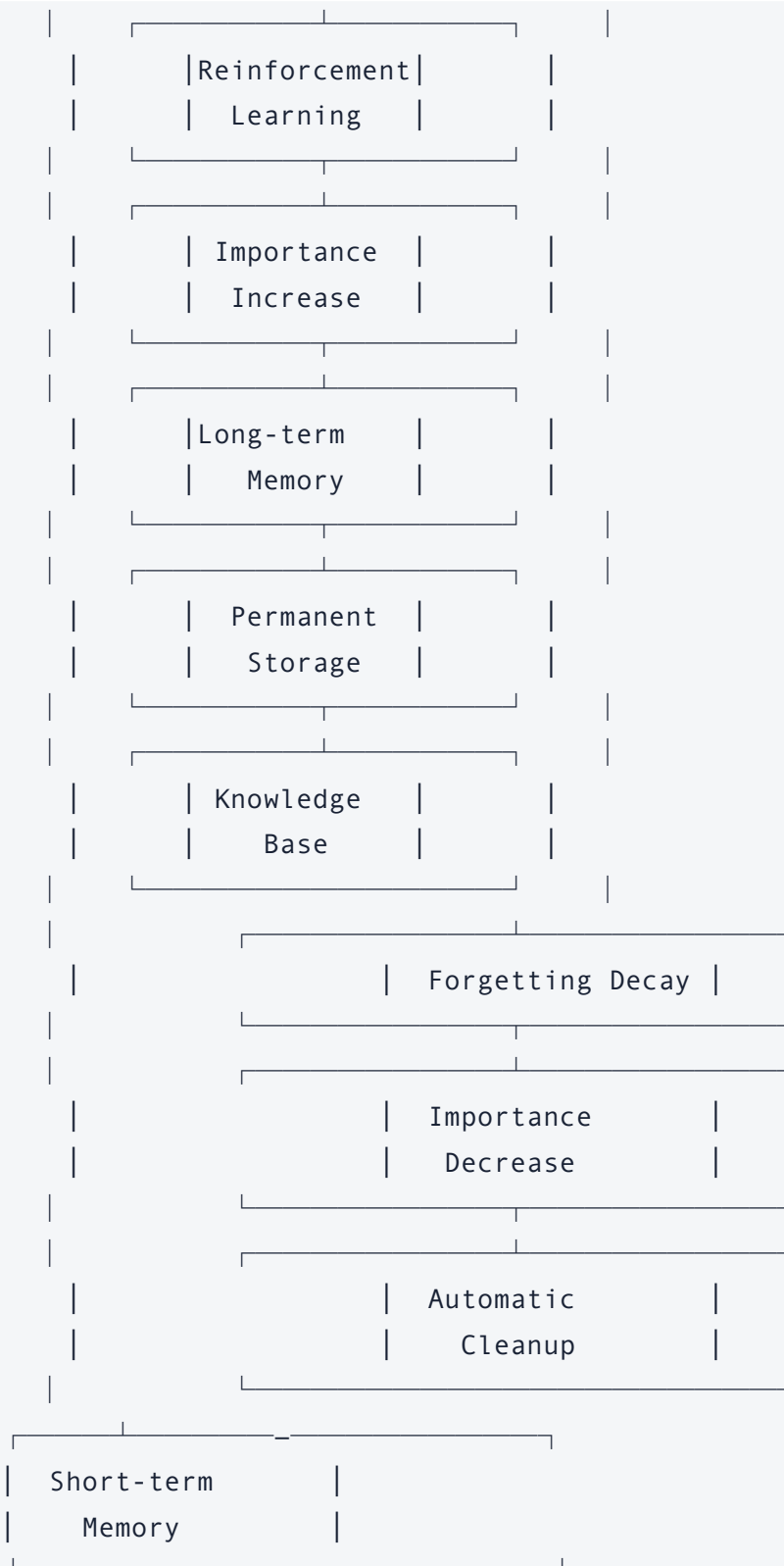
(qwen_sparse)

↓



记忆生命周期（沿用官方 docs/architecture/overview.md 原图）





模块分层

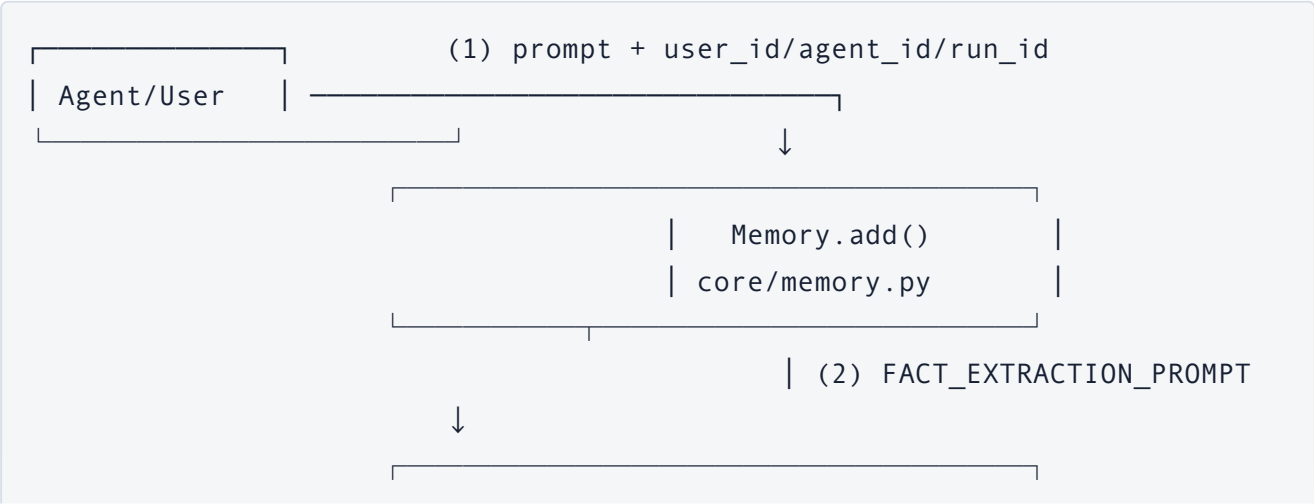
层 / 模块	职责
API · Python SDK	公开 Memory / AsyncMemory / create_memory(auto_config()); CRUD 主循环; 装配所有子组件
API · CLI (pmem)	pmem memory add/search/list/delete、pmem config init、pmem shell REPL
API · HTTP Server	FastAPI; 启动期 lifespan 初始化 service 单例; CORS / 限流 / API key
API · mcp	独立包 powermem-mcp (uvx powermem-mcp sse) , sse/stdio/streamable-http 三种 transport
Core · Memory Engine	配置归一化; 模块级 3-worker 线程池跑后台任务
Core · Intelligence	LLM 重要性打分; ebbinghaus-forgetting-curve 衰减 ($R = e^{(-t/S)}$), 阈值 0.3/0.6/0.8 ; 离线晋升/清理; 插件点
Core · Agent 多租户	5 种 scope; scope/permission/privacy/collaboration 4 个 Controller; 3 种 implementation
Core · User Memory	用户档案抽取; 基于档案的查询改写
Prompts	集中托管所有 LLM prompt; 支持 custom_*_prompt 配置覆盖
Model · LLM	12 provider: openai / anthropic / qwen / gemini / deepseek / ollama / vllm / langchain / azure / siliconflow / zai / qwen_asr
Model · Embedder	15 provider + sparse: openai / qwen / gemini / vertexai / huggingface / ollama / azure_openai / aws_bedrock / lmstudio / langchain / together / siliconflow / zai / mock + sparse(qwen_sparse)
Model · Rerank	4 provider: qwen / jina / zai / generic
Storage · 适配层	VectorStoreBase 抽象; StorageAdapter / SubStorageAdapter (sub-store 路由); Factory 创建后端
Storage · oceanbase	5 种向量索引 (HNSW/HNSW_SQ/IVFFLAT/IVFSQ/IVFPQ) ; 5 种 FTS parser; hybrid-search-rrf; Snowflake ID; REPLACE INTO upsert; 3-hop graph
	PostgreSQL 中型部署 / 单机文件部署 (开发测试默认)

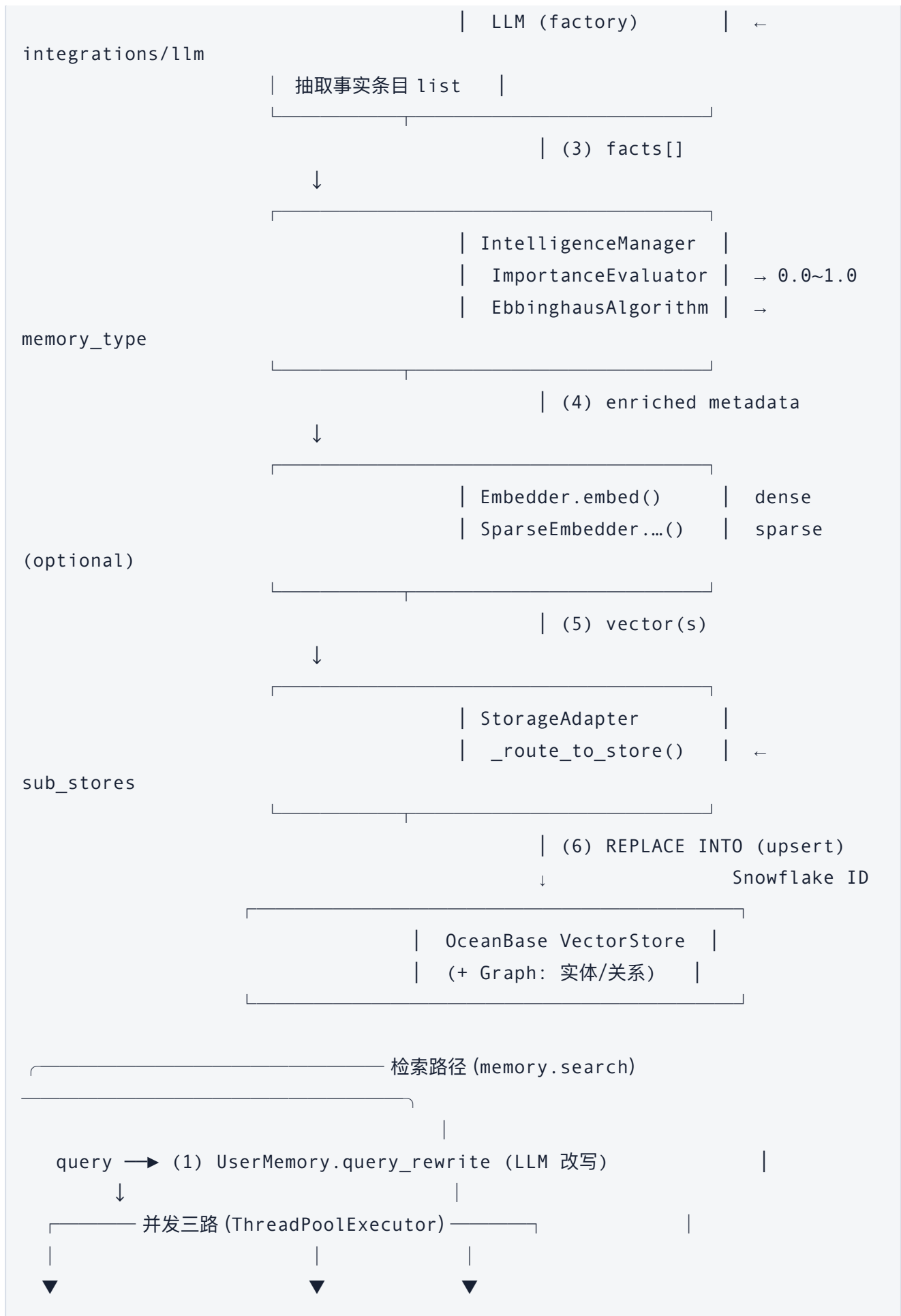
层 / 模块	职责
Storage · pgvector / SQLite	
Apps · claude-code-plugin	Go 二进制 powermem-hook + bash/PowerShell wrapper + hooks.json; HTTP 模式默认; UserPromptSubmit hook 自动检索注入
Apps · VS Code 扩展	"Link to AI tools" 把 PowerMem 接到 Cursor / Claude / Codex / Copilot / Windsurf
Dashboard	React 19 + TanStack Router + Tailwind 4 + Radix UI; 构建产物嵌入 server wheel, 挂载到 /dashboard/
Benchmark	LOCOMO 长对话评测; benchmark/server 评测服务
Examples	go / langchain / langgraph / moonbit 跨语言跨框架接入示例

分层关键约束:

- API 层不持状态: 所有 service 在 lifespan 起点初始化为 app.state.*_service 单例; HTTP / CLI / SDK 都最终经过同一个 Memory 实例。
- Core 与 Storage 通过 Adapter 隔离: StorageAdapter 持有 vector_store + embedding_service + sparse_embedder_service; Core 不直接调用 OceanBase SDK。新增后端只需实现 VectorStoreBase。
- Intelligence 不直接改库: ebbinghaus-forgetting-curve 把 importance_score / retention_strength / next_review 写进 metadata.JSON 列; MemoryOptimizer 离线扫描决定晋升/清理。
- Prompts 集中托管: 运行时可通过 custom_fact_extraction_prompt / custom_update_memory_prompt / custom_importance_evaluation_prompt 覆盖。

关键数据流







回退路径: embedder 失败 → mock [0.1] * 1536 向量; LLM 抽取失败 → 容错跳过单条 fact; reranker 配置失败 → 退化为纯 RRF; storage service 启动失败 → API route 抛 503。

设计决策与哲学

- 持久记忆做成跨形态统一中间件: 一个 .env 同时供 SDK / CLI / FastAPI / mcp / Dashboard / claude-code-plugin / VS Code 扩展使用 —— 不复制状态给每个客户端, 这是它能同时铺到 SDK / IDE / Agent 框架的关键。
- 认知科学抽象 + 数学公式解耦存储: working / short_term / long_term 三层 + ebbinghaus-forgetting-curve $R = e^{(-t/S)}$ 。Intelligence 不直接改库, 只在 metadata 写分数, MemoryOptimizer 离线扫描 —— 避免"每次写入都触发昂贵 LLM 调用"。
- oceanbase 优先但绝不绑死: VectorStoreFactory + StorageAdapter 是关键解耦点; v1.1.0 引入嵌入式 SeekDB (pyseekdb) 让用户用 OceanBase 语义但零部署。
- hybrid-search-rrf + 自适应权重是检索灵魂: 3 路 (向量/全文/稀疏) 并发, $score = \sum w_i \times 1/(60 + rank_i)$, _normalize_weights_adaptively 按实际命中路径重新归一权重, 避免"全命中 doc 因权重总和大而占优"的不公平 —— 这是 LOCOMO 78.7% 准确率的算法贡献。
- 后台线程池是 perf 关键: 模块级 _BACKGROUND_EXECUTOR = ThreadPoolExecutor(max_workers=3); LLM 抽取前台同步, telemetry/audit/optimization 后台跑 —— 解释了 p95 = 1.44s (vs baseline 17.12s 的 11.8 倍提速)。
- 插件式 Provider 矩阵 + pydantic-settings: 15 embedder + 12 LLM + 4 reranker; 用户切换只改 .env 里的 provider —— 是它能跑通"Qwen + DashScope + OceanBase 全栈国产化"路径的工程基础。

- **Multi-agent** 是组件级别: scope/permission/privacy/collaboration 4 个 Controller; 5 种 scope (private/agent_group/user_group/public/restricted) —— 是 enterprise tenant isolation 的基建。
- **Dashboard** 打包进 Python wheel: `pip install powermem + powermem-server` 一条命令就有完整 Web UI, 无需单独部署前端。
- **claude-code-plugin** 用 Go 二进制做 hook: 跨平台原生编译, 无 Python 依赖; `UserPromptSubmit` hook 自动检索注入 `additionalContext` —— 让 Claude Code 用户"不需要写 prompt 也能用上长期记忆"。

关键组件深入解读

storage/oceanbase/oceanbase.py 的 hybrid-search-rrf 融合

`_reciprocal_rank_fusion` (oceanbase.py:1626-1740) 三路 RRF 累加 (k=60) :

- 命中已有 doc → 累加 `rrf_score`; 只命中单路 → 单路得分新增 doc;
- `_normalize_weights_adaptively` 按实际参与路径重新归一化权重;
- 用 heap 维护 Top-K (避免全排序);
- `_calculate_quality_score` 用原始 similarity 做阈值过滤, 避免 RRF 排名高但绝对相似度低的长尾噪声。

相关页面

- [powermem](#)
- oceanbase
- ebbinghaus-forgetting-curve
- hybrid-search-rrf
- claude-code-plugin
- mcp

RBG (RoleBasedGroup)

RBG 是 sgl-project 的 Kubernetes workload API，用于编排多角色、有状态、分布式 AI inference workload。它把 gateway → router → prefill → decode 这类服务表达成 RoleBasedGroup，而不是把多个 Deployment/StatefulSet 当成互不相关的对象。

核心抽象

- **Role**: 每个角色拥有自己的 spec、生命周期和策略，例如 prefill/decode。
- **RoleBasedGroup**: 组成一个逻辑推理服务的多角色整体。
- **RoleInstance**: 与一组 Pod 绑定的状态与升级单元。
- **CoordinatedPolicy**: 协调跨角色升级、扩缩、maxSkew 和 progression。

架构边界

RBG 解决的是 workload topology、状态、原子操作、gang scheduling、硬件/网络亲和和跨角色协调；它不是完整 API gateway，也不是推理 engine。它适合作为 [dynamo](#)、[llm-d](#) 或自研 router/engine 上方的 Kubernetes workload substrate。

适用与限制

适合 P/D、multi-role、需要成组部署/升级/扩缩和稳定实例语义的场景。若只需要一个简单模型 Deployment + Service，RBG 的协调模型可能过重；若需要完整模型路由、限流、canary 和 API 入口，还要组合专用 serving/router 组件。

SGLang

2026-09-14 本地代码分析快照: [src-sglang-architecture](#), HEAD 2fd835b9c1。当前 upstream 证据维护在 [llm-inference-serving-project-map](#) 的“当前上游核验 (2026-09-22)”小节。

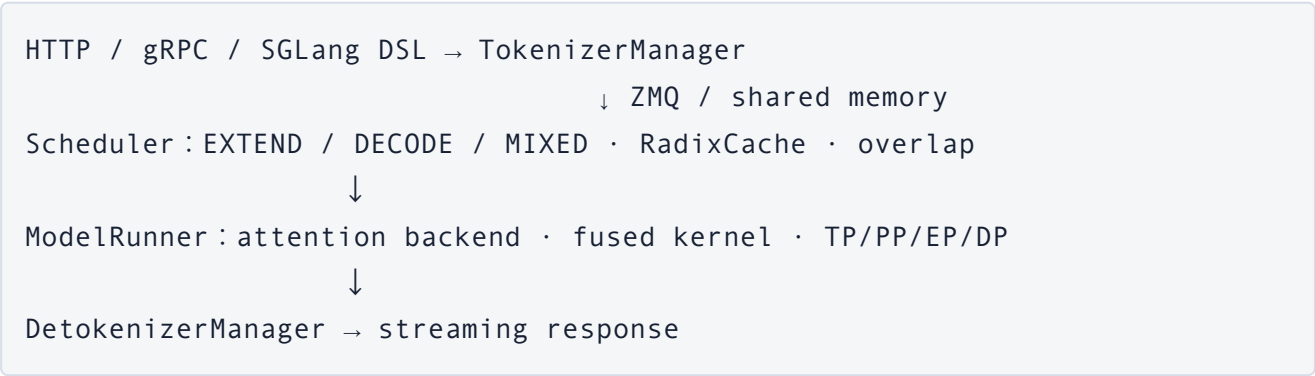
LMSYS / sglang-project 开源的高性能 LLM 推理与 serving 引擎。Apache 2.0, Python 3.10+, 主仓库 [github.com/sgl-project/sglang](#)。

下述能力数量、默认值与性能数字反映上述 2026-09-14 本地快照; 当前 release/backend 需重新核验。

一句话定位

把 LLM 推理引擎的所有"差异化轴"做到接近开源极致: [radix-attention](#) 取代 [paged-attention](#) 把 KV 复用做到 token 级; 4 进程异步流水线 + Scheduler 内 overlap 把 GPU 利用率拉到 95%+; 7 套投机解码 + 5 KV transfer backend + 10+ attention backend + 4 grammar backend 全部可插拔; 统一 OpenAI / Anthropic / Ollama 协议入口。论文里还提出"SGLang DSL" (fork / gen / select) 做结构化生成的前端编译。

最小架构图



关键能力

维度	能力
KV 缓存	radix-attention —— token 级 radix 树; 5 RadixCache 变体 (vanilla / hi / mamba / swa / cpp)
批量化	连续批 + chunked prefill + EXTEND/DECODE/MIXED 三态调度 + CUDA Graph 替换 decode 路径

维度	能力
多进程流水线	HTTP / TokenizerManager (主) / Scheduler (GPU subprocess) / DetokenizerManager (subprocess) , ZMQ pyobj 三段管道
投机解码	7 算法: EAGLE-2 / EAGLE-v2 / 多层 EAGLE / FrozenKV-MTP / NGRAM / DFLASH / Standalone, 走 BaseSpecWorker + spec_registry
prefill-decode-disaggregation	5 transfer backend: mooncake / NIXL / Mori / Ascend / Fake; prefill 与 decode 节点独立扩容
Attention 后端	10+: FlashInfer (默认) / FA3-4 / Triton / FlashMLA / NSA / DSV4 / FlexAttention / TorchNative / Wave / AITER / Intel-AMX
结构化输出	xgrammar / outlines / llguidance / reasoner, sampling 前 apply vocab mask
协议入口	OpenAI / Anthropic / Ollama / gRPC / 原生 Engine SDK
分布式	TP / PP / DP / EP (专家并行) + Elastic-EP + 专家分布记录器
国产硬件	Ascend NPU / Wave / AITER 一等公民 (不是实验路径)
模型库	100+ 模型: LLaMA / Qwen / DeepSeek / Mixtral / Gemma / GPT-OSS / 多模态
多模态	image / audio (Whisper / Qwen-ASR) / video 预处理 + KV 缓存
前端 DSL	SGLang DSL (fork / gen / select) —— 编译到 RadixCache 友好的执行计划

接入形态

- HTTP server: `python -m sglang.launch_server --model-path ...` → FastAPI/uvicorn 默认 8000 端口; 推荐用 `sglang serve` CLI
- gRPC server: `--grpc-mode`, 走 `entrypoints/grpc_server.py` 适合内部高吞吐 RPC
- 离线 Engine API: `from sglang import Engine; engine.generate(...)`, 编程式调用不起 HTTP
- encoder-only 模式: `--encoder-only` 启动专用 encoder 实例 (给 P/D disagg 用)
- Ray 模式: `--use-ray` 走 Ray cluster scheduler
- SGLang DSL: 在 Python 程序里 `@sgl.function def chain_of_thought(s, q): s += sgl.user(q); s += sgl.gen("answer")` —— 编译成多步骤请求, RadixCache 共享 system prompt

设计哲学 (与 [vllm](#) 等同类对照)

- Token 级 KV 复用 vs [vllm](#) block 级: vLLM 使用固定大小 block, 具体大小取决于配置、backend 和版本, 前缀复用按 block 边界对齐; SGLang 用 token-level radix 树 + flat KV pool 支持任意 token 边界

split & share。历史 RadixAttention 论文仅在其 LLaMA-7B tree-of-thought / few-shot 工作负载上报告 $1.6 - 6.4\times$ over vLLM，不代表当前通用吞吐结论

- 4 进程异步流水线：Tokenize / Forward / Detokenize 拆到不同 OS 进程，ZMQ 串联；任何一环堵塞都不卡其他环，GPU 维持高占用
- Mixin 拼装 Scheduler：4000+ 行的 scheduler.py 通过 10+ Mixin 把 disagg/PP/DPAtn/Dllm/Profiler/UpdateWeights 等横切关注点解耦，open-closed 友好；新加 disagg 后端不改主类
- 可插拔哲学贯穿整栈：attention backend / spec algorithm / KV transfer / grammar / quantization / model 全可注册可替换；服务启动时按 server_args 选择
- 国产硬件一等公民：Ascend NPU 不是 fallback 路径，专门有 disagg/ascend、attention/ascend、platforms/ascend，跟 NVIDIA 路径并行

工程数据

指标	实际表现
prefix 缓存收益	历史 RadixAttention 论文在 LLaMA-7B tree-of-thought / few-shot 特定工作负载上报告 throughput $1.6 - 6.4\times$ over vLLM，非当前通用结论
decode latency	CUDA graph replay $\rightarrow$ 单步 $\approx$ kernel-only
投机解码加速	EAGLE-2 默认 topk=5 step=5：典型 $1.5-2.5\times$ decode 加速
P/D 分离收益	长 prefill / 长 decode 场景吞吐 $1.3-2\times$ over collocated
流水线 overlap	4 进程异步 + Scheduler overlap $\rightarrow$ 单 GPU 占用 95%+

学术与起源

- 来源论文：Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs" (NeurIPS 2024) —— 论文里 RadixAttention + SGLang DSL 是核心贡献
- 组织起源：LMSYS / UC Berkeley Sky Computing Lab 团队（FastChat / Chatbot Arena / vLLM 都来自相近社区）
- 生态：在 DeepSeek 官方推荐推理引擎之一；DeepSeek-V3 的 MTP / MLA 实现是 SGLang 主导贡献

在 M4 模块地图中的位置

SGLang 位于 engine/runtime 层，负责 Scheduler、RadixCache 和模型执行，通过 P/D 与分布式集成接口连接外围 serving 层。职责边界见 [llm-inference-serving-project-map](#)，组合选择见 [llm-serving-engine-selection-map](#)。

与 KVCacheD 的集成关系

[kvcached](#) 保留 SGLang Scheduler、ScheduleBatch、[radix-attention](#) 和 attention backend，只替换底层 token/page allocator 与 MHA/MLA/Mamba/hybrid KV pool 的 buffer allocation。RadixCache node 仍持有 token-slot indices；只有 cache evict 或请求释放使 page 内全部slot空闲后，KVCacheD 才能归还physical VRAM。

当前集成让每个 SGLang TP worker 本地拥有自己的 KVCacheD pool，避免把一个rank的pool operation重复广播给peer；这一点与vLLM的EngineCore→worker fan-out模型不同。完整对照见 [kvcached-sglang-vllm-knowledge-system](#)，源码证据见 [src-kvcached-architecture](#)。

相关页面

- 架构详解: [src-sglang-architecture](#)
- 核心算法: [radix-attention](#)、speculative-decoding、prefill-decode-disaggregation
- 同类系统: [vllm](#) (最直接对标)
- 依赖: flash-attention (FlashInfer / FA3 / FlashMLA)、mooncake (KV transfer)
- 相关概念: [paged-attention](#) (vLLM 的对照系统)
- 弹性 KV: [kvcached](#)、[elastic-kv-cache](#)

SGLang 架构与推理优化

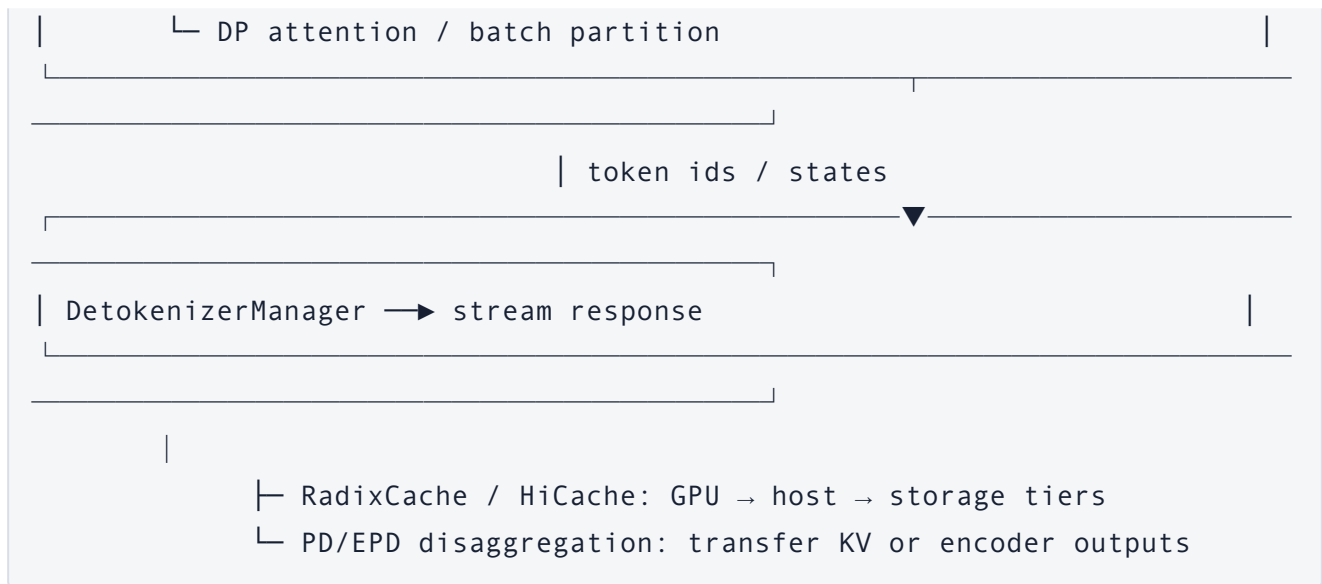
原文: raw/sglang-architecture-analysis.md · 仓库: /Users/zhenyu.jiang/
sglang · 版本: local HEAD 2fd835b9c1

一句话定位

[sglang](#) 是高性能 LLM serving runtime，核心差异是 [radix-attention](#)：用 radix tree 做 token-level KV prefix sharing；再以连续批、chunked prefill、融合 kernel、投机解码和 TP/PP/EP/DP 构成执行管线。

核心架构图





请求与缓存数据流



推理优化要点

- KV: RadixCache 做 token-level 前缀共享; HiCache 提供 GPU/host/storage 分层; 量化 KV 降低容量; P/D 分离搬运 KV。
- 连续批: EXTEND/DECODE/MIXED 与 chunked prefill 在 TTFT、ITL、吞吐间调节。
- 融合: attention backend、sgl-kernel、JIT/torch.compile、CUDA Graph、fused MoE/量化 kernel 减少 launch 和中间读写。
- 并行: TP 切权重, PP 切层, EP 分布 MoE 专家, DP 分区请求/批/KV; DP attention 可复制 attention、保留 FFN/MoE 的 TP/EP。

关键设计

- Radix tree 换取更细粒度 prefix reuse, 同时增加树维护、eviction 与跨 rank locality 复杂度。
- Scheduler 只生成紧凑 batch/attention metadata, ModelRunner 专注 GPU 执行, backend 可替换。
- 结构化程序、会话缓存、投机解码和 P/D/EPD 复用同一套 runtime 扩展点。

代码级关键路径

```
run_event_loop()
  → get_next_batch_to_run()
  → ScheduleBatch(EXTEND / DECODE / MIXED)
  → match_prefix() + alloc_for_extend/decode()
  → run_batch() / ModelRunner
  → Attention Backend (按 ForwardMode dispatch)
  → process_batch_result()
  → cache_finished_req() / next batch
```

```
run_event_loop()
├─ 接收 request / abort / control message
├─ get_next_batch_to_run()
│   └─ 选择 running batch 继续 decode
│   └─ 从 waiting queue 取新请求
│       └─ match prefix / check memory
│           └─ 形成 EXTEND / DECODE / MIXED ForwardMode
├─ run_batch(ScheduleBatch)
│   └─ TP worker / ModelRunner forward
├─ process_batch_result()
│   └─ 更新 Req output / seq_lens
│   └─ 处理 stop / abort / grammar
│       └─ verify speculative result
│           └─ cache_finished_req()
└─ 继续下一轮 event loop
```

SGLang 当前以 ScheduleBatch、UnifiedRadixCache、ForwardMode 和 BaseSpecWorker 连接调度、KV、Attention 与投机解码。P/D 侧通过 DecodeRequest、transfer queue 和 connection contract 等状态对象等待/接收 KV；HiCache 可继续从 host/storage tier 恢复 prefix。

与 vLLM 的代码实现对比

维度	SGLang	vLLM
主循环	run_event_loop → get_next_batch_to_run → run_batch → process_batch_result	EngineCore.step → schedule → execute_model → update_from_output
	ScheduleBatch + mixins	SchedulerOutput + Request state

维度	SGLang	vLLM
Batch		
KV	UnifiedRadixCache + token/page pools	KVCacheManager + block coordinator/table
Spec	BaseSpecWorker + SpecInput + target verify	proposer + lookahead slots + scheduled spec tokens
Attention	registry 创建, ForwardMode 二次选择	selector 选择 backend class/enum

TP / PP / EP / DP 与 Spec KV 修正

```

bootstrap rank geometry
  ↓
ParallelState(TP / PP / DP-attn / EP / MoE-DP)
  ↓
ForwardBatch / ScheduleBatch metadata
  ├── TP Linear / Attention head shard → all-reduce/gather
  ├── PP stage → activation send/recv
  ├── EP MoE gate → token all-to-all → local experts → reduce
  └── DP attention → request/batch partition + rank-consistent padding
    ↓
tp_worker.forward_batch_generation()
  ↓
merge output / next Scheduler step

```

```

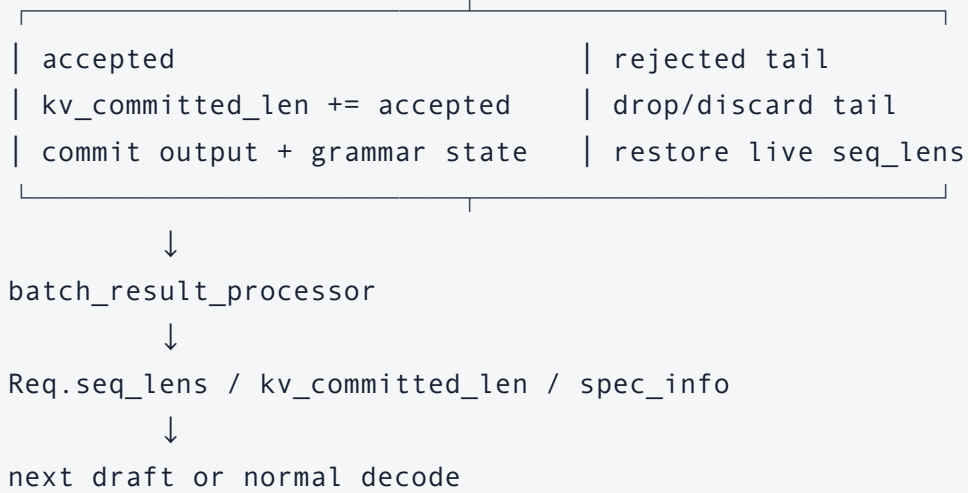
draft worker propose → optimistic draft KV / spec_info
  ↓ target verify (TARGET_VERIFY)
accepted draft count + bonus token
  ├── commit kv_committed_len / output / grammar
  └── discard rejected tail / restore live seq_lens
    ↓
batch_result_processor → next draft or normal decode

```

```

draft worker propose
  ↓ optimistic draft KV / spec_info
target verify (TARGET_VERIFY)
  ↓ accepted draft count + bonus token

```



代码入口: distributed/bootstrap.py:271-290、distributed/
parallel_state.py:649/1155/1280、scheduler_components/
batch_result_processor.py:776-804、speculative/dflash_info_v2.py:160-218。
| P/D | DecodeRequest + transfer queue/receiver | KVConnector metadata + worker hooks |

相关页面

- [sglang](#)
- [vllm](#)
- [radix-attention](#)
- [paged-attention](#)
- [llm-inference](#)

SkyPilot

SkyPilot 是面向 AI/ML 工作负载的多云算力控制平面。用户通过 YAML / Python API 声明 Task、Dag、Resources，SkyPilot 负责选择可用且便宜的 GPU/CPU/TPU 资源，完成 provisioning、文件同步、setup、run、日志流和自动回收。详见 [src-skypilot-architecture](#)。

架构边界

SkyPilot 不优化 LLM 推理内核，也不直接管理 KV cache。它位于 [llm-inference](#) 更上层：决定 workload 应该跑在哪个云、哪个区域、哪种实例、哪个集群，以及失败时如何 failover。

关键设计

- Client/SDK 把 YAML、CLI override、env/secrets、workdir 转成 Task/Dag。
- API server 负责鉴权、RBAC、request queue、blob storage 和日志流。
- Optimizer 按资源、价格、容量和失败 blocklist 选择 cloud/region/instance。
- CloudVmRayBackend 把资源选择落到 Ray cluster、Kubernetes、Slurm 或云 VM。
- Managed jobs / SkyServe / pools 在 launch/run 之上提供作业恢复和服务控制器。

选型判断

需要跨云 GPU 资源经济、failover 和 AI job/serve 控制时看 SkyPilot。需要单机推理引擎看 [vllm](#) / [sglang](#)；需要多节点 LLM serving 编排看 [dynamo](#) / [llm-d](#)；需要 Kubernetes GPU 设备层看 [k8s-gpu-device-stack](#)。

SkyPilot 架构与设计思路分析

原文: raw/skypilot-architecture-analysis.md · 仓库: [googs1025/skypilot](#) · 分析版
本 master HEAD 55b9185 (2026-06-01)

一句话定位

SkyPilot 是一个面向 AI/ML 工作负载的多云算力控制平面：用户用 YAML / Python API 声明 Task、Dag、Resources，SkyPilot 负责选择可用且便宜的 GPU/CPU/TPU 基础设施，完成 provisioning、文件同步、setup、run、日志流和自动回收。它的关键手段是把“用户声明”和“云厂商细节”之间拆成 API server request 队列、optimizer、CloudVmRayBackend、cloud/provision provider 接口、controller (managed jobs / serve / pools) 几层。

核心架构图




```
confirmation |
| └─ upload workdir/file_mounts
blob |
| └─ POST /launch, /jobs/*, /serve/* + stream request
logs |
┌───────────────────────────────────────────────────────────────────────────────────┐
│                                     | authenticated REST request + request_id
▼
┌───────────────────────────────────────────────────────────────────────────────────┐
| API
server
| FastAPI
server.py |
| └─ auth: loopback / oauth2-proxy / service-account JWT / basic
auth |
| └─ RBAC + workspace
permission |
| └─ request DB + blob storage + log
streaming |
| └─ executor: LONG queue vs SHORT
queue |
┌───────────────────────────────────────────────────────────────────────────────────┐
│                                     | scheduled long-running request
▼
┌───────────────────────────────────────────────────────────────────────────────────┐
| Execution
orchestration |
| sky/
execution.py |
| └─ validate + admin policy + secrets/volumes
resolve |
| └─ OPTIMIZE: Optimizer.optimize /
optimize_job_group |
| └─ PROVISION:
CloudVmRayBackend.provision |
| └─ SYNC_WORKDIR / SYNC_FILE_MOUNTS /
SETUP |
```



模块分层

层 / 模块	职责
用户入口	Click CLI、Python SDK、异步 SDK、公开 API facade。把 shell/YAML/Python 调用变成 Task/Dag 和 API server request。
声明模型	用户声明的中间表示：任务、DAG、资源、volume、storage、service/pool。负责 YAML schema、env/secrets、workdir、file mounts、autostop、hooks。
API server / 控制面	FastAPI API server、request 排队/执行/日志、blob 上传、鉴权、RBAC、workspace 权限。
执行编排	统一执行 stage：validate、optimize、provision、sync、setup、pre_exec、exec、down；维护 cluster 状态、事件和 handle。
优化器	把抽象资源要求映射成具体 cloud/region/zone/instance_type，按 cost/time 选择，支持 job group 和 failover blocklist。
后端	以 Ray + skylet 作为远端运行时，创建/复用集群、同步文件、运行 setup/run、提交 job、设置 autostop/hook。
provider 实现	每家云/集群的能力声明、区域/价格/规格查询、实例生命周期 API、kubernetes/Slurm/SSH 特化实现。
高层服务	在基础 launch/exec/down 之上实现 managed jobs 自动恢复、SkyServe replica/autoscaler、batch/pools、SSH node pools、volume。
插件/策略/观测	admin policy、server plugins、RBAC 扩展、Prometheus metrics、usage telemetry、外部故障源。

关键约束是：Task/Resources 只表达用户需求，不做云 API 调用；Cloud 类描述能力和候选资源，不直接成为长期状态对象；provision/* 做具体生命周期；CloudVmRayBackend 负责把 optimizer 选出的 launchable resources 落到远端 runtime；API server 负责请求排队和鉴权，不把所有重任务塞在 FastAPI event loop 上。

关键数据流

sky launch 主链路



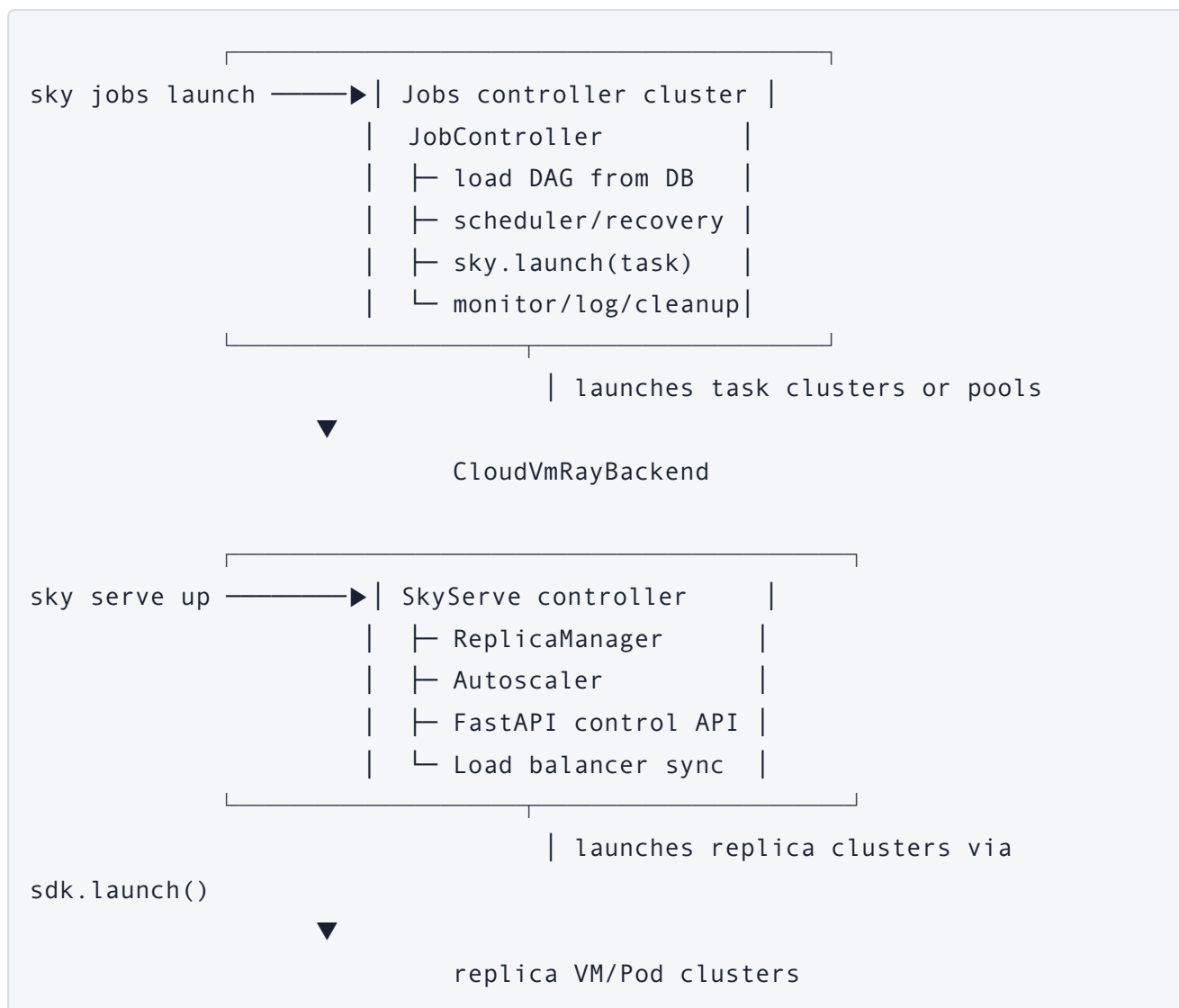
```

|   |─ load_chain_dag_from_yaml / load_job_group_from_yaml
|   └─ apply CLI resource/env/secret/workdir overrides
▼
command.py:launch()
|   └─ sdk.launch(task, cluster_name, backend=CloudVmRayBackend, ...)
▼
sdk.py:launch()
|   |─ convert_entrypoint_to_dag()
|   |─ apply client-side admin policy
|   |─ upload_mounts_to_api_server()
|   └─ POST /launch -> request_id
▼
server.py:/launch
|   └─ executor.schedule_request_async(..., func=execution.launch, LONG)
▼
execution.py:launch() / _execute_dag()
|   |─ validate, resolve volumes/secrets
|   |─ Optimizer.optimize()
|   |─ backend.register_info(dag, requested_features, planner)
|   |─ backend.provision()
|   |─ backend.sync_workdir() / sync_file_mounts()
|   |─ backend.setup()
|   |─ backend.set_autostop()
|   └─ backend.execute()
▼
CloudVmRayBackend
|   |─ distributed cluster lock
|   |─ RetryingVmProvisioner.provision_with_retries()
|   |─ cloud/provision provider APIs
|   └─ Ray job queue + skylet + logs
▼
Remote VM / Pod / Slurm allocation

```

错误和回退路径很关键：如果第一次 optimizer 无可行资源，`ResourcesUnavailableError.failover_history` 为空；如果 cloud feature check、region/zone provisioning 或 capacity 失败，`RetryingVmProvisioner` 会把失败资源加入 blocklist，调用 `Optimizer.optimize(..., blocked_resources=...)` 重新选资源，再继续尝试。

Managed Jobs / Serve 控制器路径



Managed jobs 与 serve 都不是直接改底层后端，而是复用同一套 `sdk.launch()` /

`CloudVmRayBackend`。区别是它们把自己变成“控制器集群”上的长期进程，维护 job/service 状态，并用恢复/扩缩容策略反复发起普通 launch/down。

设计决策与哲学

- API server 是所有重操作的边界：用户侧 `sky.client.sdk.launch()` 上传 mounts 后 POST / launch, server 只调度 request, 真正执行在 request worker 中。这个边界让 CLI、dashboard、远端 API server、async SDK 都围绕 request_id 和日志流工作。
- 长短请求分队列：API server 明确区分 cluster launch / job submission 这类 LONG request 和 status/logs 这类 SHORT request, 避免长 provisioning 阻塞控制面读请求。
- Task/Dag/Resources 是稳定 IR：CLI 会先把 YAML、shell command、JobGroup、CLI overrides 规整成统一声明模型；kubernetes、Slurm、AWS/GCP/Azure 等 provider 不直接影响用户接口。

- **optimizer** 在上层，**provider** 只回答能力和可行性：执行层先调用 **optimizer**，再把可重新规划的 **planner** 注入 **backend**；**provider** 层通过 **feature gates**、**catalog** 和 **provision hooks** 暴露能力。
- **跨云 failover** 是失败反馈后的重新优化：**capacity/feature/identity** 失败后，**backend** 把失败 **resources** 加入 **blocklist**，再重新调用 **optimizer**，而不是固定候选列表盲试。
- **managed jobs / serve** 是控制器模式：**jobs controller** 和 **SkyServe controller** 都通过 **sdk.launch()** / **sdk.down()** 管理实际集群，自己只维护恢复、扩缩容、状态和日志策略。
- **安全模型** 偏控制面托管 + **RBAC**：状态库记录 **clusters/users/volumes/events**；**API server** 支持 **OAuth2 proxy**、**basic auth**、**service-account JWT**、**Casbin RBAC** 和 **workspace permission**。**viewer** 是 **allowlist**，新 **endpoint** 默认不可见。

关键组件深入解读

sky launch 执行编排

execution._execute() 是 **SkyPilot** 的 **stage runner**：**validate**、**admin policy**、**volume/secrets resolve**、**storage construct** 后，进入 **_execute_dag()**。普通 **launch** 目前只支持单 **task**；**stage** 默认包含 **OPTIMIZE**、**PROVISION**、**SYNC_WORKDIR**、**SYNC_FILE_MOUNTS**、**SETUP**、**PRE_EXEC**、**EXEC**、**DOWN**。它先在锁外运行 **Optimizer.optimize()**，同时把一个 **planner** 回调注入 **backend**，供 **backend** 在拿到同名 **cluster lock** 后按最新状态重新规划，降低并发状态变化带来的竞态。

CloudVmRayBackend 与 failover

CloudVmRayBackend 是核心后端：远端执行语义来自 **Ray cluster + skylet + job queue**。**provision** 前检查 **rsync** 和 **owner identity**，再拿同名 **cluster lock**；锁内决定复用/重建/新建，随后 **build SkyPilot wheel** 并交给 **RetryingVmProvisioner**。如果 **provisioning** 失败且允许 **failover**，它会把当前资源加入 **blocklist**，再调用 **optimizer** 重新选择 **best_resources**，这就是 **SkyPilot** 能在 **capacity error** 后自动换 **region/cloud** 的关键。

与 [dynamo](#) / [vllm](#) / [sglang](#) 的关系

SkyPilot 不是推理引擎，也不直接实现 [llm-inference](#) 的 **batching**、**KV cache** 或模型执行。它更像 **AI compute control plane**：可以用统一 **YAML** 启动 [vllm](#)、[sglang](#)、训练任务、**RAG app**、**Jupyter**、**batch inference** 或 [disaggregated-serving](#) 相关组件，并在 **kubernetes**、**Slurm**、公有云 **GPU provider** 之间做资源选择、**failover** 和生命周期治理。

相关页面

- [kubernetes](#)
- [vllm](#)
- [sglang](#)
- [dynamo](#)
- [llm-inference](#)

- [disaggregated-serving](#)

substrate

Agent Substrate 是运行在 kubernetes 之上的高密度 agent-like workload substrate。它把 Kubernetes 当作低频容量和模板控制面，把 Actor 的高频 resume/suspend、worker 分配、状态锁和路由唤醒放到 Redis/ValKey、ateapi、atelet、ateom-gVisor 和 atenet router 里完成。详见 [src-substrate-architecture](#)。

架构边界

Substrate 不是普通的 Agent framework，也不是只创建一个 Pod 的 sandbox CRD。它的核心目标是让大量 idle actor multiplex 到较少 warm worker 上，通过 golden snapshot、runtime checkpoint/restore、actor DNS 和 Envoy ext_proc 唤醒降低激活延迟。

它和 [agent-sandbox](#) / agentcube 的区别在层级：[agent-sandbox](#) 提供有状态 Sandbox 原语，agentcube 做会话路由和 warm pool，Substrate 更底层，直接把 worker pool、actor template、snapshot 和路由恢复做成 substrate。

关键抽象

- WorkerPool：声明 worker 容量和运行模板，由 controller 物化为 privileged ateam pods。
- ActorTemplate：生成 golden actor，再 checkpoint 成 golden snapshot，供后续 actor 快速恢复。
- Actor：高频运行状态不进 K8s API，而是存 Redis/ValKey。
- atelet / ateam-gvisor：节点侧 supervisor 和 gVisor worker，负责 runsc create/start/checkpoint/restore。
- atenet：通过 actor hostname 触发 ResumeActor，再把请求转发到已分配 worker。

选型判断

适合研究或构建高密度、低空闲成本、可暂停恢复的 agent runtime。若只是给单个 agent 提供隔离，优先看 [agent-sandbox](#) 或 OpenShell；若需要团队级会话编排和 warm pool，优先看 agentcube；若要把 sandbox 出口流量纳入治理，再叠加 agentgateway / ai-gateway。

vLLM

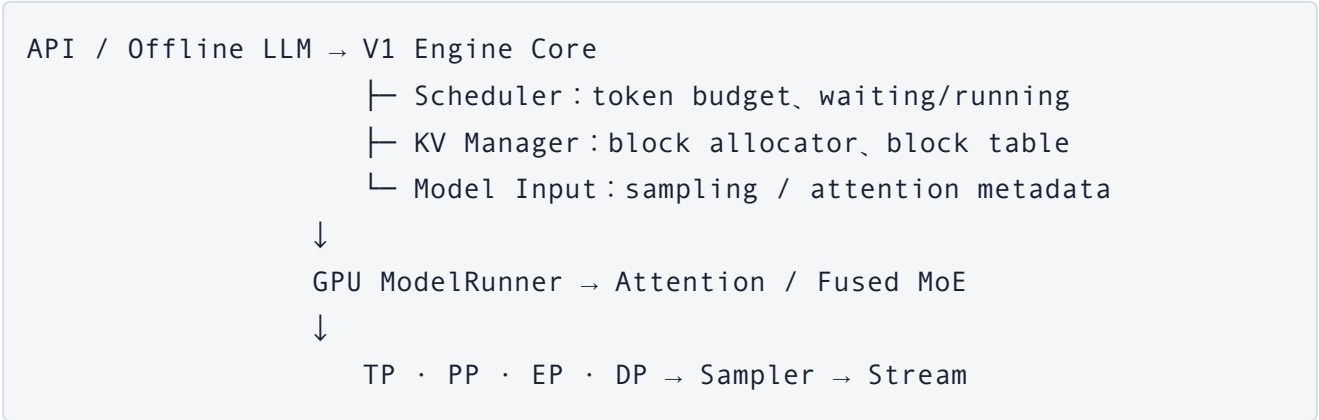
2026-09-14 本地代码分析快照: [src-vllm-architecture](#), HEAD dc36fcce90。当前 upstream 证据维护在 [llm-inference-serving-project-map](#) 的“当前上游核验 (2026-09-22)”小节。

UC Berkeley Sky Computing Lab 开源的 LLM 推理与 serving 引擎。Apache 2.0，最早把 [paged-attention](#) 引入开源界 (SOSP 2023 论文)，是目前最广泛使用的 LLM serving 框架之一。

一句话定位

LLM serving 的"事实标准基线": 用 [paged-attention](#) 将 KV 缓存组织为固定大小的逻辑/物理 block (类比 OS 虚存分页)，并通过 block table 建立映射；具体 block 大小取决于配置、attention backend 和版本。这把 GPU 显存从"按最大 seq_len 预分配"改成"按需 block 分配 + block table 映射"，让吞吐量数倍于 HuggingFace transformers。后来的 [sglang](#) / TensorRT-LLM / TGI 都把 vLLM 当对标。

最小架构图



关键能力（与 [sglang](#) 对照）

维度	vLLM	sglang
KV 缓存粒度	固定大小的逻辑/物理 block + block table (大小依配置/backend/版本)	token 级 (radix-attention)
前缀共享	按 block 边界共享；末尾未填满的 partial block 在完整前可能无法复用	任意分叉点自动 share
投机解码	EAGLE / Medusa (少量)	7 算法 (EAGLE / NGRAM / MTP / DFLASH / Standalone / 多层 EAGLE / v2)

维度	vLLM	sglang
P/D 分离	通过 KV connector 等 backend-specific 集成接入；成熟度/功能覆盖需按所选 release/backend 验证	通过 transfer backend 等 backend-specific 集成接入；成熟度/功能覆盖需按所选 release/backend 验证
Attention 后端	FlashAttn / xFormers / TorchSDPA	10+ 后端
结构化输出	outlines	4 backend
协议入口	OpenAI	OpenAI / Anthropic / Ollama / gRPC / Engine
国产硬件	实验	Ascend NPU 一等公民
生态广度	最大（HF 模型几乎全支持）	追赶中

历史与影响

- 2023-09: vLLM 0.1 发布，论文 "Efficient Memory Management for Large Language Model Serving with PagedAttention" (SOSP 2023)
- 首创性：把虚存分页思想引入 LLM KV cache，是行业转折点
- 采用度：HuggingFace TGI、Ray Serve、Anyscale、Together AI 等都基于 vLLM 或受其启发；几乎所有 "LLM as a Service" 产品的 baseline

与 SGLang 的差异点（基于 sglang 架构分析）

- vLLM PagedAttention 使用 block table：将固定大小的逻辑 KV block 映射到物理 KV block；具体 block 大小和未用容量行为随配置、attention backend 与版本而异
- vLLM PrefixCache 按 block 边界共享：末尾未填满的 partial block 在完整前可能无法复用；SGLang radix 树支持在任意 token 边界 split
- vLLM 单进程主导：scheduler + tokenizer + worker 多线程；SGLang 4 进程异步流水线
- vLLM 投机解码生态较窄：EAGLE + Medusa；SGLang 7 算法

在 M4 模块地图中的位置

vLLM 位于 engine 层，负责请求调度、模型执行、attention backend 和本地 KV 管理；外部流量路由与自动扩缩由外围 serving 层承担。职责边界见 [llm-inference-serving-project-map](#)，组合选择见 [llm-serving-engine-selection-map](#)。

与 KVCacheD 的集成关系

[kvcached](#) 位于 vLLM 的 KV physical backing 下层，不替代 EngineCore、Scheduler、[paged-attention](#) 或 attention backend。它通过版本化 patch：

- 用 ElasticBlockPool 接住 BlockPool 的 block/APC/ref-count 契约；
- 在 GPUModelRunner 中建立 VMM-backed KV tensor 并继续调用原生 `bind_kv_cache`；
- 在 EngineCore/coordinator 与 GPU worker 分进程时，经 TP/PP Unix socket同步map/unmap；
- 把共享物理池的瞬时耗尽转成 `allocate_slots() -> None`，让原生 scheduler preempt/retry；
- queued/async batch 下先做 worker barrier，再释放 physical page。

因此 vLLM 管“request需要哪些逻辑block”，KVCacheD 管“这些block所在VMM page当前是否有真实VRAM backing”。完整对象映射见 [kvcached-sglang-vllm-knowledge-system](#)，源码证据见 [src-kvcached-architecture](#)。

相关页面

- 核心算法：[paged-attention](#)
- 主要对标：[sglang](#)
- 概念对照：[radix-attention](#)
- 依赖：flash-attention
- 弹性 KV：[kvcached](#)、[elastic-kv-cache](#)

vLLM Semantic Router

Semantic Router 是 vLLM 生态的 programmable Mixture-of-Models routing and control layer。应用通过稳定的 OpenAI/Anthropic-compatible endpoint 访问它，Router 根据请求特征选择或组合能力路径。

解决的问题

应用不应把“简单问题走便宜模型、复杂问题升级、敏感内容走安全路径、需要工具/检索/验证时组合能力”硬编码在业务代码中。Semantic Router 将 intent、difficulty、context、modality、identity、risk 和 system state 统一放在请求路径。

核心抽象

Recipe/virtual model 可以指向单模型、specialist、cascade 或 bounded multi-model workflow，并附加 retrieval、memory、tools、caching、guardrails 和 verification。

边界与选型

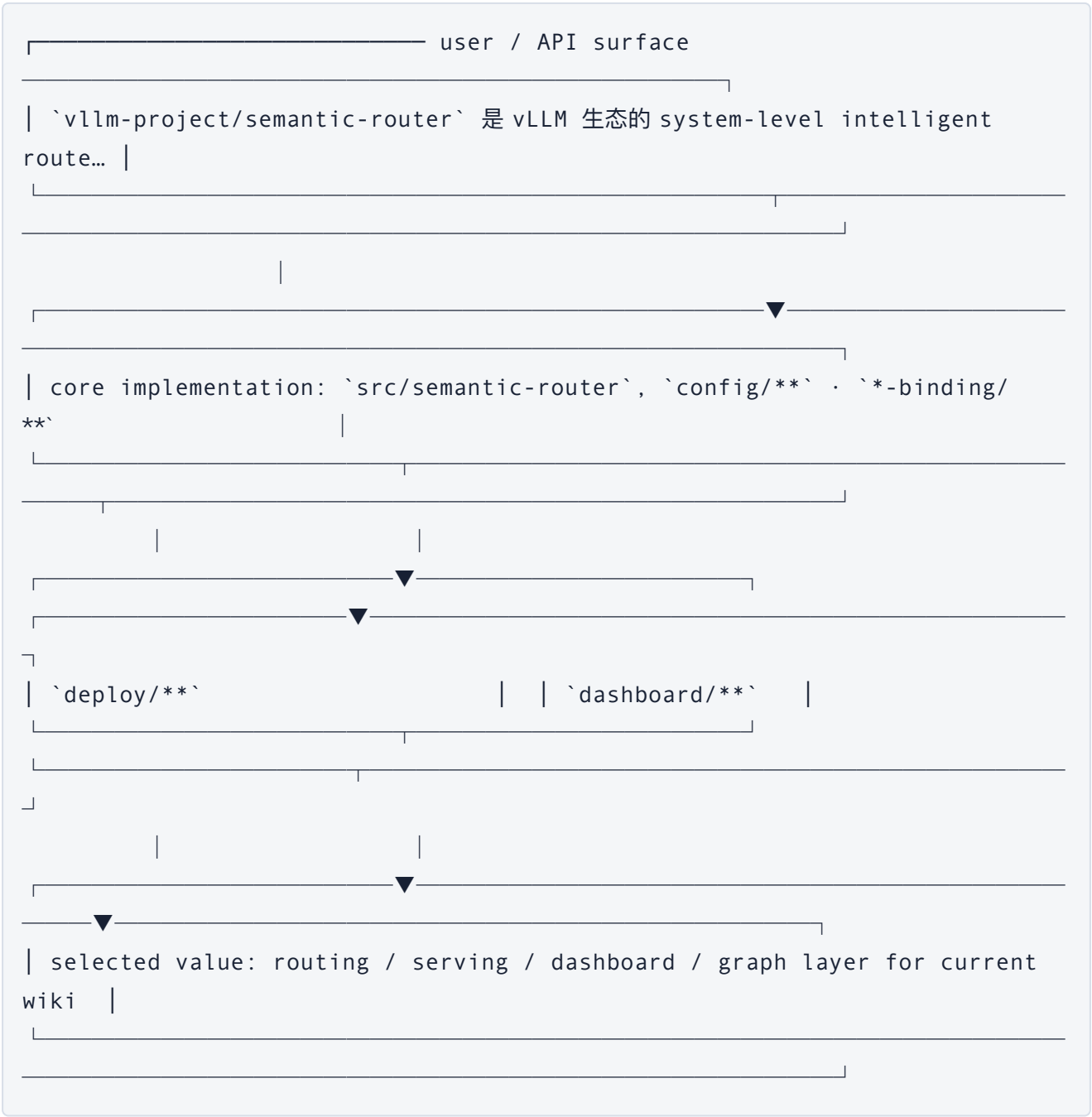
它决定 model/ability/recipe，不替代 Envoy 或模型 server，也不直接负责 K8s pod endpoint picking。需要成本/质量算法基线看 routellm；需要 KV、队列、InferencePool endpoint 看 [llm-d-router](#)。

详见 [src-semantic-router-architecture](#)、[src-k8s-gateway-routing-comparison](#)。

vLLM Semantic Router 架构与设计思路分析

vllm-project/semantic-router 是 vLLM 生态的 system-level intelligent router，目标不是 KV cache locality，而是按请求语义/规则/模型能力做 mixture-of-models 路由。仓库超过 200MB，本次按 ingest-codebase 缩小到 src/semantic-router、Go/Rust bindings、config、deploy/operator、dashboard/README 等核心。

核心架构图



模块分层

层/目录	责任
src/semantic-router, config/**	语义路由主逻辑和决策配置。
*-binding/**	Candle/ML/NLP/ONNX/OpenVINO 等加速/模型绑定。
deploy/**	K8s/operator/helm/kserve/local 部署。
dashboard/**	管理界面，最近 first-admin setup 修复相关。

关键数据流

1. 请求进入 router。
2. router 读取 config 中 algorithm/decision/signal/knowledge base。
3. embedding/classifier/binding 产生语义信号，选择模型/endpoint。

设计决策

- 语义路由和 KV-aware routing 是不同维度，可叠加但不能混淆。
- 多 binding 说明它追求系统级低延迟/可部署性。
- dashboard/operator 表明它正从 library 走向平台组件。

对比定位

和 RouteLLM 相比，semantic-router 更工程化/系统化；和 llm-d-router 相比，它按语义/模型选择，不按 pod load/KV locality。

相关链接

- GitHub 当前状态底稿：[src-github-stars-backlog-current-state](#)
- 选型地图：[github-stars-backlog-implementation-map](#)

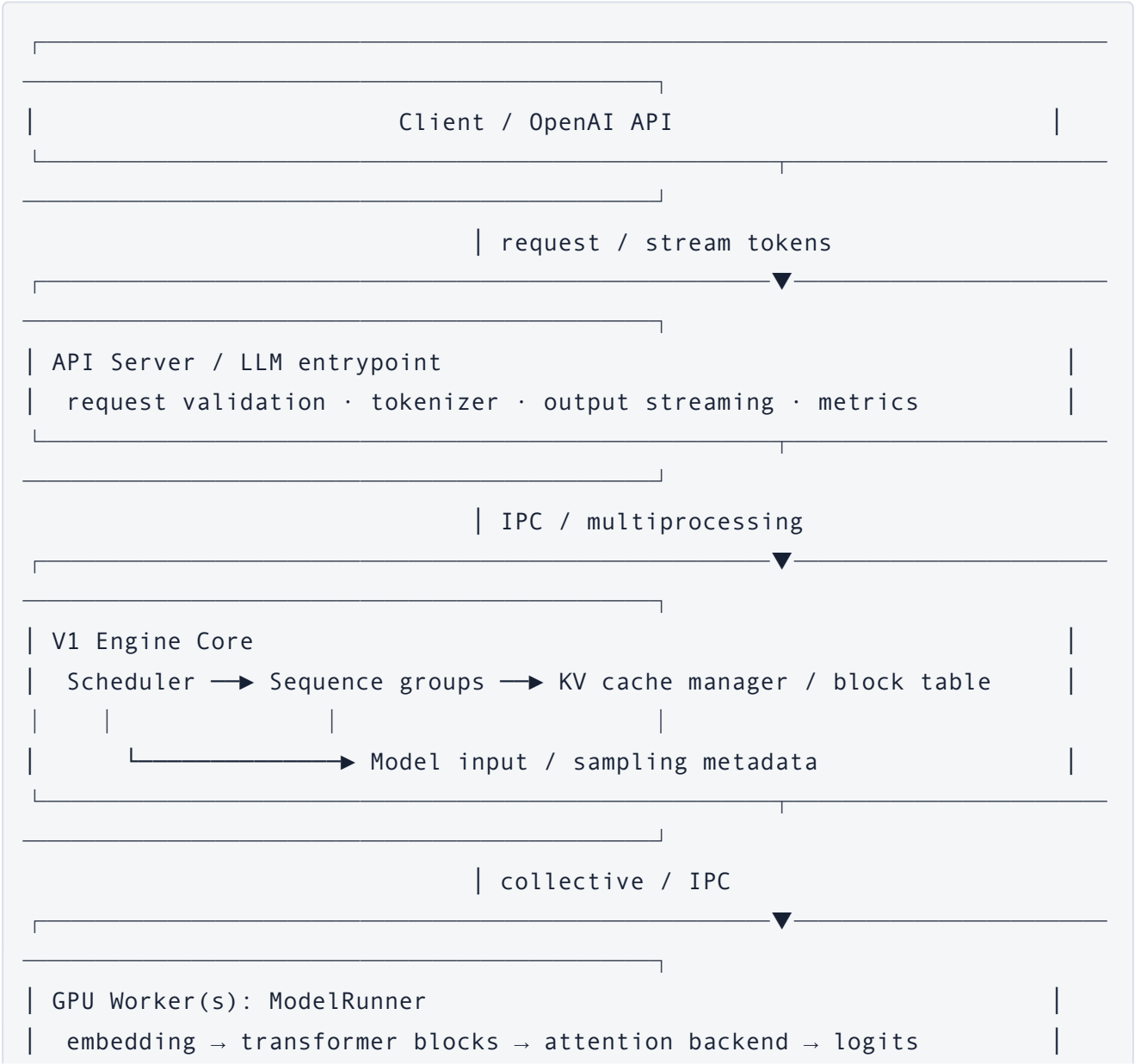
vLLM 架构与推理优化

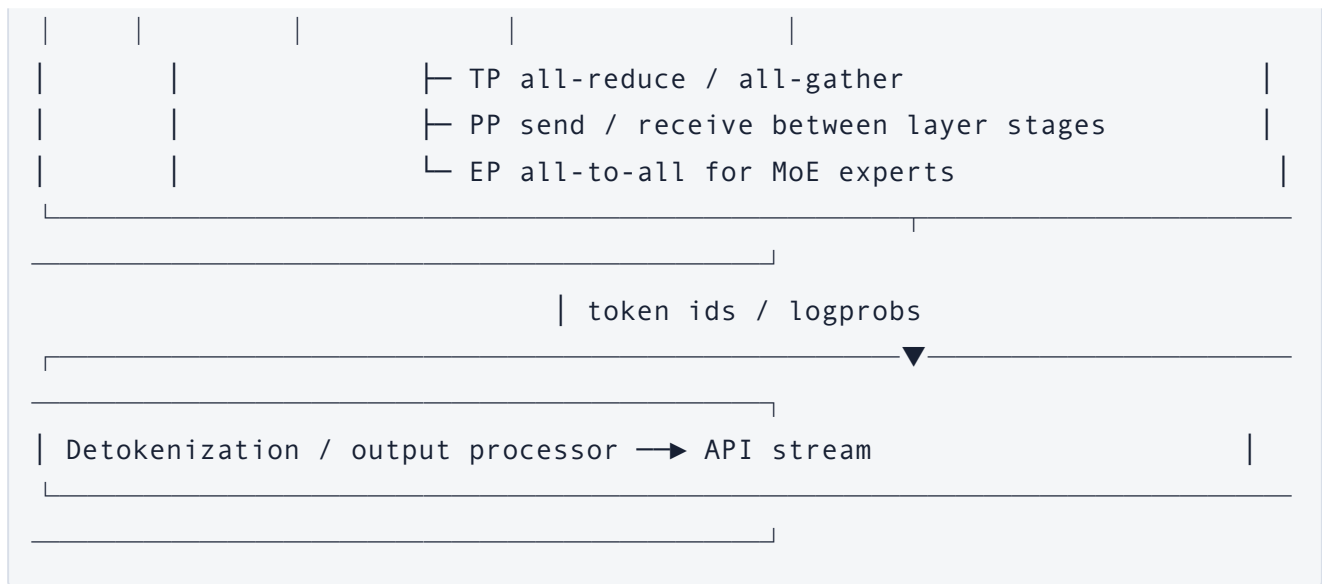
原文: raw/vllm-architecture-analysis.md · 仓库: /Users/zhenyu.jiang/vllm
· 版本: local HEAD dc36fcce90

一句话定位

vllm 是高吞吐 LLM serving 引擎：以 [paged-attention](#) 管理 KV block，以 continuous batching、编译/融合 kernel 和 TP/PP/EP/DP 扩展推理容量与吞吐。

核心架构图





请求与 KV 数据流

新请求

```

|
| └─ tokenize, 计算 prompt token 与可复用 prefix hash
|
| └─ Scheduler 按 token budget 选择 waiting/running 请求
|   └─ 命中 block table → 只 prefill 未命中的 suffix
|     └─ 未命中 → 分配 KV blocks; 空间不足则 eviction / 等待
|
| └─ Prefill: 批量处理 prompt, 写入 paged KV cache
|
| └─ Decode: 每轮为每条活跃序列生成一个 token; 新序列可随时加入, 完成序列释放 blocks
|   └─ continuous batching + chunked prefill 平衡 TTFT 与 ITL
|
| └─ attention backend 通过 block table gather K/V, 执行 fused attention
|
| └─ logits → sampler → detokenize → stream; 停止/异常时回收 sequence 与 KV blocks
  
```

推理优化要点

- **KV**: PagedAttention 解决碎片与动态增长; Prefix Caching 复用完整 KV block; offload/connector 扩展 GPU 之外的容量; FP8 等 KV dtype 以精度换容量。
- **连续批**: 每轮按 token budget 调度; chunked prefill 与 decode 交错, 平衡 TTFT、ITL、吞吐和队列等待。

- 融合：attention backend、CUDA Graph、torch.compile fusion、RMSNorm/ROPE/KV 写入和 Fused MoE 降低 launch 与中间 tensor 成本。
- 并行：TP 切层内权重，PP 切层，DP 复制模型，EP 分布 MoE 专家；通信拓扑和 batch shape 决定真实收益。

关键设计

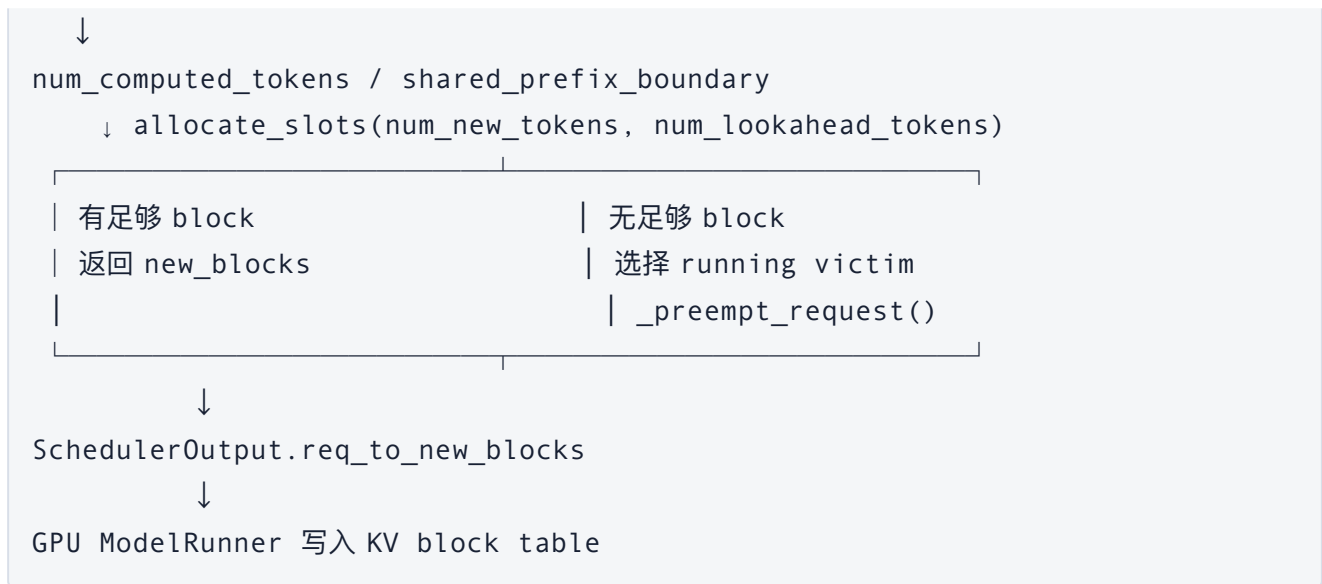
- Engine Core 管理状态、预算和 KV；GPU worker 专注执行，形成清晰的调度/执行边界。
- 固定 block 换取可预测的内存管理；代价是 prefix sharing 受 block 边界影响。
- 后端可替换，允许按硬件、shape 和 prefill/decode 阶段选择不同 kernel。

代码级关键路径

```
EngineCore.step()
  → Scheduler.schedule()
  → KVCacheManager.allocate_slots()
  → SchedulerOutput
  → GPUModelRunner.execute_model()
  → _prepare_inputs() / attention metadata
  → Attention backend
  → sampler
  → Scheduler.update_from_output()
```

```
EngineCore.step()
  └ scheduler.has_requests()
  └ Scheduler.schedule()
    └ new_step_starts()
      └ 先遍历 running requests
      └ 再从 waiting queue admission
        └ KVCacheManager.allocate_slots()
        └ 空间不足 → preempt lowest-priority / tail request
        └ 生成 SchedulerOutput
  └ model_executor.execute_model(non_block=True)
  └ future.result() / sample_tokens()
  └ scheduler.update_from_output()
```

```
waiting Request
  ↓ get_computed_blocks(request)
local prefix hit + external connector hit
```

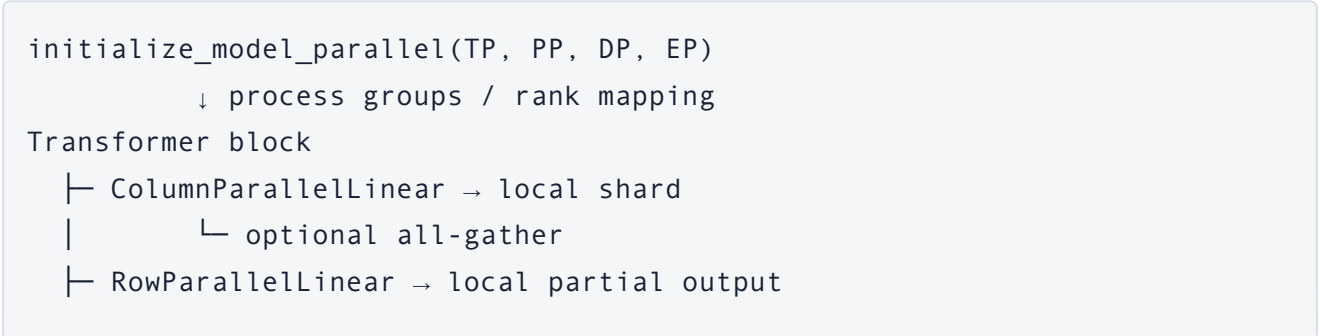


vLLM V1 用 request 的 `num_computed_tokens` 统一描述 chunked prefill、decode 和 speculative lookahead；KV admission 与 preemption 在 `schedule()` 内完成。Attention 通过 selector 选择 FlashAttention、FlashInfer、Triton 等后端；P/D、CPU offload 和远端 KV 通过 `KVConnectorBase_V1` 的 `scheduler/worker metadata` 接口接入。

与 SGLang 的代码实现对比

维度	vLLM	SGLang
主循环	<code>EngineCore.step()</code>	Scheduler event loop + <code>run_batch/process_batch_result</code>
Batch 状态	SchedulerOutput + request state	ScheduleBatch + mixin/组件
KV	block table / coordinator	radix/unified tree + token pool
Spec	lookahead slots + proposer/verify	BaseSpecWorker + <code>spec_info</code> + target verify
Attention	selector/enum/class	registry + forward-mode backend selection

TP / PP / EP / DP 代码路径



```
|      └─ tensor_model_parallel_all_reduce
├─ PP stage boundary → send/recv intermediate tensors
└─ MoE gate → expert token dispatch all-to-all
    ↓
logits / sampler → DP shard gather or rank-local result
```

代码入口: distributed/parallel_state.py:1977、distributed/
communication_op.py:12-31、model_executor/layers/linear.py:428/1621、v1/
worker/gpu_worker.py:1179/1216、layers/fused_moe/all2all_utils.py。

相关页面

- [vllm](#)
- [paged-attention](#)
- [radix-attention](#)
- [llm-inference](#)
- [inference-routing](#)